

CS 6290 — Georgia Tech OMSCS

High Performance Computer Architecture

講義ノート

kinginu

2026 年秋学期

目次

1	イントロダクション	1
1.1	コンピュータアーキテクチャとは何か	1
1.2	技術トレンド	1
1.3	速度・電力・コスト	2
1.4	消費電力	2
1.5	製造コスト	3
2	性能指標と評価	4
2.1	性能	4
2.2	性能の比較	4
2.3	性能の測定	4
2.4	性能の要約	5
2.5	性能の鉄則	5
2.6	Amdahl の法則	6
3	パイプライン	8
3.1	パイプライン化の考え方	8
3.2	パイプライン CPI	8
3.3	ストールとフラッシュ	9
3.4	制御依存	9
3.5	データ依存	10
3.6	依存とハザード	10
3.7	ハザードの処理	10
3.8	ステージ数はいくつにすべきか	11
4	分岐	12
4.1	パイプラインにおける分岐	12
4.2	予測精度と性能	13
4.3	不成立予測	13
4.4	よりよい予測の必要性	14
4.5	分岐先バッファ	15
4.6	方向予測	15
4.7	2 ビット予測器	16
4.8	履歴ベース予測器	18
4.9	共有カウンタ: PShare と GShare	19
4.10	トーナメント予測器と階層型予測器	20
4.11	リターンアドレススタック	22
5	プレディケーション	24
5.1	予測とプレディケーション	24
5.2	if 変換	25
5.3	条件付きムーブ命令	26
5.4	if 変換の性能	27
5.5	条件付きムーブの利点とコスト	28
5.6	完全プレディケーション	28
6	ILP	30
6.1	すべての命令を同一サイクルで	30
6.2	同一サイクル実行における依存	30
6.3	偽の依存の除去	31
6.4	レジスタリネーミング	31
6.5	リネーミング後の依存	32
6.6	命令レベル並列性	33
6.7	構造依存と制御依存	34
6.8	ILP と IPC	35

7 命令スケジューリング	37
7.1 IPC の向上	37
7.2 Tomasulo のアルゴリズム	37
7.3 構成	38
7.4 発行	39
7.5 ディスパッチ	40
7.6 結果書き込み	40
7.7 1 サイクル内のステップ	42
7.8 ロード命令とストア命令	42
7.9 完全な例	43
7.10 構造を追跡しないタイミングの算出	44
8 リオーダーバッファ	47
8.1 アウトオブオーダー実行における例外	47
8.2 分岐予測ミス	47
8.3 正しいアウトオブオーダー実行	48
8.4 リオーダーバッファ	49
8.5 分岐予測ミス回復	50
8.6 リオーダーバッファのもとでの例外	52
8.7 コミット時の RAT 更新	52
8.8 リオーダーバッファの例	53
8.9 リオーダーバッファのもとでのタイミング	54
8.10 統合リザベーションステーション	55
8.11 スーパースカラプロセッサ	56
8.12 用語	56
8.13 何がアウトオブオーダーなのか	56
9 メモリオーダーリング	58
9.1 メモリを介した依存	58
9.2 メモリが書かれるとき	58
9.3 ロードストアキュー	59
9.4 ロードストアのアウトオブオーダー実行とインオーダー実行	60
9.5 ストアロードフォワードリング	62
9.6 ロードストアキューの例	62
9.7 LSQ と ROB とリザベーションステーション	63
10 コンパイラによる ILP	65
10.1 コンパイラはどのように IPC を高められるか	65
10.2 木の高さ削減	66
10.3 命令スケジューリング	67
10.4 スケジューリングと if 変換	68
10.5 ループ展開	69
10.6 インライン展開	71
10.7 その他のコンパイラ技法	73
11 VLIW	74
11.1 スーパースカラプロセッサと VLIW プロセッサ	74
11.2 利点と欠点	75
11.3 VLIW 命令	76
11.4 実際の VLIW プロセッサ	77
12 キャッシュの復習	79
12.1 局所性の原理	79
12.2 キャッシュ	80
12.3 キャッシュの性能	80
12.4 ブロックとライン	81
12.5 ブロック番号, タグ, 有効ビット	82
12.6 ブロックを置ける場所	83
12.7 ダイレクトマップキャッシュ	83

12.8 セットアソシアティブキャッシュ	84
12.9 フルアソシアティブキャッシュ	85
12.10 キャッシュの置換	86
12.11 書き込み方針	87
12.12 全体のまとめ	87
13 仮想記憶	90
13.1 なぜ仮想記憶が必要か	90
13.2 物理メモリと仮想アドレス空間	90
13.3 ページとページテーブル	91
13.4 ページフォールト	92
13.5 アドレス変換	93
13.6 単一レベルページテーブルの大きさ	94
13.7 多段ページテーブル	94
13.8 ページサイズの選択	96
13.9 変換を伴うメモリアクセス時間	97
13.10 TLB	97
13.11 TLB の構成と性能	99
14 高度なキャッシュ	100
14.1 キャッシュ性能の改善	100
14.2 パイプライン化キャッシュ	100
14.3 アドレス変換とヒット時間	101
14.4 エイリアシング	103
14.5 ウェイ予測	104
14.6 置換アルゴリズムとヒット時間	105
14.7 ミスの原因	106
14.8 キャッシュブロックの大型化	107
14.9 プリフェッチ	108
14.10 ループ交換	109
14.11 ミスの重なり	110
14.12 キャッシュ階層	111
14.13 局所ミス率と大域ミス率	112
14.14 包含性	112
15 メモリ	114
15.1 メモリ技術	114
15.2 SRAM セル	114
15.3 DRAM セル	115
15.4 メモリチップの構成	117
15.5 リフレッシュ	118
15.6 アドレスの多重化	118
15.7 ファストページモード	118
15.8 DRAM とプロセッサの接続	119
参考文献	121
索引	123

イントロダクション

講義動画	Lesson 1, 動画 1-19
教科書	H&P [1] §1.1-1.6
キーワード	コンピュータアーキテクチャ, ムーアの法則, メモリウォール, 動的電力, 静的電力, 製造コスト, 歩留まり

この章では、コンピュータアーキテクチャとは何かを定義し、それが独立した学問分野として必要とされる理由を述べ、以降のすべての設計判断を制約する技術トレンドと3つの量—性能・電力・コスト—を導入する。

1.1 コンピュータアーキテクチャとは何か

コンピュータアーキテクチャとは、利用可能な製造技術を用いて、目的に適したコンピュータを設計する学問分野である。この定義には強調すべき点が2つある。

- 目的に適していること。携帯電話、ノート PC、データセンタのサーバはいずれもコンピュータだが、最適化の対象が異なる。一方では電池寿命・大きさ・重さ、他方では生の性能やコスト当たりのスループットである。したがってアーキテクチャとは唯一の最良設計を求めるのではなく、トレードオフを扱うことである。
- 利用可能な技術。アーキテクトはトランジスタそのものを設計しないが、現在のプロセスで何が実現できるか—1 チップに載るトランジスタ数、スイッチング速度、消費電力—を知っていなければならない。それらが達成可能な範囲を定めるからである。

1.1.1 なぜ必要か

コンピュータアーキテクチャの目標は2つある。

1. **性能の向上**（広い意味で）：実行速度、電池寿命、大きさ、重さ、エネルギー効率。
2. **能力の向上**：これまでできなかったことを可能にする。例えば3D グラフィックス、デバッグのハードウェア支援、セキュリティ機能。

もう1つの理由はタイミングである。今日設計するプロセッサが製造されるのは数年後なので、アーキテクトは現在のパラメータではなく、その時点で技術がどこにあるかを予測して設計しなければならない。

1.2 技術トレンド

1.2.1 ムーアの法則

ムーアの法則とは、一定面積のチップに載せられるトランジスタ数が約 18~24 か月ごとに倍増するという経験則である。物理法則ではなくトレンドだが、数十年にわたって成り立ち、業界全体の期待を形作ってきた。講義では、同じ 18~24 か月の倍増周期で次の帰結が導かれる。

- プロセッサの速度が倍になる（トランジスタが増えれば並列性を高められ、ゲートは小さく速くなる）。
- 演算当たりのエネルギーが半分になる。
- メモリ容量が倍になる。

例 1.1. 速度が2年ごとに倍になるなら、10年で $2^5 = 32$ 倍の向上が期待される。逆に言えば、4年かけて従来品の2倍しか速くないプロセッサを作った設計チームは、トレンドに追いついたにすぎない。

▷ computer architecture

出典 Lesson 1, 動画 2-3 ; H&P §1.1-1.3.

講義では「性能の向上」と「能力の向上」とまとめられている。

ヒント 後の章で過剰設計に見える判断があったら、それがどの技術トレンドを見越したものかを考えるとよい。

▷ Moore's law

出典 [2] ; Lesson 1, 動画 6 ; H&P §1.4.

1.2.2 メモリウォール

すべての構成要素が同じ曲線に従うわけではない。プロセッサ性能が 18~24 か月で倍増する一方、メモリのレイテンシは同じ期間に約 1.1 倍しか改善しなかった。その結果、プロセッサがデータを消費できる速さとメモリがそれを供給できる速さの差は世代ごとに広がる。この差を**メモリウォール**と呼び、キャッシュ（第 12~14 章）が重要である理由はここにある。

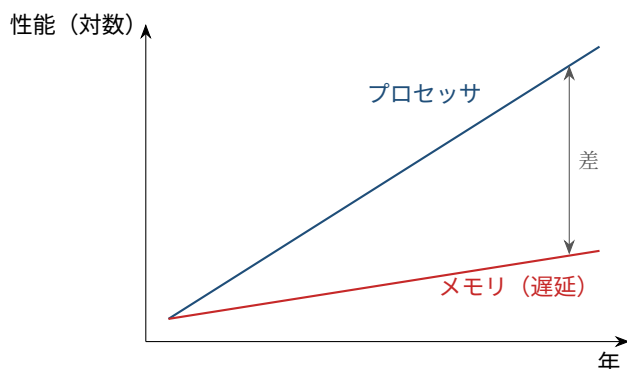


図 1.1: プロセッサ性能とメモリレイテンシは異なる速さで改善する。その差がメモリウォールである。

要点

メモリ容量はムーアの法則に追随するが、メモリのレイテンシは追随しない。本講義のメモリシステム設計の大半は、このレイテンシを隠すために存在する。

1.3 速度・電力・コスト

長い間「より良い」とは単に「より速い」を意味した。今日のアーキテクトは 3 つの量のバランスを取らなければならない。

性能 想定するワークロードをどれだけ速く実行できるか。

電力 どれだけのエネルギーを消費するか。携帯機器では電池寿命を、サーバでは冷却コストを決める。

コスト 製造にいくらかかるか。

どれが支配的かは製品による。携帯電話なら重さと電池寿命、組込みコントローラならコスト、スーパーコンピュータなら性能である。

1.4 消費電力

プロセッサが消費する電力は 2 つの成分からなる。

1.4.1 動的電力（アクティブ電力）

動的電力は、信号が切り替わるたびに配線やゲートの容量を充放電することで消費される。

$$P_{\text{dyn}} = \frac{1}{2} CV^2 f \alpha \quad (1.1)$$

ここで C はスイッチングされる総容量、 V は電源電圧、 f はクロック周波数、 α は活性化率、すなわち各サイクルで実際に切り替わる容量の割合である。

式 (1.1) から直ちに 2 つのことが分かる。

- 電力は電圧の 2 乗に比例するので、 V を下げるのが最も効果的な手段である。ただし回路が維持できる最大周波数も V とともに下がるため、実際には V と f を同時に下げることになり、電力はおおよそ V^3 に比例して減る。

出典 [3] ; Lesson 1, 動画 8 ; H&P §2.1.
▷ memory wall

出典 Lesson 1, 動画 9-10.

注意 ワークロードと電力・コスト予算を伴わない「より速く」は、意味のある設計目標ではない。

出典 Lesson 1, 動画 11-13 ; H&P §1.5.

▷ dynamic (active) power

C はチップ面積とトランジスタ数とともに増え、 α はワークロードと、どれだけの論理が遊んでいるかに依存する。

- ・電圧一定で周波数を倍にすると電力も倍になる。同じ高速化をトランジスタ数の倍増（並列ハードウェアの追加）で得る場合、電力の増加はCの増分だけである。

例 1.2. C, f , α を固定したまま電源電圧を15%下げると、動的電力は元の $0.85^2 \approx 0.72$ 倍、すなわち約28%減る。

1.4.2 静的電力

静的電力は、信号が切り替わらなくても消費される。トランジスタは完全なスイッチではなく、オフでも電流が漏れるからである。リーク電流は、電源電圧—それに伴ってトランジスタのしきい値電圧—を下げるほど大きくなる。したがってトレードオフがある。Vを下げると動的電力は減るが静的電力は増え、合計 $P = P_{\text{dyn}} + P_{\text{static}}$ はある中間の電圧で最小になる（図 1.2）。

要点 – 電力

動的電力 $\propto CV^2f\alpha$ は低電圧を好み、静的電力は低電圧を罰する。現代の設計は両者の和の最小点付近にあり、さらなる改善はVよりもC, α , f の削減から得ている。

1.5 製造コスト

チップは円形のシリコン**ウェハ**上に製造され、各ウェハは多数の同一の長方形の**ダイ**に切り分けられる。ウェハ1枚を処理するコストはほぼ一定なので、ダイ1個のコストは次式で与えられる。

$$\text{ダイ当たりコスト} = \frac{\text{ウェハ当たりコスト}}{\text{ウェハ当たりダイ数} \times \text{歩留まり}} \quad (1.2)$$

1.5.1 歩留まり

すべてのダイが動作するわけではない。ウェハ上には欠陥が散在し、欠陥を含むダイは通常廃棄される。**歩留まり**とは、機能するダイの割合である。欠陥はランダムな位置に生じるため、大きなダイほど欠陥を含みやすく、歩留まりはダイ面積が増えるにつれて低下する。

1.5.2 帰結

式(1.2)と歩留まりの振る舞いを合わせると、この節の中心的な結論が得られる。ダイ面積を半分にすると、ダイ当たりコストは半分以下になる。ウェハ当たりのダイ数が倍になり、かつ歩留まりも上がるからである。ムーアの法則と合わせて読むと、アーキテクトには技術世代ごとに2つの選択肢がある。

- ・ダイ面積を一定に保ち、2倍のトランジスタを詰め込む—ほぼ同じコストでより高機能なチップ。
- ・設計を一定に保ち、面積を半分に縮小する—同じチップを半分よりずっと安いコストで。

例 1.3. ウェハが\$5000で、ある設計のダイが100個取れ、歩留まりが50%だとする。動作するダイ1個のコストは $5000/(100 \times 0.5) = \100 である。縮小によりウェハ当たり200個取れ、歩留まりが70%に上がると、コストは $5000/(200 \times 0.7) \approx \36 と半分以下になる。

要点 – 章のまとめ

アーキテクチャとは、技術トレンド（ムーアの法則）とその例外（メモリウォール）に駆動されながら、性能・電力・コストという3つの制約の下でトレードオフを設計することである。次章では「性能」を厳密に定義する。

▷ static (leakage) power

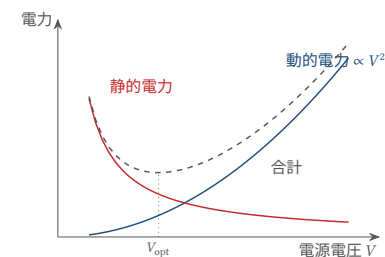


図 1.2: 電源電圧を変えると、動的電力と静的電力は逆方向に動く。

▷ wafer

▷ die

出典 Lesson 1, 動画 15–17; H&P §1.6.

▷ yield

H&P §1.6 には経験的モデル 歩留まり $\approx 1/(1 + \text{欠陥密度} \times \text{面積})^N$ が示されている。

性能指標と評価

講義動画	Lesson 2, 動画 1-25
教科書	H&P [1] §1.8-1.9
キーワード	レイテンシ, スループット, 高速化率, ベンチマーク, 幾何平均, 性能の鉄則, CPI, Amdahl の法則, 収穫逡減

性能を改善するには、まず性能を測定し、2つのマシンを比較し、変更の効果を予測できなければならない。この章では用語（レイテンシ、スループット、高速化率）、方法論（ベンチマークとその要約法）、そして本講義を通じて繰り返し現れる2つの解析ツール—性能の鉄則と Amdahl の法則—を導入する。

2.1 性能

「性能」は、何を測るのかを言わない限り曖昧である。

定義 2.1 (レイテンシ). タスクの**レイテンシ**とは、開始から完了までの時間である。

定義 2.2 (スループット). システムの**スループット**とは、単位時間当たりに完了するタスク数である。

スループットは単純にレイテンシの逆数ではない。タスクが重なって（パイプライン化、第3章）あるいは並列に（マルチプロセッサ）実行される場合、個々のタスクに長い時間がかかっても、システムは毎秒多数のタスクを完了できる。

例 2.3. 2つのプログラムを順に実行し、それぞれに 10 s かかるプロセッサは、レイテンシがプログラム当たり 10 s、スループットが 0.1 programs/s である。プログラム当たり 15 s かかるが2つを同時に実行できる別のプロセッサは、レイテンシは劣るがスループットは $2/15 \approx 0.13$ programs/s となる。

2.2 性能の比較

マシン X がマシン Y より「N 倍速い」と言うために**高速化率**を用いる。

$$N = \frac{\text{speed}(X)}{\text{speed}(Y)} = \frac{\text{latency}(Y)}{\text{latency}(X)} = \frac{\text{throughput}(X)}{\text{throughput}(Y)} \quad (2.1)$$

レイテンシは逆向きに現れることに注意する。時間が短いほど速いからである。高速化率が 1 より大きければ X が優れ、1 未満なら劣る（「高速化率 0.8」は減速である）。高速化率は乗法的に合成される。X が Y の 2 倍速く、Y が Z の 3 倍速ければ、X は Z の 6 倍速い。

2.3 性能の測定

理想的な測定は、ユーザの実際のワークロードを実際のマシンで走らせることである。これが可能なことは稀で—マシンはまだ存在しないかもしれず、ワークロードは通常不明である—そこで**ベンチマーク**に頼る。ベンチマークとは、実ワークロードの代理としてコミュニティが合意したプログラムと入力データの組である。そのようなプログラムの集まりをベンチマークスイートと呼ぶ。

2.3.1 ベンチマークの種類

代表性の高い順に:

実アプリケーション 現実的な入力を伴う完全なプログラム。最も代表的だが、準備が最も難しく、シミュレータや未完成のプロトタイプでは実行できない。

出典 Lesson 2, 動画 2-3 ; H&P §1.8.

▷ latency

▷ throughput

注意 スループット = $1/\text{レイテンシ}$ が成り立つのは、タスクを一度に 1 つずつ、厳密に順番に処理する場合だけである。

▷ speedup

出典 Lesson 2, 動画 4-7.

ヒント 比は常に 旧時間/新時間 の形で保つこと。本講義で生じる符号の誤りの大半は、この比の逆転が原因である。

▷ benchmark

出典 Lesson 2, 動画 8-10 ; H&P §1.8.

カーネル 実アプリケーションの最も時間を消費する部分（例えば行列積）を切り出し、単独で実行できるようにしたもの。実行しやすく、実際の処理を反映している。

合成ベンチマーク 実アプリケーションと同様に振る舞う（命令やメモリアクセスの構成が似ている）ように書かれた、より実行の簡単なプログラム。設計の初期段階で有用。

ピーク性能 ハードウェアが理論上維持できる最大レート（例えば全ユニットが常に稼働した場合の毎秒命令数）。実プログラムが到達することのないマーケティング用の数値。

注 2.4. ピーク性能はベンチマーク結果ですらない。実プログラムがどれだけそれに近づけるかを何も語らない上限値である。

2.3.2 ベンチマークの標準

ベンダは自社製品に有利なベンチマークを選ぶため、独立した組織が標準スイートを定めている。SPEC (Standard Performance Evaluation Corporation) は CPU, グラフィックス, サーバのスイートを公開し, TPC はトランザクション処理ワークロードに, EEMBC は組み込みシステムに対して同様のことを行う。これらの組織の会員は, プログラム, 入力, 報告規則について合意する。

出典 Lesson 2, 動画 12 ; [4].

2.4 性能の要約

スイートからはプログラムごとに 1 つの数値が得られるが, 通常はマシンごとに 1 つの数値が欲しい。

- ・ **実行時間** は通常の算術平均で合成してよい。平均時間は well-defined であり, 2 台のマシンの平均時間の比は総実行時間の高速化率に等しい。
- ・ **高速化率 (比)** を算術平均してはならない。比の算術平均はどちらのマシンを基準にするかに依存し, 平均時間の比とも一致しない。幾何平均を用いる。

$$\bar{S} = \left(\prod_{i=1}^n S_i \right)^{1/n} \quad (2.2)$$

例 2.5. プログラム A はマシン X で 10 s, Y で 20 s かかり, プログラム B は X で 20 s, Y で 10 s かかる。X の Y に対するプログラムごとの高速化率は 2 と 0.5 である。算術平均 1.25 は X が速いことを示唆するが, Y を基準にして計算した算術平均も 1.25 となり, Y が速いことを示唆する一矛盾である。幾何平均はどちら向きでも $\sqrt{2 \times 0.5} = 1$ となり, このスイートでは両マシンが同等であることを正しく報告する。

要点

時間は算術平均, 高速化率は幾何平均で平均する。比の平均が平均の比に等しくなる平均は幾何平均だけである。

出典 Lesson 2, 動画 13–14 ; H&P §1.8.

▷ geometric mean

2.5 性能の鉄則

性能の鉄則 は, CPU 時間を, 計算機システムの異なる部分に対応する 3 つの因子に分解する。

$$\text{CPU 時間} = \underbrace{\text{命令数}}_{\text{IC}} \times \underbrace{\frac{\text{サイクル数}}{\text{命令}}}_{\text{CPI}} \times \underbrace{\frac{\text{秒}}{\text{サイクル}}}_{T_{\text{clk}}} \quad (2.3)$$

命令数 (IC) アルゴリズム, コンパイラ, 命令セット (ISA) で決まる。

命令当たりサイクル数 (CPI) ISA とプロセッサのマイクロアーキテクチャ (パイプライン化, アウトオブオーダー実行, キャッシュなど) で決まる。本講義の大半は CPI を下げることに関するものである。

▷ iron law of performance

出典 Lesson 2, 動画 15–18 ; H&P §1.9.

クロックサイクル時間 (T_{clk}) マイクロアーキテクチャ (2 つのラッチの間にとれだけの論理があるか), 回路設計, 製造技術で決まる。

この法則の価値は, トレードオフを露わにすることにある。1 つの因子を減らす変更は, しばしば別の因子を増やす。深いパイプラインは T_{clk} を下げるが CPI を上げる。高性能な ISA は IC を下げるが CPI や T_{clk} を上げうる。

2.5.1 命令ごとに時間が異なる場合

命令クラスごとに必要なサイクル数が異なる場合, CPI は重み付き平均になる。 n 個のクラスについて,

$$\text{CPU 時間} = \left(\sum_{i=1}^n \text{IC}_i \times \text{CPI}_i \right) \times T_{\text{clk}} \quad (2.4)$$

ここで IC_i , CPI_i はクラス i の命令数と命令当たりサイクル数である。

例 2.6. あるプログラムが 2 GHz のプロセッサで 10^9 命令を実行する。60% が ALU 演算 (1 サイクル), 30% がロード/ストア (2 サイクル), 10% が分岐 (3 サイクル) である。平均 CPI は $0.6 \cdot 1 + 0.3 \cdot 2 + 0.1 \cdot 3 = 1.5$ なので, CPU 時間は $10^9 \times 1.5 \times 0.5 \text{ ns} = 0.75 \text{ s}$ である。

2.6 Amdahl の法則

Amdahl の法則は, 実行の一部だけを高速化したときの全体の高速化率を予測する。

定義 2.7 (Amdahl の法則). ある改善が元の実行時間のうち割合 f_{enh} を s_{enh} 倍速くするとき, 全体の高速化率は

$$S_{\text{overall}} = \frac{1}{(1 - f_{\text{enh}}) + \frac{f_{\text{enh}}}{s_{\text{enh}}}} \quad (2.5)$$

である。

例 2.8. 浮動小数点命令が実行時間の 40% を占める。新しい FP ユニットによりそれらが 4 倍速くなった。全体の高速化率は $1/(0.6 + 0.4/4) = 1/0.7 \approx 1.43$ である。FP ユニートを無限に速くしても $1/0.6 \approx 1.67$ にしか届かない。

2.6.1 含意

式 (2.5) で $s_{\text{enh}} \rightarrow \infty$ とすると, 上限 $S_{\text{overall}} \leq 1/(1 - f_{\text{enh}})$ が得られる。ここから 2 つの設計原則が導かれる。

要点 – 頻出ケースを速くせよ

時間の大きな部分をそこそこ速くする方が, 小さな部分を劇的に速くするより効く。プログラムが実際に時間を費やしている部分を最適化すること。

講義では冗談めかして「Lhadma の法則」と呼ばれる逆命題も述べられる。頻出ケースを速くする際に, 稀なケースを犠牲にしすぎてはならない。ある最適化で手を付けなかった 10% の部分が副作用で 10 倍遅くなれば, その部分だけで元のプログラム全体と同じ時間がかかり, 全体としては減速になる。

2.6.2 収獲逡減

実行の同じ部分を繰り返し改善すると**収獲逡減**が生じる。その部分が縮んでしまえば, もはや頻出ケースではなく, 残りの時間はそれ以外のすべてに支配される。改善のたびにアーキテクトは再測定し, 今の頻出ケースは何かを問い直すべきである。

注意 変更を単一の因子で評価してはならない。CPI と IC が不変でない限り, 「クロック周波数が高い」は「速い」ではない。

▷ Amdahl's law

出典 [5]; Lesson 2, 動画 19–22; H&P §1.9.

注意 f_{enh} は元の時間に対する割合であって, 変更後の時間に対する割合でも, 命令数に対する割合でもない。

出典 Lesson 2, 動画 21–24.

▷ diminishing returns

例 2.9. 前の例 (FP = 時間の 40%) から始めると, 最初の FP 2 倍高速化で $S = 1/(0.6 + 0.2) = 1.25$ が得られる. 新しいマシンでは FP は時間の $0.2/0.8 = 25\%$ しか占めないで, 2 回目の FP 2 倍高速化はわずか $1/(0.75 + 0.125) \approx 1.14$ にしかない.

要点 – 章のまとめ

状況に応じてレイテンシまたはスループットで測り, 高速化率で比較し, 幾何平均で要約する. 性能の鉄則 (式 (2.3)) は時間を IC・CPI・クロック周期に帰着させ, Amdahl の法則 (式 (2.5)) は部分的改善で達成できる上限を定める.

パイプライン

講義動画	Lesson 3, 動画 1-18
教科書	H&P [1] 付録 C.1-C.3
キーワード	パイプライン, ステージ, パイプライン CPI, ストール, フラッシュ, 制御依存, データ依存, RAW, WAR, WAW, ハザード, フォワードリング, パイプライン深さ

パイプライン化はスループットを高めるための最初にして最も基本的な技法である。1つの命令を終えてから次を始めるのではなく、プロセッサは複数の命令の実行を、それぞれ異なる進行段階で重ね合わせる。この章では、パイプライン化されたプロセッサの動作を復習し、理想的な性能を定量化した後、理想の達成を妨げる命令間の依存を調べる。

3.1 パイプライン化の考え方

命令の実行は、異なるハードウェアを使う一連のステップからなる。ステップをラッチで区切られた**ステージ**として配置すれば、命令 I_1 がステージ 1 を離れてステージ 2 に進んだ瞬間に、命令 I_2 がステージ 1 に入れる。k ステージなら、最大 k 個の命令が同時に処理中となる。

第2章の2つの指標に対する帰結は異なる。

- **スループット**は最大 k 倍に向上する。定常状態では、k ステージ時間ごとではなく 1 ステージ時間ごとに 1 命令が完了する。
- 個々の命令の**レイテンシ**は改善しない。総作業量は同じであり、ステージ間のラッチがわずかなオーバーヘッドを加えるため、命令当たりのレイテンシはむしろ少し増える。

3.1.1 プロセッサにおけるパイプライン

古典的な 5 段パイプライン（図 3.1）を、本講義を通じて基準となる設計として用いる。



図 3.1: 5 段の命令パイプライン。

IF 命令フェッチ: プログラムカウンタ (PC) が指す命令を命令メモリから読み、PCを進める。

ID 命令デコードとレジスタ読み出し: 演算の種類を決定し、ソースオペランドをレジスタファイルから読む。

EX 実行: ALU 演算を行う、あるいはロード/ストアのメモリアドレスを計算する、あるいは分岐条件を評価する。

MEM メモリ: ロード/ストアのためにデータメモリにアクセスする。

WB 書き戻し: 結果をレジスタファイルに書く。

図 3.2 は独立な 5 命令がパイプラインを流れる様子を示す。最初の 4 サイクルでパイプラインが満たされた後は、毎サイクル 1 命令が完了する。

3.2 パイプライン CPI

図 3.2 の理想的な場合、パイプラインは毎サイクル 1 命令を完了するので、性能の鉄則（式 (2.3)）の CPI は 1 である。これを 1 より大きくする効果が 2 つある。

▷ stage

出典 Lesson 3, 動画 2-3 ; H&P §C.1.

実際のプロセッサは 10~20 段だが、5 段モデルでここで重要な問題はすべて捉えられる。

出典 Lesson 3, 動画 6 ; H&P §C.1.

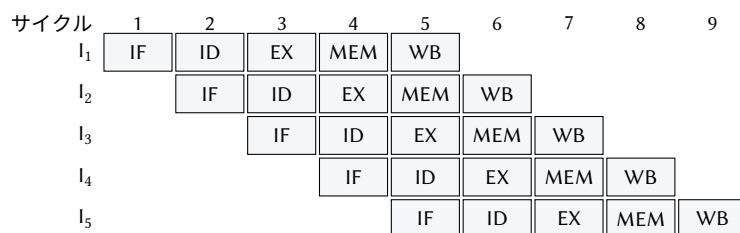


図 3.2: 5 段パイプラインの命令タイミング。サイクル 5 以降は毎サイクル 1 命令が完了する。

- 充填と排出。最初の命令が完了するまでに k サイクルかかる。 n 命令のプログラムでは合計 $k + (n - 1)$ サイクル，すなわち $CPI = (k + n - 1)/n \rightarrow 1$ (n が大きいとき) となる。実プログラム (n は数十億) では無視できる。
- ストール。パイプラインが命令を次のステージへ進められないとき，命令が完了しないままサイクルが過ぎる。パイプライン化されたプロセッサの CPI を実際に決めるのはこのサイクルである。

$$CPI = 1 + \frac{\text{ストールサイクル数}}{\text{命令}} \quad (3.1)$$

3.3 ストールとフラッシュ

パイプラインでサイクルを失う機構は 2 つある。

定義 3.1 (ストール)。 ストールとは，先行する命令は進み続ける一方で，ある命令（とその後続すべて）を現在のステージに 1 サイクル以上留めることである。その代わりに，バブル—空きスロット—がパイプラインを流れていく。

定義 3.2 (フラッシュ)。 フラッシュとは，すでにパイプラインにフェッチされた命令を，そもそも実行すべきでなかったという理由で破棄することである。典型的には，結果を誤って予測した分岐の後続命令が対象となる。

c サイクルのストールは c サイクルを， m 命令のフラッシュは m サイクルを失わせる。フラッシュされた各命令は，有用な命令が完了できたはずのスロットを占めていたからである。

3.4 制御依存

制御依存とは，ある命令がそもそも実行されるかどうか，先行する分岐の結果に依存することである。5 段パイプラインでは，分岐の条件と飛び先は EX の終わりまで分からない。その時点で後続の 2 命令はすでにフェッチされている（図 3.3）。分岐が成立していれば，それらをフラッシュし，飛び先からフェッチをやり直さなければならない。

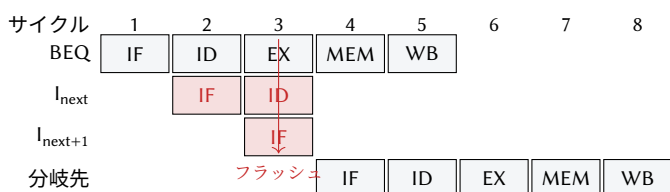


図 3.3: EX で解決される成立分岐は，2 つのフェッチスロットを無駄にする。

これが CPI に与えるコストは，どれだけ頻繁に起こるかに依存する。講義で使われる典型的な数値は，命令の 20% が分岐で，分岐の 60% が成立するというものである。常に次の命令をフェッチする単純な方針（「不成立予測」）では，成立した分岐だけがフラッシュを引き起こす。

$$CPI = 1 + 0.2 \times 0.6 \times 2 = 1.24 \quad (3.2)$$

出典 Lesson 3，動画 7-8；H&P §C.2.

▷ stall

▷ flush

▷ control dependence

出典 Lesson 3，動画 9-10；H&P §C.2.

例 3.3. 分岐がステージ 10 で解決されるより深いパイプラインでは、成立分岐ごとに 9 命令がフラッシュされる。同じ分岐統計で CPI は $1 + 0.2 \times 0.6 \times 9 = 2.08$ となり、プロセッサは 2 サイクルに 1 命令も完了できない。パイプラインが深いほど分岐予測（第 4 章）が重要になるのはこのためである。

3.5 データ依存

データ依存は、2つの命令が同じレジスタまたはメモリ位置にアクセスし、少なくとも一方がそれを書き込むときに存在する。プログラム順でのアクセスの順序にちなんで名付けられた 3 種類がある。

RAW (read after write) 後の命令が、先の命令が書く値を読む。真の依存あるいはフロー依存とも呼ばれる。データが実際に一方から他方へ流れなければならない。

WAR (write after read) 後の命令が、先の命令が読む位置に書く。逆依存。

WAW (write after write) 両命令が同じ位置に書く。後の書き込みが残らなければならない。出力依存。

WAR と WAW は**偽の依存**あるいは**名前依存**と呼ばれる。命令間でデータは移動せず、単に同じレジスタ名を再利用しているだけだからである。これらはリネーミング（第 6 章）で除去できる。RAW 依存は除去できない。

例 3.4. I1: ADD R1, R2, R3 ; R1 <- R2 + R3
I2: SUB R4, R1, R5 ; I1 と R1 について RAW
I3: MUL R1, R6, R7 ; I1 と R1 について WAW, I2 と R1 について WAR

3.6 依存とハザード

依存はプログラムの性質であり、それが問題を起こすかどうかはパイプラインに依存する。

定義 3.5 (ハザード). **ハザード**とは、特定のパイプラインがそのまま進行すると依存に違反してしまう状況—依存する命令が古い値を読んだり、実行すべきでないときに実行されたりする状況—である。

5 段パイプラインでは:

- **RAW 依存**は、命令が十分近いときだけハザードになる。レジスタは WB で書かれ ID で読まれるので、ある命令は直前の命令とその前の命令に対してハザードを持つが、3 つ以上後ろの命令は書き込み後にレジスタを読む。
- **WAR** と **WAW 依存**はここではハザードにならない。命令はプログラム順に ID でレジスタを読み WB で書くので、後の命令が先の命令を追い越すことはないからである。これらはアウトオブオーダーのパイプライン（第 6 章）でハザードになる。
- **制御依存**は常にハザードである。パイプラインは分岐が解決する前に後続命令をフェッチするからである。

要点

依存はプログラムが決め、ハザードはパイプラインが決める。同じプログラムでも、より深い、あるいはアウトオブオーダーのパイプラインではハザードが増える。

3.7 ハザードの処理

ハザードを検出したら、パイプラインには 3 つの選択肢がある。

1. 依存する命令をフラッシュする—誤った命令がフェッチされた制御ハザードの場合。

ヒント 制御依存で失うサイクル = (分岐命令の割合) × (予測を外す割合) × (予測ミス当たりのフラッシュ命令数)。

▷ data dependence

出典 Lesson 3, 動画 11–12; H&P §C.2.

▷ false dependence

RAR—同じレジスタの 2 回の読み出し—は依存ではない。順序は問題にならない。

出典 Lesson 3, 動画 13–14; H&P §C.2.

▷ hazard

サイクルの前半に書き、後半に読むレジスタファイルなら、この距離は 1 つ縮む。

出典 Lesson 3, 動画 15–16; H&P §C.2.

- 必要な値が書き込まれるまで依存する命令を**ストール**する—常に正しいが、サイクルを失う。
- フォワーディング**（バイパス）で値を**補う**：結果をレジスタファイルに到達するのを待たずに、生成したステージから必要とするステージへ直接送る（図 3.4）。

▷ forwarding

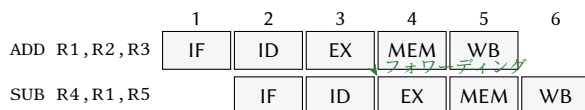


図 3.4: フォワーディング: ADD の ALU 結果を SUB の EX ステージへ直接送り，ストールを回避する。

フォワーディングは RAW ストールの大半を除去するが、すべてではない。値を時間を遡って送ることはできないからである。ロードの結果は MEM の終わりにならないと得られないが、次の命令はそれを自身の EX の開始時—同じサイクル—to必要とする。したがってロード使用の組では、完全なフォワーディングがあっても 1 サイクルのストールが避けられない。

例 3.6.

```

LW  R1, 0(R2)      ; R1 は MEM の終わり（サイクル 4）に得られる
ADD R3, R1, R4      ; EX の開始（サイクル 4）に R1 が必要: 1 サイクル
                        ストール

```

フォワーディングありでストールなしだと ADD は古い R1 を使ってしまう。フォワーディングなしなら 2 サイクルストールする。

3.8 ステージ数はいくつにすべきか

k 段にパイプライン化するとスループットが最大 k 倍になるなら、なぜ数百段にしないのか。深さを増やすと 3 つの効果異なる方向に働く。

- ・ **クロック周期が短くなる**（ステージ当たりの論理が減る）。これは性能の鉄則の T_{clk} を下げる、意図された利点である。しかしステージ当たりのラッチのオーバーヘッドは縮まないで、利得は頭打ちになる。
- ・ **CPI が増える**：ステージが増えると、分岐とその解決の間に処理中の命令が増え、予測ミス当たりのフラッシュ命令数が増える。また RAW ハザードにストールが必要な距離も伸びる。
- ・ **電力が増える**：毎サイクル切り替わるラッチが増え、周波数の上昇が動的電力（式 (1.1)）を押し上げる。

性能のみ（ $\text{CPI} \times T_{\text{clk}}$ の積）を考えると最適は深く、30～40 段程度である。電力を考慮に入れると最適はおよそ 10～15 段に移り、これが現在の設計が位置する範囲である。

要点 — 章のまとめ

パイプライン化はスループットを上げるがレイテンシは上げない。理想 CPI は 1 で、すべてのストールとフラッシュがそれに加算される（式 (3.1)）。制御依存と RAW 依存はサイクルを失わせるハザードとなる。フォワーディングは RAW ストールの大半を除去し、分岐予測（次章）が制御ハザードに対処する。パイプラインの深さはクロック周期と CPI・電力とのトレードオフである。

出典 Lesson 3, 動画 17; H&P §C.1, §1.5.

分岐

講義動画	Lesson 4, 動画 1-48
教科書	H&P [1] §3.3, §3.9, §C.2
キーワード	分岐予測, 予測ミスペナルティ, 不成立予測, 分岐先バッファ, 分岐履歴テーブル, 2 ビット予測器, 履歴ベース予測器, PShare, GShare, トーナメント予測器, 階層型予測器, リターンアドレススタック, プリデコード

第 3 章では、制御依存がどのパイプラインでもハザードになることを見た。プロセッサは、分岐の行き先が分かるよりずっと前に分岐の後続命令をフェッチしなければならない、選択を誤るたびに作業がフラッシュされる。この章では、フェッチステージがその選択をどのように行うかを調べる。最も単純な方針—常に次の命令に進む—から出発して、実際のプロセッサで使われる構造を順に導入する。すなわち、分岐先バッファ、飽和カウンタ、分岐履歴からパターンを学習する予測器、複数の予測器の組合せ、そして関数からのリターン専用の予測器である。その過程で、パイプラインが深いほど予測精度が重要になる理由も示す。

4.1 パイプラインにおける分岐

第 3 章の 5 段パイプラインで、条件分岐 BEQ R1,R2,L を考える。R1 と R2 が等しければ、実行はラベル L から続く。このとき分岐は**成立分岐**であり、L のアドレスがその**分岐先アドレス**である。

そうでなければ、実行はメモリ上の次の命令、すなわち PC+4 から続き、分岐は**不成立分岐**となる。どちらの場合であるかをパイプラインが知るのは、分岐が 3 つのステージを通過した後である。IF が分岐をフェッチし、ID が R1 と R2 を読み、EX がそれらを比較して分岐先を計算する。分岐は EX の終わりに解決される (図 4.1)。

フェッチステージはそれを待てない。分岐が ID にあるサイクル 2 にも、続くサイクル 3 にも、フェッチステージは何らかの命令を読み込まなければならない。分岐が解決されるとき、その後ろにはすでに 2 命令がパイプラインに入っている。それらが本当に分岐の後に続く命令であれば何も失われないが、そうでなければフラッシュされ、2 サイクルが無駄になる。この間に何もフェッチしなければ同じ 2 サイクルが無条件に無駄になるので、正しい可能性が最も高い命令をフェッチすることが常に得策である。

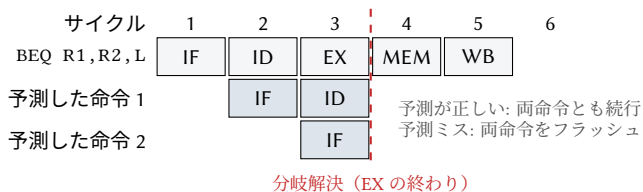


図 4.1: 5 段パイプラインにおける分岐。分岐が解決される前にフェッチされる 2 命令は予測であり、予測が誤っていればフラッシュされる。

定義 4.1 (分岐予測). 現在の命令がデコードも実行もされないうちに、次の命令をフェッチするアドレスをフェッチステージで選ぶことを**分岐予測**という。

定義 4.2 (予測ミス). **分岐予測ミス**は、解決された分岐が予測とは別の場所へ進むと判明したときに生じる。誤ったアドレスからフェッチされた命令はフラッシュされ、正しいアドレスからフェッチがやり直される。

出典 Lesson 4, 動画 1-3 ; H&P §C.2.

- ▷ taken branch
- ▷ branch target address
- ▷ not-taken branch

- ▷ branch prediction
- ▷ branch misprediction

4.1.1 予測器が知っていること

フェッチの時点で得られる情報は、フェッチ中の命令の PC だけである。命令そのものはまだデコードすらされていない。このアドレスだけから、予測器は 3 つの問いに答えなければならない。

1. その命令はそもそも分岐か。
2. 分岐であれば、成立するか。
3. 成立するならば、分岐先アドレスはどこか。

答えは次の PC という 1 つの数である。分岐でない命令と不成立分岐では PC+4、成立分岐では分岐先アドレスとなる。

4.2 予測精度と性能

予測器があれば、制御ハザードで失われるサイクルは、予測ミスした経路で費やされたサイクルだけになる。式 (3.1) を分岐に特化すると次式を得る。

$$\text{CPI} = 1 + \frac{\text{予測ミス数}}{\text{命令}} \times \frac{\text{ペナルティ}}{\text{予測ミス}} \quad (4.1)$$

定義 4.3 (精度とペナルティ). **分岐予測精度**とは、結果が正しく予測される分岐の割合である。したがって

$$\frac{\text{予測ミス数}}{\text{命令}} = \frac{\text{分岐数}}{\text{命令}} \times (1 - \text{精度})$$

となる。ここで、分岐でない命令は次の PC を PC+4 として常に正しく予測されると仮定している。**予測ミスペナルティ**とは、予測ミス 1 回当たりに失われるサイクル数である。1 ステージに 1 命令を保持するパイプラインでは、フラッシュされる命令数に等しい。ステージ r で解決される分岐のペナルティは $r-1$ サイクルである。

式 (4.1) の 2 つの因子は由来が異なる。第 1 の因子は予測器（およびプログラムが実行する分岐の数）で決まり、第 2 の因子はパイプライン、すなわち分岐がどれだけ遅く解決されるかで決まる。よりよい予測器は第 1 の因子を小さくし、より深いパイプラインは第 2 の因子を大きくする。そのため同じ予測器でも、深いパイプラインほどコストが大きくなる。完全な予測器であれば、深さによらず理想の CPI 1 に到達する。

例 4.4. あるプログラムでは命令の 15 % が分岐であり、予測器の精度は 92 % である。命令当たりの予測ミス数は $0.15 \times 0.08 = 0.012$ である。分岐が EX（ステージ 3、ペナルティ 2）で解決されるなら $\text{CPI} = 1 + 0.012 \times 2 = 1.024$ となる。分岐をステージ 11（ペナルティ 10）で解決するパイプラインでは、同じ予測器で $\text{CPI} = 1 + 0.012 \times 10 = 1.12$ となる。

4.3 不成立予測

予測に代わる方法は、予測を拒むことである。フェッチステージは次の PC が分かるまで単に待つ。命令が分岐かどうかさえ ID の後にならないと分からないので、この待ちはずべての命令に影響する。分岐でない命令の次のフェッチはその命令自身のフェッチの 2 サイクル後になり、分岐の次のフェッチは、成立・不成立にかかわらず 3 サイクル後になる。

表 4.1: 5 段パイプラインにおいて、ある命令のフェッチから正しい次の命令がフェッチされるまでのサイクル数。

命令	予測を拒む	不成立予測
成立分岐	3	3
不成立分岐	3	1
分岐以外	2	1

無条件ジャンプ、コール、リターンは常に成立する分岐であり、これらに関するのは問い 1 と 3 だけである。

出典 Lesson 4, 動画 4-5 ; H&P §C.2.

▷ branch prediction accuracy

▷ misprediction penalty

出典 Lesson 4, 動画 6-8 ; H&P §C.2.

最も単純な実用的予測器は**不成立予測**，すなわち常にPC+4からフェッチすることである。フェッチステージはどのみちPCをインクリメントするので，これにはメモリも，フェッチステージがすでに持つもの以外のハードウェアも必要ない。分岐でない命令と不成立分岐に対しては常に正しく，誤るのは成立分岐だけであり，そのコストは予測を拒む場合と同じく3サイクルである。表4.1は2つの方針を比較している。不成立予測が劣ることは決していない。

しかし，その精度は低い。大半の分岐は成立するループの分岐はたいいてい後方へジャンプし，無条件ジャンプ，コール，リターンは常に成立する一ので，不成立予測は分岐の過半数を予測ミスする。第3章で用いた典型的な統計では，全命令の12%で予測ミスが起こり，式(3.2)のCPIとなる。

注 4.5 (処理中の複数の予測)。深いパイプラインでは，IFと解決ステージの間に複数の分岐が同時に存在する。それぞれの分岐は，フェッチ時に行われた予測を伴っている。ある分岐が予測ミスと判明すると，その後にフェッチされたものは，後続の分岐とその予測も含めてすべてフラッシュされる。したがって，誤った経路でフェッチされた分岐がそれ自身のフラッシュを引き起こすことはない。サイクルを失わせるのは，正しい経路上にある分岐の予測ミスだけである。

4.4 よりよい予測の必要性

よりよい予測器にどれだけの価値があるかは，パイプラインに依存する。1ステージ当たりw命令を扱うプロセッサは，同じ深さで幅が1命令のパイプラインと比べて，予測ミス1回当たりw倍の命令をフラッシュする。したがって，完全な予測に対する相対実行時間は

$$\frac{T}{T_{\text{perfect}}} = 1 + \frac{\text{予測ミス数}}{\text{命令}} \times (\text{予測ミス当たりのフラッシュ命令数}) \quad (4.2)$$

であり，w=1のときはちょうど式(4.1)のCPIになる。

例 4.6。命令の15%が分岐で，そのうち70%が成立するとする。このとき不成立予測の精度は30%である。表4.2は，3種類の予測器と3種類のパイプラインについて式(4.2)を評価したものである。パイプラインは，5段パイプライン（フラッシュ2命令），分岐をステージ13で解決するパイプライン（フラッシュ12命令），および同じ深さで1ステージ当たり3命令を扱うパイプライン（フラッシュ36命令）である。

表 4.2: 例 4.6 の分岐統計における，完全な予測に対する相対実行時間。

予測器	予測ミス/命令	ステージ 3	ステージ 13	ステージ 13, 幅 3
不成立 (30%)	0.105	1.21	2.26	4.78
精度 90%	0.015	1.03	1.18	1.54
精度 99%	0.0015	1.003	1.018	1.054

ここから3つのことが分かる。浅いパイプラインでは，不成立予測でさえコストは21%にすぎず，精度を90%から99%に改善しても利得は3%に満たない。深いパイプラインでは，不成立予測は実行時間を2倍以上にし，同じ改善は1.18/1.018 ≈ 1.16の高速化に相当する。深く幅も広いパイプラインでは1.54/1.054 ≈ 1.46に相当する。現代のプロセッサは深く，かつ幅も広い。だからこそ分岐予測に多大な投資をしているのである。

要点

重要なのは精度ではなく予測ミス率である。精度90%から99%への向上は小さな一歩に見えるが，予測ミスの90%を取り除く。そしてパイプラインが深く幅広いほど，残った予測ミス1回当たりのコストは大きくなる。

▷ predict not taken

出典 Lesson 4, 動画 9-11 ; H&P §3.3.

4.5 分岐先バッファ

不成立予測は、フェッチ中の命令に関する情報を何も使わない。よりよい予測器はその履歴、すなわちこの PC にある命令が過去に実行されたときに何をしたかを用いる。次の PC は、PC とこの履歴の関数として予測される。あるアドレスに格納された命令は変化しないので、それが分岐かどうか、直接分岐がどこへジャンプするかは毎回同じである。また、ほとんどの分岐はたいいてい同じ方向に進む。

定義 4.7 (分岐先バッファ). 分岐先バッファ (BTB) とは、フェッチ中の命令の PC でインデックスされ、その命令の予測された次の PC を格納するテーブルである。

IF では現在の PC で BTB を読み、その出力を次の PC として用いる。分岐が解決されると、実際の次の PC を予測と比較する。一致しなければ、誤ってフェッチされた命令をフラッシュし、実際の分岐先を BTB のエントリに書き込む。こうして、同じ命令の次の実行は新しい分岐先で予測される。

4.5.1 現実的な BTB

BTB は、あり得るすべての命令アドレスに 1 つずつエントリを持つことはできない。BTB はフェッチそのものに加えて同じフェッチサイクル内で読まなければならないので、小さなテーブル—数百から数千エントリ—に限られる。そこで PC は単純なハッシュ、すなわちその下位の数ビットによってエントリに対応付けられる。ループの命令のように時間的に近接して実行される命令は、メモリ上でも近接している。それらは上位ビットを共有するが下位ビットが異なるので、別々のエントリに分散される。2 つの命令が衝突するのはコード上で遠く離れている場合だけであり、そのような命令が同時に活発に実行される可能性は低い。

ワード境界に整列した 4 バイト固定長命令では、すべての PC の下位 2 ビットはゼロであり情報を持たない。そのため 2^k エントリの BTB はビット $k+1$ から 2 でインデックスされる (図 4.2)。可変長命令 (x86) では、インデックスはビット 0 から始まる。

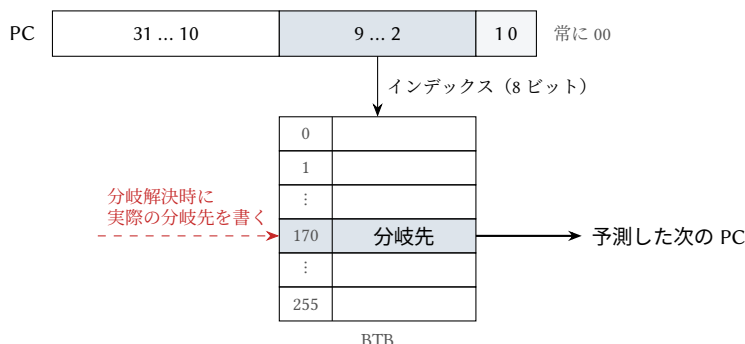


図 4.2: PC のビット 9-2 でインデックスされる 256 エントリの BTB。エントリは IF で読まれ、分岐が解決されると実際の分岐先で書き込まれる。

例 4.8. 256 エントリの BTB では、0x004012A8 にある命令は PC のビット 9-2 を用いる。 $(0x12A8 \gg 2) \& 0xFF = 0x4AA \& 0xFF = 0xAA$, すなわちエントリ 170 である。次の命令 (0x004012AC) はエントリ 171 を用い、256 命令先の 0x004016A8 にある命令はエントリ 170 を共有する。

4.6 方向予測

BTB だけでも、方向の問いには暗黙のうちに答えている。PC+4 を保持するエントリは「不成立」を意味する。これでは、分岐でない命令と不成立分岐のすべてに分岐先アドレス 1 つ分の記憶を費やすことになる。予測を分割する方が安価である。方向予測器が、その命令が成立分岐であるかどうかを判断し、成立分岐と判断したときにだけ BTB を参照する。

出典 Lesson 4, 動画 12-15 ; H&P §3.9 ; [6].

▷ branch target buffer

出典 Lesson 4, 動画 16-21 ; H&P §C.2.

▷ direction predictor

定義 4.9 (分岐履歴テーブル). 分岐履歴テーブル (BHT) とは, PC の下位ビットでインデックスされるテーブルの形をとる方向予測器である. 最も単純な形では, 各エントリは分岐の前の実行結果を保持する 1 ビットであり, 0 なら次の PC として PC+4 を, 1 なら BTB の分岐先を選ぶ. これが **1 ビット予測器 (1BP)** である.

分岐が解決されると, その BHT エントリには実際の結果が設定され, 分岐が成立した場合には分岐先が BTB に書き込まれる (図 4.3). この構成には 3 つの利点がある. 分岐でない命令の BHT エントリは書き込まれることがなく 0 のままなので, PC+4 を正しく予測する. BTB に必要なのは成立分岐のためのエントリだけであり, 成立分岐は全命令よりはるかに少ない. そして BHT のエントリは 1 ビットなので, 同じコストで BHT は BTB よりはるかに多くのエントリを持つことができ, 異なる分岐がエントリを共有することはまれになる.

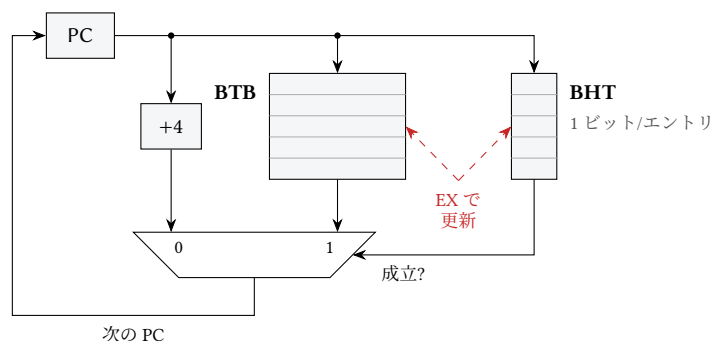


図 4.3: BHT と BTB による次の PC の予測. BHT のビットが PC+4 と BTB の分岐先のどちらかを選ぶ. 分岐が解決されると両方のテーブルが更新される.

例 4.10. ある内側ループは, 入るたびに配列の 3 要素を処理する.

```
0x0400  Inner: LW    R2, 0(R3)    ; 要素をロード
0x0404          ADD  R5, R5, R2    ; 合計に加算
0x0408          ADDI R3, R3, 4     ; 次の要素
0x040C          ADDI R1, R1, -1    ; カウントを減らす
0x0410          BNE  R1, R0, Inner ; 1 回の実行で T, T, N
```

64 エントリの BHT (インデックスはビット 7-2) では, 5 つの命令は BHT エントリ 0 から 4 を用い, BNE はエントリ 4 を用いる. 8 エントリの BTB (ビット 4-2) では, BNE は BTB エントリ 4 を用い, これを 8 命令前の 0x03F0 にある命令と共有する. その命令が分岐でなければ, その BHT エントリ (エントリ 60) は 0 のままなので, BTB エントリを使うのはこの分岐だけである. 表 4.3 は, すべてのテーブルがクリアされた状態から始めて, ループ 2 回の実行にわたり BNE を追跡したものである.

表 4.3: 例 4.10 の BNE (BHT エントリ 4) の, ループ 2 回の実行にわたる 1 ビット予測.

実行	1	2	3	4	5	6
実行前の BHT[4]	0	1	1	0	1	1
予測した次の PC	0x414	0x400	0x400	0x414	0x400	0x400
結果	T	T	N	T	T	N
正しいか	いいえ	はい	いいえ	いいえ	はい	いいえ

最初の実行では, BTB エントリ 4 に 0x400 も書き込まれる. それ以降, 分岐先は常に正しいが, 方向は 6 回の実行のうち 4 回で誤っている.

4.7 2 ビット予測器

1 ビット予測器は常に同じ方向に進む分岐にはよく機能するが, 2 つの弱点がある.

▷ branch history table

▷ one-bit predictor

- ほとんど常に一方向に進む分岐は、例外 1 回につき 2 回の予測ミスを被る。1 回は例外そのもので、もう 1 回はその次の通常の結果である。例外によってビットが反転してしまうからである。
- 方向が頻繁に変わる分岐はうまく予測できない。交互的分岐 (T, N, T, N, ...) は毎回予測ミスし、表 4.3 の短いループは 3 回に 2 回予測ミスする。

定義 4.11 (2 ビット予測器). 2 ビット予測器 (2BP) は、2 ビットカウンタ (2BC) とも呼ばれ、各 BHT エントリに、分岐が成立すると増加し、不成立だと減少する 2 ビットカウンタを格納する。

このカウンタは**飽和カウンタ**である。すなわち、11 で増加させても 11 に留まり、00 で減少させても 00 に留まる。上位ビットは**予測ビット**であり、1 なら成立を予測する。下位ビットは**ヒステリシスビット**である。下位ビットは予測ビットと合わせて、カウンタが自らの予測をどれだけ強く信じているかを記録する。2 ビットが等しいときカウンタは強い状態に、異なるときは弱い状態にあるので、4 つの状態は強い不成立 (00)、弱い不成立 (01)、弱い成立 (10)、強い成立 (11) である (図 4.4)。強い状態から予測を変えるには、予測に反する結果が 2 回続く必要があるので、単発の例外による予測ミスは 1 回で済む。

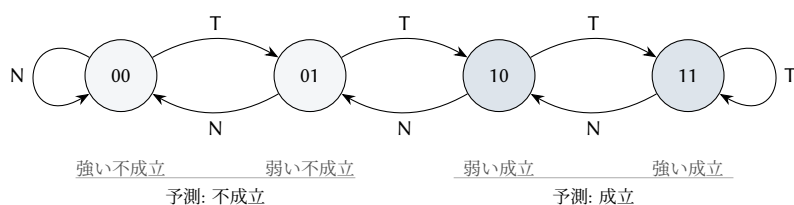


図 4.4: 2 ビット飽和カウンタの状態遷移図。T (成立) で右へ、N (不成立) で左へ移る。上位ビットが予測である。

表 4.4: 例 4.10 のループ分岐に対する、00 から始まる 2 ビットカウンタによる予測 (ループ 3 回の実行)。予測ミスを × で示す。

実行	1	2	3	4	5	6	7	8	9
結果	T	T	N	T	T	N	T	T	N
実行前のカウンタ	00	01	10	01	10	11	10	11	11
予測	N	N	T	N	T	T	T	T	T
2BP	×	×	×	×		×			×
1BP (比較用)	×		×	×		×	×		×

表 4.4 は短いループに対する効果を示している。カウンタがいったん強い成立 (11、実行 6 の前) に達すると、1 回のループ脱出ではカウンタは弱い成立 (10) に下がるだけで、次の成立の結果で元に戻る。そのため予測ミスするのは脱出だけであり、予測ミスはループ 1 回の実行につき 2 回ではなく 1 回になる。

4.7.1 初期化とビット数の増加

強い状態に初期化されたカウンタは、分岐が実際には逆方向に進む場合に 2 回の予測ミスを招く。弱い状態に初期化されたカウンタなら 1 回で済む。しかし弱い状態は、カウンタが最も脆弱な状態でもある。交互的分岐 T, N, T, N, ... は、カウンタが 01 から始まると 01 と 10 の間を行き来して毎回予測ミスするが、00 から始まれば予測ミスは 1 回おきで済む。どちらの選択もすべての分岐にとって最善ではなく、どの初期状態にも、それが不利になる結果の系列が存在する。そこでカウンタは、単に最も単純な状態である 00 にリセットされる。

3 ビット以上のカウンタはヒステリシスを増やす。すなわち、予測を変えるのにより多くの例外が連続する必要がある。これが役立つのは例外がまとまって生じる分岐だけであり、振る舞いを本当に変えた分岐への追従を遅くし、ビット数も増える。1 ビットから 2 ビットへの変更は大きな改善だが、それ以上ビットを増やしても得るものは少ない。2 ビットカウンタが標準的な構成要素であるのはこのためである。

▷ two-bit predictor

▷ saturating counter

どの予測器も万能ではない。どんな予測器に対しても、常にその時点の予測に反する結果の系列が存在する。

4.8 履歴ベース予測器

分岐の中には、カウンタでは捉えられない規則的なパターンに従うものがある。カウンタはどちらの方向が多いかを追跡するだけだからである。ループインデックスが偶数かどうかを判定する分岐は T, N, T, N と交互に変わり、例 4.10 のループ分岐は T, T, N を繰り返す。どちらも直近の結果が分かれば完全に予測できる。

定義 4.12 (履歴ベース予測器). 履歴ベース予測器は、分岐の直近の結果（その履歴）を記録し、それをを用いて複数のカウンタから 1 つを選ぶ。各カウンタは、ある特定の履歴の後に分岐が何をするかを学習する。

4.8.1 1 ビット履歴

最も単純な構成では、各 BHT エントリに 1 ビットの履歴（分岐の前の結果）と 2 つの 2 ビットカウンタを格納する。一方は前回の結果が N のときに、他方は T のときに使われる。予測では、履歴ビットがカウンタを選び、そのカウンタの上位ビットが予測となる。更新では、選ばれたカウンタを実際の結果に従って増加または減少させ、その結果を新しい履歴ビットとする。エントリには $1 + 2 \times 2 = 5$ ビットが必要である。

交互に変わる分岐では、N の後に選ばれるカウンタは T だけを、T の後に選ばれるカウンタは N だけを見る。それぞれの学習が済めば、分岐は毎回正しく予測される。パターン T, T, N にはこれでは足りない。T の後で分岐は成立することもしないこともあるので、T の後に選ばれるカウンタは予測ミスを繰り返す。

4.8.2 2 ビット履歴と N ビット履歴

履歴 2 ビットとカウンタ 4 つ（エントリ当たり 10 ビット、図 4.5）では、可能な 4 つの履歴 NN, NT, TN, TT のそれぞれが専用のカウンタを持つ。T, T, N では、履歴 TT の後は常に N であり、TN と NT の後は常に T である。表 4.5 はループ分岐を追跡したものである。カウンタの学習が済むと、すべての実行が正しく予測される。NN のカウンタは二度と使われない。

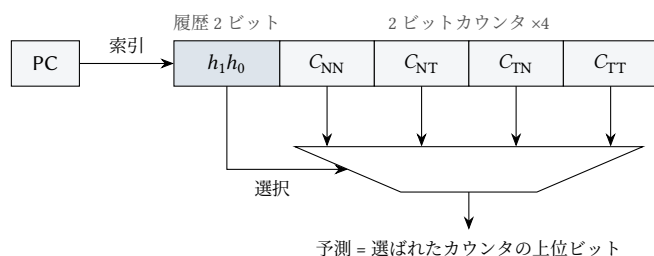


図 4.5: 2 ビット履歴予測器のエントリ。履歴ビットが 4 つの 2 ビットカウンタから 1 つを選ぶ。分岐が解決されると、選ばれたカウンタが更新され、結果が履歴にシフトインされる。

表 4.5: 例 4.10 のループ分岐に対する 2 ビット履歴予測器。履歴 NN, 全カウンタ 00 から始める。履歴は古い結果から順に書き、予測ミスを × で示す。

実行	1	2	3	4	5	6	7	8	9	10	11	12
履歴	NN	NT	TT	TN	NT	TT	TN	NT	TT	TN	NT	TT
使うカウンタ	00	00	00	00	01	00	01	10	00	10	11	00
予測	N	N	N	N	N	N	N	T	N	T	T	N
結果	T	T	N	T	T	N	T	T	N	T	T	N
予測ミス	×	×		×	×		×					

定義 4.13 (N ビット履歴予測器). N ビット履歴予測器は、各分岐の直近 N 回の結果と、エントリ当たり 2^N 個の 2 ビットカウンタ（可能な履歴ごとに 1 つ）を保持する。1 エントリのコストは次のとおりである。

$$N + 2 \cdot 2^N \text{ ビット} \quad (4.3)$$

出典 Lesson 4, 動画 27–33; H&P §3.3; [8].

▷ history-based predictor

▷ N-bit history predictor

学習後、 N ビット履歴予測器は、周期が $N+1$ 以下のあらゆる繰り返しパターンを正しく予測できる。そのようなパターンでは、直近 N 回の結果が次の結果を決めるからである。コストは N に対して指数的に増大し、その大半は無駄になる。周期 p の分岐は、 2^N 個のカウンタのうち高々 p 個しか使わない。また、使われるカウンタはそれぞれ個別に学習しなければならないので、履歴が長いほど学習にも時間がかかる。

例 4.14. 常に 10 回反復するループの分岐は、周期 10 のパターン TTTTTTTTTN を持つ。これを完全に予測するには $N=9$ 、すなわち BHT エントリ当たり $9+2 \cdot 512 = 1033$ ビットが必要であり、分岐が使うのは 512 個のカウンタのうち 10 個だけである。そのようなエントリ 1024 個からなる BHT は 1 メガビットを超える。

要点

履歴により、予測器は偏りだけでなくパターンを学習できる。しかし、分岐ごとに専用の 2^N 個のカウンタを与えると、長い履歴は法外に高価になり、しかもそれらのカウンタのほとんどは一度も使われない。

4.9 共有カウンタ: PShare と GShare

カウンタを分岐専用にしなければ、この無駄はなくなる。BHT はエントリごとに N ビットの履歴だけを保持し、すべての分岐が 2 ビットカウンタからなる 1 つの大きなテーブルを共有する。

定義 4.15 (パターン履歴テーブル). **パターン履歴テーブル (PHT)** とは、すべての分岐が共有する 2 ビットカウンタのテーブルであり、分岐の履歴と PC の組合せ、典型的にはそれらのビットごとの XOR でインデックスされる。

分岐が占めるカウンタは、その分岐が持つ異なる履歴の数だけになる。常に成立する分岐の履歴は TT...T であり、カウンタを 1 つしか使わない。交互に変わる分岐は 2 つ、例 4.14 の 10 回反復ループの分岐は 10 個を使う。PC のビットを混ぜることで、同じ履歴を持つ異なる分岐が異なるカウンタに送られる。衝突の可能性は残るが、PHT が大きければまれである。

例 4.16. 12 ビットの履歴を保持する 1024 エントリの BHT は 12 288 ビットを、 $2^{12} = 4096$ 個のカウンタからなる PHT は 8192 ビットを占め、合計 20 480 ビットとなる。12 ビット履歴と専用カウンタを持つ 1024 エントリの予測器では $1024 \times (12 + 2 \cdot 4096) = 8\,400\,896$ ビットが必要であり、これは約 410 倍である。

4.9.1 ローカル履歴とグローバル履歴

PHT をインデックスする履歴の選び方は 2 通りある (図 4.6)。

定義 4.17 (PShare). 分岐の**ローカル分岐履歴**とは、その分岐自身の直近の結果の系列であり、BHT エントリに保持される。**PShare 予測器**は、ローカル履歴と PC の XOR で共有 PHT をインデックスする。

定義 4.18 (GShare). **グローバル分岐履歴**とは、すべての分岐の直近の結果の系列であり、解決された各分岐が結果をシフトインする単一のシフトレジスタに保持される。**GShare 予測器**は、グローバル履歴と PC の XOR で共有 PHT をインデックスする。

PShare は、独自のパターンを持つ分岐、すなわち交互に変わる分岐や短いループに適している。GShare は、直前に実行された他の分岐の結果に依存する相関のある分岐に適している。

出典 Lesson 4, 動画 34–39 ; H&P §3.3 ; [9].

▷ pattern history table

▷ private branch history
▷ PShare predictor

▷ global branch history

▷ GShare predictor

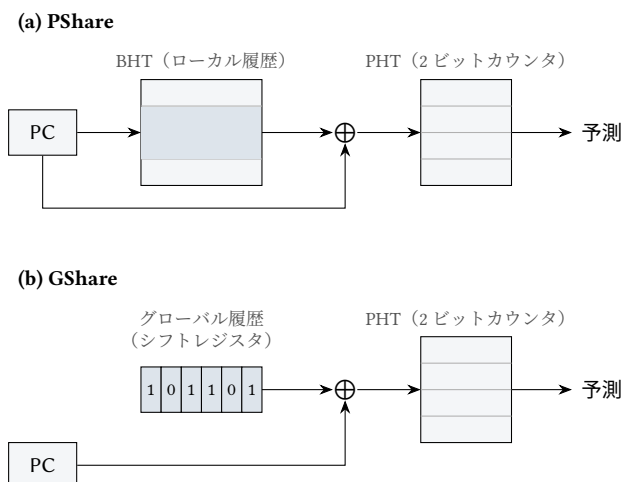


図 4.6: 共有カウンタを用いる予測器. (a) PShare は分岐自身の履歴を PC と組み合わせ、(b) GShare は全分岐のグローバル履歴を PC と組み合わせる。

例 4.19. 次の断片では、2 番目の分岐は、1 番目の分岐が不成立のときに成立し、成立のときに不成立となる。

```

        BNE  R1, R0, Skip1    ; R1 == 0 でなければスキップ
        ADDI R2, R2, 1        ; ゼロの個数を数える
Skip1:   BEQ  R1, R0, Skip2    ; R1 == 0 ならスキップ
        ADD  R3, R3, R1       ; 非ゼロの値を合計
Skip2:   ...

```

R1 の値がデータに依存するなら、どちらの分岐のローカル履歴もランダムに見え、PShare はうまく予測できない。2 番目の分岐から見たグローバル履歴には 1 番目の分岐の結果が含まれるので、これらの履歴に対応する PHT カウンタの学習が済めば、PHT でのエイリアシングがない限り、GShare は 2 番目の分岐を完全に予測する。

どちらもすべての分岐に対して優れているわけではない。GShare も原理的には分岐自身のパターンを学習できるが、その間に実行された他の分岐の結果がグローバル履歴に混ざり込むので、同じパターンに PShare よりかなり長い履歴—そしてより大きな PHT—が必要になる。プログラムには両方の種類の分岐が含まれるので、自然な次の一步は両方の予測器を使うことである。

4.10 トーナメント予測器と階層型予測器

分岐によって最適な予測器が異なるので、プロセッサは複数の予測器を組み合わせ、分岐ごとにそれに適したものを使うことができる。組合せ方には対称なものと階層的なものがある。

4.10.1 トーナメント予測器

トーナメント予測器は、2 つの予測器、例えば GShare と PShare を並行して動作させ、どちらを信頼するかを分岐ごとに個別に決める。

定義 4.20 (メタ予測器). トーナメント予測器の**メタ予測器**とは、PC でインデックスされる 2 ビットカウンタのテーブルであり、ある分岐に対応するカウンタが、2 つの構成予測器のどちらが正しいかを予測する (図 4.7)。

両方の構成予測器は、通常どおりすべての分岐で学習する。メタ予測器のカウンタが変化するのは構成予測器の正誤が分かれたときだけであり、そのときは正しかった方へ向かって動く。両方とも正しいとき、または両方とも誤ったときは変化しない。

出典 Lesson 4, 動画 40–43 ; H&P §3.3 ; [9].

▷ tournament predictor

▷ meta-predictor

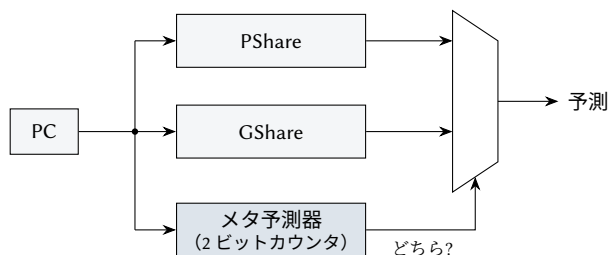


図 4.7: トーナメント予測器。メタ予測器が、どの構成予測器の予測を使うかを分岐ごとに選ぶ。

4.10.2 階層型予測器

トーナメント予測器は、よい—したがって高価な—予測器を 2 つ組み合わせ、すべての分岐で両方を更新する。しかし、ほとんどの分岐は予測が容易である。

定義 4.21 (階層型予測器)。 階層型予測器は、すべての分岐を扱う安価な予測器と、安価な予測器がうまく扱えない分岐に対してだけ使用・学習される、より優れた高価な 1 つ以上の予測器とを組み合わせる。

高価な予測器が学習すべき分岐ははるかに少ないので、同じ精度ならトーナメント予測器の場合よりずっと小さくでき、同じサイズなら精度をより高くできる。

典型的な構成は 3 レベルからなる (図 4.8)。PC でインデックスされる 2 ビットカウンタの大きなテーブルが、すべての分岐を予測する。その上に、小さなローカル履歴予測器と小さなグローバル履歴予測器があり、選ばれた分岐のエントリだけを保持する。それらのエントリには PC のビットからなるタグが付けられており、参照時にその分岐のエントリがあるかどうか分かる。予測は、一致するエントリを持つ最も上位のレベルから得る。カウンタテーブルは常に更新され、分岐に一致するローカルまたはグローバルのエントリも同様に更新される。カウンタテーブルが予測ミスした分岐にはローカル予測器のエントリが与えられ、ローカル予測器が予測ミスした分岐にはグローバル予測器のエントリが与えられる。カウンタでよく予測できる分岐が高価なレベルのエントリを占めることはまれなので、それらの小さなテーブルは、エントリを必要とする分岐のために残される。

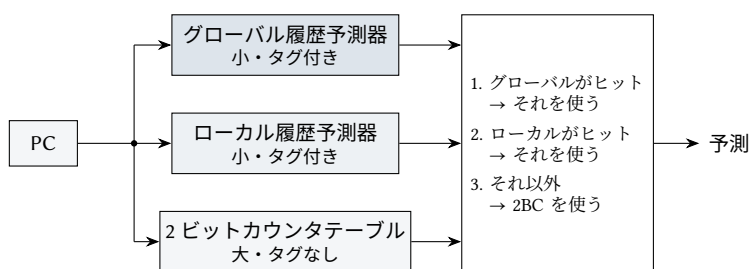


図 4.8: 階層型予測器。タグ付きの上位レベルは、保持している分岐について 2 ビットカウンタテーブルの予測を置き換える。

表 4.6 は、どの種類の分岐をどの種類の予測器が扱うかをまとめたものである。

表 4.6: 分岐の振る舞いと、それをうまく扱える最も単純な予測器。

分岐の振る舞い	予測器
ほとんど常に同じ方向	2 ビットカウンタ
独自の繰り返しパターン (交互, 短いループ)	ローカル履歴 (PShare)
先行する分岐と相関がある	グローバル履歴 (GShare)
1 つのプログラム内で上記が混在	トーナメントまたは階層型
関数からのリターン	リターンアドレススタック (次節)

▷ hierarchical predictor

商用プロセッサでも、このような階層型の予測器が使われてきた。例えば Intel の Pentium M である。

4.11 リターンアドレススタック

関数からのリターン (RET) は無条件分岐なので、方向の予測は自明であり、常に成立と予測すればよい。しかし、その分岐先は関数がどこから呼ばれたかに依存する。BTBは最新の分岐先しか覚えていないので、関数が異なる場所から代わる代わる呼ばれると、リターンは毎回予測ミスする。

例 4.22. ある関数が2か所から呼ばれる。

```

0x1000      CALL Func      ; 戻りアドレス 0x1004
...
0x1040      CALL Func      ; 戻りアドレス 0x1044
...
0x2000  Func:  ...
0x2030      RET              ; まず 0x1004 へ、次に 0x1044 へ

```

1回目のリターンが解決されると、RETのBTBエントリには0x1004が入る。このリターン自体は、エントリがまだ空だったので予測ミスする。0x1044へ戻る2回目のリターンは予測ミスし、エントリを上書きする。2か所からの呼び出しが交互に起こると、リターンは毎回予測ミスする。

定義 4.23 (リターンアドレススタック). **リターンアドレススタック (RAS)** とは、フェッチステージにある小さなハードウェアスタックである。コールがフェッチされると、その戻りアドレス (PC+4) をプッシュする。リターンがフェッチされると、先頭のエントリをポップし、予測した分岐先として用いる。

▷ return address stack

コールとリターンは入れ子になるので、RASはリターンをほぼ完全に予測する (図 4.9)。RASは小さいので、コールが深く入れ子になると満杯になりうる。RASが満杯のときにコールがフェッチされると、プッシュはラップアラウンドして最も古いエントリを上書きする。リターンはコールの逆順に起こるので、最も早く必要になるのは最新のエントリであり、またほとんどのコールとリターンはコールツリーの深い位置で起こる。最も古いエントリを上書きしても、失われるのは最も外側の関数からのリターンの予測だけであり、それはまれである。一方、新しい戻りアドレスを捨てれば、最も深いレベルで頻繁に起こるコールの予測が失われる。

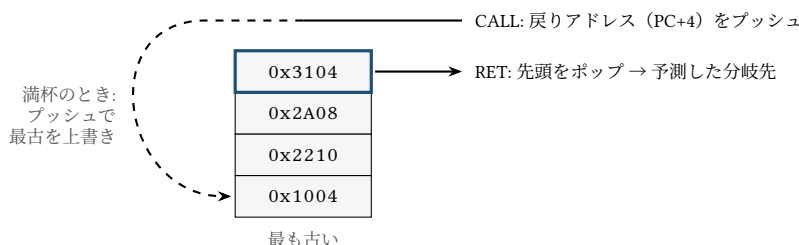


図 4.9: 4 エントリのリターンアドレススタック。コールでプッシュし、リターンでポップする。スタックが満杯のときは、プッシュがラップアラウンドして最も古いエントリを上書きする。

4.11.1 リターンの識別

RASは、RETがデコードされる前、すなわちフェッチされている最中に使わなければならない。命令がリターンであることを知る方法は2つある。

- **予測する。** 予測器のテーブルに、PCごとにその命令がリターンかどうかを格納する。エントリは、その命令が初めてデコードされたときに設定される。あるアドレスにある命令は変化しないので、この予測はほぼ常に正しい。
- **プリデコードする。** **プリデコード**では、プロセッサは各命令をメインメモリからプロセッサのキャッシュに取り込む際に部分的にデコードし、例えばそれが分岐かリターンかといった数ビットの追加情報を、キャッシュされた命令とともに格納する。フェッチステージはこれらのビットを命令と一緒に読むので、命令の種類がただちに分かる。

▷ predecoding

要点 – 章のまとめ

すべてのフェッチは予測である。予測ミスは、それがフラッシュするすべての命令の作業を無駄にし（式 (4.2)）、フラッシュされる命令数はパイプラインの深さと幅とともに増える。不成立予測はコストがかからないが不正確である。BTB は分岐先を、BHT は方向を予測し、BHT は 2 ビット飽和カウンタを用いると最もよく機能する。履歴ベース予測器はパターンを学習する。そのカウンタを PHT で共有すれば長い履歴も現実的なコストで実現でき、自己反復する分岐にはローカル履歴（PShare）、相関のある分岐にはグローバル履歴（GShare）を用いる。トーナメント予測器と階層型予測器は複数の予測器を組み合わせ、リターンアドレススタックは関数からのリターンを予測する。

ブレイケーション

講義動画	Lesson 5, 動画 1-12
教科書	H&P [1] §A.6, §C.2, §H.4, §H.6
キーワード	ブレイケーション, 予測困難な分岐, if 変換, 条件付きムーブ, MOVZ, MOVN, CMOVcc, 条件コード, 損益分岐予測ミス率, 完全ブレイケーション, Itanium, 述語レジスタ

第 4 章の予測器はほとんどの分岐を正しく予測するが、どの予測器にとっても予測が困難な分岐が残る。典型的には、分岐の方向が処理対象のデータに依存するためである。また深いパイプラインでは、予測ミスのたびに数十命令がフラッシュされる。この章では、そのような分岐に対する代替手段であるブレイケーションを調べる。ブレイケーションは分岐の両方向の処理を実行し、正しい方向の結果だけを残すので、予測ミスの対象となるものが何も残らない。どの分岐がその恩恵を受けるか、コンパイラが if 変換によって分岐をどのように除去するか、それが前提とする条件付きムーブ命令はどのようなものか、この変換が有利になるのはどのような場合か、そしてすべての命令を条件付きにできる命令セットはどのようなものかを、順に調べる。

5.1 予測とブレイケーション

分岐予測器は、分岐がどこへ進むかをフェッチステージで推測し、プロセッサは推測した経路に沿ってフェッチを続ける。推測が正しければコストはかからない。推測が誤っていれば、分岐の後にフェッチされたすべての命令がフラッシュされる。幅が数命令で段数も多い現代のプロセッサでは、これは数十命令に達する (例 4.6)。したがって分岐のコストは、ほとんどの場合ゼロであり、ときどき大きくなる。

定義 5.1 (ブレイケーション). 分岐の両方の方向の処理を実行し、誤りと判明した方向の結果を捨てることを**ブレイケーション**という。

ブレイケーションは何も予測しないので予測ミスペナルティがないが、不要な方向の処理を常に無駄にする。両方向の命令数が等しければ、処理の半分が捨てられる。分岐の実行 1 回当たり、ブレイケーションは不要な方向の命令を無駄にし、予測は予測ミスペナルティに予測ミスの確率を掛けた分を無駄にする。どちらが小さいかは分岐の種類に依存する (表 5.1)。ブレイケーションがよりよい選択となりうるのは、各方向の命令数が予測ミスでフラッシュされる命令数よりはるかに少ない、小さな if-then-else だけである。

表 5.1: 分岐の種類ごとの予測とブレイケーション。

分岐	予測	ブレイケーション	有利な方
ループ	正確	反復のたびにループの後のコードも実行する	予測
コール、リターン	常に成立。BTB と RAS が分岐先を与える	もう一方の経路の処理が常に無駄になる	予測
大きな if-then-else	ときどき数十命令をフラッシュする	大きな方向を毎回無駄にする	予測
小さな if-then-else	予測ミスごとに数十命令をフラッシュする	数命令を毎回無駄にする	予測困難ならブレイケーション

小さな if-then-else では、どちらを選ぶかは分岐がどれだけよく予測されるかに依存する。予測ミスが 40 命令をフラッシュし、不要な方向が 4 命令からなるなら、実行の 10 回に 1 回より多く予測ミスが起こる時点で、ブレイケーションの無駄は予測の無駄より少なくなる。5.4 節では、コンパイラが実際に生成す

出典 Lesson 5, 動画 1-2 ; H&P §C.2.

▷ predication

るコードについてこの比較を厳密に行う。

要点

予測は、正しければコストがかからず、誤れば数十命令のコストがかかる。プレディケーションは不要な方向を常に無駄にする。そのため分岐は原則として予測され、プレディケーションを検討する価値があるのは、予測が困難な小さな if-then-else の分岐だけである。

5.2 if 変換

プレディケーションには、両方向の処理が、間に分岐を挟まずに次々と実行される命令の形になっている必要がある。そのようなコードを生成するのはコンパイラの役割である。次の if-then-else を考える。これはしきい値未満の値としきい値以上の値を別々に合計し、しきい値未満の値の個数を数える。v の値が t のどちら側になるかに規則性がなければ、この分岐は予測が困難である。

```
if (v < t) {
    lo = lo + v;
    n  = n + 1;
} else {
    hi = hi + v;
}
```

定義 5.2 (if 変換). 両方向の文の制御依存 (第 3 章) を分岐条件に対するデータ依存に変換することで、そのような分岐を除去するコンパイラの変換を **if 変換** という。

通常の命令を用いる場合、変換後のコード (図 5.1) は両方向の文を無条件に実行してその結果を一時変数に書き込み、次に条件 c をデータとして用いて、いずれかの方向で代入されるすべての変数の新しい値を選択する。両方向が変数 x に代入する場合、選択は 2 つの一時変数のどちらかを選ぶ ($x = c ? x1 : x2$)。この例の各変数のように一方の方向だけが代入する場合、もう一方の選択肢は元の値である。変換後のコードには分岐がない。

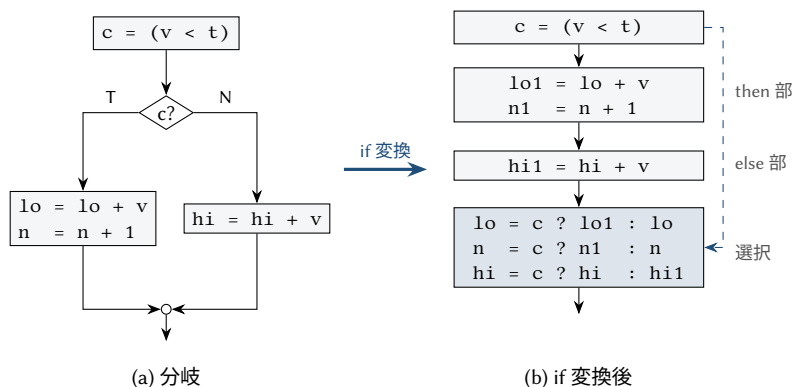


図 5.1: 小さな if-then-else の if 変換. (a) 元のコードは c で分岐する. (b) 変換後のコードは両方向を一時変数に計算し, c をデータとして用いて結果を選択する。

この変換で何かが得られるのは、選択を分岐なしで実行できる場合だけである。通常の分岐命令に翻訳すると、各選択はそれ自体が c による if-then-else となり、除去した分岐と同じく予測が困難である。そのため変換後のコードは依然として予測ミスを起こし、そのうえ両方向の処理も実行することになる。通常の算術演算でも、両方の選択肢を AND と OR でマスクすれば分岐なしに選択できるが、選択 1 つ当たり数命令を要する。if 変換が安価になるのは、命令の効果を条件付きにできる命令セットの場合だけである。

出典 Lesson 5, 動画 3; H&P §H.4; [11].

▷ if-conversion

注意 不要な方向も実行されるので、その方向がメモリにストアしたり、例えば無効なポインタを介したロードでフォールトを起こしたりしてはならない。

5.3 条件付きムーブ命令

そのような命令のうち最も単純なものは、コピーだけを条件付きにする。

定義 5.3 (条件付きムーブ). 条件付きムーブとは、条件が成り立てば値を宛先にコピーし、成り立たなければ宛先を変更しない命令である。

この命令はどちらの場合も実行され、書き込むかどうかだけが条件に依存する。制御の流れを変えないので、予測すべきものは何もない。

MIPS の命令セットには、レジスタの内容を条件とする条件付きムーブが 2 つある。命令 **MOVZ** (move if zero) はレジスタ R_t がゼロを保持していればレジスタ R_s をレジスタ R_d にコピーし、命令 **MOVN** (move if not zero) は R_t がゼロでなければコピーする。

```
MOVZ  $R_d$ ,  $R_s$ ,  $R_t$       ; if ( $R_t == 0$ )  $R_d \leftarrow R_s$ 
MOVN  $R_d$ ,  $R_s$ ,  $R_t$       ; if ( $R_t \neq 0$ )  $R_d \leftarrow R_s$ 
```

選択 $x = c ? x_1 : x_2$ は x のレジスタへの 2 つのムーブとなり、そのうちちょうど 1 つが書き込む。 c が R_3 , x_1 と x_2 が R_1 と R_2 , x が R_4 にあるとすると次のようになる。

```
MOVN  $R_4$ ,  $R_1$ ,  $R_3$       ; if  $c \neq 0$ :  $x \leftarrow x_1$ 
MOVZ  $R_4$ ,  $R_2$ ,  $R_3$       ; if  $c == 0$ :  $x \leftarrow x_2$ 
```

条件を保持するレジスタは通常、SLT のような比較によって設定される。SLT は条件が成り立てば 1 を、成り立たなければ 0 を書き込む。

例 5.4. v が R_1 , t が R_2 , lo , n , hi が R_4 , R_5 , R_6 にあるとすると、5.2 節のコードは分岐を使うと次のようにコンパイルされ、

```
SLT   $R_3$ ,  $R_1$ ,  $R_2$       ;  $R_3 \leftarrow (v < t)$ 
BEQ   $R_3$ ,  $R_0$ , Skip      ;  $v \geq t$  なら else 部へ
ADD   $R_4$ ,  $R_4$ ,  $R_1$       ;  $lo \leftarrow lo + v$ 
ADDI  $R_5$ ,  $R_5$ , 1        ;  $n \leftarrow n + 1$ 
J    Cont                  ; else 部を飛ばす
Skip: ADD  $R_6$ ,  $R_6$ ,  $R_1$  ;  $hi \leftarrow hi + v$ 
Cont: ...
```

if 変換すると次のようになる。

```
SLT   $R_3$ ,  $R_1$ ,  $R_2$       ;  $R_3 \leftarrow (v < t)$ 
ADD   $R_7$ ,  $R_4$ ,  $R_1$       ; then 部を  $R_7$ ,  $R_8$  に計算
ADDI  $R_8$ ,  $R_5$ , 1
ADD   $R_9$ ,  $R_6$ ,  $R_1$       ; else 部を  $R_9$  に計算
MOVN  $R_4$ ,  $R_7$ ,  $R_3$       ;  $lo \leftarrow R_7$  if  $v < t$ 
MOVN  $R_5$ ,  $R_8$ ,  $R_3$       ;  $n \leftarrow R_8$  if  $v < t$ 
MOVZ  $R_6$ ,  $R_9$ ,  $R_3$       ;  $hi \leftarrow R_9$  if  $v \geq t$ 
```

一時変数 R_7 – R_9 が必要なのは、どれを変更するかをムーブが決めるまで、 R_4 – R_6 が元の値を保持しなければならないからである。ここでは各変数が一方の方向でしか代入されないので、変数当たり 1 つのムーブで足りる。

他の命令セットでは、条件付きムーブの条件はレジスタの内容ではなくフラグに基づく。

定義 5.5 (条件コード). 条件コード (フラグ) とは、算術命令や比較命令が副作用として設定し、後続の命令が検査する 1 ビットの状態値である。例えばゼロ、負、キャリー、オーバーフローがある。

x86 の命令セットには条件付きムーブの一群 **CMOVcc** があり、 cc は条件コードに対する条件を表す。CMOVZ はゼロフラグがセットされていればムーブし、CMOVNZ はクリアされていればムーブする。CMOVG は直前の比較の結果が「より大きい」ならば、CMOVL は「より小さい」ならばムーブする、といった具合である。よく使われるパターンは、一方の選択肢を無条件に代入し、1 つの条件付きムーブでそれをもう一方の選択肢で上書きするものである。 $m = \max(a, b)$ の次の計算がその例である。

出典 Lesson 5, 動画 4–5; H&P §H.4, §A.6.

▷ conditional move

▷ MOVZ

▷ MOVN

▷ condition code

▷ CMOVcc

```
MOV    EAX, EBX    ; m ← a
CMP    EBX, ECX    ; a - b でフラグを設定
CMOVL EAX, ECX    ; if a < b: m ← b
```

5.4 if 変換の性能

if 変換にはコストがかかる。変換後のコードは、毎回の実行で両方向の処理と条件付きムーブを実行する。コード断片の 2 つの版を比較するため、第 3 章と同様にパイプラインのスロットを数える。実行される命令はそれぞれ 1 スロットを占め、予測ミスはそれがフラッシュする命令数と同じだけのスロットを無駄にする。分岐版は実行 1 回当たり平均 n_b 命令を実行し、1 回当たり m 回の予測ミスを起こし、予測ミス 1 回のペナルティは P 命令であるとする。また変換版は n_c 命令を実行するとする。if 変換が有利になるのは次の場合である。

$$n_c < n_b + mP \quad \Leftrightarrow \quad m > \frac{n_c - n_b}{P}. \quad (5.1)$$

右辺が損益分岐予測ミス率であり、変換後のコードで増える命令数を、予測ミスペナルティを単位として測ったものである。5.1 節の大まかな比較と比べると、増える命令には条件付きムーブが含まれ、変換で除去される分岐命令とジャンプ命令の分が差し引かれる。

例 5.6. 例 5.4 の分岐版は、then 部が実行されるときは 5 命令 (SLT, BEQ, ADD, ADDI, J)、そうでないときは 3 命令 (SLT, BEQ, ADD) を実行する。両方の部分が同じ頻度で実行されるなら $n_b = 4$ であり、変換版は常に $n_c = 7$ 命令を実行する。この分岐に対する予測器の精度が 75% で、 $P = 20$ であるとする。分岐版のコストは実行 1 回当たり $4 + 0.25 \times 20 = 9$ スロット、変換版は 7 スロットであり、この断片の高速化率は $9/7 \approx 1.29$ である。式 (5.1) より損益分岐率は $m = 3/20 = 0.15$ であり、分岐版が速いのは予測器の精度が 85% を超える場合だけである。 $P = 40$ なら損益分岐率は半分の 0.075、精度にして 92.5% となる。

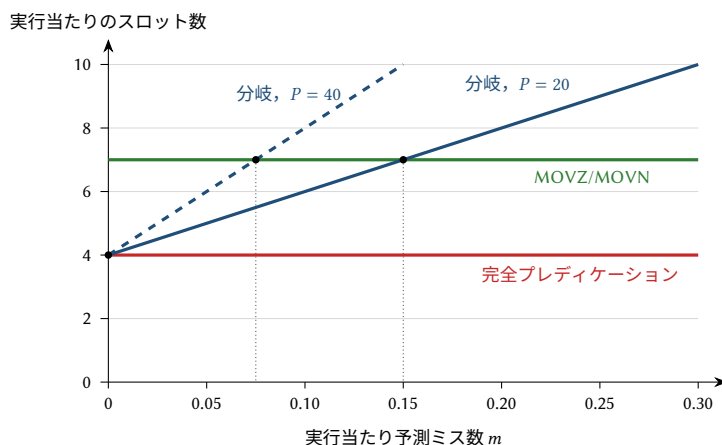


図 5.2: 例 5.6 の断片の実行当たりスロット数を、予測ミス率の関数として示す。点は損益分岐率を表す。完全プレディケーション版については 5.6 節で述べる。

図 5.2 は、この比較を m の関数として示す。分岐版のコストは予測ミス率に対して線形に増え、その傾きはペナルティに等しい。一方、変換版のコストはどの m でも同じである。パイプラインが深く、あるいは幅広くなるとペナルティが増えるので if 変換が有利になり、then 部と else 部が大きくなると $n_c - n_b$ が増えるので分岐が有利になる。

要点

if 変換は、ときどき生じる大きなペナルティを一定のオーバーヘッドに置き換える。実行当たりの予測ミス数とペナルティの積が、変換後のコードで増える命令数を上回るとき、if 変換は有利になる (式 (5.1))。

出典 Lesson 5, 動画 6-7; H&P §C.2.

5.5 条件付きムーブの利点とコスト

条件付きムーブで分岐を除去することには、1つの利点といくつかのコストがある。

- ・ **利点**. 予測困難な分岐が消え、それとともにその予測ミスもなくなる。
- ・ **コンパイラのサポート**が必要である。プロセッサは与えられたコードを実行するだけであり、分岐が除去されるのは、コンパイラが適切な if-then-else を認識して変換した場合だけである。
- ・ **より多くのレジスタ**が必要である。すべての結果はまず一時変数に計算される。
- ・ **より多くの命令**が実行される。分岐が正しく予測されたはずの実行でも両方向の処理が実行され、さらに結果を選択する条件付きムーブも実行される。

このうち、コンパイラのサポート、分岐の除去、両方向の実行はプレディケーションに本質的なものだが、追加のレジスタとムーブはそうではない。これらが生じるのは、条件付きなのがムーブだけだからである。結果を残すかどうかを条件付きムーブが決められるように、すべての結果を一時変数に計算しておかなければならない。結果を計算する命令そのものを条件付きにできれば、一時変数もムーブも不要になる。

出典 Lesson 5, 動画 8 ; H&P §H.4.

5.6 完全プレディケーション

条件付きムーブは独立したオペコードであり、それが条件付きにする操作はコピーだけである。その代わりとなるのは、すべての命令を条件付きにする機能を命令セットでサポートすることである。

定義 5.7 (完全プレディケーション). すべての命令に条件ビットが付加され、各命令はその条件が成り立つときにだけ効果を持ち、成り立たなければ何の効果も持たないとき、その命令セットは**完全プレディケーション**を提供するという。

Intel の Itanium プロセッサの IA-64 命令セットがその例である (図 5.3)。IA-64 のほとんどの命令では、41 ビットの命令の下位 6 ビットが**限定述語** (qualifying predicate) を保持する。これは 64 個ある 1 ビットのレジスタのうちの 1 つの番号であり、それぞれのレジスタを**述語レジスタ**と呼ぶ。命令はどちらの場合もフェッチされてパイプラインのスロットを占めるが、効果を持つのは選択された述語レジスタが 1 を保持するときだけである。そうでなければ命令は無効化され、結果を書き込まず、例外も発生させない。述語レジスタ 0 は常に 1 を保持するので、限定述語が 0 の命令は無条件に実行される。述語レジスタは比較命令によって設定される。比較命令は、条件に対するものとその否定に対するものという、互いに相補的な値を持つ述語の対を書き込む。

▷ full predication

▷ predicate register

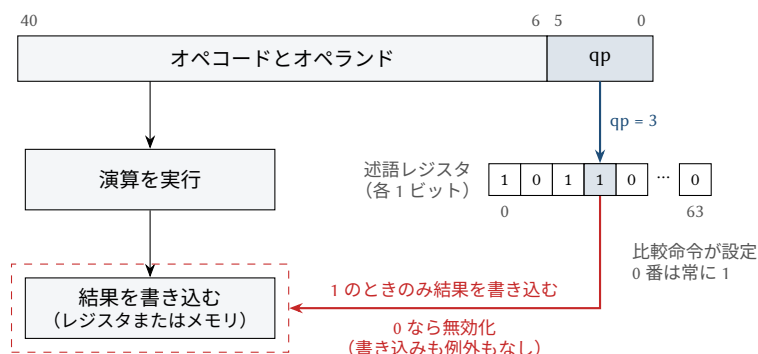


図 5.3: IA-64 における完全プレディケーション。命令の限定述語 (qp) が、64 個の 1 ビット述語レジスタのうち 1 つを選択する。命令はどちらの場合もパイプラインを通過するが、結果が書き込まれるのはそのレジスタが 1 を保持するときだけである。そうでなければ命令は無効化され、書き込みも例外も起こらない。

出典 Lesson 5, 動画 9–12 ; H&P §H.4, §H.6 ; [12].

例 5.8. 完全プレディケーションがあれば、if 変換は両方向の各文を条件またはその否定によって直接条件付けし、5.2 節のコードは次のようになる。

```

CMPLT P1, P2, R1, R2 ; P1 <- (v < t), P2 <- !P1
(P1) ADD R4, R4, R1 ; lo <- lo + v if v < t
(P1) ADDI R5, 1 ; n <- n + 1 if v < t
(P2) ADD R6, R6, R1 ; hi <- hi + v if v >= t

```

ここで限定述語は命令の前の括弧内に書かれており、CMPLT は、R1 が R2 より小さければ P1 を 1、P2 を 0 に設定し、そうでなければその逆に設定する比較命令を表す。加算そのものが条件付きなので、結果は変数に直接書き込まれ、比較命令が SLT と分岐の両方を置き換える。表 5.2 は、例 5.6 の数値を用いて 3 つの版を比較したものである。

表 5.2: 精度 75% の予測器と $P = 20$ における、例の断片の 3 つの版。

版	命令数	一時変数	ムーブ	予測ミス	スロット
分岐	5 または 3	0	0	0.25	9
MOVZ/MOVN	7	3	3	0	7
完全プレディケーション	4	0	0	0	4

完全プレディケーション版が実行するのは、比較命令と両方向の命令だけである。分岐版と比べると、不要な方向を実行する代わりに分岐もジャンプも実行しない。ここでは両者が釣り合い、その $n_c = 4$ は分岐版の平均 n_b に等しい。式 (5.1) による損益分岐率は $m = (4 - 4)/20 = 0$ である。すなわち、完全な予測を用いても分岐版は速くならず、予測ミスが起こるたびに遅くなる (図 5.2)。一般に、完全プレディケーション版は条件付きムーブ 1 つ当たり MOVZ/MOVN 版より 1 命令少なく実行するので、ムーブが M 個なら損益分岐率は M/P だけ低い。条件付きムーブが条件をすでに保持しているレジスタを検査する場合には、条件付きムーブ版は比較を必要としないのに対し、完全プレディケーションは依然として比較を必要とするので、その差は $(M - 1)/P$ となる。損益分岐率を下げることで、完全プレディケーションは、条件付きムーブでは引き合わないほどよく予測される分岐に対しても if 変換を有利にする。

要点 – 章のまとめ

分岐予測は、正しければコストがかからず、誤れば数十命令のコストがかかる。プレディケーションは分岐の両方向を実行して予測ミスを起こさないが、その代償として不要な処理を常に行う。そのため分岐は原則として予測され、コンパイラがプレディケーションを使うのは小さく予測困難な if-then-else の分岐に対してだけであり、ループ、コール、リターン、大きな if-then-else の分岐には使わない。if 変換は、制御依存を条件に対するデータ依存に変換することでそのような分岐を除去する。通常の命令を用いる場合、if 変換は両方向を一時変数に計算し、MIPS の MOVZ と MOVN や、x86 の条件コードに基づく CMOVcc といった条件付きムーブで結果を選択する。実行当たりの予測ミス数とペナルティの積が増える命令数を上回る時、if 変換は有利になり (式 (5.1))、パイプラインが深いほど有利になる。IA-64 のようにすべての命令に条件を付ける完全プレディケーションは、一時変数とムーブを不要にし、損益分岐予測ミス率を下げる。

ILP

講義動画
教科書
キーワード

Lesson 6, 動画 1–20

H&P [1] §3.1, §3.4 ; [13, 14]

RAW 依存, WAW 依存, 偽の依存, アーキテクチャレジスタ, 物理レジスタ, レジスタリネーミング, レジスタエイリアステーブル, 命令レベル並列性, 理想プロセッサ, サイクル当たり命令数, プロセッサ幅, インオーダ実行, アウトオブオーダ実行

分岐予測 (第 4 章) とプレディケーション (第 5 章) は, 制御依存によってパイプラインが失うサイクルの大半を取り除き, CPI を理想値の 1 に近づける. 1 を下回るには複数の命令を同じサイクルで実行しなければならず, それを妨げるのが命令間のデータ依存である. この章では, 本質的な依存は RAW 依存だけであること, それ以外の依存はレジスタリネーミングで除去されること, そして残った依存がプログラムの命令レベル並列性—実際のプロセッサのサイクル当たり命令数を測る基準となる上限—を定めることを示す.

6.1 すべての命令を同一サイクルで

ストールもフラッシュもしないパイプラインは毎サイクル 1 命令を完了するので, その CPI は 1 である (式 (3.1)). CPI がそれより低いということは, 1 サイクルに複数の命令が完了するということである. 極端な場合, プログラムの N 個の命令すべてが同じサイクルでフェッチされ, 次のサイクルで一斉にデコードされ, その次のサイクルで一斉に実行され, 以下同様に進む (図 6.1). 5 段プロセッサはプログラム全体を 5 サイクルで完了し, CPI は $5/N$ となる. これは長いプログラムでは 0 に近づく.

実際のプログラムはこのようには実行できない. 命令が他の命令の結果を使うからである. この章の前半を通じて用いる例として, 次の 5 命令を考える.

```
I1:  ADD R3, R1, R2      ; R3 <- R1 + R2
I2:  SUB R5, R3, R4      ; R5 <- R3 - R4
I3:  AND R6, R1, R4      ; R6 <- R1 AND R4
I4:  OR  R4, R2, R7      ; R4 <- R2 OR R7
I5:  XOR R5, R1, R7      ; R5 <- R1 XOR R7
```

5 命令すべてが同じサイクルで実行ステージにあるとき, I2 は, I1 がまだ R3 を計算している間に $R3 - R4$ を計算するので, R3 の古い値を使ってしまう. フォワーディング (図 3.4) ではこれを正せない. フォワーディングは, あるサイクルで計算された結果をより後のサイクルで実行される命令に届けるものであるが, ここでは結果とその使用が同じサイクルに属するからである. したがって, プロセッサがどこまで理想に近づけるかは, プログラムのデータ依存によって決まる.

6.2 同一サイクル実行における依存

I1 に対する I2 の RAW 依存は守らなければならない. I2 を実行できるのは早くても I1 の 1 サイクル後, すなわち I1 の結果をフォワーディングで受け取れるときである. I3, I4, I5 は, 先行する命令が書き込むレジスタを読まないのだから, 依然として I1 と同じサイクルで実行できる. 図 6.2 はその結果のタイミングを示す. I2 は 1 サイクル待ち, I3 から I5 は, プログラム上では I2 の後にあるにもかかわらず I2 より先に実行される. したがって RAW 依存は, 幅に制限がなくても CPI をどこまで下げられるかを制約する. すべての命令が直前の命令の結果を読むなら, 毎サイクル 1 命令しか完了せず, CPI は 1 のままである.

先行する命令を追い越して命令を実行すると, I2 と I5 の間の WAW 依存が破られる. 両者はともに R5 に書き込み, プログラム順では I5 が最後に書くので, これらの命令の後で R5 は I5 の結果を保持していなければならない. R5 を読む後続の命令はすべてその値を前提としている. しかし図 6.2 では, I5 がサイクル 5

出典 Lesson 6, 動画 1–3 ; H&P §3.1.

サイクル	1	2	3	4	5
I1	IF	ID	EX	MEM	WB
I2	IF	ID	EX	MEM	WB
I3	IF	ID	EX	MEM	WB
⋮	⋮	⋮	⋮	⋮	⋮
IN	IF	ID	EX	MEM	WB

図 6.1: 理想的な場合. N 個の命令すべてが同じ 5 サイクルで各ステージを通過する.

出典 Lesson 6, 動画 4–6 ; H&P §3.1.

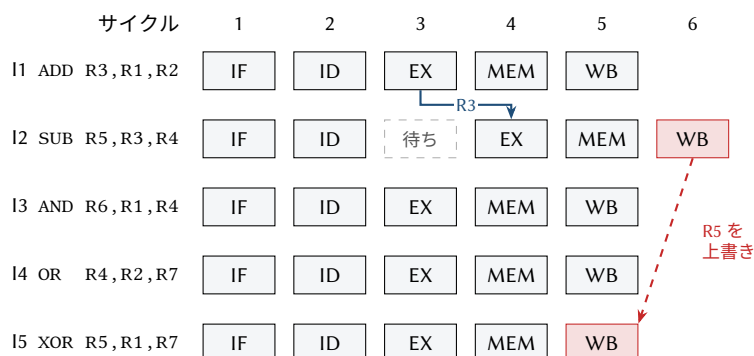


図 6.2: 例の 5 命令の同一サイクル実行. R3 に関する RAW 依存が I2 を 1 サイクル遅らせる. その結果 I2 は I5 より後に R5 に書き込み, 両者の WAW 依存に違反する.

で R5 に書き込み, 遅延した I2 がサイクル 6 でそれを上書きする. 正しい値を保つには, I5 の計算は I2 から何も必要としないにもかかわらず, I5 も I2 の書き込みの後まで遅らせなければならない.

WAR 依存も同じように危うくなる. I4 は I2 が読む R4 に書き込み, プログラム順では I2 は R4 の古い値を使わなければならない. I2 が遅延すると, I2 の実行時には I4 が生成した新しい値がすでに存在しており, I2 がそれを使わないようにしなければならない. このように, 1 つの RAW 遅延は波及する. 遅延した命令が読み書きするレジスタに書き込む後続の命令は, すべて同様に遅らせなければならない. そうしてはじめて, 各命令はプログラム中の自分の位置に対応するレジスタ値を見ることができる.

6.3 偽の依存の除去

第 3 章の 3 種類のデータ依存は性質が異なる. RAW 依存では, 後の命令が先の命令の計算した値を使う. データは実際に一方の命令から他方へ流れるのであり, どのようなハードウェアでも, 読む側の命令を書く側の命令より先に実行させることはできない. WAW 依存と WAR 依存は偽の依存である. 2 つの命令の間で値は受け渡されない. これらの依存は, プログラムが異なる値を同じレジスタに格納するというだけの理由で存在する. それは, レジスタの数に限りがあること, そして繰り返し実行される命令—例えばループの各反復で実行される命令—が常に同じレジスタに書き込むことの帰結である. この例では, I2 と I5 の結果は, たまたま R5 という名前が付いた無関係な値である.

したがって, 異なる値を異なる場所に保持すれば偽の依存は消える. その方法の 1 つは, レジスタの複数のバージョンを保持することである. I2 と I5 がともに R5 に書き込むとき, プロセッサは一方で他方を上書きするのではなく両方の結果を格納し, R5 を読む各命令はプログラム順によって選ばれるバージョンを使う. I2 と I5 の間にある命令は I2 の結果を, I5 の後にある命令は I5 の結果を使う. 2 つの書き込みが実際に起こる順序はもはや問題にならず, I5 は I2 が存在しない場合と同じだけ早く実行できる. 同様に, I2 は, I4 が新しいバージョンをいつ生成するかにかかわらず, R4 の古いバージョンを読む.

難しいのはその管理である. 読み出しのたびに, プロセッサは正しいバージョン, すなわちプログラム順で最も近い先行する書き込み命令が生成したバージョンを見つけなければならない. レジスタリネーミングはこの管理を系統的に行う.

6.4 レジスタリネーミング

リネーミングは, プログラムが名前前で指定するレジスタと, プロセッサがその値を保持する記憶領域とを分離する.

定義 6.1 (アーキテクチャレジスタと物理レジスタ). 命令セットで定義され, したがってプログラム中で名前前で指定されるレジスタ (例えば R0 から R31) を **アーキテクチャレジスタ** という. プロセッサがレジスタの値を実際に保持する記憶位

出典 Lesson 6, 動画 7-8 ; H&P §3.1.

出典 Lesson 6, 動画 9-11 ; H&P §3.1, §3.4 ; [13].

▷ architectural register

置を**物理レジスタ**という。リネーミングを行うプロセッサは、アーキテクチャレジスタよりはるかに多くの物理レジスタを持つ。

定義 6.2 (レジスタリネーミング). 各命令をデコード時に書き換え、アーキテクチャレジスタの代わりに物理レジスタを読み書きさせ、結果ごとに新たに割り当てた物理レジスタを用いるようにすることを**レジスタリネーミング**という。

定義 6.3 (レジスタエイリアステーブル). **レジスタエイリアステーブル (RAT)** は、アーキテクチャレジスタごとに 1 つのエントリを持つ。各エントリは、それまでにリネーミングされた命令の中でプログラム順に最も新しい、そのアーキテクチャレジスタの値を保持する物理レジスタを示す。

各命令は 3 つのステップでリネーミングされる (図 6.3)。

1. 各ソースレジスタについて、その RAT エントリを読む。リネーミング後の命令はその物理レジスタを読む。
2. 結果のために、空いている物理レジスタ、すなわちまだ必要な値を保持していない物理レジスタを割り当てる。リネーミング後の命令はそこに書き込む。
3. 宛先レジスタの RAT エントリを新しい物理レジスタで上書きする。これにより、このアーキテクチャレジスタを読む後続の命令は新しい結果を使う。ソースの参照は、宛先のエントリを上書きする前に行わなければならない。そうしないと、ADD R1, R1, R2 のような命令が R1 の以前の値ではなく自分自身の結果を読んでしまう。

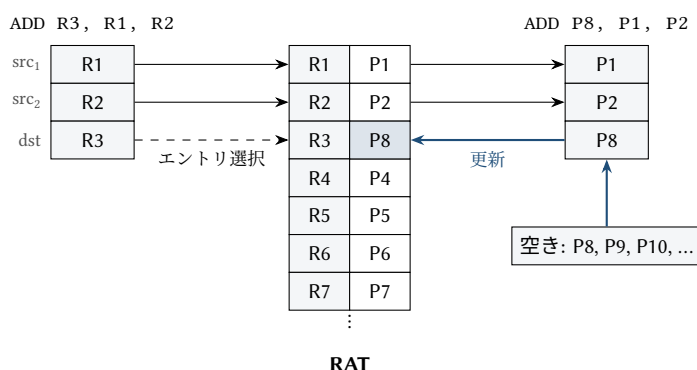


図 6.3: ADD R3, R1, R2 のリネーミング. R1 と R2 を RAT で参照し、結果には空いている物理レジスタ P8 を与え、P3 を保持していた R3 の RAT エントリを P8 で上書きする。

物理レジスタは、6.3 節で述べたバージョンにはかならない。書き込みのたびに新しいバージョンが作られ、RAT はプログラム順でどのバージョンが現在のものかを記録するので、読み出す命令は自動的に、最も近い先行する書き込み命令が生成したバージョンを使う。

例 6.4. 例の 5 命令をリネーミングする。I1 の前には、RAT は例で使うレジスタ R1 から R7 を P1 から P7 に、その他のアーキテクチャレジスタを例で使わない物理レジスタに対応付けている。P8, P9, ... は空いており、番号の小さい順に割り当てられる。表 6.1 は、リネーミング後の各命令と、その命令の後の RAT エントリを示す。I2 が I1 の結果である P8 を読むのは、I1 がすでに R3 のエントリを更新しているからである。I3 は、I4 が R4 を P11 に対応付ける前に、古い R4 である P4 を読む。R5 への 2 つの書き込みは P9 と P12 に向かい、最後に RAT は R5 を P12 に対応付けるので、後続の命令はプログラム順が要求するとおり I5 の結果を読む。

6.5 リネーミング後の依存

リネーミングされたプログラムでは、すべての結果がそれぞれ専用の物理レジスタに書き込まれる。2 つの命令が同じ位置に書き込むことはなく、先行する命令が読む位置に書き込む命令もないので、レジスタアクセス間の WAW 依存と

▷ physical register

▷ register renaming

▷ register alias table

講義では RAT を *register allocation table* の略としている。どちらの名前も同じテーブルを指す。

実際の設計でリネーミングがアウトオブオーダー実行とどのように組み合わせられるかは、第 7 章と第 8 章で示す。

表 6.1: 例の 5 命令のリネーミング. 命令によって変更されたエントリを太字で示す. R1, R2, R7 は終始 P1, P2, P7 のままである.

命令	リネーミング後	R3	R4	R5	R6
(初期状態)		P3	P4	P5	P6
I1 ADD R3, R1, R2	ADD P8, P1, P2	P8	P4	P5	P6
I2 SUB R5, R3, R4	SUB P9, P8, P4	P8	P4	P9	P6
I3 AND R6, R1, R4	AND P10, P1, P4	P8	P4	P9	P10
I4 OR R4, R2, R7	OR P11, P2, P7	P8	P11	P9	P10
I5 XOR R5, R1, R7	XOR P12, P1, P7	P8	P11	P12	P10

WAR 依存はもはや存在しない. 一方, すべての読み出しは, 元のプログラムでその値を生成した命令の結果を依然として参照するので, RAW 依存はすべて保たれる. 表 6.1 で残る依存は, P8 を介した I1 に対する I2 の依存だけである.

同一サイクル実行が守らなければならないのは, 今やこの 1 つの依存だけである. I1, I3, I4, I5 は最初のサイクルで, I2 は 2 番目のサイクルで実行され, I1 の結果は I2 にフォワーディングされる. I2 と I5 はそれぞれ P9 と P12 に書き込むので書き込みの順序は無関係であり, I2 が読む P4 に I4 は触れない. 5 命令は 2 サイクルで実行され, 後続の読み出し命令はすべて RAT を通じて正しい値を見つける.

要点

RAW 依存は命令間の唯一の真の依存である. レジスタリネーミングはレジスタに関するすべての WAW 依存と WAR 依存を除去するので, 命令の並列実行に残る制約はデータの流れそのものだけである.

6.6 命令レベル並列性

偽の依存を除去すれば, プログラムの実行に必要なサイクル数はその RAW 依存だけで決まる. この数はプログラム自体の性質であり, 特定のプロセッサの性質ではない. これは仮想的な**理想プロセッサ**上で測定される. 理想プロセッサは次の性質を持つ.

- レジスタをリネーミングするので, 偽の依存は存在しない.
- 各命令を, それが読む値がすべて得られる最初のサイクルで実行する. それは, 先行する命令に依存しなければ最初のサイクル, 依存すれば依存先の命令のうち最も遅いものの次のサイクルである.
- 1 サイクルに実行する命令数に制限がない.
- ハードウェアに制限がないので, 使用中の実行ユニットを待つ命令はない. すなわち構造依存は存在しない.
- 各命令を, 演算の種類によらずちょうど 1 サイクルで実行する.

定義 6.5 (命令レベル並列性). N 命令を実行するプログラムの**命令レベル並列性** (ILP) を次式で定義する.

$$\text{ILP} = \frac{N}{C}, \quad (6.1)$$

ここで C は, 理想プロセッサがそれらの命令を実行するサイクル数である.

理想プロセッサが命令 I を実行するサイクル $c(I)$ は, RAW 依存から直接求まる.

$$c(I) = 1 + \max_{J \in D(I)} c(J), \quad C = \max_I c(I), \quad (6.2)$$

ここで $D(I)$ は I が結果を読む命令の集合であり, 空集合に対する最大値は 0 とする. したがって C は, プログラム中の RAW 依存の最長の連鎖の長さである. どのプロセッサも, 命令が必要とする値が存在する前にその命令を実行することはできないので, ILP は, 任意のプロセッサがこのプログラムを実行できる速度 (サイクル当たりの命令数) の上限である (6.8 節).

メモリ位置を介した依存はレジスタリネーミングの影響を受けない. それは第 9 章で扱う.

出典 Lesson 6, 動画 13–15 ; H&P 第 3 章 ; [14].

▷ ideal processor

▷ instruction-level parallelism

ヒント 紙の上でリネーミングを行う必要はない. 各レジスタの読み出しについて, 最も近い先行する書き込み命令を探せばよい. 後の書き込みは関係しない.

例 6.6. 次の 8 命令の ILP を求める。

```

I1:  ADD R1, R2, R3
I2:  MUL R4, R1, R5      ; R1 は I1 から, R5 は後で書かれる
I3:  SUB R6, R2, R7      ; R2, R7 は後でしか書かれない
I4:  ADD R1, R6, R8      ; R6 は I3 から
I5:  AND R9, R1, R8      ; R1 は I1 からではなく I4 から
I6:  OR  R2, R8, R3
I7:  XOR R7, R4, R2      ; R4 は I2 から, R2 は I6 から
I8:  ADD R5, R3, R8

```

I1, I3, I6, I8 は、それより前に書き込まれていないレジスタだけを読むので、サイクル 1 で実行される。I2 と I4 はサイクル 1 の命令に依存し、サイクル 2 で実行される。I5 と I7 はそれぞれサイクル 2 の命令に依存し、サイクル 3 で実行される (図 6.4)。よって $C = 3$, $ILP = 8/3 \approx 2.67$ である。ここには 2 つの落とし穴が見られる。R1 には I1 と I4 の両方が書き込み、I5 が読むのは I4 の値である。I1 を生成元とみなすと、I5 を誤ってサイクル 2 に置いてしまう。また I3 は R2 と R7 を読むが、I6 と I7 がそれらに書き込むのは後になってからである。これらは WAR 依存であり、リネーミングで除去されるので、I3 を遅らせない。

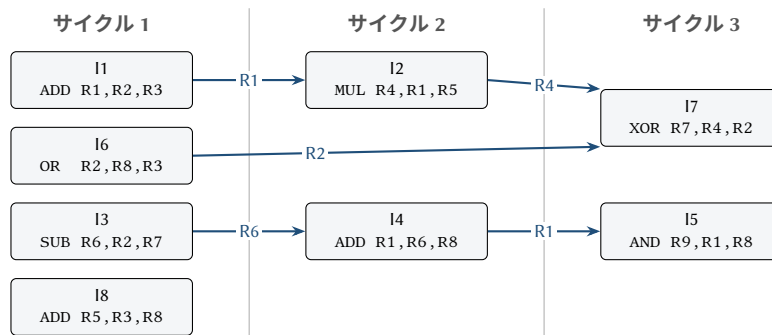


図 6.4: 例 6.6 の RAW 依存。矢印には値を運ぶレジスタを付した。各命令は、理想プロセッサがそれを実行するサイクルに置かれている。最長の連鎖の長さは 3 である。

6.7 構造依存と制御依存

構造依存、すなわちある命令が別の命令の使用しているハードウェアを待つ依存は、準備のできたすべての命令に十分なハードウェアを持つ理想プロセッサでは生じない。しかし、分岐は制御依存を加える。分岐の後の命令が実行されるかどうかは、分岐の結果に依存する。ILP を定義する理想プロセッサは、すべての分岐を完全に予測すると仮定する。したがって、どの命令が実行されるかを最初から知っており、分岐の後の命令は分岐の実行を待たない。分岐自体は他の命令と同じく 1 つの命令であり、 N に数えられ、そのオペランドを生成する命令の 1 サイクル後に実行される。分岐の結果によって飛ばされる命令は実行されず、数えられない。

例 6.7. 次の断片では分岐が成立するので、I4 は実行されない。

```

I1:  ADD R1, R2, R3      ; サイクル 1
I2:  SUB R4, R1, R5      ; サイクル 2: R1 は I1 から
I3:  BEQ R4, R6, Skip    ; サイクル 3: R4 は I2 から
I4:  ADD R7, R8, R9      ; 実行されない
I5:  Skip: XOR R8, R2, R6 ; サイクル 1
I6:  AND R9, R8, R1      ; サイクル 2: R8 は I5 から

```

分岐を完全に予測すれば、分岐がサイクル 3 になってようやく実行されるにもかかわらず、I5 はサイクル 1 で実行される。5 命令が 3 サイクルで実行されるので、 $ILP = 5/3 \approx 1.67$ である。I5 と I6 が分岐を待たなければならないとすると、それらはサイクル 4 と 5 で実行され、ILP は 1 に下がる。

出典 Lesson 6, 動画 16; H&P §3.1.

6.8 ILP と IPC

ILP は理想プロセッサ上でのプログラムを特徴づける量であり、実際のプロセッサを特徴づけるのは、それが実際に達成する速度である。

定義 6.8 (サイクル当たり命令数). プロセッサが 1 サイクルあたりに完了する命令の平均数を、そのプロセッサの**サイクル当たり命令数** (IPC) という。IPC は性能の鉄則 (式 (2.3)) の CPI の逆数であり、 $IPC = 1/CPI$ である。

すべてのプロセッサとプログラムについて $IPC \leq ILP$ が成り立つ。理想プロセッサは、各命令を依存が許す限り早く実行するからである。実際のプロセッサは、いくつかの理由で理想に届かない。

幅 1 サイクルに実行できる命令数は有限の w に限られる。幅の狭いプロセッサは 1 つか 2 つ、幅の広いプロセッサは 4 つ以上の命令を実行する。プログラムによらず $IPC \leq w$ である。

実行ユニット 加算器、乗算器などの実行ユニットの数は限られており、乗算のような演算には複数サイクルかかることがある。必要な種類のユニットが使用中である命令は、入力が揃っていても待つ。これは構造依存であり、理想プロセッサにはないものである。

実行順序 **インオーダー実行**を行うプロセッサは、プログラム上で先行する命令より先に命令を実行することは決してない。連続する独立な命令は同じサイクルで実行できるが、入力を待たなければならない命令は、その後ろにあるすべての命令を引き止める。アウトオブオーダー実行を行うプロセッサは、入力が揃った任意の命令を実行できる (第 7 章)。

アウトオブオーダープロセッサがそうするにはリネーミングが必要である。リネーミングがなければ、6.2 節の WAW 依存と WAR 依存も守らなければならない。インオーダープロセッサはレジスタをプログラム順に読み書きするので、リネーミングを必要としない。

例 6.9. 例 6.6 のプログラムを 4 つのプロセッサで実行する。いずれのプロセッサも各命令を 1 サイクルで実行し、あらゆる種類の実行ユニットを十分に持つ。アウトオブオーダーのプロセッサはレジスタをリネーミングし、準備のできた命令が幅を超えるときは古い命令から実行する。表 6.2 は各サイクルで実行される命令を示す。インオーダーのプロセッサはサイクル 1 で I1 しか実行しない。I2 が R1 を待たなければならない、I3 から I8 をせき止めるからである。同じせき止めが I4 (R6 待ち)、I5 (I4 からの R1 待ち)、I7 (R2 待ち) でも繰り返されるので、これらのプロセッサは幅によらず 5 サイクルを要する。幅 2 のアウトオブオーダープロセッサは毎サイクル準備のできた 2 命令を実行し、4 サイクルを要する。ILP に到達するのは、幅 4 のアウトオブオーダープロセッサだけである。

表 6.2: 例 6.6 の命令のうち、各サイクルで実行されるもの。理想プロセッサのスケジュールは、幅 4 のアウトオブオーダープロセッサのものと等しい。

プロセッサ	サイクル					IPC
	1	2	3	4	5	
インオーダー, 幅 2	1	2, 3	4	5, 6	7, 8	1.60
インオーダー, 幅 4	1	2, 3	4	5, 6	7, 8	1.60
アウトオブオーダー, 幅 2	1, 3	2, 4	5, 6	7, 8		2.00
アウトオブオーダー, 幅 4	1, 3, 6, 8	2, 4	5, 7			2.67

この例では、2 つのインオーダープロセッサはいずれも順序に制約されており、幅を 2 から 4 に広げても何も得られない。アウトオブオーダー実行は、幅 2 のプロセッサをその幅まで (1.60 から $w = 2$ へ) しか引き上げないが、幅 4 のプロセッサは ILP まで (1.60 から 2.67 へ) 引き上げる。一般には (表 6.3), 幅の狭いプロセッサを主に制約するのは幅である ($w = 1$ ならどちらの順序でも $IPC \leq 1$) ので、アウトオブオーダー実行による IPC の向上は限定的である。幅の広いインオーダープロセッサでは、待っている命令が後ろの命令をせき止め、幅の大半が使われないままになる。したがって、幅の広いプロセッサはその幅を活かすためにアウトオブオーダー実行を必要とし、アウトオブオーダー実行は幅の広いプロセッサでは

出典 Lesson 6, 動画 17-20 ; H&P 第 3 章.

▷ instructions per cycle

▷ in-order execution

じめて十分に効果を発揮する。

表 6.3: プログラムの ILP に対して、プロセッサの IPC を制限するもの。

	インオーダー	アウトオブオーダー
幅が狭い (幅 1-2)	主に幅	主に幅
幅が広い (幅 4 以上)	主に順序 (幅は使われない)	わずか (IPC は ILP に近づく)

要点 – 章のまとめ

多くの命令を同じサイクルで実行すれば CPI は 1 を大きく下回るはずだが、データ依存がそれを妨げる。真の依存は RAW 依存だけであり、WAW 依存と WAR 依存はレジスタ名の再利用から生じる。レジスタエイリアステーブルに基づくレジスタリネーミングは、アーキテクチャレジスタを新たに割り当てた物理レジスタに対応付け、レジスタアクセス間の偽の依存をすべて除去する。プログラムの ILP (式 (6.1)) は、リネーミングを行い、幅に制限がなく、構造依存がなく、分岐を完全に予測する理想プロセッサのサイクル当たり命令数、すなわち命令数を RAW 依存の最長の連鎖の長さで割った値である。これはプログラムの性質であり、あらゆるプロセッサの IPC の上限となる。実際のプロセッサは、幅、実行ユニット、インオーダー実行に制約されてこれを下回る。ILP に近づくには幅の広いアウトオブオーダープロセッサが必要である。次章では、リネーミングを伴って命令をアウトオブオーダーに実行する Tomasulo のアルゴリズムを紹介する。

命令スケジューリング

講義動画	Lesson 7, 動画 1-30
教科書	H&P [1] §3.4-3.5 ; [15]
キーワード	アウトオブオーダー実行, Tomasulo のアルゴリズム, 命令キュー, リザベーションステーション, レジスタエイリアステーブル, タグ, 共通データバス, 発行, ディスパッチ, 結果書き込み, 古くなった結果, 最古優先

第 6 章では、プログラムにはインオーダープロセッサが利用できるよりはるかに多くの命令レベル並列性が含まれていること、そしてそれに近づくには、レジスタをリネーミングし命令をアウトオブオーダーに実行するプロセッサが必要であることを示した。この章では、そのようなプロセッサが、どの命令を実行するかをサイクルごとにどのように決めるかを述べる。その方法が Tomasulo のアルゴリズムである。これはレジスタリネーミングと、オペランドの揃った命令の検出とを組み合わせたもので、今日のアウトオブオーダープロセッサでも依然として基礎となっている。命令がたどる 3 つのステップを追い、複数の命令や結果が同じハードウェアを奪い合うときに生じる選択を調べ、完全な例をトレースし、最後に各命令がいつ実行されるかを求める簡便法を導く。

7.1 IPC の向上

実際のプログラムの ILP は 1 を大きく上回るので、IPC をそれに近づけようとするプロセッサは、命令間の 4 種類の依存に対処しなければならない。それぞれに対策がある。

制御依存 分岐に続く命令が分岐の実行前にパイプラインに入れるのは、分岐の結果を推測する場合だけである。分岐予測（第 4 章）と、予測の難しい分岐に対するプレディケーション（第 5 章）がこれに対処する。

偽のデータ依存 WAW 依存と WAR 依存は、命令がレジスタ名を再利用するというだけの理由で存在する。レジスタリネーミング（第 6 章）がこれらを除去する。

真のデータ依存 RAW 依存は除去できない。命令は、自分が読む値が計算されるまで待たなければならない。避けられるのは、その命令に依存しない命令までが一緒に待つことである。

構造依存 命令はハードウェアも奪い合う。リザベーションステーション、実行ユニット、そして後述するバスである。対策は、これらの資源をより多く備えた幅の広いプロセッサ（第 6 章）であり、それによって同じサイクルにより多くの命令を発行し実行できる。

定義 7.1 (アウトオブオーダー実行). プログラム順で先行する命令がまだ待っている間でも、各命令を、それが読む値が得られ次第実行することを**アウトオブオーダー実行**という。

アウトオブオーダー実行はリネーミングと組み合わせて用いられる。リネーミングがなければ、命令が追い越せるのは WAW 依存も WAR 依存も持たない先行命令だけとなり、ILP の多くが失われる。残るのはスケジューリングの問題である。プロセッサは毎サイクル、どの命令のオペランドがすべて揃っているかを判定し、そのうちどれを実行するかを決めなければならない。

7.2 Tomasulo のアルゴリズム

アウトオブオーダープロセッサで用いられるスケジューリング手法は、IBM System/360 Model 91 で導入された。

定義 7.2 (Tomasulo のアルゴリズム). Tomasulo が 1967 年に発表したハードウェアアルゴリズムで、レジスタをリネーミングし、オペランドが揃った命令を判定

出典 Lesson 7, 動画 1-2 ; H&P §3.1.

▷ out-of-order execution

出典 Lesson 7, 動画 3 ; H&P §3.4 ; [15].

することによって、各命令をそのオペランドが計算され次第実行させるものを **Tomasulo のアルゴリズム** という。

現在のプロセッサも本質的に同じ方法で命令をスケジューリングする。相違点を表 7.1 に示す。最初の 2 つはアルゴリズムの適用範囲に関わるだけだが、3 つ目はその機構を変える。Tomasulo のアルゴリズムには例外への備えがない。命令がアウトオブオーダーに実行された後では、レジスタの内容は、例外を受け付けられるプログラム中のどの時点にも対応しているとは限らない。例外をサポートするには追加の構造が必要であり、それが第 8 章の主題である。

表 7.1: Tomasulo のアルゴリズムと現代のアウトオブオーダープロセッサ。

	Tomasulo のアルゴリズム	現代のプロセッサ
スケジューリングする命令	浮動小数点演算	すべての命令
同時に処理中の命令	数個	数百個
例外	サポートしない	サポートする (第 8 章)

▷ Tomasulo's algorithm

7.3 構成

図 7.1 は、Tomasulo のアルゴリズムを実行するハードウェアを示す。フェッチステージは、フェッチした各命令を**命令キュー**に追加する。命令キューは先入れ先出しのバッファであり、フェッチされた命令を、リザベーションステーションへ移せるようになるまでプログラム順に保持する。

出典 Lesson 7, 動画 4 ; H&P §3.4.

▷ instruction queue

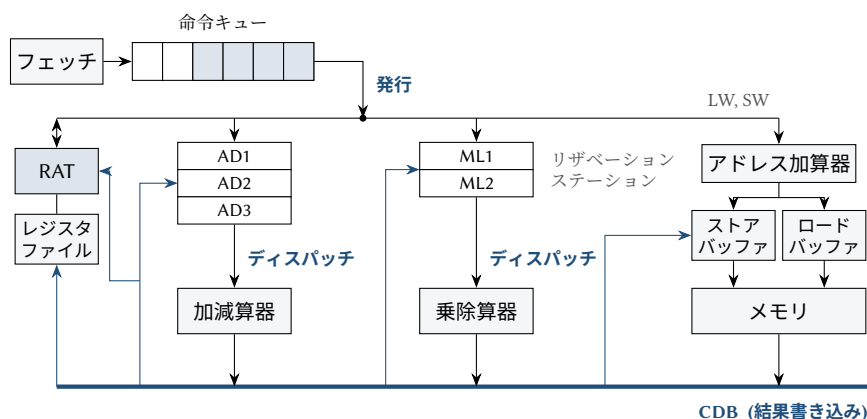


図 7.1: Tomasulo のアルゴリズムを用いるプロセッサの構成。発行は、RAT とレジスタファイルを用いて、キューの先頭の命令をリザベーションステーションに移す。ディスパッチは、準備のできた命令をその実行ユニットに送る。結果は共通データバスを通じて、リザベーションステーション、RAT、レジスタファイル、ストアバッファに戻る。ロードとストアは、アドレス加算器とロードバッファ・ストアバッファを経由する経路をとる。

定義 7.3 (リザベーションステーション). リザベーションステーション (RS) は、1 つの命令を、それが命令キューを離れた時点から結果がブロードキャストされるまで保持する。RS は、使用中かどうか、演算の種類、各オペランドについてその値あるいは値がまだ計算されていないならばその値のタグ、そして命令がすでにディスパッチされたかどうかを記録する。

▷ reservation station

リザベーションステーションは、それを担当する実行ユニットごとにまとめられる。この章の例では、1 つの加減算器が担当する加算・減算用の 3 つのステーション AD1, AD2, AD3 と、1 つの乗除算器が担当する乗算・除算用の 2 つのステーション ML1, ML2 を用いる。ステーションの名前は、その命令の結果がブロードキャストされるときタグである。

第 6 章のレジスタエイリアステーブルは、レジスタごとに 1 つのエントリを持つ。エントリが空であれば、レジスタファイルがそのレジスタの現在の値を保持

している。そうでなければ、エントリは、その値を生成する命令が入っているステーションのタグを保持する。したがってタグは、第6章の物理レジスタの役割を果たす。すべての結果が新しい名前を受け取り、レジスタを読む命令は、最も近い先行する書き込み命令の結果を参照する。

定義 7.4 (共通データベース). **共通データベース (CDB)** は、実行ユニットが計算した各結果を、そのタグとともに、すべてのリザベーションステーション、RAT、レジスタファイルに運ぶ。

ロードとストアは別の経路を用いる。加算器がメモリアドレスを計算した後、ロードはアドレスを保持するロードバッファで、ストアはアドレスと格納する値 (CDB から受け取ることもある) を保持するストアバッファで待つ。ロードが読んだ値は、他の結果と同様に CDB に載せられる (7.8 節)。

すべての命令は、図 7.1 に示した 3 つのステップを経る。

発行 命令キューの先頭の命令を、RAT を通じてオペランドと宛先を解決したうえで、空いているリザベーションステーションに置く。このステップを**発行**という。

ディスパッチ オペランドがすべて揃った命令を、そのリザベーションステーションから実行ユニットに送る。これを命令の**ディスパッチ**という。

結果書き込み 結果を CDB に載せ、そこから待機中のリザベーションステーション、RAT、レジスタファイルに届ける。このステップを**結果書き込み**、あるいはブロードキャストという。

▷ common data bus

▷ issue

▷ dispatch

▷ write result

7.4 発行

発行は、命令キューの先頭の命令を取り出し、次のステップを実行する。この章の例では 1 サイクルに 1 命令を発行する。幅の広いプロセッサは 1 サイクルに複数の命令を、やはりプログラム順に発行する。

1. プログラム順で次の命令を、キューの先頭から取り出す。
2. 各オペランドの出所を決める。オペランドのレジスタの RAT エントリが空であれば、値はレジスタファイルにある。そうでなければ、エントリがその値のタグを示す。
3. その演算用の空いているリザベーションステーションを探す。その種類のステーションがすべて使用中であれば、命令はこのサイクルには発行されない。命令はキューの先頭に留まり、その後の命令も待つ。
4. 命令をステーションに置く。演算の種類と、各オペランドについてレジスタファイルの値あるいは RAT のタグを格納する。
5. 宛先レジスタの RAT エントリを、そのステーションの名前に設定する。

命令は厳密にプログラム順に発行される。そのため RAT は、発行中の命令の位置から見えるレジスタ値を表し、各オペランドは最も近い先行する書き込み命令を参照する。リネーミングと同様に、オペランドの参照は宛先のエントリを上書きする前に行う。

例 7.5. 次のプログラムを、この章の実行例とする。

```
I1:  MUL R1, R2, R3      ; R1 <- R2 * R3
I2:  ADD R4, R1, R5      ; R4 <- R1 + R5
I3:  SUB R6, R2, R5      ; R6 <- R2 - R5
I4:  ADD R1, R5, R3      ; R1 <- R5 + R3
I5:  DIV R2, R6, R3      ; R2 <- R6 / R3
I6:  MUL R3, R4, R1      ; R3 <- R4 * R1
I7:  SUB R5, R3, R2      ; R5 <- R3 - R2
```

初期状態では RAT は空で、レジスタファイルは $R1 = 1$, $R2 = 14$, $R3 = 4$, $R4 = 3$, $R5 = 6$, $R6 = 7$ を保持する。サイクル 1 で、I1 がオペランド値 14 と 4 とともに ML1 に発行され、R1 の RAT エントリは ML1 になる。サイクル 2 で、I2 が AD1 に発行される (図 7.2)。R1 の RAT エントリは ML1 を保持しているので、第 1 オペランドはタグ ML1 となる。R5 のエントリは空なので、第 2 オペランドは値 6 となる。R4 の RAT エントリは AD1 になる。I3 と I4 がサイクル 3 と 4 で続く。I4 は再び R1 に書き込み、R1 のエントリの ML1 を AD3 で置き換える。これ以降に発行される命令のうち R1 を読むものはすべて、I1 ではなく I4 の結果を使う。

出典 Lesson 7, 動画 5-7; H&P §3.5.

注意 発行はオペランドを待つことはなく、空いているリザベーションステーションだけを待つ。

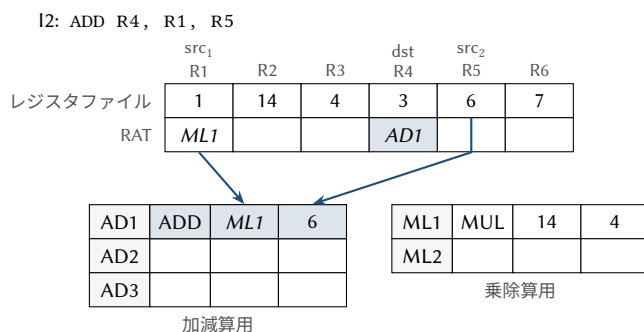


図 7.2: 例 7.5 のサイクル 2 における I2 の発行. R1 の RAT エントリからタグ ML1 が得られ (タグは斜体), R5 のエントリは空なので値 6 がレジスタファイルから取られ, R4 のエントリは AD1 に設定される. 網掛けのセルがこのステップで書き込まれる.

7.5 ディスパッチ

命令は, すべてのオペランドが値になるまでリザーベーションステーションで待つ. CDB に結果が現れるたびに, すべてのリザーベーションステーションが自身のオペランドのタグをバス上のタグと比較し, 一致したところではタグをバスの値で置き換える. これを値の取り込みという. オペランドがすべて値になったステーションは, 準備のできた命令を保持している. そのステーションを担当する実行ユニットが空いていれば, 準備のできた命令はディスパッチされる. すなわち, その演算の種類とオペランド値がユニットに送られ, ユニットが実行を開始する. 命令の実行中もステーションは使用中のままである. ステーションの名前が, 結果をブロードキャストするときのタグだからである.

出典 Lesson 7, 動画 8 ; H&P §3.5.

7.5.1 準備のできた命令が複数ある場合

実行ユニットが空いたときに複数の命令が準備できていることがあり, その中から 1 つを選ばなければならない. 準備のできた命令は必要な値をすべて持っているのだから, どれを選んでも正しい. 選択が影響するのは性能とハードウェアコストだけである.

最古優先 最初に発行された命令をディスパッチする. 他の条件が同じなら, 古い命令ほど, 処理中の命令のうちその後に発行されてその結果を待っている可能性のあるものが多い. したがってこの方針は次の方針を近似し, しかも発行の順序を記録するだけで済む.

依存命令最多優先 待機中の命令のうち最も多くがその結果を必要とする命令をディスパッチし, できるだけ多くの命令を早く準備完了にする. 最良の性能が期待できるが, その命令を見つけるには, すべての候補のタグをすべてのリザーベーションステーションのオペランドと比較する必要がある. 遅すぎるうえにコストも大きすぎる.

ランダム 選択論理が最も単純である. 多くの命令が依存する命令を, 重要でない命令の後ろで待たせることがあるので, 性能が低下する.

最古優先は他の 2 つの方針のよい折衷であり, この章の例で用いる規則である. 実行例では, サイクル 7 で加減算器が空いたとき, I2 と I4 がともに準備できている (7.9 節). I4 のほうが 2 サイクル長く準備完了の状態にあるが, 最古優先により I2 がディスパッチされる.

出典 Lesson 7, 動画 9–10.

7.6 結果書き込み

実行ユニットが命令の実行を終えると, 結果書き込みのステップがその結果をブロードキャストする.

1. 結果と, 命令のリザーベーションステーションのタグを CDB に載せる.
2. このタグを保持するすべてのオペランドが値を取り込む (7.5 節).
3. あるレジスタの RAT エントリがそのタグを保持していれば, 値をレジスタファイルのそのレジスタに書き込み, RAT エントリを空にする. これによ

出典 Lesson 7, 動画 11 ; H&P §3.5.

り、後で発行される命令はレジスタファイルから値を読む。

4. リザーベーションステーションを解放する。

ステーションは宛先レジスタを記録する必要がない。タグを保持する RAT エントリがそれを特定するからである。図 7.3 は、実行例における I2 の結果のブロードキャストを示す。

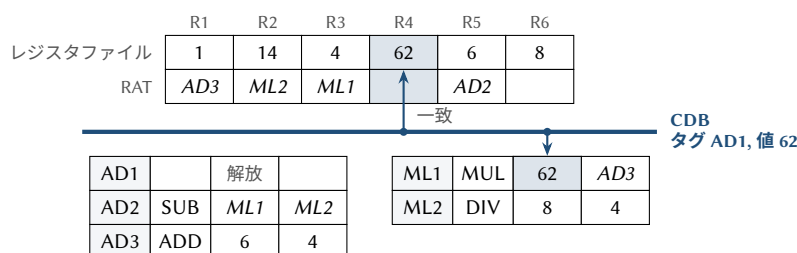


図 7.3: 実行例のサイクル 9 における I2 の結果書き込み。ML1 (I6) はタグ AD1 の代わりに 62 を取り込む。R4 の RAT エントリは AD1 を保持しているので、R4 は 62 になり、エントリは空になる。AD1 は解放される。AD1 を保持するオペランドや RAT エントリは他にない。

7.6.1 1 サイクルに複数の結果がある場合

2 つの実行ユニットが同じサイクルに実行を終えることがあるが、1 本のバスは一度に 1 つの結果しか運べない。1 つの解決法は、ユニットごとに別々のバスを設けることである。その場合、すべてのリザーベーションステーションのすべてのオペランドが自身のタグを各バスと比較し、一致したバスから値を選ばなければならない。レジスタファイルにはバスごとに書き込みポートが必要になる。もともと大きい比較論理が、バスの数に比例して増大する。もう 1 つの方法は、バスを 1 本のままとし、一方の結果を次のサイクルまで待たせることである。ここで用いる規則は、遅いほうのユニットの結果を優先する。その命令はより前にディスパッチされたので古い命令である傾向があり、その結果を待っている命令も多い可能性が高いからである。例では、結果がバスを待っているユニットは次の命令を開始できないと仮定する。

出典 Lesson 7, 動画 12.

7.6.2 古くなった結果

結果がブロードキャストされる時点で、同じレジスタに書き込む後の命令がすでに発行されていることがある。このときそのレジスタの RAT エントリは、バス上のタグではなく後の命令のタグを保持しており、ステップ 3 は何の効果も持たない。レジスタファイルも RAT も変化しない。結果はそのレジスタにとってのみ古くなったものであり、そこでそれを無視するのは正しい。両方の命令の後では、レジスタは後の結果を保持していなければならない。その結果は後の命令がブロードキャストするときに届く。先の結果もなお必要だが、それを必要とするのは 2 つの書き込み命令の間に発行された命令だけである。それらはそのタグを保持しており、ステップ 2 で値を取り込む。後の書き込み命令より後に発行された命令は、後のタグを参照する。

出典 Lesson 7, 動画 13; H&P §3.5.

例 7.6. 実行例では、I1 と I4 がともに R1 に書き込む。サイクル 6 で I1 がタグ ML1 のもとで 56 をブロードキャストするとき、I4 がサイクル 4 で発行されているので、R1 の RAT エントリはすでに AD3 を保持している。レジスタファイルの R1 は古い値 1 のままであり、エントリも AD3 のままである。一方、I2 を保持する AD1 は 56 を取り込む。R1 が 10 になるのは、サイクル 11 で I4 が AD3 のもとでブロードキャストするときである。

要点

ブロードキャストは、そのタグを待つすべてのオペランドに値を届けるが、レジスタファイルと RAT を更新するのは、宛先の RAT エントリがまだそのタグを保持している場合だけである。

7.7 1 サイクル内のステップ

1つの命令から見ると、Tomasulo のアルゴリズムは、発行、リザベーションステーションでの待機と取り込み、ディスパッチ、実行、結果書き込みという一連の流れである。プロセッサ全体から見ると、毎サイクルすべてのステップが、それぞれ異なる命令に対して行われる。同じサイクルに、1つの命令が発行され、複数のステーションがブロードキャストされた値を取り込み、準備のできた命令が空いているユニットにディスパッチされることがあり、そして1つの結果がブロードキャストされる。

同じ命令の2つのステップが同じサイクルに行われうるかどうかは、回路設計の問題である。最も単純な設計では、各ステップはサイクル開始時点の構造の状態を調べ、すべての更新はサイクルの終わりに反映される。このとき次のようになる。

- ・サイクル C で発行された命令がリザベーションステーションに入っているのはサイクル $C+1$ からであり、ディスパッチされるのは早くても $C+1$ である。
- ・サイクル C で最後に残っていたオペランドを取り込んだ命令は、サイクル $C+1$ から準備完了となり、ディスパッチされるのは早くても $C+1$ である。
- ・サイクル C のブロードキャストで解放されたステーションは、サイクル $C+1$ に新しい命令を受け入れられる。

取り込みとディスパッチの間の遅延は、追加の論理、すなわち現在のサイクルにバス上に到着するオペランドも考慮するディスパッチによって取り除くことができ、ブロードキャストで解放されたステーションも同様に同じサイクルで再利用できる。追加の論理は、それが拡張するステップと同じサイクルに置かれるので、クロックサイクルを長くする。ただし、命令が発行されたサイクルにディスパッチされることはない。発行が命令をステーションに書き込んでいる間、ディスパッチからはその命令がまだ見えないからである。

同じサイクルにおける発行と結果書き込みの間の2つの相互作用には注意が必要である。

- ・両者が同じレジスタの RAT エントリを変更することがある。古い書き込み命令のブロードキャストはエントリを空にし、新しい書き込み命令の発行はエントリを新しいタグに設定する。発行はプログラム順で後の命令に属するので、発行が優先されなければならない。言い換えれば、結果書き込みを先に、発行を後に適用する。そうしないと、後続の命令はレジスタファイルを読み、新たに発行された命令の結果を見落としてしまう。
- ・ある命令が、そのオペランドの1つがブロードキャストされるサイクルに発行されることがある。最も単純な設計では、エントリはサイクルの終わりにならないと空にならないので、命令は RAT にタグを見つけるが、ステーションに入るのは値がバスを通過した後である。したがって発行はオペランドのタグをバスと比較し、値を直接取らなければならない。そうしないと、命令は決して来ないブロードキャストを待ち続ける。ブロードキャストによる更新が同じサイクルで発行から見えるなら、発行は単にレジスタファイルから新しい値を読む。

要点

最も単純な設計では、命令は1サイクルに高々1ステップしか進まない。サイクル C で発行された命令がディスパッチされるのは早くても $C+1$ であり、サイクル W でブロードキャストされた結果を待つ命令がディスパッチされるのは早くても $W+1$ である。

7.8 ロード命令とストア命令

ロードとストアは、レジスタを介した依存と同様の、メモリを介した依存を生む。

```
SW  R1, 0(R2)      ; MEM[R2] <- R1
LW  R3, 8(R4)      ; R3 <- MEM[R4 + 8]
```

$R4+8$ が $R2$ に等しければ、ロードは直前に格納された値を読む。これはメモリ

出典 Lesson 7, 動画 14–19 ; H&P §3.5.

出典 Lesson 7, 動画 20 ; H&P §3.5.

を介した RAW 依存である。同じアドレスへの 2 つのストアは WAW 依存を持ち、先行するロードが読むアドレスへのストアは WAR 依存を持つ。RAT はこれらのうち偽の依存を除去できない。RAT がリネーミングするのはレジスタであってメモリ位置ではなく、また 2 つのメモリアクセスが衝突するかどうかは、両方のアドレスが計算されてはじめて分かるからである。

そのため Tomasulo のアルゴリズムは、ロードとストアをプログラム順に実行する。ロードとストアは他の命令と同様に、オペランドを RAT を通じて解決して発行され、アドレス加算器がアドレスを計算する。その後、ロードはロードバッファで、ストアはストアバッファで待つ (図 7.1)。格納する値がまだ計算されていないストアは、それを CDB から取り込む。メモリアクセスは、ロードとストアが発行された順に、これらのバッファから行われる。ロードは読んだ値を、演算の結果とまったく同じように、自身のタグのもとで CDB にブロードキャストする。現代のプロセッサはロードとストアもアウトオブオーダーに実行するが、そのためにはプロセッサがメモリを介した依存を検出して処理する必要がある。これは第 9 章で扱う。

7.9 完全な例

ここで例 7.5 のプログラムを最後まで実行する。前提は、7.7 節の最も単純な設計に次の仮定を加えたものである。

- 1 サイクルに発行する命令は高々 1 つであり、必要な種類の空いているステーションのうち番号の最も小さいものに発行する。
- 加減算器は ADD と SUB を 2 サイクルで、乗除算器は MUL と DIV を 4 サイクルで実行する。サイクル D でディスパッチされ、実行時間が k サイクルの命令は、サイクル D から $D+k-1$ まで実行され、サイクル $D+k$ でブロードキャストする。そのサイクルには、ユニットはすでに次の命令を受け入れられる。
- CDB は 1 本であり、乗除算器が加減算器より優先される (7.6.1 節)。結果がバスを待っているユニットは新しい命令を受け入れない。準備のできた命令は最古優先でディスパッチされる。

表 7.2 は各サイクルに起こることを列挙し、表 7.3 はサイクル 5, 6, 7 の後のリザベーションステーションと RAT を示す。

表 7.2: 完全な例の各サイクルの事象。取り込みは、それを引き起こすブロードキャストとともに示す。

サイクル	発行	ディスパッチ	結果書き込みと備考
1	I1 → ML1		
2	I2 → AD1	I1	
3	I3 → AD2		I2 は ML1 を待つ
4	I4 → AD3	I3	R1 の RAT エントリが AD3 になる
5	I5 → ML2		全ステーションが使用中；I4 は準備完了、加減算器は使用中
6			I1: ML1 = 56, R1 には古くなった結果；AD1 が取り込む；I3 は実行終了、バス待ち；I6 用の空きステーションなし
7	I6 → ML1	I2	I3: AD2 = 8, R6 ← 8；ML2 が取り込む；I2 と I4 が準備完了, I2 が古い
8	I7 → AD2	I5	
9		I4	I2: AD1 = 62, R4 ← 62；ML1 が取り込む
10			
11			I4: AD3 = 10, R1 ← 10；ML1 が取り込む
12		I6	I5: ML2 = 2, R2 ← 2；AD2 が取り込む
13-15			I6 を実行中
16			I6: ML1 = 620, R3 ← 620；AD2 が取り込む
17		I7	
18			
19			I7: AD2 = 618, R5 ← 618

出典 Lesson 7, 動画 21-26；H&P §3.5.

表 7.3: サイクル 5, 6, 7 の終了時のリザーベーションステーションと RAT. タグは斜体で示し, 「実行中」はディスパッチ済みの命令を表す. 空の RAT エントリはレジスタファイルの値で示す.

	サイクル 5 の後	サイクル 6 の後	サイクル 7 の後
AD1	I2: <i>ML1</i> , 6	I2: 56, 6	I2: 56, 6 (実行中)
AD2	I3: 14, 6 (実行中)	I3: 14, 6 (実行中)	空き
AD3	I4: 6, 4	I4: 6, 4	I4: 6, 4
ML1	I1: 14, 4 (実行中)	空き	I6: <i>AD1</i> , <i>AD3</i>
ML2	I5: <i>AD2</i> , 4	I5: <i>AD2</i> , 4	I5: 8, 4
R1	<i>AD3</i>	<i>AD3</i>	<i>AD3</i>
R2	<i>ML2</i>	<i>ML2</i>	<i>ML2</i>
R3	4	4	<i>ML1</i>
R4	<i>AD1</i>	<i>AD1</i>	<i>AD1</i>
R5	6	6	6
R6	<i>AD2</i>	<i>AD2</i>	8

サイクル 1-5. 1 サイクルに 1 命令ずつ発行される. I1 と I3 は発行時にすべてのオペランドが揃っており, 次のサイクルで I1 は乗除算器に, I3 は加減算器にディスパッチされる. I4 はサイクル 5 から準備完了だが, 加減算器は I3 を実行中である. I2 はタグ ML1 を, I5 は AD2 を待つ. サイクル 5 の後, すべてのステーションが使用中となる.

サイクル 6. 両ユニットはサイクル 5 で実行を終えている. バスは遅いほうの乗除算器の結果, すなわち ML1 のもとでの 56 を運ぶ. これは R1 にとっては古くなった結果である (例 7.6). AD1 がそれを取り込み, ML1 は解放される. I3 の結果は加減算器で待つ. サイクルの開始時点で乗除算用のステーションが 2 つとも使用中なので, I6 は発行されない.

サイクル 7. I3 が AD2 のもとで 8 をブロードキャストする. R6 の RAT エントリは AD2 を保持しているので, R6 は 8 になり, エントリは空になる. ML2 は I5 のためにこの値を取り込み, AD2 は解放される. 加減算器は新しい命令を受け入れられ, I2 と I4 がともに準備できているので, 最古優先により I2 がディスパッチされる. I6 は前のサイクルで解放された ML1 に発行され, AD1 と AD3 の 2 つのタグを受け取る.

サイクル 8-19. I7 は AD2 に発行され, ML1 と ML2 を待つ. I5 はサイクル 8 に乗除算器で実行を始め, I4 は I2 が加減算器を解放するサイクル 9 に加減算器で実行を始める. I6 は I2 (サイクル 9) と I4 (サイクル 11) の結果, そして I5 がサイクル 12 に解放する乗除算器を必要とする. I7 は, サイクル 16 にブロードキャストされる I6 の結果を必要とする. サイクル 19 の最後のブロードキャストの後, レジスタファイルは $R1 = 10$, $R2 = 2$, $R3 = 620$, $R4 = 62$, $R5 = 618$, $R6 = 8$ を保持する. これはプログラムを順に実行して得られる値とまったく同じである.

要点

この例の命令は I1, I3, I2, I4, I5, I6, I7 の順にブロードキャストし, 各命令はオペランド, 空いているステーション, 空いているユニットが許す限り早く実行される. すべてのオペランドは発行時にタグを通じて解決され, 古くなった結果はレジスタファイルに届かないので, 最終的なレジスタ値は逐次実行の場合と同じになる.

7.10 構造を追跡しないタイミングの算出

すべての構造をサイクルごとにトレースするのは手間がかかる. タイミングだけが問題であれば, 各命令の発行, ディスパッチ, 結果書き込みのサイクルを求めれば十分である. 命令 n のこれらのサイクルを I_n , D_n , W_n で, その実行時間を k_n で表す. 7.9 節の仮定のもとで, 次が成り立つ.

出典 Lesson 7, 動画 27-29 ; H&P §3.5.

発行 I_n は, I_{n-1} より後で, 必要な種類のステーションが空いている最初のサイクルである. 命令 m の結果書き込みで解放されたステーションは, サイクル $W_m + 1$ から空いている.

ディスパッチ D_n は, 次式を満たし, かつ実行ユニットが空いている最初のサイクルである.

$$D_n \geq 1 + \max(I_n, \max_{p \in P(n)} W_p), \quad (7.1)$$

ここで $P(n)$ は, n のオペランドを生成する命令 (最も近い先行する書き込み命令) の集合である. ユニットが空いているとは, 他の命令 m に占有されておらず (m はサイクル D_m から $W_m - 1$ までユニットを占有する), 準備のできたより古い命令にも取られていないことである.

結果書き込み $W_n = D_n + k_n$ である. ただし, そのサイクルのバスが優先権を持つ結果に取られていれば, それより後になる.

ユニットやバスは, 先に準備のできた後の命令に取られることがあるので, 行を単純にプログラム順に埋めていくことはできない. ユニットとバスは, すべての命令にわたってサイクル順に割り当てる. 各サイクルで, 空いているユニットはそのサイクルに準備のできている命令のうち最も古いものを取り, バスは優先権を持つ結果を取る. まだ準備のできていない古い命令のためにユニットが空いたまま待つことはない.

ヒント $P(n)$ は, n が読む各レジスタについて, 最も近い先行する書き込み命令からなる (第 6 章).

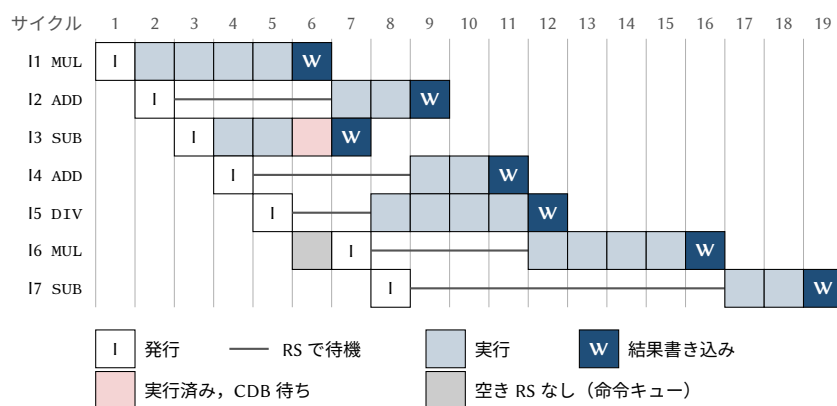


図 7.4: 実行例のタイミング. アウトオブオーダー実行により, I2 が待っている間に I3 が実行され, I4 が待っている間に I5 が実行を始める. I3 (バス), I4 (加減算器), I6 (リザベーションステーション) の遅延は, 例 7.7 で導いたものである.

表 7.4: 実行例の発行, ディスパッチ, 結果書き込みのサイクル.

命令	I_n	D_n	W_n	タイミングを決める要因
I1 MUL R1, R2, R3	1	2	6	発行時に準備完了
I2 ADD R4, R1, R5	2	7	9	$W_1 = 6$; $W_3 = 7$ まで加減算器は I3 が使用
I3 SUB R6, R2, R5	3	4	7	発行時に準備完了; サイクル 6 はバスが使用中
I4 ADD R1, R5, R3	4	9	11	加減算器を I3, 次いでより古い I2 が使用
I5 DIV R2, R6, R3	5	8	12	$W_3 = 7$
I6 MUL R3, R4, R1	7	12	16	$W_1 = 6$ までステーションなし; $W_4 = 11$; $W_5 = 12$ まで乗除算器が使用中
I7 SUB R5, R3, R2	8	17	19	$W_6 = 16$

例 7.7. 表 7.4 はこれらの規則を実行例に適用したものであり, 図 7.4 はその結果を図示する. I3, I4, I6 にすべての規則が現れている. I3 は発行の 1 サイクル後のサイクル 4 にディスパッチされ, サイクル 5 に実行を終える. 結果書き込みはサイクル 6 になるはずだが, そのときバスは I1 の結果を運ぶので, $W_3 = 7$ となる. したがって, サイクル 7 に加減算器が I2 のために空いているかどうか

は、後の命令 I3 の行に依存する。I4 はサイクル 5 から準備完了だが、加減算器は、結果がバスを待っている I3 にその結果書き込みのサイクル 7 まで占有され、続いてより古い I2 にサイクル 9 まで占有されるので、 $D_4 = 9$, $W_4 = 11$ となる。I6 は、 $W_1 = 6$ で ML1 が解放されるまで乗除算用のステーションが 2 つとも使用中なので、 $I_6 = 7$ となる。そのオペランドは I2 と I4 から来るので、式 (7.1) より $D_6 \geq 1 + \max(7, 9, 11) = 12$ であり、これは I5 が乗除算器を解放するサイクルでもあるので、 $D_6 = 12$, $W_6 = 16$ となる。この表は表 7.2 の完全なトレースと一致する。

要点 – 章のまとめ

Tomasulo のアルゴリズムは、レジスタリネーミングと、各命令をオペランドが揃い次第実行するアウトオブオーダー実行とを組み合わせる。発行は、命令をプログラム順に命令キューからリザベーションステーションへ移し、各オペランドをレジスタファイルの値または RAT のタグとして取得し、宛先の RAT エントリがそのステーションを指すようにする。ステーションは命令の準備ができるまで共通データバスから値を取り込み、その後ディスパッチが命令を空いている実行ユニットに送る。複数の命令が準備できていれば最も古いものを選ぶ。結果書き込みは値とタグをブロードキャストし、ステーションを解放し、結果が古くなっていない場合に限りレジスタファイルと RAT を更新する。1 本のバスは 1 サイクルに 1 つの結果を運び、遅いユニットが優先される。ロードとストアはプログラム順に実行される。1 サイクルに 1 ステップという前提のもとでは、各命令の発行、ディスパッチ、結果書き込みのサイクルは、そのオペランド、利用可能なステーションとユニット、そしてバスから決まる。Tomasulo のアルゴリズムは例外をサポートしない。次章のリオーダーバッファがこれを加える。

リオーダーバッファ

講義動画	Lesson 8, 動画 1-40
教科書	H&P [1] §3.6, §3.8, §C.4; [16]
キーワード	例外, 正確な例外, ファントム例外, リオーダーバッファ, コミットポインタ, 発行ポインタ, コミット, 分岐予測ミス回復, レジスタエイリアステーブル, 統合リザベーションステーション, スーパースカラ, アウトオブオーダー実行

第 7 章の Tomasulo のアルゴリズムは、各命令をそのオペランドが揃い次第実行し、結果がブロードキャストされた時点でそれをレジスタファイルに書き込む。リネーミングによって計算される値そのものは正しく保たれるが、実際のプロセッサがどれも扱わなければならない 2 つの事象が、この仕組みを破綻させる。例外と分岐予測ミスである。どちらの場合も、問題を起こした命令よりプログラム順で後ろにある命令がすでにレジスタを上書きしている可能性があり、その書き込みは取り消せない。この章では、命令をアウトオブオーダーに実行しながらレジスタの更新は厳密にプログラム順に行う、リオーダーバッファを導入する。そうして得られるプロセッサの動作を詳細に追い、最後に統合リザベーションステーション、スーパースカラ実行、そしてアウトオブオーダープロセッサのどのステップが実際に順序どおりでないのかを正確に述べる。

8.1 アウトオブオーダー実行における例外

例外とは、0 除算や不正なメモリアドレスへのアクセスのように、命令が実行時に発生させる事象である。プロセッサは例外を起こした命令のアドレスを保存し、通常はオペレーティングシステムの一部である例外ハンドラに制御を移す。ハンドラが原因に対処し終えると、保存されたアドレスに復帰することがあり、そのとき例外を起こした命令が再び実行されて、プログラムはその後を続行する。ハンドラが正しく働いたため、そして復帰後にプログラムが正しく続行するためには、レジスタとメモリが、命令を 1 つずつ実行するプロセッサでの姿と同じように見えなければならない。

定義 8.1 (正確な例外). ハンドラが開始する時点で、例外を起こした命令よりプログラム順で先行する命令がすべて完了してレジスタとメモリを更新しており、かつ例外を起こした命令もそれより後続の命令もそれらを変更していないとき、その例外を**正確な例外**という。

Tomasulo のアルゴリズムは正確な例外を提供しない。次の例を考える。

```

I1:  DIV R1, R2, R3      ; R3 = 0: 0 除算
I2:  ADD R2, R4, R5      ; I1 が読む R2 を上書きする
I3:  ADD R6, R6, R7      ; R6 <- R6 + R7

```

I2 と I3 は I1 に依存しないので、発行から数サイクルのうちに実行され、R2 と R6 に書き込む。除算は多数のサイクルを要し、ここで用いるモデルでは、除算器は演算が完了した時点で 0 除算を報告する (図 8.1)。そのときには後続の 2 つの結果はすでにレジスタファイルに入っており、3 つの不都合が生じる。

- ・ハンドラは、I1 で停止した逐次実行では生じえない R2 と R6 の値を見る。
- ・復帰後、I1 が再び実行され、今度は I2 が書いた R2 の値を読む。プログラム順では決して見てはならない値である。
- ・I2 と I3 も 2 度目の実行を受ける。I2 は同じ値を計算し直すだけだが、I3 は R6 に R7 を 2 回加えてしまう。

8.2 分岐予測ミス

分岐予測ミスも同じ問題を引き起こす。次のプログラムでは、I2 が除算の結果を

出典 Lesson 8, 動画 1-2; H&P §C.4, §3.6.

▷ precise exception

出典 Lesson 8, 動画 3; H&P §3.6.

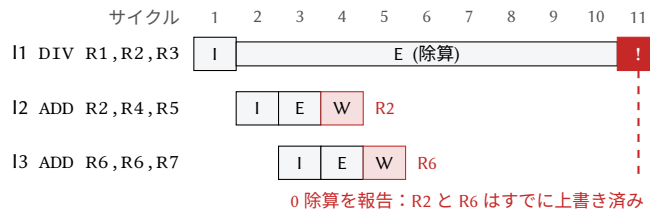


図 8.1: Tomasulo のアルゴリズムのもとでの例外 (I: 発行, E: 実行, W: 結果書き込み). I2 と I3 は、除算 I1 が 0 除算を報告するよりはるかに早く R2 と R6 に書き込む。

待っている間に、分岐予測器のおかげでプロセッサは I3 から I6 までをフェッチし発行できる。

```

I1:      DIV R1, R2, R3      ; 多数のサイクルを要する
I2:      BEQ R1, R4, Skip    ; 不成立と予測される
I3:      ADD R5, R6, R7      ; I3-I6: 分岐が不成立の
I4:      LW  R8, 0(R5)       ;   ときだけ実行される
I5:      MUL R9, R5, R5
I6:      SUB R6, R9, R7
      ...
Skip: ...

```

I2 は R1 を必要とするので除算が完了するまで実行できないが、I3 はただちにオペランドが揃い、I4 もその 1 サイクル後に揃う。Tomasulo のアルゴリズムのもとでは、R5 と R8 は分岐が解決されるよりはるかに早く書き込まれる。分岐が成立すると判明した場合、誤った経路の命令はリザベーションステーションから取り除き、Skip からフェッチをやり直せるが、R5 と R8 はプログラムが決して計算しない値で上書きされており、正しい値は失われている。

第 2 の問題は、誤った経路の命令が例外を発生させるときに生じる。プログラムが R5 をアドレスとして使うのは分岐が不成立のときだけであり、成立するときには $R6 + R7$ が有効なアドレスである必要はない。したがってロード I4 は、I2 が実行されるよりはるかに早く例外を発生させる。

定義 8.2 (ファントム例外). 予測を外した経路上にあり、したがってプログラムの逐次実行では決して実行されない命令が発生させる例外を **ファントム例外** という。

▷ phantom exception

ファントム例外はハンドラに届いてはならない。プログラム順では、それを発生させた命令は存在しないからである。

8.3 正しいアウトオブオーダー実行

2つの問題の原因は同じである。命令が結果を計算し次第レジスタを変更してしまい、先行するすべての命令が例外も予測ミスもなく完了したと判明する前に書き込みが行われるのである。対策は、結果を計算することと、それを可視にすることとを分離することである。

- 命令はこれまでどおり、オペランドが揃い次第アウトオブオーダーに実行される。
- 結果もこれまでどおりアウトオブオーダーにブロードキャストされ、待機中の命令は遅れなくそれを受け取る。
- しかし結果がレジスタに書き込まれるのはプログラム順であり、しかもその命令がプログラムの実行に属すると確実にってからである。

書き込みが許されるまで結果を保持するには、発行された後の命令のプログラム順を覚えておく構造が必要である。リザベーションステーションにはこれがない。命令は結果がブロードキャストされ次第ステーションを離れ、またステーションは特定の順序で割り当てられるわけでもないからである。

要点

実行はアウトオブオーダー、ブロードキャストもアウトオブオーダー、しかしレ

出典 Lesson 8, 動画 4; H&P §3.6; [16].

ジスタへの書き込みはプログラム順に行う。

8.4 リオーダーバッファ

プログラム順を覚えておく構造は、発行済みの命令のキューである。

定義 8.3 (リオーダーバッファ). **リオーダーバッファ (ROB)** は、発行済みで、かつ結果をレジスタファイルに書き込む **定義 8.4 のコミットステップ**をまだ通過していない命令を、すべてプログラム順に保持する。各エントリは3つのフィールドを持つ。

Reg 命令の宛先レジスタ。

Value 計算された後の、その結果。

Done Value が結果を保持していることを示すビット。

講義が描くのはこの3つのフィールドだけである。実際にはエントリは、命令が例外を発生させたか、あるいは予測を外した分岐であるかと、命令のアドレスも記録しなければならない。コミットが例外を受け付けたりフェッチをやり直したりするのに、これらが必要だからである (8.5 節, 8.6 節)。

ROB は2つのポインタで管理される循環バッファである (図 8.2)。 **発行ポインタ**は次に発行される命令が受け取るエントリを示し、命令が発行されるたびに1エントリ進む。まだコミットしていない最も古い命令は **コミットポインタ**の位置にある。コミットポインタから発行ポインタの手前までのエントリが使用中であり、命令をプログラム順に保持する。それ以外はすべて空きである。2つのポインタが等しい状態は空のバッファと満杯のバッファの両方を表しうるので、使用中エントリ数か満杯ビットで両者を区別する。通常の動作では両ポインタは進むだけで、最後のエントリから先頭へ巻き戻る。発行ポインタが戻るのは回復のときだけである (8.5 節)。すべてのエントリが使用中のときは、最も古い命令が ROB を離れるまで命令を発行できない。発行される命令はすべてエントリを受け取り、レジスタに書き込まない分岐も例外ではない。分岐のエントリはプログラム順における位置を保ち、後述するように予測を外したかどうかを記録する。

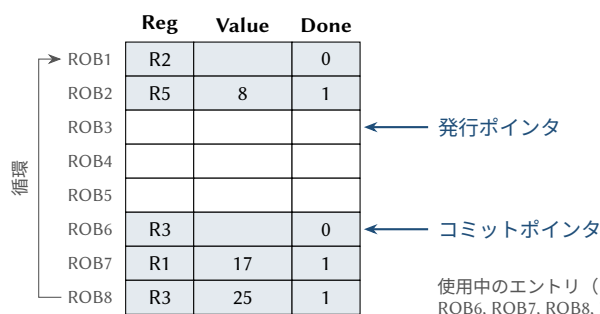


図 8.2: 8 エントリのリオーダーバッファ。コミットポインタから発行ポインタの手前までの網掛けのエントリが使用中である。ここではこの範囲が ROB8 から ROB1 へ巻き戻っている。

8.4.1 発行, ディスパッチ, 結果書き込み

ROB を備えると、Tomasulo のアルゴリズムの3つのステップは次のように変わる。

発行 命令は、空いているリザーベーションステーション (RS) と、発行ポインタの位置にある空き ROB エントリを必要とする。どちらかが欠ければ発行は待つ。ROB エントリは宛先レジスタを受け取り、Done ビットはクリアされ、発行ポインタが進む。続いて各オペランドを RAT で調べる。RAT のエントリは、いまや RS ではなく ROB エントリを示す。

- RAT エントリがレジスタファイルを指していれば、値をレジスタから複製する。

出典 Lesson 8, 動画 5-10; H&P §3.6.

▷ reorder buffer

▷ issue pointer

▷ commit pointer

- Done ビットの立った ROB エントリを示していれば、値をそのエントリから複製する。
- そうでなければ、RS はその ROB エントリをオペランドのタグとして記録し、そのタグを伴うブロードキャストを待つ。

最後に、宛先レジスタの RAT エントリを新しい ROB エントリに設定し、RS はそのエントリを自身の結果のタグとして記録する。

ディスパッチ これまでと同様に、オペランドがすべて揃った命令が実行を開始する。RS はこの時点で解放される。

結果書き込み 結果は、そのタグである ROB エントリとともに CDB にブロードキャストされる。待機中の RS はこれまでどおり値を取り込み、ROB エントリは値を格納して Done ビットを立てる。レジスタファイルも RAT も変更されない。

2 つ目の参照のケースは新しい。結果は結果書き込みからコミットまで ROB に留まり、その間 RAT はその ROB エントリを示し続ける。この区間に発行される命令は、値をそこで見つけなければならない。見つけられなければブロードキャストを待つことになるが、そのブロードキャストはすでに起きてしまっているからである。

RS をより早く解放する点も新しい。Tomasulo のアルゴリズムでは、ステーション自体が結果をブロードキャストするときのタグなので、RS は結果書き込みまで確保され続けなければならない。ROB があればタグは ROB エントリなので、演算が始まってしまえば RS は不要になり、同じ数の RS でより多くの待機中の命令を保持できる。

8.4.2 コミット

すべての命令を完了させる第 4 のステップがある。

定義 8.4 (コミット). コミットのステップでは、プロセッサはコミットポインタの位置にあるエントリを調べる。その Done ビットが立っていれば、値を宛先レジスタに書き込み、そのレジスタの RAT エントリがまだこの ROB エントリを示していれば、RAT エントリをレジスタファイルを指すように戻す。続いてコミットポインタが進み、エントリが解放される。

コミットポインタの位置にある命令が完了していなければ、後続のエントリが完了していても何もコミットされない。結果がレジスタファイルに入るのはプログラム順にかぎられる。RAT エントリを戻すのは、それがまだコミットするエントリを示している場合だけである。そうでなければ、後の命令が同じレジスタに書き込むということであり、RAT はその命令のエントリを示し続けなければならない (8.7 節)。コミットはリタイアとも呼ばれる。

8.4.3 ハードウェア構成

図 8.3 は、第 7 章の構成に ROB を加えたものを示す。CDB はいまや結果を RS と ROB に届け、RAT は ROB エントリを示し、レジスタファイルに書き込むのはコミットだけである。命令は、フェッチ、デコード、発行、実行 (ディスパッチから演算の完了まで)、結果書き込み、コミットという 6 つのステップを経る。

注 8.5. 任意の時点で、これまでに発行された命令をプログラム順に見たときの、あるアーキテクチャレジスタの最新の値は、3 つの場所のいずれかにある。RAT がレジスタファイルを指していればレジスタファイルの中、Done ビットの立った ROB エントリの中、あるいはまだ計算されておらず、Done ビットがクリアされた ROB エントリに入るのを待っている状態である。レジスタファイル自体は、常に最後にコミットした命令の時点での値を保持する。

8.5 分岐予測ミス回復

ROB があれば、予測を外した経路の命令は、コミットする前に取り除かれるかぎりレジスタファイルを変更できない。8.2 節のプログラムでは、I2 が R1 を待っている間に I3 から I6 が発行され、まだ実行中の I6 を除くすべてが結果を自身の ROB エントリに書き込む。I2 がようやく実行され、分岐の予測が外れていたと

▷ commit

出典 Lesson 8, 動画 11–12 ; H&P §3.6.

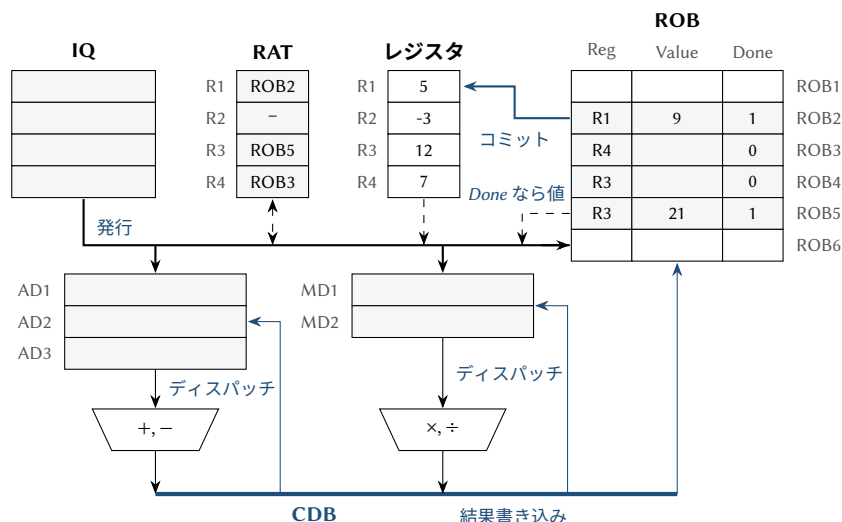


図 8.3: リオーダーバッファを備えた Tomasulo の構成. 発行は RS と ROB エントリを確保し, RAT を通じてオペランドをリネームする. オペランドの値はレジスタファイルから, あるいは Done の立った ROB エントリから得られる. 結果は CDB から RS と ROB へ送られ, コミットはコミットポインタの位置の値をレジスタファイルに複製する.

分岐と, 予測ミスが I2 の ROB エントリに記録され, その時点では他に何も起きない. 分岐より前の命令は影響を受けず, 実行とコミットを続ける.

分岐がコミットポインタに達したとき, それより前の命令はすべてコミットしているの, レジスタファイルは, 逐次実行が分岐を実行し終えた時点で持つ値をちょうど保持している. プロセッサ内に残る命令はすべて分岐より後ろにあり, 誤った経路の上にある. そこでプロセッサは**分岐予測ミス回復**を行う. これは次の 4 つの動作からなる (図 8.4).

1. 発行ポインタをコミットポインタと等しく設定してすべての ROB エントリを破棄し, ROB を空にする.
2. すべての RAT エントリを, レジスタファイルを指すように戻す.
3. すべての RS を解放し, 実行ユニット内の演算を放棄する. それらの結果は, 破棄された ROB エントリにしか行き場がない.
4. 分岐の正しい後続命令からフェッチをやり直す.

予測が当たった分岐は単にコミットする. コミットポインタが進むだけで, 他には何も起きない.

▷ branch misprediction recovery



図 8.4: 予測を外した分岐 I2 がコミットポインタに達したときの回復. I1 はすでにコミットしている. I3 から I6 の ROB エントリは, I4 に記録された例外もとも破棄され, RAT のエントリはすべてレジスタファイルに戻る.

注 8.6. コミット時に回復するのは最も単純な方式であり, 講義で用いられるのもこれである. 実際のプロセッサは, 予測を外した分岐が実行され次第回復を開始し, 分岐より後ろの ROB エントリだけを破棄して直ちにフェッチをやり直す. こうすれば, 分岐がコミットを待つ間にサイクルを失わずに済む (H&P §3.6). そ

の場合 RAT は分岐の時点の状態に戻さなければならない。たとえば分岐の発行時に保存しておいた複製から戻す。

8.6 リオーダーバッファのもとでの例外

例外は、それを発生させた命令のもう 1 つの結果として扱うことで処理する。命令が実行中に例外を発生させると、プロセッサはハンドラを呼び出す代わりに、その例外を命令の ROB エントリに記録する。そのエントリがコミットポイントに達したとき、例外が受け付けられる。プロセッサは、予測ミス回復とまったく同様に ROB エントリを破棄し、RAT を戻し、RS と実行ユニットを解放したうえで、例外を起こした命令のアドレスを保存し、例外ハンドラのフェッチを開始する。

これにより 8.1 節、8.2 節の 2 つの問題がともに解消される。

正確な例外 例外を起こした命令がコミットポイントに達した時点で、先行する命令はすべてコミットしており、後続の命令はレジスタファイルに書き込んでいない。ストアも同様にコミット時にはじめてメモリに書き込む（第 9 章）ので、メモリも後続の命令によって変更されていない。ハンドラは定義 8.1 の状態をちょうど見ることであり、復帰後には後続の命令がその状態から実行される。

ファントム例外は起きない 予測を外した経路上の例外は、予測を外した分岐より後ろのエントリに記録される。先に分岐がコミットし、その回復がそのエントリを例外もろとも破棄するので、例外が受け付けられることはない。図 8.4 では、I2 がコミットするときに I4 の例外が消える。

8.6.1 実行の外部からの見え方

ROB は、命令がいつ実行されたのかという 2 つの捉え方を分離する。プロセッサの内部では、命令は実行ユニットがその演算を行ったときに、順序を問わず実行される。一方、プログラマ、オペレーティングシステム、例外ハンドラに見えるのはレジスタファイル、メモリ、プログラムカウンタだけであり、これらにとって命令が実行されるのはコミットしたときである。コミットはプログラム順に 1 命令ずつ行われ、その様子は、各命令を完了させてから次を始めるプロセッサとまったく変わらない。結果が ROB にあってもまだコミットしていない命令は、外から見ればまだ実行されていない。その結果が破棄されれば、まったく実行されなかったことになる。8.8 節の例は、最後にその実例を示す。

8.7 コミット時の RAT 更新

定義 8.4 の RAT 更新に条件が付いているのは、1 つのレジスタに書き込む命令が同時に複数 ROB の中にありうるからである。ROB2 と ROB5 の命令がともに R4 に書き込み、ROB5 がまだ ROB にある状態で ROB2 がコミットするとする。R4 の RAT エントリは ROB5 を示している。そちらの命令のほうが後で発行されたからである。

- コミットは ROB2 の値を R4 に書き込む。これは ROB2 の後のアーキテクチャ上正しい値であり、ROB2 と ROB5 の間の命令が例外を発生させたときに例外ハンドラが見なければならない値でもある。
- それでも RAT エントリは ROB5 を示し続けなければならない。いま発行される命令はプログラム順で ROB5 より後ろにあり、レジスタファイルに入ったばかりの古い値ではなく、ROB5 の値を使わなければならない。

ROB5 がコミットするとき、その間にさらに後の書き込み命令が発行されていなければ、R4 の RAT エントリは依然として ROB5 を示しており、そのときにかぎってエントリが戻される。逆に、コミット済みのエントリを示したまま RAT エントリを放置することも、同じく誤りである。エントリはコミット時に解放され、まもなく無関係な命令に与えられうるからである。

出典 Lesson 8, 動画 13–15 ; H&P §3.6.

出典 Lesson 8, 動画 16 ; H&P §3.6.

8.8 リオーダーバッファの例

ここで、ROB を備えたプロセッサでプログラムを最後まで追う。このプロセッサは、加算・減算用に 3 つの RS (AD1-AD3)、乗算・除算用に 2 つの RS (MD1, MD2)、そして 8 エントリの ROB を持つ。ADD と SUB は 1 サイクル、MUL は 4 サイクル、DIV は 6 サイクルを要し、実行ユニットは十分にあって、準備のできた命令がユニットを待つことはない。各ステップは次の規約に従う。

- 1 サイクルに 1 命令を発行し、必要な種類の空いている RS のうち番号の最も小さいものに入れる。
- サイクル C のディスパッチで解放された RS と、サイクル C のコミットで解放された ROB エントリは、サイクル $C+1$ から新しい命令に与えられる。
- 命令がディスパッチされるのは、発行の次のサイクル以降であり、かつ待っている各オペランドのブロードキャストの次のサイクル以降である。
- サイクル D にディスパッチされ、実行時間が k サイクルの演算は、サイクル D から $D+k-1$ まで実行され、サイクル $D+k$ に結果を書き込む。CDB は 1 本であり、この例では 2 つの結果が同じサイクルに書き込まれることはない。
- 命令がコミットするのは早くとも結果書き込みの次のサイクルであり、1 サイクルにコミットする命令は高々 1 つである。

例 8.7. プログラムは次のとおりである。

```
I1:  DIV R1, R2, R3      ; R1 <- R2 / R3
I2:  ADD R4, R5, R6      ; R4 <- R5 + R6
I3:  MUL R2, R4, R4      ; R2 <- R4 * R4
I4:  SUB R5, R1, R6      ; R5 <- R1 - R6
I5:  ADD R4, R4, R3      ; R4 <- R4 + R3
I6:  ADD R6, R2, R4      ; R6 <- R2 + R4
```

初期値は $R1 = 5$, $R2 = 48$, $R3 = 4$, $R4 = 2$, $R5 = 7$, $R6 = 3$ である。RAT のエントリはすべてレジスタファイルを指し、ROB の 2 つのポインタはともに ROB1 にある。

サイクル 1-6：発行。 1 サイクルに 1 命令が発行され、 I_k は ROB_k に入る。I1 はレジスタファイルから 48 と 4 を MD1 に複製し、R1 の RAT エントリを ROB1 に設定する。サイクル 2 で I1 がディスパッチされて MD1 を解放し、I2 が値 7 と 3 とともに AD1 に発行される。サイクル 3 で発行される I3 は、R4 の RAT エントリに ROB2 を見つける。ROB2 は完了していないので、I3 の両オペランドは ROB2 を待つ。I3 は MD1 を再び使える。Tomasulo のアルゴリズムだけであれば、MD1 はサイクル 8 まで I1 に占有されたままだったはずである。I4 は、サイクル 3 で I2 が解放した AD1 に発行される。第 1 オペランドは ROB1 を待ち、第 2 オペランドは 3 である。サイクル 4 で I2 が結果を書き込み、ROB2 が 10 を受け取って I3 がそれを取り込む。サイクル 5 で I5 が AD2 に発行され、R4 の RAT エントリがまだ ROB2 を示しているのを見つめる。ROB2 は完了しているので、I5 は 10 を ROB から複製する。古い値 2 をまだ保持しているレジスタファイルからではない。続いて I5 は R4 を ROB5 にリネームする。サイクル 6 で I6 は AD3 に発行される。同じサイクルに I5 のディスパッチで解放される AD2 は、サイクル 7 からしか使えないからである。I6 は ROB3 と ROB5 を待つ。表 8.1, 8.2 はサイクル 6 の終了時の状態を示す。AD1 が I4 を、AD3 が I6 を保持し、他の RS はすべて空きで、レジスタファイルは変化していない。

サイクル 7-8：結果は出るがコミットはない。 I5 はサイクル 7 に 14 を ROB5 に、I1 はサイクル 8 に 12 を ROB1 に書き込み、I6 と I4 がこれらの値を取り込む。ROB2 はサイクル 4 から、ROB5 はサイクル 7 から完了しているが、何もコミットされない。コミットポインタの位置にある ROB1 が、サイクル 8 の終わりまで完了しないからである。

サイクル 9-10：最初のコミット。 サイクル 9 で I1 がコミットする。R1 は 12 になり、R1 の RAT エントリはまだ ROB1 を示しているので、レジスタファイルを指すように戻される。同じサイクルに I3 が 100 を ROB3 に書き込み、I4 がディス

出典 Lesson 8, 動画 17-30 ; H&P §3.6.

表 8.1: 例 8.7 のサイクル 6 と 10 の終了時の ROB. サイクル 6 の終了時にはコミットポインタは ROB1 に, サイクル 10 の終了時には ROB3 にある. 発行ポインタはどちらでも ROB7 にある.

エントリ	命令	Reg	サイクル 6		サイクル 10	
			Value	Done	Value	Done
ROB1	DIV R1, R2, R3	R1		0	コミット済み	
ROB2	ADD R4, R5, R6	R4	10	1	コミット済み	
ROB3	MUL R2, R4, R4	R2		0	100	1
ROB4	SUB R5, R1, R6	R5		0	9	1
ROB5	ADD R4, R4, R3	R4		0	14	1
ROB6	ADD R6, R2, R4	R6		0		0

表 8.2: 例 8.7 の RAT とレジスタファイル. ダッシュは, RAT エントリがレジスタファイルを指すことを表す.

レジスタ	サイクル 6		サイクル 10		最終
	RAT	値	RAT	値	
R1	ROB1	5	–	12	12
R2	ROB3	48	ROB3	48	100
R3	–	4	–	4	4
R4	ROB5	2	ROB5	10	14
R5	ROB4	7	ROB4	7	9
R6	ROB6	3	ROB6	3	114

パッチされる. サイクル 10 で I2 がコミットして R4 は 10 になるが, R4 の RAT エントリは ROB5 を示しているので, 8.7 節の要求どおりそのままにされる. I4 が 9 を書き込み, I6 がディスパッチされる.

サイクル 11–14: 残りのコミット. I6 はサイクル 11 に 114 を書き込む. I3, I4, I5, I6 はサイクル 11 から 14 に 1 サイクル 1 つずつコミットする. いずれも自身のレジスタへの最後の書き込み命令なので, それぞれが自分の RAT エントリを戻す. 最終的に ROB は空になり, RAT のエントリはすべてレジスタファイルを指し, レジスタは $R1 = 12$, $R2 = 100$, $R3 = 4$, $R4 = 14$, $R5 = 9$, $R6 = 114$ という逐次実行の値を保持する.

サイクル 10 の終了時, レジスタファイルの R4 は 10 を保持する一方で, RAT は R4 を読む命令を, 14 を保持する ROB5 に導く. どちらも正しい. 10 は最後にコミットした命令 I2 の後の R4 の値であり, 14 は最後に発行された書き込み命令 I5 の後の値である.

この例は 8.6.1 節の外部からの見え方も示している. I4 がサイクル 9 の実行時に例外を発生させたとする. 例外は ROB4 に記録され, I2 と I3 はそのままサイクル 10 と 11 にコミットし, サイクル 12 に I4 がコミットポインタに達して例外が受け付けられる. ハンドラが見るのは $R1 = 12$, $R2 = 100$, $R4 = 10$ であって, $R4 = 14$ ではない. I5 は I1 が終わるより前のサイクル 7 に 14 を ROB5 に書き込んでいたにもかかわらずである. 外から見れば, 実行はちょうど I4 の直前で停止したのである.

8.9 リオーダーバッファのもとでのタイミング

ROB を備えたプロセッサのサイクルごとの状態は, 第 7 章の Tomasulo のアルゴリズムと同様に, 各命令が各ステップを通過するサイクルによって要約でき, そこにコミットの列が 1 つ加わる. 記法は第 7 章のままとする. I_n , D_n , W_n は命令 n の発行, ディスパッチ, 結果書き込みのサイクル, k_n はその実行時間, $P(n)$ はそのオペランドを生成する命令の集合であり, C_n をそのコミットのサイクルとする. 8.8 節の規約のもとでは, I_n は I_{n-1} より後で, 必要な種類の RS と ROB エントリがともに空いている最初のサイクルである. 命令 m のディスパッチで解放された RS はサイクル $D_m + 1$ から, m のコミットで解放された ROB エント

出典 Lesson 8, 動画 31–34; H&P §3.6.

りはサイクル $C_m + 1$ から空いている．実行ユニットが十分にあり CDB の奪い合いもなければ，残りのサイクルは次式から定まる．

$$D_n = 1 + \max\left(I_n, \max_{p \in P(n)} W_p\right), \quad W_n = D_n + k_n, \quad C_n = \max(W_n + 1, C_{n-1} + 1), \quad (8.1)$$

ここで第 1 式は，実行ユニットが空いている場合の式 (7.1) の下限である．実行とは異なり，コミットは 2 つの命令がデータを共有しない場合でも直前の命令に依存する．RS をいつ解放するか，1 サイクルに何命令コミットするかは設計上の仮定であり，これを変えれば表も変わる．第 7 章のように RS が結果書き込みではじめて解放されるなら，RS はサイクル $W_m + 1$ から空く．2 つの命令が同じサイクルにコミットしうるなら， $C_n = \max(W_n + 1, C_{n-1}, C_{n-2} + 1)$ となる．

表 8.3: 例 8.7 のタイミング．実行の列は，演算が行われるサイクル D_n から $D_n + k_n - 1$ までを示す．

	命令	発行	実行	結果書き込み	コミット
I1	DIV R1, R2, R3	1	2-7	8	9
I2	ADD R4, R5, R6	2	3	4	10
I3	MUL R2, R4, R4	3	5-8	9	11
I4	SUB R5, R1, R6	4	9	10	12
I5	ADD R4, R4, R3	5	6	7	13
I6	ADD R6, R2, R4	6	10	11	14

表 8.3 は例に対する結果を示す．実行の列と結果書き込みの列は順序どおりでない．I5 は，先行する I1, I3, I4 より先に結果を書き込む．コミットの列は増加している．I2 と I5 は結果書き込みの 6 サイクル後にコミットする．ROB の先頭にある遅い除算に，そしてそれがサイクル 9 にコミットした後は 1 サイクル 1 命令というコミットの上限に阻まれるからである．それ以降の命令は，いずれも直前の命令のちょうど 1 サイクル後にコミットする．この待ちほどの計算も遅らせない．I3 と I5 は I2 の結果を，I6 は I3 と I5 の結果を，コミットを待つことなく得ている．ROB が遅らせるのは，結果がアーキテクチャ状態の一部になる時点だけである．

とはいえ ROB は，小さすぎれば実行を制限しうる．エントリが 4 つしかなければ，I5 は I1 がコミットした次のサイクルであるサイクル 10 より前には発行できず，オペランドがサイクル 5 から揃っているにもかかわらず，表 8.3 より数サイクル遅れて実行されることになる．したがって ROB は，遅い命令がコミットを待つ間に発行されるすべての命令を保持できるだけの大きさを持たなければならない．

8.10 統合リザベーションステーション

図 8.3 では，実行ユニットの種類ごとに専用の RS の組があり，各組の大きさは設計時に固定される．プログラムによって演算の構成は異なるので，加算の多いプログラムでは，乗算器の RS が空いているのに加算器の RS がすべて使用中になり，発行が待たされることがある．命令はプログラム順に発行されるので，待たされている加算より後ろにある乗算でさえ，空いている乗算器の RS を使えない．

定義 8.8 (統合リザベーションステーション)．すべての演算で共有される単一の RS の集まりを**統合リザベーションステーション**という．命令は，その集まりの中の空いている任意の RS に発行できる．

このとき発行が待たされるのは，すべての RS が使用中のときだけであり，同じ数の RS がより有効に使われる．代償はディスパッチにある．1 つの実行ユニットのために 1 つの組の RS から選ぶのではなく，ディスパッチ論理は，空いている実行ユニットごとに，そのユニットが担当する演算の，準備のできた命令を集まり全体から探し出し，その中から選ばなければならない．

ヒント 表はプログラム順に 1 行ずつ，各行は左から右へ，コミットを最後にして埋めるとよい．

出典 Lesson 8，動画 35；H&P 第 3 章．

▷ unified reservation station

8.11 スーパースカラプロセッサ

第 6 章で見たように、プロセッサの IPC は、1 サイクルに扱える命令数によって制限される (6.8 節)。

定義 8.9 (スーパースカラプロセッサ). スーパースカラプロセッサは、すべてのステップを 1 サイクルに複数の命令に対して行う。すなわち、複数の命令をフェッチ、デコード、発行し、複数の実行ユニットにディスパッチし、複数の結果を書き込み、各サイクルに複数の命令をコミットする。

そのためには、各ステップにそれぞれのハードウェアが必要になる。フェッチとデコードは複数の命令を一度に扱わなければならない。発行は同じサイクルに複数の命令をリネームし、そのそれぞれが自分より前の命令による RAT の更新を見なければならない。ディスパッチには複数の実行ユニットが要る。複数の結果を書き込むには複数の CDB が必要で、すべての RS のオペランドが自身のタグをそのすべてと比較しなければならない。コミットは、コミットポイントの位置の複数のエントリを調べ、複数のレジスタに書き込まなければならない。

これらのステップは、すべての命令が通る 1 本の鎖をなす。したがってプロセッサが持続できる 1 サイクル当たりの命令数は、最も制約の厳しいステップが扱える数を超えない。1 サイクルに 4 命令をフェッチ、デコード、ディスパッチしながら発行は 3 命令、コミットは 2 命令というプロセッサは、プログラムにどれだけ ILP があっても、長い目で見れば 1 サイクルに高々 2 命令しか完了させられない。1 つのステップを広げることに意味があるのは、そのステップがボトルネックである場合だけである。

8.12 用語

アウトオブオーダープロセッサのステップに対して本講義で用いる名称は、普遍的なものではない。論文、教科書、プロセッサの資料では、いくつもの別名が使われている (表 8.4)。その中にはコミットを *complete* と呼ぶものもある。これに対し本章では、演算が完了するのはその実行が終わったときである。

表 8.4: 本講義と他の文献におけるステップの名称。

本講義	他の文献での名称
発行	issue, allocate, dispatch
ディスパッチ	dispatch, execute, issue
コミット	commit, complete, retire, graduate

8.13 何がアウトオブオーダーなのか

アウトオブオーダー実行を行うプロセッサでも、すべてのステップが順序どおりでないわけではない (図 8.5)。

フェッチとデコード 予測された経路に沿って、命令をプログラム順に取り込む。

発行 プログラム順である。リネーミングがそれを要求する。各命令は、自分より前のすべての命令が残した RAT を見なければならず、ROB エントリもプログラム順に割り当てられるからである。

実行と結果書き込み 順序どおりでない。命令はオペランドが揃い次第開始し、その演算が終わったときに終了する。後の命令が先の命令を追い越すのは、この 2 つのステップだけである。

コミット 再びプログラム順である。実行が緩めた順序を ROB が回復する。

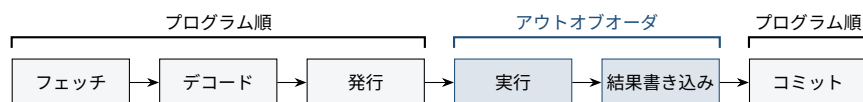


図 8.5: ROB を備えたアウトオブオーダープロセッサのステップ。プログラム順でないのは実行と結果書き込みだけである。

出典 Lesson 8, 動画 36; H&P §3.8.

▷ superscalar

出典 Lesson 8, 動画 37; H&P 第 3 章.

注意 同じ語が別のステップを指すことがある。文献によっては *dispatch* が本講義でいう発行を、*issue* が本講義でいうディスパッチを指す。

出典 Lesson 8, 動画 38–40; H&P §3.6.

要点 – 章のまとめ

Tomasulo のアルゴリズムは、結果がブロードキャストされ次第それをレジスタに書き込むので、例外は正確でなく、予測を外した命令がレジスタを壊し、誤った経路の命令がファントム例外を発生させうる。リオーダーバッファは、発行済みの命令を、コミットポイントと発行ポイントの間にプログラム順に保持する。RAT は ROB エントリを示し、結果は RS と ROB にブロードキャストされ、RS はディスパッチ時に解放される。レジスタファイルに書き込み、コミットするエントリをまだ示している RAT エントリを戻すのは、プログラム順に ROB エントリを 1 つずつ処理するコミットだけである。予測を外した分岐や例外を起こした命令への処置は、それがコミットするときに行う。ROB を空にし、RAT をレジスタファイルに戻し、RS と実行ユニットを解放して、正しいアドレスからフェッチをやり直す。タイミングの表にはコミットの列が加わり、これは式 (8.1) に従う。統合リザベーションステーションは、1 つの RS の集まりをすべての演算で共有する。スーパースカラプロセッサはすべてのステップで 1 サイクルに複数の命令を扱い、最も狭いステップによって制限される。順序どおりでないのは、実行と結果書き込みだけである。

メモリアーダリング

講義動画	Lesson 9, 動画 1-14
教科書	H&P [1] §3.5-3.6
キーワード	メモリを介した依存, ロード, ストア, コミット, ロードストアキュー, LSQ, スタロードフォワードリング, ロードストアのアウトオブオーダー実行, ロードストアのインオーダー実行, 再実行, リオーダーバッファ, リザベーションステーション

第8章のリオーダーバッファによって、アウトオブオーダープロセッサはレジスタを介するあらゆる依存を処理し、例外を正確に保てるようになった。しかしロードとストアは、メモリを介しても互いに依存する。あるロードが、先行するストアの書き込む位置を読むかどうか分かるのは、両方のアドレスが計算された後である。第7章がロードとストアをプログラム順に実行していたのはこのためである。この章では、それらをアウトオブオーダーに実行する方法を示す。ストアはコミット時にはじめてメモリに書き込み、ロードストアキューが未コミットのロードとストアをプログラム順に保持し、各ロードは自分の値をメモリから、あるいはそれを書いたストアから直接得る。

9.1 メモリを介した依存

第8章のプロセッサは、制御依存と、レジスタを介するあらゆる依存に対処する。分岐予測（第4章）は制御依存のコストの大半を取り除き、ROBは予測を外した経路の命令を取り消す。リネーミング（第6章）は、同じレジスタへの書き込みの間の偽の依存を除去する。Tomasuloのアルゴリズムによるスケジューリング（第7章）は、各命令を、それが読むレジスタ値が計算されるまで待たせる。そしてROBは正確な例外を実現する。

残るのはロードとストアである。これらは、命令がレジスタを介して互いに依存するのとまったく同じように、メモリ位置を介して互いに依存する（7.8節）。

```
SW  R1, 0(R2)      ; MEM[R2] <- R1
ADD R4, R5, R6      ; R4 <- R5 + R6
LW  R3, 0(R4)      ; R3 <- MEM[R4]
```

R4がR2に等しいとき、かつそのときに限り、ロードはストアに対してRAW依存を持つ。このような依存をレジスタを介した依存と区別する性質が2つある。

- 命令そのものからは見えない。レジスタ名はメモリ位置について何も語らず、メモリ位置はアドレスが計算されてはじめて、多くの場合その命令が発行された数サイクル後に分かる。
- メモリをリネーミングするものは存在しない。RATが対応付けるのはレジスタ名だけなので、プロセッサはロードごとに、その値を読むべきストアを見つけるか、そのようなストアがないと判定しなければならない。

この章の残りでは2つの問いに答える。ストアが実際にメモリに書き込むのはいつか、そしてロードはどこで自分の値を見つけるか、である。

9.2 メモリが書かれるとき

ストアも他の命令と同様にオペランドが揃えば実行されるが、その時点ではメモリに書き込まない。ストアの実行はアドレスを計算し書き込む値を確定するだけであり、書き込み自体はストアがコミットするときに行われる。メモリへの書き込みは、レジスタへの書き込みと同様に取り消せない。ストアが実行時にメモリに書き込むとすると、その後に先行命令が例外を発生させたとき、ハンドラは、プログラム順ではまだ書かれていないはずの位置を見ることになり、また予測を外した経路のストアは、プログラムが変更しないメモリを変更してしまう。これに対しコミット時には、そのストアがプログラムの実行に属することが確定して

出典 Lesson 9, 動画 1-2 ; H&P §3.5-3.6.

出典 Lesson 9, 動画 3 ; H&P §3.6.

おり、しかもストアはプログラム順にコミットする。

書き込みを遅らせることは、ロードにとって問題を生む。ロードはアドレスを計算し次第メモリを読むことができるが、プログラム順で先行するストアが、実行を終えてコミットを待っている、あるいはまだアドレスすら計算していないことがある。そのストアがロードの読む位置に書き込むなら、メモリはまだ古い値を保持しており、ロードは代わりにそのストアの値を得なければならない。したがってプロセッサは、まだコミットしていないすべてのストアのアドレスと値をプログラム順に保持し、すべてのロードがそれらを参照しなければならない。

要点

ストアはアウトオブオーダーに実行されるが、メモリへの書き込みはコミット時にプログラム順で行う。それまでその値はプロセッサの内部にしか存在せず、ロードはそこを探さなければならない。

9.3 ロードストアキュー

まだコミットしていないロードとストアは、ROB と同じように働くキューに保持される。

定義 9.1 (ロードストアキュー). **ロードストアキュー (LSQ)** は、発行済みでまだコミットしていないすべてのロードとストアを、プログラム順に保持する。各エントリは4つのフィールドを持つ。

L/S その命令がロードかストアか。

Addr メモリアドレス。計算された後に記録される。

Value ストアでは書き込む値、ロードでは得た値。

C エントリが完了したときに立つ。ロードでは値を得たとき、ストアではアドレスと値の両方を記録したときである。

ROB と同様に、LSQ は発行ポインタとコミットポインタを持つ循環バッファである (図 9.1)。ロードやストアは、発行されるとときに ROB エントリに加えて発行ポインタの位置のエントリを受け取り、その LSQ エントリはコミット時に解放される。他の命令は LSQ を使わない。ロードはアドレスを記録し、続いて値を記録する。ストアは、ベースレジスタが得られ次第アドレスを、値レジスタが得られ次第値を、どちらの順でも記録する。ストアはコミットするまでエントリを保持し、そのときにはじめてメモリに書き込むので、LSQ にあるストアは、値がまだメモリに入っていないストアにちょうど一致する。

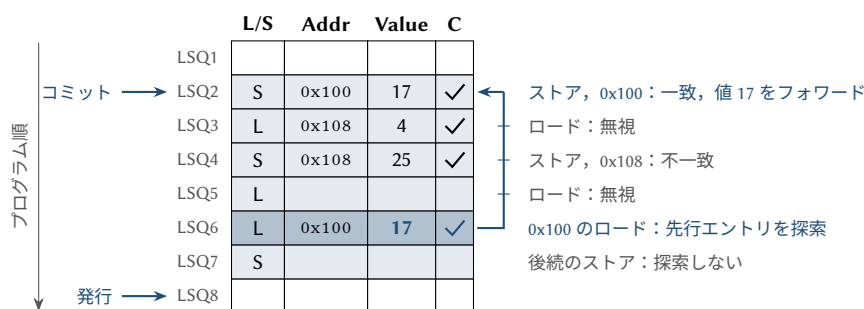


図 9.1: 8 エントリのロードストアキュー。LSQ2 から LSQ7 までが使用中である。LSQ6 のロードはアドレス 0x100 を計算した。その探索はロードと 0x108 へのストアを飛ばし、0x100 への最も近い先行ストアから値 17 を取る。

ロードはアドレスを計算すると、それを自身の LSQ エントリに記録し、自分のエントリからコミットポインタに向かって先行するエントリを探索する。位置を読むことはその内容を変えないので、ロードは飛ばされ、ストアだけが問題になる。探索は、同じアドレスを持つ最初のストアで、あるいはコミットポインタで終わる。

1. 同じアドレスのストアが見つければ、この最も近い一致するストアが、プログラム順でロードの前にその位置へ書き込む最後の命令なので、その値こそロードが読むべき値である。ロードはそのストアの LSQ エントリから

第3章のパイプラインでは、MEM ステージがロードとストアをプログラム順に行う。

出典 Lesson 9, 動画 4-5 ; H&P §3.6.

▷ load-store queue

ROB もロードとストアを保持するが、キューを分けることでアドレスの探索を短く保てる。

値を複写し、ストアがまだ値を記録していなければそれを待ち、メモリにはまったくアクセスしない。このように未コミットのストアの値をロードに与えることを、**ストアロードフォワーディング**という。

2. 一致するストアもアドレスの分からないストアも見つからないままコミットポイントに達すれば、ロードの前にその位置へ書き込む未コミットのストアは存在せず、メモリがすでに正しい値を保持しているので、ロードはメモリを読む。

探索が終わる前に、まだアドレスの計算されていないストアを通過した場合、そのストアはロードの読む位置に書き込むかもしれないし書き込まないかもしれない、プロセッサにはまだどちらとも分からない。これが難しい場合であり、次節ではその扱い方を示す。

9.4 ロードストアのアウトオブオーダ実行とインオーダ実行

アドレスの分からないストアを探索が通過してしまうロードに対しては、3つの選択肢がある。

1. 第7章と同様に、ロードとストアを互いにプログラム順に実行する。
2. 先行するすべてのストアがアドレスを計算するまでロードを待たせ、その探索がアドレスの分からないストアに出会わないようにする。
3. ロードをそのまま実行し、誤った値は後で検出して訂正する。

最初の2つはロードを待たせ、3つ目は待たせない代わりに検査を必要とする。ストアのアドレスはベースレジスタが得られるまで分からず、そのレジスタを遅い命令が計算する場合には長くかかりうる。次のプログラムでは、ストアのアドレスは乗算に依存するのに対し、ロードのアドレスはほとんど直ちに得られる。

```
I1: MUL R1, R2, R3      ; R1 <- R2 * R3
I2: ADD R1, R1, R4      ; R1 <- R1 + R4 (アドレス)
I3: SW  R5, 0(R1)       ; MEM[R1] <- R5
I4: ADD R6, R6, R7      ; R6 <- R6 + R7 (アドレス)
I5: LW  R8, 0(R6)       ; R8 <- MEM[R6]
I6: SUB R9, R8, R5      ; R9 <- R8 - R5
```

タイミングの規約は8.8節のものを用いる。MULは4サイクル、ADD、SUB、ロード、ストアは1サイクルを要し、その1サイクルでロードはアドレスを計算して値を得、ストアはアドレスを計算する。値R5は最初から利用できる。I1は結果をサイクル6に、I2はサイクル8に書き込むので、ストアI3が実行され、そのアドレスが分かるのはサイクル9である。I4は結果をサイクル6に書き込むので、ロードI5はサイクル7に実行できる(図9.2)。

9.4.1 ロードストアのアウトオブオーダ実行

定義 9.2 (ロードストアのアウトオブオーダ実行)。**ロードストアのアウトオブオーダ実行**(選択肢3)では、ロードは、先行するストアがアドレスを計算しているかどうかにかかわらず、自身のアドレスが計算でき次第実行される。そのLSQ探索は、アドレスの分からないストアを、アドレスが異なるものとみなして通過する。

例ではI5はサイクル7に、その値を使うI6はサイクル9に実行される。その代わり、すべてのストアは自身のアドレスを、自分を追い越したロードと照合しなければならない。I3はサイクル9にアドレスを計算すると、後続のLSQエントリの中から、同じアドレスを持ちCビットの立ったロード、すなわちすでに値を得たロードを探す。

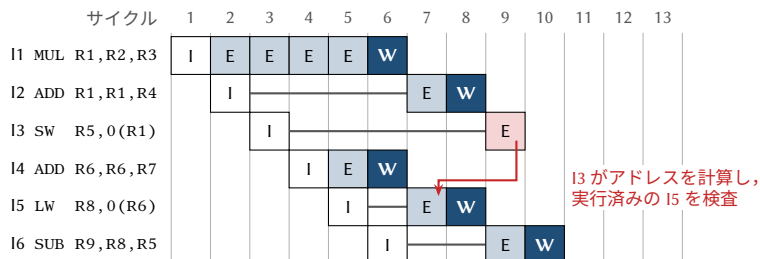
- I5のアドレスが異なれば何も起こらない。I5は正しい値を得ており、I6はサイクル10に結果を書き込む。
- アドレスが等しければ、I5は、プログラム順ではI3が上書きする値を得てしまっているので、その値を直接または間接に使ったすべての命令が誤った結果を計算したことになる。プロセッサはI5とそれ以降のすべての命令を取り消し、I5のフェッチからやり直して再実行する。I3がそれらに先行

▷ store-to-load forwarding

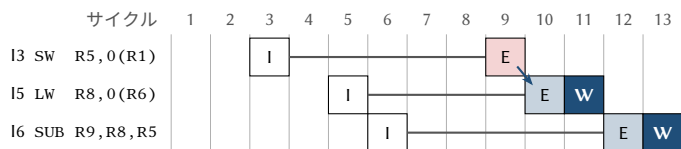
出典 Lesson 9, 動画 5-8 ; H&P §3.6.

▷ out-of-order load-store execution

(a) ロードストアのアウトオブオーダー実行



(b) ロードストアのインオーダー実行



[I] 発行 — 待機 [E] 実行 [] ストア実行 [W] 結果書き込み

図 9.2: 9.4 節のプログラムを 2 つの方針で実行した場合。(a) I5 はストア I3 がアドレスを知る前に実行され、I3 はサイクル 9 に I5 を検査する。(b) I5 は I3 がアドレスを計算するまで待つ。I1, I2, I4 は (a) と同じである。

するので、いずれもコミットしておらず、ROB は予測を外した経路の命令と同じようにそれらを破棄する (8.5 節)。I5 が再び実行されると、その探索は I3 を見つけ、フォワーディングによって値を受け取る。

9.4.2 ロードストアのインオーダー実行

定義 9.3 (ロードストアのインオーダー実行). ロードストアのインオーダー実行 (選択肢 1) では、ロードとストアは互いにプログラム順に実行される。すなわち、ロードやストアは、先行するすべてのロードとストアが実行されるまで待つ。ここでストアは、アドレスが分かった時点で実行されたとみなす。他の命令は依然としてアウトオブオーダーに実行される。

▷ in-order load-store execution

例では I5 は I3 を待ってサイクル 10 に実行され、I6 はサイクル 12 に実行されてサイクル 13 に結果を書き込む。どのロードも誤った値を得ることはなく、再実行も起こらない。しかし I5 は、そのアドレスが I3 のアドレスと異なる場合にも 3 サイクル待ち、そのときこの待ちは純粋な損失である。ここでは選択肢 2 も同じタイミングになる。I5 に先行するロードがないからである。一般には選択肢 2 のほうが待ちは短い。ロードは先行するロードには依存しないからである。

9.4.3 比較

ロードストアのアウトオブオーダー実行は、ロードが、アドレスのまだ分からない先行ストアの書き込む位置を読まない限りサイクルを稼ぎ、読む場合にはサイクルを失う。ロードとそれ以降の命令を取り消して再実行するコストは、待った場合よりも大きいからである。選択肢 1 と 2 は、アドレスの分からないストアに続くすべてのロードを、両者が衝突するかどうかにかかわらず遅らせる。実際のプログラムでは、そのようなストアの書き込む内容を本当に読むロードはごくわずかなので、ロードストアのアウトオブオーダー実行のほうが性能がよく、高性能プロセッサが採用しているのはこの方針である。

要点

ロードストアのインオーダー実行はロードを再実行することがない代わりに、通常は依存しないストアをロードに待たせる。ロードストアのアウトオブオーダー実行はロードを直ちに実行させ、その正しさは、各ストアがアドレスを計算するときに完了済みの後続ロードを検査することに依存する。

9.5 スタロードフォワーディング

スタロードフォワーディングは、各ロードと、その値を読むべきストアとを対応付ける。ロードストアのアウトオブオーダー実行を行うプロセッサは、この対応を両側から用いる。

ロードにとって 値は、同じアドレスを持つ最も近い先行ストア、すなわちプログラム順でロードの前にその位置へ書き込む最後のストアから来る。同じアドレスへのより古いストアは該当しない。その値はより近いストアに上書きされるからである。LSQ にこのアドレスの先行ストアがなければ、値はメモリから来る。

ストアにとって 値は、同じアドレスを持つ後続のすべてのロードに、そのアドレスへの次のストアの手前まで届く。そのストアより後のロードは、代わりにそのストアの値を受け取る。

ロードは探索 (9.3 節) で第 1 の規則を、ストアは完了済みの後続ロードの検査 (9.4 節) で第 2 の規則を適用する。図 9.3 は、2 つのアドレスに対する 7 つのロードとストアについてこの対応を示す。

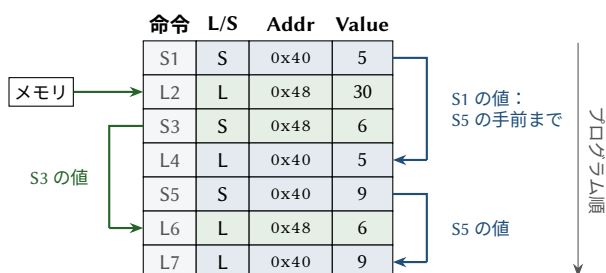


図 9.3: 0x40 と 0x48 への 7 つのロードとストアに対するスタロードフォワーディング。値は逐次実行の場合のものである。L4 は S1 の値を受け取り、その後ろにある S5 の値は受け取らない。L7 は S1 の値を上書きする S5 の値を受け取る。L2 は S3 に先行するのでメモリを読む。

9.6 ロードストアキューの例

ここで、図 9.3 のロードとストア (例 9.4) を、ロードストアのアウトオブオーダー実行のもとでトレースする。

例 9.4. あるプログラムのロードとストアは、プログラム順に次のとおりである。

```

S1:  SW  R1, 0(R2)      ; MEM[0x40] <- R1
L2:  LW  R3, 8(R2)      ; R3 <- MEM[0x48]
S3:  SW  R4, 8(R10)     ; MEM[R10+8] <- R4
L4:  LW  R5, 0(R2)      ; R5 <- MEM[0x40]
S5:  SW  R6, 0(R7)      ; MEM[R7] <- R6
L6:  LW  R8, 8(R2)      ; R8 <- MEM[0x48]
L7:  LW  R9, 0(R2)      ; R9 <- MEM[0x40]
```

ここで R2 = 0x40, R1 = 5, R4 = 6, R6 = 9 は最初から利用できる。R10 = 0x40 と R7 = 0x40 は S3 と S5 に先行する遅い命令が計算するので、この 2 つのストアがアドレスを知るのは遅く、その順序は表 9.1 のとおりである。メモリは最初 0x40 に 20 を、0x48 に 30 を保持している。

L4 と L6 は、アドレスのまだ分からないストアを通過するが、その後の S3 と S5 の検査では、これらとの衝突は見つからない。L7 も S5 を通過するが、ステップ 7 で S5 が 0x40 を計算すると、このアドレスで完了している L7 が見つかり、しかも両者の間に 0x40 へのストアはない。L7 が S1 から得た値 5 は、プログラム順では S5 に上書きされるので、L7 とそれ以降のすべての命令が取り消されて再実行され、ステップ 8 で L7 は S5 から 9 を受け取る。

エントリがプログラム順にコミットした後、メモリは 0x40 に 9 を、0x48 に 6 を保持し、ロードは R3 = 30, R5 = 5, R8 = 6, R9 = 9 を返している。これは逐次実行の結果と同じである。ロードストアのインオーダー実行であれば、L4 は S3

出典 Lesson 9, 動画 9; H&P 第 3 章。

注意 探索はロードから遡って行う。コミットポイントから順に前向きに探索すると、一致するストアのうち最も古いものがみつかる。

出典 Lesson 9, 動画 10; H&P §3.6。

表 9.1: 例 9.4 の実行. 各ステップで1つのエントリがアドレスを計算する. ロードは先行するストアを探索し, ストアはCビットの立った後続ロードを検査する.

ステップ	エントリ	Addr	探索または検査	結果
1	S1	0x40	完了した後続ロードなし	値 5 を記録
2	L2	0x48	S1 は相違	メモリから 30
3	L4	0x40	S3 は不明, S1 が一致	S1 から 5
4	S3	0x48	L4 は相違	衝突なし
5	L6	0x48	S5 は不明, S3 が一致	S3 から 6
6	L7	0x40	S5 は不明, S3 は相違, S1 が一致	S1 から 5
7	S5	0x40	L6 は相違, L7 が一致	L7 を再実行
8	L7	0x40	S5 が一致	S5 から 9

を, L6 と L7 は S5 を待ち, 二度実行される命令はなかったはずである.

9.7 LSQ と ROB とリザベーションステーション

ロードとストアも他の命令と同じステップを経るが, LSQ が各ステップに作業を加える (表 9.2). ロードやストアが発行されるのは, ROB エントリと LSQ エントリの両方が空いているときだけである. LSQ エントリは, 他の命令が ROB エントリとともに必要とするリザベーションステーションの代わりとなる. ロードは, 算術演算の結果とまったく同じように, 自身の ROB エントリの名前のもとで値を CDB にブロードキャストする. ストアはレジスタの値を生成しないので何もブロードキャストせず, その ROB エントリは, ストアがアドレスと値の両方を記録した時点で完了となる.

出典 Lesson 9, 動画 11–14 ; H&P §3.6.

表 9.2: LSQ を持つプロセッサにおけるロードとストアのステップ.

ステップ	ロード	ストア
発行	空いている ROB エントリと LSQ エントリを必要とする (RS は不要). ベースレジスタを RAT で引き, 宛先レジスタの RAT エントリをそのロードの ROB エントリに設定する	空いている ROB エントリと LSQ エントリを必要とする (RS は不要). ベースレジスタと値レジスタを RAT で引く. RAT エントリは変化しない
実行	アドレスを計算して LSQ に記録し, 先行するストアを探索し, フォワードされた値を複写するかメモリを読み, C を立てる	ベースレジスタが得られたときアドレスを記録し, C の立った後続ロードを検査する. 値は得られたときに記録する. 両方を記録した時点で C を立てる
結果書き込み	値を CDB にブロードキャストし, ROB エントリがそれを受け取る	ブロードキャストするものはない
コミット	値をレジスタファイルに書き込み, RAT エントリがまだこの ROB エントリを指していればそれを消す. ROB エントリと LSQ エントリを解放する	値をメモリに書き込む. ROB エントリと LSQ エントリを解放する

どちらのキューもプログラム順なので, ROB のコミットポインタにあるロードまたはストアは LSQ の最も古いエントリでもあり, 2つのエントリは同時に解放される. ストアが LSQ を離れるのは, その値がメモリに書き込まれるのと同じ時点なので, 後続のロードは, ロードが探す2つの場所のいずれかで必ずその値を見つける. コミット前はストアの LSQ エントリに, コミット後はメモリにある.

要点 – 章のまとめ

ロードとストアは、実行されてはじめて分かるメモリアドレスを介して互いに依存し、メモリはリネーミングされない。そのためストアは、コミット時にプログラム順でメモリに書き込む。ロードストアキューは、未コミットのロードとストアを、そのアドレス、値、完了ビットとともにプログラム順に保持する。ロードは、ストアロードフォワーディングによって、同じアドレスを持つ最も近い先行ストアから値を取り、そのようなストアがなければメモリから取る。アドレスの分からない先行ストアの扱いには2つの方針がある。ロードストアのインオーダー実行ではロードがそれらを待ち、ロードストアのアウトオブオーダー実行ではロードが先に進み、後に一致するアドレスを計算したストアが、そのロードとそれ以降のすべての命令を再実行させる。そのような衝突は稀なので、後者のほうが速い。ロードとストアは発行時に ROB エントリと LSQ エントリを必要とし、他の命令は ROB エントリと RS を必要とする。

コンパイラによる ILP

講義動画	Lesson 10, 動画 1-20
教科書	H&P [1] §3.2, 付録 H
キーワード	命令レベル並列性, サイクル当たり命令数, 依存の連鎖, 木の高さ削減, 命令スケジューリング, インオーダー実行, if 変換, ループ展開, インライン展開, コード肥大化, 命令数, ソフトウェアパイプラインニング, トレーススケジューリング

第 6~9 章では、ハードウェアによってプロセッサのサイクル当たり命令数 (IPC) を高める手法を見てきた。レジスタリネーミング、リオーダーバッファを伴うアウトオブオーダー実行、ロードとストアのアウトオブオーダー実行により、IPC はプログラムの命令レベル並列性 (ILP) に近づく。しかしハードウェアが利用できる並列性は、プログラムが実際に含んでいて、かつハードウェアの手の届く範囲にあるものに限られる。この章では、コンパイラがその両方をどのように改善するかを示す。木の高さ削減は ILP を制限する依存の連鎖を短くし、命令スケジューリングは独立な命令を隣り合わせに配置し、if 変換・ループ展開・インライン展開は、それらを隔てる分岐・呼び出し・リターンを除去する。

10.1 コンパイラはどのように IPC を高められるか

どのプロセッサでも、IPC は実行するプログラムの ILP によって上限が定まる (6.8 節)。IPC に対する 2 つの制限はプロセッサだけで決まるものではなく、コンパイラはそのどちらにも影響する。

プログラムの ILP ILP は、命令数を RAW 依存の最長の連鎖の長さで割ったものである (式 (6.1), (6.2))。連鎖がどれだけ長くなるかは、コンパイラがソースプログラムをどう翻訳したかに依存する。同じ計算を、より短い連鎖をなす命令で表現できることが多い。

プロセッサが見つけれられる並列性 プロセッサが実行する命令を探すのは、最も古い未実行の命令に続く限られた数の命令の中だけである。アウトオブオーダープロセッサはリオーダーバッファに収まる命令を対象とし (第 8 章)、インオーダープロセッサは次の数命令だけを見て、待たなければならない最初の命令で止まる (第 6 章)。プログラム順でこのウィンドウより離れた位置にある独立な命令は、ILP がどれほど高くても同じサイクルには実行できない。コンパイラはそのような命令を近くに配置でき、また両者の間にある分岐、呼び出し、リターンを除去できる。

コンパイラはプログラムの一部のウィンドウではなく全体を見ることができ、しかもプログラムの実行前に動作するので、プロセッサが 1 クロックサイクルの中では到底行えない解析にも時間をかけられる。この章で扱う技法を表 10.1 に示す。

出典 Lesson 10, 動画 1-2 ; H&P §3.2.

表 10.1: IPC に対する制限と、それに対処するコンパイラ技法。

制限	コンパイラ技法	節
長い依存の連鎖が ILP を下げる	木の長さ削減	10.2
独立な命令が離れている	命令スケジューリング	10.3
分岐が命令を隔てる	if 変換, トレーススケジューリング	10.4, 10.7
ループの分岐が反復を隔てる	ループ展開, ソフトウェアパイプラインニング	10.5, 10.7
呼び出しとリターンが命令を隔てる	インライン展開	10.6

10.2 木の高さ削減

コンパイラは算術式を式木を経由して翻訳する。式木では、各演算がノードであり、その子がオペランドである。各演算は 1 つの命令になり、あるノードの命令はその子の命令の結果を読む。したがって、これらの命令の間の RAW 依存の最長の連鎖は木の高さ、すなわち根から葉への最長経路上の演算の個数に等しく、第 6 章の理想プロセッサでは木の 1 段が 1 サイクルを要する。

素直な翻訳は、ソースプログラムがオペランドを書いた順に従う。R1 から R6 に保持された 6 つの変数の和を R10 に求める場合、次のコードが生成される。

```
ADD R10, R1, R2      ; サイクル 1
ADD R10, R10, R3     ; サイクル 2
ADD R10, R10, R4     ; サイクル 3
ADD R10, R10, R5     ; サイクル 4
ADD R10, R10, R6     ; サイクル 5
```

どの加算も直前の加算の結果を使う。木は高さ 5 の連鎖に退化し (図 10.1a), 5 つの命令はどのプロセッサでも 5 サイクルかかり、その ILP は 1 である。

定義 10.1 (木の高さ削減) 演算の結合法則と交換法則を用いて、式を、より段数の少ない式木を持つ等価な式に書き換えることを**木の高さ削減**という。演算の個数、したがって命令数は変わらない。

例 10.2. $((R1 + R2) + (R3 + R4)) + (R5 + R6)$ とグループ化すると、同じ和は次のようになる。

```
ADD R10, R1, R2      ; サイクル 1
ADD R11, R3, R4      ; サイクル 1
ADD R12, R5, R6      ; サイクル 1
ADD R10, R10, R11     ; サイクル 2
ADD R10, R10, R12     ; サイクル 3
```

対をなす 3 つの加算はどれも何にも依存せず、サイクル 1 で実行される。最初の 2 対の和がサイクル 2 に、最後の加算がサイクル 3 に続く (図 10.1b)。命令は依然として 5 つだが、それが 3 サイクルで済む。ILP は 1 から $5/3 \approx 1.67$ に上がる。その代償は、中間結果のための 2 つの追加のレジスタ R11 と R12 である。

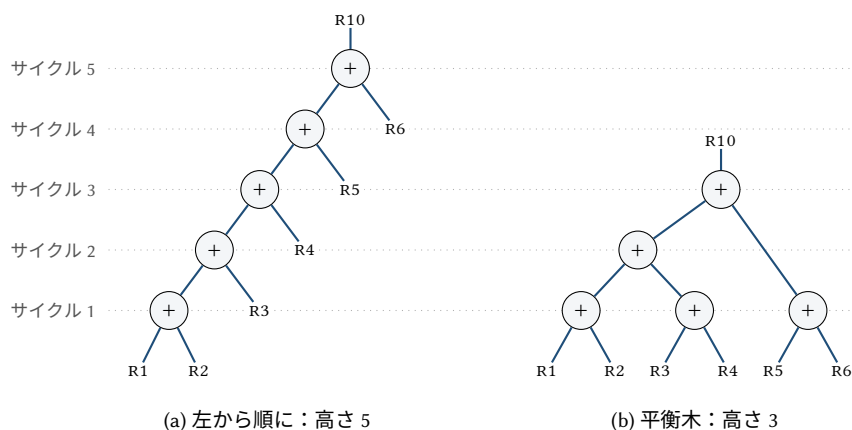


図 10.1: R1 から R6 までの和の式木。(a) 左から順に翻訳すると、5 つの加算の連鎖になる。(b) 例 10.2 の平衡木は高さ 3 である。加算の各段は理想プロセッサの 1 サイクルで実行される。

1 つの結合的な演算で結ばれた n 個のオペランドについて、左から順に翻訳すると高さは $n-1$ になるが、平衡木の高さは $\lceil \log_2 n \rceil$ である。加算と乗算は結合的であり、AND, OR, XOR も同様である。減算は $(a-b)-c \neq a-(b-c)$ なので結合的ではないが、符号を調整すれば減算を含む式も再グループ化できる。 $a-b = a+(-b)$ であるから、各項が自身の符号を保つ限り項は自由にグループ化でき、マイナス符号に続くグループの中の符号は反転する。例えば $a-b-c-d = (a-b)-(c+d)$ となり、高さが 3 から 2 に減る。

出典 Lesson 10, 動画 3-4 ; H&P 付録 H.

▷ tree height reduction

注意 浮動小数点の加算は (丸めのため) 結合的ではない。コンパイラがこれを再グループ化するのは、許可された場合だけである。

要点

木の高さ削減は、命令間の依存を変えるのであって命令数を変えるのではない。ILP を制限する連鎖を短くするので、1 サイクルに複数の命令を実行できるプロセッサで効果がある。

10.3 命令スケジューリング

プロセッサが独立な命令を同じサイクルに実行するのは、それらを見つけられた場合だけである。コンパイラは通常プログラムを文ごとに翻訳するので、ある文の命令の後に次の文の命令が続く。各文が 1 つの連鎖をなす場合、異なる文に属する独立な命令は連鎖の残りによって隔てられる。ウィンドウが両方の連鎖を覆うアウトオブオーダープロセッサならそれでも見つけられるが、ウィンドウがより短いアウトオブオーダープロセッサは見つけられず、待たなければならない最初の命令までしか後続を実行しないインオーダープロセッサも同様である。独立な命令を見つけやすくするコンパイラ技法が 3 つある。命令スケジューリング、ループ展開 (10.5 節)、トレーススケジューリング (10.7 節) である。

定義 10.3 (命令スケジューリング) プログラムの結果を変えずに命令を並べ替え、同じサイクルに実行できる命令がプログラム順で隣り合うようにすることを **命令スケジューリング** という。

インオーダープロセッサでは、スケジューリングは、そのままではプロセッサが待つことになるサイクルを、待っている命令に依存しない命令で埋める。1 サイクルに最大 w 命令を実行するプロセッサ向けにスケジューリングするには、コンパイラは各命令について理想プロセッサがそれを実行するサイクルを求め (式 (6.2)), そのサイクルの順に、高々 w 命令ずつのグループにして命令を並べればよい。このとき各命令は、自分が読む結果を生成する命令より後に置かれたままであるが、それに加えて、並べ替えたプログラムは元のプログラムと同じ結果を計算しなければならない。WAW 依存や WAR 依存を破る移動は、ある命令が使う値を変えてしまう。そこでコンパイラは、元の順序を保つか、あるいは次のように補正する。

宛先レジスタ 移動する命令が、後の命令がまだ読む古い値を持つレジスタを上書きしてしまう場合、その命令は代わりに、使われていない別のレジスタに結果を書き、その結果を読む命令もそれに合わせて変更する。

アドレスのオフセット 添字の増分のようにレジスタに定数を加える命令が、そのレジスタをアドレスのベースとして使う命令より前に移動する場合、アドレスの変位をその定数だけ減らす。SW R3, 0(R1) は ADDI R1, R1, 4 の後に置かれると SW R3, -4(R1) になる。

例 10.4. a から h を R1 から R8 に保持するとき、文 $x = (a+b) \times c - d$ と $y = (e-f) \times g + h$ は次のように翻訳される。

```
I1:  ADD R9, R1, R2      ; a + b
I2:  MUL R9, R9, R3      ; (a + b) * c
I3:  SUB R9, R9, R4      ; x <- (a + b) * c - d
I4:  SUB R10, R5, R6     ; e - f
I5:  MUL R10, R10, R7    ; (e - f) * g
I6:  ADD R10, R10, R8    ; y <- (e - f) * g + h
```

これらの命令は I1-I3 と I4-I6 の 2 つの連鎖をなすので、理想プロセッサはサイクル 1 に I1 と I4 を、サイクル 2 に I2 と I5 を、サイクル 3 に I3 と I6 を実行する。ILP は 2 である。幅 2 のインオーダープロセッサは、I2 が I1 を待たなければならないのでサイクル 1 には I1 だけを、サイクル 2 には I2 だけを実行する。サイクル 3 では I3 を I4 と一緒に実行するが、その後は I5 と I6 が再び 1 サイクルずつ必要になる。合計 5 サイクル、IPC は $6/5 = 1.2$ である (図 10.2a)。I1, I4, I2, I5, I3, I6 の順にスケジューリングすると、隣り合う命令の組はすべて独立になり、同じプロセッサが 3 サイクルで済む。IPC は 2 であり、ILP に等しい (図 10.2b)。2 つの文は異なるレジスタを使うので、この並べ替えは WAW 依存も WAR 依存も破らない。

出典 Lesson 10, 動画 5-7 ; H&P §3.2.

▷ instruction scheduling

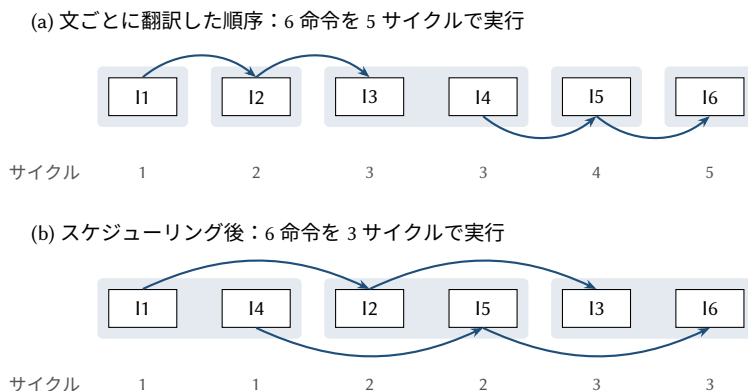


図 10.2: 幅 2 のインオーダープロセッサにおける例 10.4 のプログラム。弧は RAW 依存であり、網掛けのグループは同じサイクルに実行される。(a) 文の順序のままでは、連鎖 I1-I3 が I4 を足止める。(b) スケジューリング後は、両方の連鎖が毎サイクル進む。

例 10.5. 次のループは

```
for (i = 0; i < n; i++)
    a[i] = a[i] + x;
```

整数配列の n 個の要素それぞれに x を加える。R1 が現在の要素を指し、R2 が x を保持し、R5 が最後の要素の次のアドレスを保持するとき、次のように翻訳される。

```
Loop: LW    R3, 0(R1)      ; R3 <- a[i]
      ADD   R3, R3, R2      ; R3 <- a[i] + x
      SW    R3, 0(R1)      ; a[i] <- R3
      ADDI  R1, R1, 4       ; 次の要素へ
      BNE   R1, R5, Loop   ; 末尾まで繰り返す
```

完全な分岐予測を持つ幅 4 のインオーダープロセッサは、ADD が LW を待つのでまず LW だけを、次に SW が ADD を待つので ADD だけを実行する。SW と ADDI は一緒に実行され、ADDI を待つ BNE は次の反復の LW と一緒に実行される。1 反復は 3 サイクルかかり、CPI は $3/5 = 0.6$ である。ADDI を LW の直後に繰り返上げると、ADDI はもはや連鎖の後ろで待たされず、増分の後に置かれることになった SW は変位 -4 を必要とする。

```
Loop: LW    R3, 0(R1)
      ADDI  R1, R1, 4
      ADD   R3, R3, R2
      SW    R3, -4(R1)      ; R1 はすでに進んでいる
      BNE   R1, R5, Loop
```

これは理想サイクルの順序であり、分岐だけは反復の末尾に留めてある。LW と ADDI が一緒に実行され、ADD が続き、SW と BNE、そして次の反復の LW と ADDI が次のサイクルの 4 つの枠をすべて埋める。1 反復は 2 サイクルとなり、CPI は $2/5 = 0.4$ である。

このようにスケジューリングが効くのは、インオーダープロセッサ、プログラム中の連鎖に比べてウィンドウが短いアウトオーダープロセッサ、そして幅より多くの命令が準備できているときのアウトオーダープロセッサである。どの命令が先に実行されるかを順序が決めるからである (第 6 章)。

10.4 スケジューリングと if 変換

命令スケジューリングが命令を動かせるのは、全体がひとまとまりに実行されるコードの範囲の中だけであり、条件分岐はその範囲を終わらせる。分岐に続く命令は、その一方の経路でしか実行されない。それを分岐の前に移動すれば、もう一方の経路でも実行されてしまう。また 2 つの経路の命令は、どちらか一方の経

出典 Lesson 10, 動画 8-9; H&P §3.2.

路しか実行されないのでは、同じグループに置くことはできない。したがってコンパイラは、分岐の前後の命令が独立であっても、分岐をまたいで命令をグループ化できない。

小さな if-then-else に対しては、if 変換（第 5 章）がこの境界を取り除く。変換後のコードは、条件と両方の部分が無条件に計算し、条件付きムーブで結果を選択する。全体が分岐のない 1 つの列になっており、コンパイラはそれを他のコードと同じようにスケジューリングできる。条件と 2 つの部分の計算する命令は選択に依存しないので、それら自身の依存に従って先頭のグループに置かれ、選択がその後に続く。例 5.4 の if 変換後のコードは、すでにこの形をしている。最初の 4 つの命令は互いに独立であり、3 つの条件付きムーブはそれらにしか依存しないので、幅 4 のインオーダープロセッサは 7 つの命令すべてを 2 サイクルで実行する。

10.4.1 ループ

ループは各反復の末尾に先頭へ戻る分岐を持つが、この分岐を if 変換しても役には立たない。これを除去するということは、ループが取りうる反復回数と同じだけ本体を実行するということであり、各コピーには、その反復が実行されるべきかどうかを示す述語がそれぞれ必要になる。これらの述語を計算し適用する命令のコストは、スケジューリングで得られる利得を上回る。さらに、ループの分岐は正確に予測され、これにプレディケーションを適用すると、ループの後のコードを毎反復実行することになる（表 5.1）。したがってループの分岐は残り、命令スケジューリングを 1 回の反復の中に閉じ込める。幅 4 のインオーダープロセッサで 1 反復 2 サイクルとなる例 10.5 のスケジューリング後の本体は、そのプロセッサにとって最良の順序である。本体が短い場合、ループの独立な命令の大半は異なる反復にあり、そこではコンパイラはそれらを近づけられない。ループ展開がこの制限に対処する。

10.5 ループ展開

本体の短いループでは、実行される命令の大きな割合がループのオーバーヘッドであり、本来の処理はループの分岐で隔てられた多数の反復に分散する。例 10.5 のループでは、5 つの命令のうち 3 つが本来の処理を行い、ADDI と BNE はオーバーヘッドである。

定義 10.6 (ループ展開) 本体を k 回繰り返して添字を調整し、添字を k ずつ進めることによって、新しいループの 1 反復が元のループの k 反復分の処理を行うようにループを書き換えることを**ループ展開**という。この k を**展開係数**という。1 回展開すれば $k = 2$ 、2 回展開すれば $k = 3$ である。

n が偶数のとき、 $k = 2$ で展開するとループは

```
for (i = 0; i < n; i += 2) {
    a[i]    = a[i]    + x;
    a[i+1] = a[i+1] + x;
}
```

となり、次のように翻訳される。

```
Loop: LW    R3, 0(R1)      ; a[i] <- a[i] + x
      ADD   R3, R3, R2
      SW    R3, 0(R1)
      LW    R3, 4(R1)     ; a[i+1] <- a[i+1] + x
      ADD   R3, R3, R2
      SW    R3, 4(R1)
      ADDI  R1, R1, 8      ; 2 要素先へ
      BNE   R1, R5, Loop
```

各反復は 8 つの命令で 2 要素を処理し、そのうち添字の増分と分岐は 1 回ずつしか実行されない。本体の 2 つのコピーは隣接していて間に分岐がないので、2 つ目のコピーが中間結果を自分専用のレジスタに保持するようにすれば、コンパイラは両者をまとめてスケジューリングできる（10.5.2 節）。

出典 Lesson 10, 動画 10-15 ; H&P §3.2.

▷ loop unrolling

n が k の倍数でない場合、コンパイラは残りの要素のためのコード—通常は元のループのコピー—を追加する。

10.5.1 命令数と ILP

展開はクロック周期には影響しないが、性能の鉄則（式 (2.3)）の他の 2 つの因子を変える。元のループは 1 要素当たり 5 命令を実行し、展開後のループは 2 要素当たり 8 命令、すなわち 1 要素当たり 4 命令を実行する。オーバーヘッドの命令が実行される回数は半分になり、命令数は 20% 減る。係数 k では、ループは 1 要素当たり $(3k+2)/k$ 命令を実行し、これは本来の処理だけの 3 に近づく。

ILP も上がる。理想プロセッサ（式 (6.2)）では、連続する反復の ADDI が 1 反復 1 サイクルの連鎖をなす。各反復の LW は前の反復の ADDI から R1 を読み、自分の反復の ADDI と同じサイクルに実行され、その ADD と SW は続く 2 サイクルに実行される。展開後の本体における R3 への 2 回目の書き込みは WAW 依存であり、リネーミングがこれを除去する。したがって理想プロセッサは 1 サイクルに 1 反復、すなわち k 要素を実行し、長いループの ILP は 1 反復当たりの命令数に近づく。元のループでは 5、展開後のループでは 8 である。これは、反復が互いに添字を通じてしか依存しないことを前提としている。 $s = s + a[i]$ のように累算するループでは、 s への加算がどちらの版でも 1 要素 1 サイクルの連鎖をなす。

10.5.2 命令当たりサイクル数

CPI も下がるかどうかはプロセッサに依存する。表 10.2 は、例 10.5 の幅 4 のインオーダープロセッサと理想プロセッサ（どちらも分岐予測は完全とする）について、ループの 4 つの版を比較したものである。最初の 2 つの版は例 10.5 のものである。展開しただけでスケジューリングしていない版では、ループは依然として LW-ADD-SW の連鎖を順に実行する。2 つ目の LW が 1 つ目の SW と、ADDI が 2 つ目の SW と、BNE が次の反復の LW と一緒に実行されるので、8 命令からなる 1 反復は 5 サイクルかかる。CPI は 0.6 から $5/8 = 0.625$ に上がるが、1 要素当たりの時間は命令数とともに 3 サイクルから 2.5 サイクルに減る。理想サイクルの順にスケジューリングすると、展開後の本体は次のようになる。

```

Loop: LW    R3, 0(R1)
      LW    R4, 4(R1)      ; 2 つ目のコピーは R4 を使う
      ADDI  R1, R1, 8
      ADD   R3, R3, R2
      ADD   R4, R4, R2
      SW    R3, -8(R1)     ; R1 はすでに進んでいる
      SW    R4, -4(R1)
      BNE   R1, R5, Loop

```

10.3 節の補正が両方とも必要になる。2 つ目の LW は、1 つ目のコピーの ADD と SW より前に来るが、これらはまだ R3 を必要とするので、2 つ目のコピーは自分の値を R4 で計算する。また両方のストアが増分の後に来るので、その変位を 8 だけ減らす。プロセッサは最初のサイクルに 2 つの LW と ADDI を、2 サイクル目に 2 つの ADD を実行し、3 サイクル目を 2 つの SW、BNE、そして次の反復の最初の LW が埋める。それ以降はグループが反復と一致しなくなり、定常状態では 16 命令、すなわち 2 反復が 5 サイクルかかる。1 要素当たり 1.25 サイクル、CPI は $5/16 = 0.3125$ である。スケジューリングした元のループと比べると、1 要素当たりの時間は 2 サイクルから 1.25 サイクルに減り、高速化率は 1.6 となる。そのうち $5/4 = 1.25$ が命令数に、 $0.4/0.3125 = 1.28$ が CPI に由来する。

表 10.2: $k = 2$ で展開する前と後の 例 10.5 のループを、コンパイルしたままの版とスケジューリングした版について比較する。1 要素当たりの命令数と、分岐予測が完全な 2 つのプロセッサにおける 1 要素当たりのサイクル数および CPI を示す。

版	命令数	幅 4 のインオーダー		理想	
		サイクル	CPI	サイクル	CPI
元のループ	5	3.00	0.600	1.00	0.200
スケジューリング後	5	2.00	0.400	1.00	0.200
展開後	4	2.50	0.625	0.50	0.125
展開・スケジューリング後	4	1.25	0.3125	0.50	0.125

ループの分岐の予測が外れるのは、実行 1 回につきおよそ 1 回、ループを抜けるときである。端数処理のループがあっても高々もう 1 回増えるだけである。

このようにインオーダープロセッサでは、展開はそれが可能にするスケジューリングを通じて CPI を下げる。ループの分岐が 2 つの反復の命令を隔てなくなり、コンパイラが両者を交互に並べられるからである。理想プロセッサでは命令の順序は問題にならない。そこで CPI が下がるのは ILP が 5 から 8 に上がるからであり、命令数の減少と合わせて 1 要素当たりの時間は半分になる。大きなウィンドウを持つアウトオブオーダープロセッサも、自身の幅までは同じように利益を得る。1 サイクルに 1 命令を完了するプロセッサは、どの版でも CPI が 1 であり、命令数による 1.25 倍しか得られない。したがってループ展開は、どのプロセッサでも命令数を下げ、1 サイクルに複数の命令を実行するプロセッサでは CPI も下げる。

10.5.3 展開の欠点

本体は k に比例して大きくなり、5 命令から $3k + 2$ 命令になるが、利得は頭打ちになる。1 要素当たりの命令数は 3 に近づき、実在のプロセッサは ILP がどれほど高くても IPC を自身の幅より上げられない。 $k = 4$ (14 命令, 1 要素当たり 3.5 命令) から $k = 8$ (26 命令, 1 要素当たり 3.25 命令) にすると、本体はほぼ 2 倍になるのに、実行される命令は 7% しか減らない。これは第 2 章の収穫逡減の一例である。

定義 10.7 (コード肥大化). ループ展開のような変換によってプログラムのサイズが増大することを**コード肥大化**という。

コードが大きくなればより多くのメモリを占め、キャッシュの章で見るように、その命令をフェッチする際のキャッシュミスが増えて、利得を打ち消しうる。さらに 2 つの問題が反復回数に関わる。 n が k の倍数でない場合、残りの反復のために追加のコードが必要になる。また展開は、展開後のループの 1 反復が始まる時点で残りの反復回数が分かっていることを前提とする。値が 0 である最初の要素で止まる探索のように、脱出が本体の中で決まるループでは、単純に 1 反復当たり本体の k 個のコピーを実行することはできない。後ろのコピーが、元のループなら実行しない反復を行ってしまう可能性があるからであり、追加の脱出判定が必要になる。そのためコンパイラは、小さな係数で、しかも本体の短いループだけを展開する。

▷ code bloat

要点

ループ展開は、添字の増分とループの分岐を元のループの k 反復につき 1 回だけ実行するので、どのプロセッサでも命令数を下げる。さらに、1 サイクルに複数の命令を実行するプロセッサでは CPI も下げる。インオーダープロセッサではコンパイラが複数の反復の命令をまとめてスケジューリングできるようになるからであり、並列性を自分で見つけるプロセッサでは、反復が独立なループの ILP が上がるからである。その代償は、 k とともに増大するコードと、反復回数が k の倍数でない場合や事前に分からない場合の追加のコードである。

10.6 インライン展開

分岐が if-then-else の 2 つの経路を隔てるのと同じように、呼び出しは呼び出し側の命令と呼び出される関数の命令とを隔てる。さらに、呼び出しのたびに、関数本来の処理を何も行わない命令が実行される。呼び出しとリターンそのもの、呼び出し規約で定められたレジスタへの引数のコピーとそこからの結果のコピー、そしてしばしばスタックへのレジスタの退避と復帰である。

定義 10.8 (インライン展開). 呼び出しを、呼び出される関数の本体を呼び出し側のオペランドに合わせたコピーで置き換えることを**インライン展開**という。

例 10.9. 次の関数を

```
int sqdiff(int x, int y) { int d = x - y; return d * d; }
```

出典 Lesson 10, 動画 16-18 ; H&P 付録 A.

▷ function call inlining

$c = \text{sqdiff}(a, b)$ として呼び出す。 a, b, c は R8, R9, R10 にある。呼び出し規約は引数を R4 と R5 で渡し、結果を R2 で返す。

```

    ADD R4, R8, R0      ; x <- a
    ADD R5, R9, R0      ; y <- b
    CALL SqDiff
    ADD R10, R2, R0      ; c <- 結果
    ...
SqDiff: SUB R2, R4, R5    ; d <- x - y
        MUL R2, R2, R2    ; 結果 <- d * d
        RET

```

インライン展開すると、呼び出しは次のようになる。

```

    SUB R10, R8, R9      ; c <- a - b
    MUL R10, R10, R10    ; c <- c * c

```

呼び出しは 1 回につき 7 命令を実行するのに対し、インライン展開後のコードは 2 命令である。引数と結果のコピーは、 a と b から c への依存の連鎖も長くする。理想プロセッサでは、呼び出しは 4 サイクル、インライン展開後のコードは 2 サイクルかかる (図 10.3)。

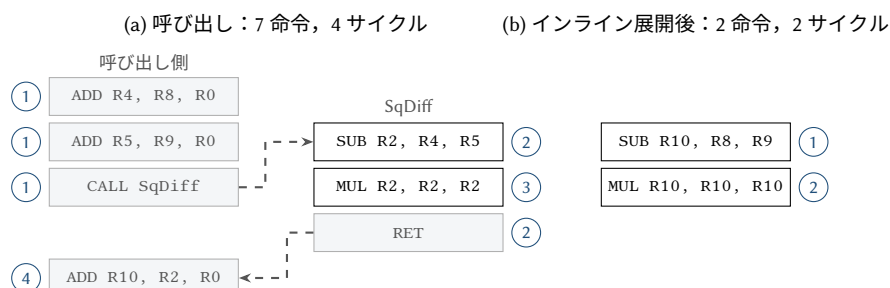


図 10.3: インライン展開の前と後の例 10.9。網掛けの命令は呼び出しを実現するだけの命令であり、破線の矢印は制御の移動を、丸で囲んだ数字は理想プロセッサが各命令を実行するサイクルを表す。

このようにインライン展開は、呼び出し、リターン、引数と結果のコピーがなくなるので命令数を下げ、本体が呼び出し側のオペランドを直接読み、結果を呼び出し側が必要とする場所に直接届けるので依存の連鎖を短くする。さらに、本体は呼び出し側のコードの一部になる。コンパイラはその命令を呼び出し側の命令とまとめてスケジューリングでき、かつての呼び出しをまたいで木の高さ削減を適用でき、プロセッサは呼び出しとリターンを予測する必要がなくなる (第 4 章)。

10.6.1 インライン展開によるコード肥大化

インライン展開は本体のコピーを呼び出し箇所ごとに別々に置くので、多数の箇所から呼ばれる大きな関数ではコードが相当に増える。

例 10.10. 本体が 20 命令とリターンからなる関数が 10 箇所から呼ばれ、各箇所が呼び出しに 4 命令 (引数のコピー 2 つ、呼び出し、結果のコピー 1 つ) を費やすとする。インライン展開しなければ、コードは $10 \times 4 + 21 = 61$ 命令を占め、呼び出し 1 回につき $4 + 21 = 25$ 命令が実行される。すべての箇所インライン展開すると、関数と呼び出しの列は消えるが、各箇所が本体の 20 命令を受け取る。合計 200 命令、コードは 3 倍以上になるのに、節約できるのは呼び出し 1 回当たり 25 命令のうち 5 命令 (20%) である。本体が 2 命令の例 10.9 の関数なら、同じ 10 箇所でも、インライン展開しない場合は $10 \times 4 + 3 = 43$ 命令、展開した場合は $10 \times 2 = 20$ 命令しか必要としない。コードは小さくなり、呼び出し 1 回につき実行される命令は 7 から 2 になる。

ループ展開と同様に、コードが大きくなるとキャッシュミスが増え、利得を打ち消しうる。そのためコンパイラがインライン展開するのは、本体に比べて呼び

出しのオーバーヘッドが大きい小さな関数と、呼び出し箇所が 1 つだけでそのコピーが元の関数を置き換える関数である。多数の箇所から呼ばれる大きな関数は、呼び出しのまま残る。

要点

インライン展開は、呼び出し、リターン、引数と結果の受け渡しを除去して命令数を下げ、依存の連鎖を短くし、さらにコンパイラが本体を呼び出し側とまとめて最適化できるようにする。小さな関数では効果があるが、多数の箇所から呼ばれる大きな関数では、コード肥大化が利得を上回る。

10.7 その他のコンパイラ技法

この章の技法は、IPC を高めるコンパイラ変換という、より大きな体系の例である。命令スケジューリングを、それを閉じ込めている境界の向こうまで押し広げる技法がさらに 2 つあるが、ここでは概略を述べるにとどめる。**ソフトウェアパイプラインニング**では、コンパイラはループをパイプラインのように扱う。反復が重なるように連続する反復の命令をスケジューリングし、各サイクルが複数の反復の独立な命令を組み合わせつつ、結果は元のループのものと同じに保たれる [17]。もう 1 つの**トレーススケジューリング**は、if 変換をさらに推し進める。コンパイラはプロファイルを用いて、複数の分岐を通る最も頻繁に実行される経路を選び、この経路を 1 つのコードブロックとしてスケジューリングし、実行がその経路から外れたときに結果を訂正する補償コードへ分岐する判定を追加する [18]。コンパイラによるスケジューリングを突き詰めると、操作を並列なステップにグループ化することを完全にコンパイラに委ねるプロセッサ、すなわち第 11 章の VLIW プロセッサに行き着く。

要点 – 章のまとめ

IPC は、プログラムの ILP と、プロセッサがウィンドウの中で見つけられる独立な命令とによって制限され、コンパイラはそのどちらにも影響する。木の高さ削減は式を再グループ化してその命令がより短い連鎖をなすようにし、命令数を変えずに ILP を上げる。命令スケジューリングは独立な命令を隣り合わせにし、命令が互いを追い越す箇所では宛先レジスタとアドレスのオフセットを変更する。これはインオーダープロセッサやウィンドウの小さいプロセッサで効く。分岐はこれを閉じ込めるが、if 変換は小さな if-then-else から分岐を除去する。ループ展開は、添字の増分とループの分岐を k 反復につき 1 回だけ実行して命令数を下げる。コンパイラが複数の反復の命令をまとめてスケジューリングできるようになると、反復が独立なループの ILP が上がることから、CPI も下げる。インライン展開は、呼び出し、リターン、引数のコピーを除去する。どちらもコード肥大化を招くので、コンパイラは短いループと小さな関数に限ってこれらを用いる。ソフトウェアパイプラインニングとトレーススケジューリングは、スケジューリングをループの反復をまたいで、また分岐をまたいで拡張する。

出典 Lesson 10, 動画 19–20 ; [17] ; [18].

▷ software pipelining

▷ trace scheduling

VLIW

講義動画	Lesson 11, 動画 1-9
教科書	H&P [1] §3.7, 付録 H
キーワード	VLIW, アウトオブオーダー・スーパースカラ, インオーダー・スーパースカラ, NOP, 後方互換性, 不規則なコード, コードの肥大化, 完全プレディケーション, 分岐ヒント, ストップビット, Itanium, EPIC, DSP

第 10 章では、コンパイラがプログラムを書き換えることで、プロセッサが 1 サイクルに完了する命令の数を増やせることを示した。コンパイラは依存の連鎖を短くし、独立な命令を隣り合わせに配置し、分岐と呼び出しを除去する。VLIW (very long instruction word) プロセッサは、この考え方を突き詰めたものである。どの操作を並列に実行するかをコンパイラだけが決め、その決定を命令に符号化するので、ハードウェアには第 7~9 章のアウトオブオーダープロセッサを持つ依存追跡の機構が一切必要ない。この章では、VLIW をスーパースカラの設計と比較し、その利点と欠点を検討し、この手法が失敗した分野と成功した分野を示す。

11.1 スーパースカラプロセッサと VLIW プロセッサ

スーパースカラプロセッサ (第 8 章) は、1 サイクルに最大 N 個の命令を実行する。設計による違いは、一緒に実行できる N 個の命令を見つけるためにいくつかの命令を調べるか、そしてその探索のどれだけをコンパイラに任せるかにある (図 11.1, 表 11.1)。アウトオブオーダーの設計は、リネーミング、ウィンドウ内のスケジューリング、リオーダーバッファのための複雑なハードウェアと引き換えに、どのようなプログラムの並列性も自力で見つける (第 6~8 章)。インオーダーの設計は、他の命令を待たなければならない最初の命令で止まるので、独立な命令を隣り合わせに配置するコンパイラの助けを必要とする (第 10 章)。VLIW プロセッサは、コンパイラが互いに独立にした最大 N 個の操作からなる 1 つの命令を、何の検査もせずに実行する。

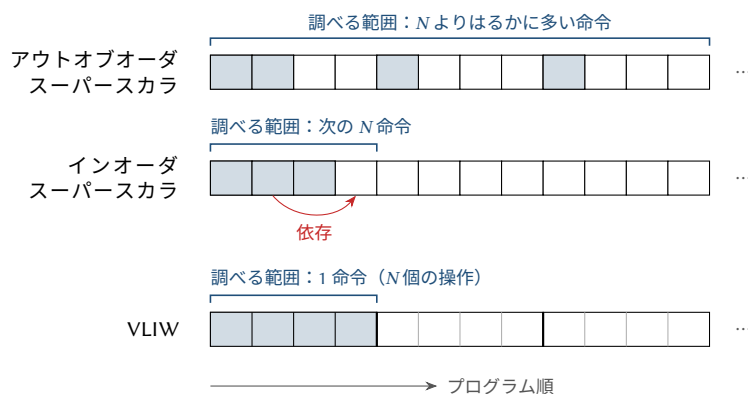


図 11.1: 1 サイクルを最大 $N = 4$ 個の操作で埋めるために、各設計が調べる範囲。網掛けの箱がこのサイクルで実行される。インオーダーの設計は、グループ内の先行する命令に依存する最初の命令で止まる。

定義 11.1 (VLIW). VLIW (very long instruction word) アーキテクチャとは、各命令が最大 N 個の操作からなり、それらの操作が同じサイクルで並列に実行される命令セットアーキテクチャである。各操作は通常の ISA の 1 命令に相当する。1 つの命令を構成する操作を選び、それらが独立であることを保証するのはコンパイラの役割であり、プロセッサは依存の検査を一切行わない。

N 個の操作を収める命令は通常の命令のおよそ N 倍の長さになり、これがこ

出典 Lesson 11, 動画 1-3 ; H&P §3.7 ; [19].

▷ very long instruction word

のアーキテクチャの名前の由来である。1 命令が最大 N 個の通常の命令の仕事をするので、1 サイクルに 1 命令を完了する VLIW プロセッサは、IPC が最大 N のスーパースカラプロセッサに相当する（第 6 章）。

表 11.1: 1 サイクルに最大 N 個の操作を実行するスーパースカラプロセッサと VLIW プロセッサ。

	アウトオブオーダー	インオーダー	VLIW
サイクル当たりの実行調べる範囲	$\leq N$ 命令 $\gg N$ 命令	$\leq N$ 命令 次の N 命令	1 命令 ($\leq N$ 操作) 1 命令
ハードウェア	複雑	より単純	最も単純
コンパイラ	助けになりうる	助けが必須	すべてを担う

例 11.2 (命令への操作の詰め込み) 次の断片は配列の 4 要素を加算し、その和をストアする。コメントは RAW 依存（第 3 章）を示す。4 つのロードは R8 を読むだけであり、互いに独立である。

```

I1: LW  R1, 0(R8)
I2: LW  R2, 4(R8)
I3: LW  R3, 8(R8)
I4: LW  R4, 12(R8)
I5: ADD R5, R1, R2 ; I1, I2 と RAW
I6: ADD R6, R3, R4 ; I3, I4 と RAW
I7: ADD R7, R5, R6 ; I5, I6 と RAW
I8: SW  R7, 16(R8) ; I7 と RAW

```

VLIW コンパイラは操作をプログラム順に取り出し、それぞれを、依存するすべての操作より後にあり、かつ空きスロットが残っている最も早い命令に配置する。余ったスロットは NOP で埋める（表 11.2）。通常の ISA なら 8 命令が必要なところ、スロットが 4 つなら 8 個の操作は 4 命令、すなわち 4 サイクルに収まり、スロットが 2 つなら 5 命令を占める。

表 11.2: 例 11.2 の断片を、操作スロットが 4 つの VLIW ISA と 2 つの VLIW ISA に向けてコンパイルした結果。各行が 1 命令である。サイズは 32 ビットのスロットを仮定している。

命令	4 スロット	2 スロット
1	I1, I2, I3, I4	I1, I2
2	I5, I6, NOP, NOP	I3, I4
3	I7, NOP, NOP, NOP	I5, I6
4	I8, NOP, NOP, NOP	I7, NOP
5	—	I8, NOP
NOP スロット	16 個中 8 個	10 個中 2 個
コードサイズ	512 ビット	320 ビット

11.2 利点と欠点

並列性の探索をコンパイラに任せることには 3 つの利点がある。

- **コンパイラには時間が十分にある。** アウトオブオーダープロセッサは、毎クロックサイクルのごく一部の時間で、しかも自身のウィンドウの中だけで独立な命令を見つけなければならない。コンパイラは一度だけ実行され、プログラム全体を解析できる。
- **ハードウェアが単純になる。** リネーミング、ウィンドウ内のスケジューリング、リオーダーバッファがないので、VLIW プロセッサは同じサイクル当たり操作数を実現するのに必要な面積と電力が少なくて済み、エネルギー効率を高くできる。
- **ループと規則的なコードに適する。** 配列をループで処理する場合、依存は少なく、コンパイル時に把握できる。ループ展開（第 10 章）が反復間の分

注意 VLIW プログラムの命令数と CPI は、通常の ISA のものとは比較できない。サイクル当たりの操作数で比較すること。

出典 Lesson 11, 動画 4-5 ; H&P §3.7, 付録 H.

岐を除去するので、独立な操作がスロットを埋める。
一方で、欠点は 4 つある。

- ・ **レイテンシがコンパイル時に分からない。** コンパイラは、各操作がそのサイクル内に完了するものとして操作をグループ化する。ある操作が 1 サイクルで終わらないと—例えばキャッシュミス（第 12 章）を起こしたロード—後続のすべての操作が、独立なものまで含めてそれを待つ。一方、アウトオブオーダープロセッサはそれに依存しない命令の実行を続ける。
- ・ **不規則なコードは難しい。** コンパイラが見るのはプログラムであって、その実行ではない。2 つの操作が独立であることを証明できないとき—同じ位置を指しうるポインタを介したストアとロードや、データに依存する分岐の両側にある操作—コンパイラはそれらを別々の命令に置かなければならない。一方、アウトオブオーダープロセッサは実行時に実際のアドレスを知り、分岐を予測する（第 4 章と第 9 章）。
- ・ **コードの肥大化。** 独立な操作が N 個未満しかないたびに NOP がスロットを埋め、プログラムが大きくなる。表 11.2 の 4 スロット版は半分が NOP で 512 ビットを占めるが、通常の 32 ビット命令 8 個なら 256 ビットで済む。命令の圧縮（11.3 節）はこのオーバーヘッドを減らす。
- ・ **新しいプロセッサには再コンパイルしたプログラムが必要である。** 操作のグループ化とスロット数はコンパイル時に固定されるので、後方互換性（後述）が損なわれる。

定義 11.3 (後方互換性). プロセッサが**後方互換性**を持つとは、同じファミリの以前のプロセッサ向けにコンパイルされたプログラムを、変更せずに実行できることである。

▷ backward compatibility

スーパースカラプロセッサでは、後方互換性に性能向上が伴う。幅の広いアウトオブオーダープロセッサは、古いバイナリからも再コンパイルなしでより多くの並列性を見つける。スロット数を増やした新しい VLIW プロセッサは、命令形式が変わったために古い VLIW バイナリをまったく実行できないか、実行できても、そのバイナリのコンパイル対象だったプロセッサよりサイクル当たりの操作数が増えない。

例 11.4 (幅の広い VLIW プロセッサ上の古いバイナリ). 2 スロット形式も受け付ける 4 スロットのプロセッサは、例 11.2 の 2 スロット版を、古いプロセッサと同じく 5 サイクルで実行する。4 サイクルで済むのは再コンパイルした 4 スロット版だけである。8 個の通常の命令は、同じ数え方をする、再コンパイルなしで、幅 2 のアウトオブオーダープロセッサでは 5 サイクル、幅 4 のものでは 4 サイクルかかる。

要点

VLIW は、ハードウェアの複雑さをコンパイラへの依存と引き換えにする。コードが規則的で、まさにそのプロセッサ向けにコンパイルされている場合には見合うが、並列性や操作のレイテンシが実行時にしか分からない場合や、古いバイナリを新しいプロセッサでより速く実行しなければならない場合には不利になる。

11.3 VLIW 命令

VLIW 命令は N 個の操作スロットの並びである（図 11.2）。各スロットは通常の ISA の命令と同じように符号化され、通常の命令コード全体から選ばれた固有の命令コードと、固有のオペランドを持つ。

1 つの命令に含まれる操作は独立でなければならない。どの操作も他の操作の結果を使ってはならず、2 つの操作が同じ位置に書き込んではいならない。ハードウェアはこれを検査しないので、正しくグループ化する責任はコンパイラにある。連続する命令の操作どうしは依存してよい。表 11.2 で I5 が I1 に依存しているのがその例である。

コンパイラは、例 11.2 のように命令スケジューリングによって並列性を引き出す。第 10 章の他の技法もこれを助ける。木の高さ削減は依存の連鎖を短くして、より多くの操作が 1 つの命令を共有できるようにし、ループ展開とインラ

出典 Lesson 11, 動画 6 ; H&P 付録 H.

注意 独立性が要求されるのは 1 つの命令の中だけであり、命令と命令の間では要求されない。

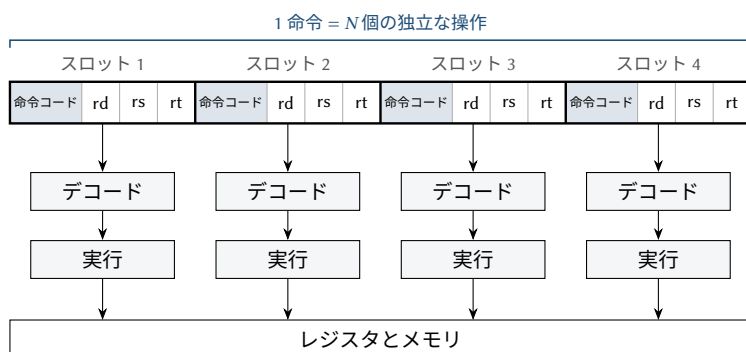


図 11.2: 4 つの操作スロットを持つ VLIW 命令. 各スロットは他のスロットと独立にデコードされ、実行される. スロット間の依存を検査するハードウェアはない.

イン展開は、操作を隔ててしまう分岐、呼び出し、リターンを除去する. VLIW プロセッサにはリネーミングのハードウェアがないので、偽の依存 (第 3 章) はコンパイラ自身が、他で使われていないレジスタに結果を書き込むことで除去する. そのためのレジスタを、命令セットは数多く用意している.

分岐の後の操作は分岐の一方の経路でしか実行されないの、分岐はグループ化を制限する. そのため VLIW の命令セットはプレディケーション (第 5 章)、理想的には完全プレディケーションを十分にサポートし、コンパイラが分岐を if 変換して両方の経路の操作をグループ化できるようにする. 残った分岐には**分岐ヒント**を付けることができ、コンパイラはこれによって、分岐がどちらに進むと予想しているかをプロセッサに伝える.

固定スロットは、独立な操作が N 個未満しかないたびに領域を無駄にする. 圧縮された命令は操作を NOP なしで順に並べて格納し、一緒に実行される各グループの最後の操作に**ストップビット**で印を付ける. 表 11.2 の 4 スロット版のプログラムでは I4, I6, I7, I8 がストップビットを持ち、操作当たり 33 ビットとすると、コードは 512 ビットではなく 264 ビットを占める.

11.4 実際の VLIW プロセッサ

VLIW の手法がどこで失敗し、どこで成功するかを、2 種類のプロセッサが示している.

11.4.1 Itanium

VLIW 風の命令を汎用コンピューティングに持ち込もうとした最も有名な試みは、Intel が IA-64 命令セットのために開発したプロセッサファミリ **Itanium** である. 設計者たちはこの手法を EPIC (explicitly parallel instruction computing) と呼んだ [20]. しかし、この命令セットには多くの追加機能が積み重なり、そのハードウェアが非常に複雑になったため、VLIW の主な利点が失われた. 不規則なコードからはやはりコンパイラがほとんど並列性を見つけられず、Itanium がアウトオブオーダープロセッサに取って代わることはなかった.

11.4.2 DSP

DSP (digital signal processor) とは、音声、映像、無線などのデジタル信号の処理に特化したプロセッサである. この市場は VLIW の長所 (11.2 節) を生かせる.

- 同じ計算を多数のサンプルに適用するループがコードの大半を占める. コンパイラが並列性を十分に見つけられる、規則的なコードである.
- プログラムは通常、それが動作する機器の特定の DSP 向けにコンパイルされるので、バイナリの後方互換性はほとんど問題にならない.
- DSP は電力の限られた機器に搭載されるので、エネルギー効率の高いハードウェアが効果を発揮する.

▷ branch hint

▷ stop bit

出典 Lesson 11, 動画 7-9; H&P §3.7, 付録 H.

▷ Itanium

IA-64 も同じようにストップを用いて、独立な命令のグループの終わりを示す (11.3 節).

▷ digital signal processor

H&P は VLIW の手法の例として TI の C6x 系 DSP を挙げている.

要点 – 章のまとめ

アウトオブオーダーのスーパースカラプロセッサは N よりはるかに多い命令を調べ、コンパイラの助けを必要としない。インオーダーのものは N 個の命令を調べ、その助けを必要とする。VLIW プロセッサは最大 N 個の独立な操作からなる 1 命令を調べ、全面的にコンパイラに頼る。VLIW のハードウェアは単純でエネルギー効率がよく、規則的なループのコードで良い性能を示す。しかし新しいプロセッサごとにバイナリを再コンパイルしなければならず、不規則なコードからはほとんど並列性が得られず、1 つのロードのキャッシュミスが後続のすべての操作を遅らせ、ストップビットで圧縮しない限り NOP がコードを肥大化させる。Itanium は汎用コンピューティングで失敗し、VLIW が適するのは DSP である。第 12 章では、サイクル当たり命令数からメモリへと話題を移す。

キャッシュの復習

講義動画	Lesson 12, 動画 1-45
教科書	H&P [1] §2.1, §B.1-B.2
キーワード	局所性の原理, 時間的局所性, 空間的局所性, キャッシュヒット, キャッシュミス, AMAT, ミスパナルティ, ブロックオフセット, タグ, 有効ビット, ダイレクトマップキャッシュ, セットアソシアティブキャッシュ, LRU, ダーティビット

第 3～11 章では、プロセッサが毎秒完了する命令の数を増やす手法を見てきた。しかし、どの命令もメモリからフェッチされなければならない、多くの命令はさらにデータをロードまたはストアする。そして第 1 章で見たように、メモリの速度はプロセッサの速度からますます引き離されていく（メモリウォール）。この章では、この差を埋める機構であるキャッシュを復習する。キャッシュは小容量で高速なメモリであり、プログラムは局所性をもってメモリにアクセスするので、大容量で低速な主記憶を待たずに大半のアクセスに応えられる。続いて、キャッシュの性能をどのように測るか、キャッシュをどのように構成し探索するか、ミスの際にどのブロックを追いつ出すか、書き込みをどのように扱うかを述べる。仮想記憶（第 13 章）と第 14 章の高度なキャッシュ技法は、この内容の上に直接成り立つ。

12.1 局所性の原理

キャッシュが機能するのは、プログラムのメモリアクセスがランダムとはほど遠いからである。

定義 12.1 (局所性の原理). プログラムが近い将来に行うことは、最近行ったことに近い可能性が高い、という観察を**局所性の原理**という。

メモリ参照において、「近い」には 2 つの意味がある。プログラムがアドレス X にアクセスしたところだとする。

定義 12.2 (時間的局所性と空間的局所性). 同じアドレス X に近いうちに再びアクセスする可能性が高いとき、プログラムは**時間的局所性**を示す。 X の近くのアドレスに近いうちにアクセスする可能性が高いとき、プログラムは**空間的局所性**を示す。

どちらの局所性もプログラムの構造から生じる。ループは同じ命令を繰り返し実行し（時間的局所性）、それらの命令は連続したアドレスからフェッチされる（空間的局所性）。反復のたびに更新される変数には同じアドレスで何度もアクセスし、順に処理される配列の要素は連続したアドレスに並んでいる。

例 12.3. 次のループは、アドレス $R4$ のバッファからアドレス $R5$ のバッファへ $R6$ バイトをコピーする。

```

Loop:  LB   R3, 0(R4)      ; R3 <- 転送元のバイト
        SB   R3, 0(R5)      ; 転送先のバイト <- R3
        ADDI R4, R4, 1      ; 次の転送元バイト
        ADDI R5, R5, 1      ; 次の転送先バイト
        ADDI R6, R6, -1     ; 残りのバイト数を 1 減らす
        BNE  R6, R0, Loop   ; 全バイトについて繰り返す

```

6 つの命令フェッチは両方の局所性をもつ。フェッチは連続したアドレスに向かい、毎回の反復で同じ 6 つのアドレスがフェッチされる。データアクセスは、各バッファを順にたどる 2 本の流れをなす。どのバイトもちょうど 1 回だけ読まれるか書かれるので、データには時間的局所性がない。それでもデータには空間的局所性がある。アクセスは遠く離れているかもしれない 2 つのバッファの間を行き来するが、各アクセスは同じバッファへの直前のアクセスに隣接している。空間的局所性が要求するのは最近アクセスしたアドレスの近くへのアクセスであ

出典 Lesson 12, 動画 1-6 ; H&P §2.1 ; [21].

▷ locality principle

▷ temporal locality

▷ spatial locality

り、最後にアクセスしたアドレスの近くである必要はない。

12.2 キャッシュ

局所性は、大容量のメモリへの高速なアクセスをプロセッサに与える方法を示唆する。フェッチ、ロード、ストアのたびに主記憶にアクセスするのは遅い。主記憶全体をプロセッサと同じ速さにすることもできない。高速なメモリはビット当たりのコストが高く、大容量のメモリほど本質的にアクセスが遅いからである。残る選択肢は、プロセッサ自身の中に少量の高速なメモリを置き、最近アクセスされたデータとその周辺のデータをそこにしておくことである。時間的局所性と空間的局所性により、大半のアクセスは目的のデータをそこで見つけれられる。

定義 12.4 (キャッシュ). **キャッシュ**とは、より大容量で低速なメモリのうち最近アクセスされた部分の写しを保持し、大半のアクセスをその写しで処理できるようにする、小容量で高速なメモリである。

プロセッサのすべてのメモリアクセスは、まずキャッシュ内でデータを探す(図 12.1)。データがあれば、そのアクセスは**キャッシュヒット**であり、データはキャッシュ内で読み書きされる。

そうでなければ、そのアクセスは**キャッシュミス**である。データを主記憶から取得してキャッシュにコピーし、そのデータへの以降のアクセスと、空間的局所性によりその近傍へのアクセスがヒットするようにする。キャッシュはすべてのアクセスで参照されるので、探索そのものが非常に高速でなければならない。これが、コストに加えてキャッシュを小さくしなければならない2つ目の理由である。小さなキャッシュにすべてを保持することはできないので、ミスは避けられない。

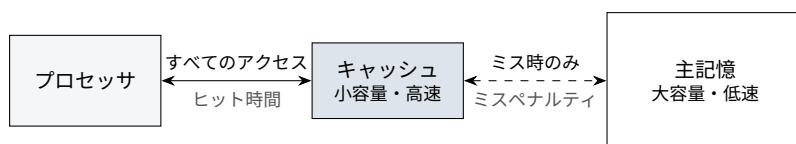


図 12.1: すべてのメモリアクセスはキャッシュに向かい、ミスだけが主記憶へ進む。

12.3 キャッシュの性能

キャッシュの効果は、メモリアクセス 1 回にかかる平均時間で測る。この時間は次の量で決まる。

定義 12.5 (ヒット率, ミス率). **ヒット率**はメモリアクセスのうちキャッシュヒットである割合、**ミス率**はミスである割合である。ヒット率 + ミス率 = 1 である。

定義 12.6 (ヒット時間, ミスペナルティ). **ヒット時間**とは、キャッシュを探索してその中のデータにアクセスするのに必要な時間である。**ミスペナルティ**とは、ミスが追加で要する時間、すなわち主記憶からデータを取り出してキャッシュに格納する時間である。

ミスを検出する前にキャッシュを探索しなければならないので、ミスもヒット時間を払う。したがってミスにはヒット時間とミスペナルティの和がかかる。2つの場合をその頻度で重み付けすると、**平均メモリアクセス時間 (AMAT)** が得られる。

$$\begin{aligned}
 \text{AMAT} &= (1 - \text{ミス率}) \times \text{ヒット時間} + \text{ミス率} \times (\text{ヒット時間} + \text{ミスペナルティ}) \\
 &= \text{ヒット時間} + \text{ミス率} \times \text{ミスペナルティ}.
 \end{aligned}
 \tag{12.1}$$

AMAT を小さくするには 3 つのことが必要である。

- ・ ヒット時間が短いこと。キャッシュは小さく単純でなければならない。
- ・ ミス率が低いこと。キャッシュは、プログラムがこれから使うデータを保持できるだけ十分に大きく、かつ巧みに構成されていなければならない。

出典 Lesson 12, 動画 7-9 ; H&P §B.1 ; [22].

▷ cache

▷ cache hit

▷ cache miss

出典 Lesson 12, 動画 10-12 ; H&P §B.2.

▷ hit rate

▷ miss rate

▷ hit time

▷ miss penalty

▷ average memory access time

注意 ミスペナルティはミスの余分な時間である。ミスの総時間からヒット時間を引くこと。

- ・ミスペナルティが小さいこと。ミスペナルティは主記憶のアクセス時間に支配され、大きい。第 14 章では、キャッシュのレベルを増やすことでこれを減らす。

最初の 2 つの目標は相反する。大きなキャッシュはミスが少ないが、探索に時間がかかる。最適なキャッシュサイズは両者のバランスをとったものであり、そのバランスを測る尺度が AMAT である。

例 12.7. 主記憶によるミスペナルティは 60 ns である。3 つの候補キャッシュを比較する。

キャッシュ	ヒット時間	ミス率	AMAT
小	1 ns	6 %	$1 + 0.06 \times 60 = 4.6 \text{ ns}$
中	1.5 ns	3 %	$1.5 + 0.03 \times 60 = 3.3 \text{ ns}$
大	3 ns	2 %	$3 + 0.02 \times 60 = 4.2 \text{ ns}$

中程度のキャッシュが最良である。大きなキャッシュはミスが少ないが、ヒット時間が長いことによるコストが、減らせるミスの分を上回る。3 つとも、毎回主記憶にアクセスする場合より 1 桁優れている。

注 12.8. キャッシュが引き合うのは、ヒットが主記憶へのアクセスよりずっと速く、かつミスが稀な場合だけである。ヒット時間が主記憶のアクセス時間に近ければ、ヒットでさえキャッシュなしのアクセスとほぼ同じくらい遅い。ミスが頻繁なら、ミスペナルティが支配的になる。どちらの場合も、AMAT は主記憶だけのアクセス時間に近づくか、それを上回ることさえある。

12.3.1 実際のプロセッサのキャッシュ

実際のプロセッサは複数のレベルのキャッシュをもつ。最初に参照され、プロセッサの読み書きに直接応えるキャッシュが最も小さく高速であり、そのミスだけが、より大きな後続のキャッシュレベル (L2, L3; 第 14 章)、そして最後に主記憶へと進む。この第 1 レベルを **L1 キャッシュ** と呼ぶ。L1 キャッシュは典型的には 16~64 KB (1 KB = 2^{10} バイト) であり、アクセスの約 90 % がヒットするだけの大きさで、ヒットが 1~3 クロックサイクルで済むだけの小ささをもつ。

出典 Lesson 12, 動画 13; H&P §2.1.

▷ L1 cache

12.4 ブロックとライン

データがキャッシュにあるかどうかを素早く判定するため、キャッシュは固定サイズのエントリからなる表として構成される。データは主記憶とキャッシュの間を 1 バイトずつではなく、ブロックと呼ばれる単位で移動する。

定義 12.9 (キャッシュブロック). **キャッシュブロック** とは、ひとまとまりとしてキャッシュにコピーされ、またキャッシュから追い出される、メモリ上の連続したバイトの並びである。ブロックのバイト数を **ブロックサイズ** という。

ブロックサイズは、空間的局所性とキャッシュの容量とのトレードオフである。

- ・ブロックが非常に小さい (極端な場合は 1 バイト) と、空間的局所性により起こりやすい次のバイトへのアクセスが再びミスし、バイトごとに管理情報が必要になる。
- ・ブロックが非常に大きいと、ブロックの大部分はすぐには使われないデータになる。それは、最近使われた他のデータを保持できたはずの領域を占め、キャッシュに入るブロックの数が減る。

典型的なブロックサイズは 32~128 バイトである。キャッシュが保持できるブロックの数は、キャッシュのサイズをブロックサイズで割ったものである。64 バイトのブロックをもつ 16 KB のキャッシュは $2^{14}/2^6 = 256$ 個のブロックを保持する。

出典 Lesson 12, 動画 14~18; H&P §B.1.

▷ cache block

▷ block size

12.4.1 ブロックの開始位置

ブロックは任意のアドレスから始まるわけではない。もしそうなら、アドレス 100~227 を覆うブロックと 200~327 を覆うブロックが同時にキャッシュに存在

しうる。すると 1 つのアドレスが複数のキャッシュ内ブロックに含まれ、あるアドレスがキャッシュされているかを調べるにはアドレス範囲の比較が必要になる。そこで、メモリはブロックサイズの倍数から始まる、重ならない固定のブロックに分割される。128 バイトのブロックなら、ブロックはアドレス 0, 128, 256, ... から始まる (図 12.2)。こうすれば、どのアドレスもちょうど 1 つのブロックに属する。

定義 12.10 (キャッシュライン). キャッシュラインとは、ブロックを 1 つ保持できるキャッシュ内のスロットである。メモリはブロックに、キャッシュはラインに分割される。ラインはブロックと同じ大きさを持ち、時によって異なるブロックを保持する。

ブロックサイズ、したがってラインのサイズは、常に 2 のべき乗である。そうして初めて、次節で示すように、アドレスが属するブロックとブロック内の位置を、除算ではなくアドレスのビットの選択によって求められる。

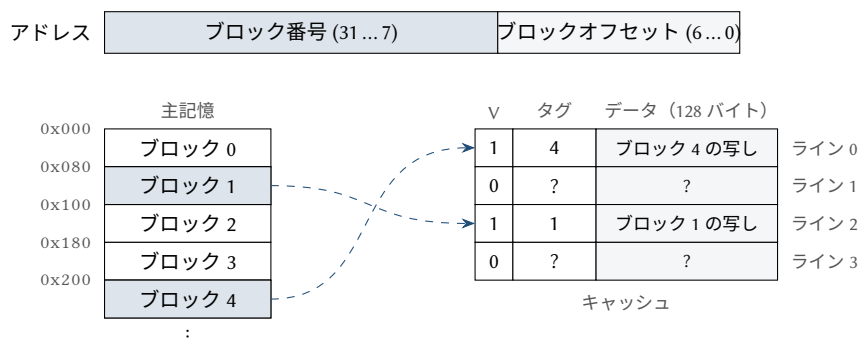


図 12.2: 128 バイトのブロックに分割された主記憶と、4 ラインのキャッシュ。ライン 0 と 2 はブロック 4 と 1 の写しを保持し、ライン 1 と 3 は有効なものを何も保持していない。

12.5 ブロック番号、タグ、有効ビット

ブロックサイズが 2^k バイトのとき、アドレス A は 2 つのフィールドに分かれる。

定義 12.11 (ブロックオフセットとブロック番号). アドレスの下位 k ビットは**ブロックオフセット**であり、そのバイトのブロック内での位置を表す。残りの上位ビットは**ブロック番号**であり、ブロックを識別する。

$$\text{ブロック番号} = \lfloor A/2^k \rfloor, \quad \text{ブロックオフセット} = A \bmod 2^k. \quad (12.2)$$

例 12.12. 128 バイトのブロック ($k = 7$) では、アドレス 0x0040A7C6 のブロックオフセットは 0xC6 & 0x7F = 0x46 = 70, ブロック番号は 0x0040A7C6 $\gg$ 7 = 0x814F である。このバイトはブロック 0x814F の位置 70 にあり、このブロックはアドレス 0x0040A780 から 0x0040A7FF までを覆う。

12.5.1 タグ

ラインは時によって異なるブロックを保持するので、各ラインは現在どのブロックを保持しているかを記録しなければならない。この記録がラインの**タグ**である。タグは、ラインにあるブロックのブロック番号、あるいは以降の節で示すように、ブロック番号のうちラインの位置からは分からない部分である。探索では、アドレスのブロック番号を計算し、そのブロックが置かれうるラインのタグと比較する。一致すればヒットであり、ブロックオフセットがライン内のバイトを選ぶ。

12.5.2 有効ビット

プロセッサの電源投入時、キャッシュのラインには任意のビット、したがって任意のタグが入っている。たまたまブロック番号と一致したタグは、一度も読み込

▷ cache line

出典 Lesson 12, 動画 19–23 ; H&P §B.1.

▷ block offset

▷ block number

▷ tag

まれていないデータに対するヒットを生んでしまう。

定義 12.13 (有効ビット). ラインの**有効ビット**は、そのラインがブロックを保持しているかどうかを示す。起動時にすべての有効ビットをクリアし、ラインにブロックを読み込んだときにそのラインの有効ビットをセットする。

したがって、ライン、あるいはキャッシュ全体を空にするには有効ビットをクリアするだけでよく、タグとデータはそのままにしておける。唯一の例外は、キャッシュ内で書き込まれたがまだ主記憶には書き込まれていないブロックであり (12.11 節)、これは先に書き戻さなければならない。

探索がヒットとなるのは、ラインが有効で、かつ格納されたタグがアドレスのタグフィールドと一致するときだけである。

$$\text{ヒット} = V \wedge (\text{格納されたタグ} = \text{アドレスのタグ}). \quad (12.3)$$

本節のように、ブロック番号のどの部分もラインの選択に使われない限り、タグフィールドはブロック番号全体である。以降の節の構成では、ブロック番号のビットの一部をラインの選択に使い、残りのビットだけをタグに保持する。

要点

アドレスはブロック番号とオフセットからなる。タグはラインがどのブロックを保持しているかを、有効ビットはそもそもブロックを保持しているかを示し、オフセットはバイトを選ぶ。次節の主題であるブロックを置ける場所が、どのラインを探索しなければならないかを決める。

12.6 ブロックを置ける場所

キャッシュは、あるブロックを置けるラインがいくつあるかによって異なる。この数をキャッシュの**連想度**という。連想度は、探索で比較しなければならないタグの数と、異なるブロックが同じラインを奪い合う頻度を決める。次の 3 つの構成が区別される (図 12.3)。

ダイレクトマップ 各ブロックはちょうど 1 つのラインにしか置けない (12.7 節)。

セットアソシアティブ 各ブロックは、セットをなす N 個のラインのどれにでも置ける (12.8 節)。

フルアソシアティブ 各ブロックはキャッシュのどのラインにでも置ける (12.9 節)。

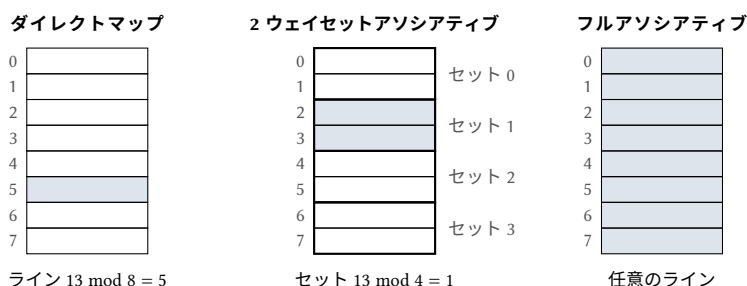


図 12.3: 各構成の 8 ラインのキャッシュにおいて、ブロック番号 13 を置けるライン。

12.7 ダイレクトマップキャッシュ

3 つの構成のうち最も単純なものは、すべてのブロックにキャッシュのラインを 1 つだけ割り当てる。**ダイレクトマップキャッシュ**では、どのブロックにもそれを置けるラインがちょうど 1 つあり、そのラインはブロック番号の下位ビットで選ばれる。

定義 12.14 (インデックス). **インデックス**とは、ブロックをキャッシュのどこに置けるかを選ぶアドレスのビット群である。 2^n 個のラインをもつダイレクトマップキャッシュでは、インデックスはブロック番号の下位 n ビットからなる。

▷ valid bit

出典 Lesson 12, 動画 24 ; H&P §B.1.
▷ associativity

出典 Lesson 12, 動画 25–28 ; H&P §B.1.
▷ direct-mapped cache

▷ index

したがってアドレスは3つのフィールドに分かれる (図 12.4). 最下位の k ビットがオフセット, その上の n ビットがインデックス, 残りのすべてのビットがタグである. ラインにはブロック番号全体ではなく, このタグのビットだけを格納する. ライン i に置けるブロックはすべてインデックス i をもち, ブロック番号のインデックス部分はライン自体から分かるからである.

探索では, インデックスで選ばれた1つのラインを読み, そのタグをアドレスのタグと比較し, 有効ビットを調べる (式 (12.3)).

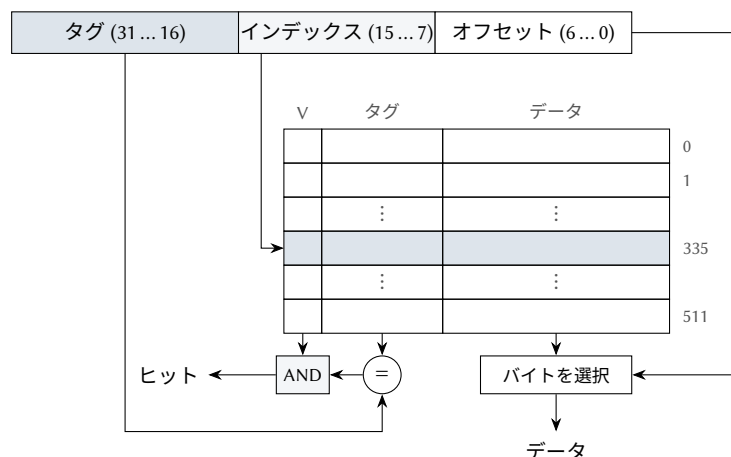


図 12.4: 128 バイトのブロックをもつ 64 KB のダイレクトマップキャッシュにおける探索. インデックスが 512 個のラインの 1 つを選び, 1 つの比較器がそのタグを調べる.

例 12.15. 128 バイトのブロックと 32 ビットのアドレスをもつ 64 KB のダイレクトマップキャッシュには $512 = 2^9$ 個のラインがあり, オフセットはビット 6-0, インデックスはビット 15-7, タグはビット 31-16 である. 例 12.12 のアドレス $0x0040A7C6$ (ブロック番号 $0x814F$) のインデックスは $0x814F \& 0x1FF = 0x14F = 335$, タグは $0x814F \gg 9 = 0x0040$ である. ライン 335 が有効でタグ $0x0040$ を保持していれば, このアクセスはヒットする.

12.7.1 利点と欠点

ダイレクトマップキャッシュの長所は, 調べるラインが 1 つだけで済むことである. 比較器は 1 つで足りるので, 探索は高速かつ安価で, エネルギー効率も良い. ヒット時間は, 与えられたサイズのキャッシュとして可能な限り短い.

短所は, 同じインデックスをもつブロックが 1 つのラインを奪い合うことである. プログラムがそのような 2 つのブロックに交互にアクセスすると, 各アクセスがもう一方のアクセスに必要なブロックを追い出し, キャッシュの残りが空であってもすべてのアクセスがミスする. こうした競合がミス率を高める.

例 12.16. 16 バイトのブロックをもつ 8 ラインのダイレクトマップキャッシュでは, オフセットはビット 3-0, インデックスはビット 6-4, タグはビット 31-7 である. あるプログラムがアドレス $0x1000$ (ブロック $0x100$) と $0x1084$ (ブロック $0x108$) に交互にアクセスする. どちらのブロックもインデックスは 0 で, タグはそれぞれ $0x20$ と $0x21$ である. 各アクセスは直前のアクセスが読み込んだブロックを置き換えるので, プログラムがどれだけ長く続いても, すべてのアクセスがミスとなる.

12.8 セットアソシアティブキャッシュ

セットアソシアティブ構成は, ラインが 1 つだけという制約を緩める.

定義 12.17 (セットアソシアティブキャッシュ). セットアソシアティブキャッシュは, ラインを N 個ずつのセットに分け, ブロックを特定の 1 つのセットのどのラインにでも置けるようにする. このようなキャッシュを N ウェイセットアソシアティブという.

出典 Lesson 12, 動画 29-31; H&P §B.1.

▷ set-associative cache

このときインデックスはラインではなくセットを選ぶ。2^s 個のセットをもつキャッシュのインデックスは s ビットであり、タグは再びインデックスより上のすべてのビットからなる。探索では、アドレスのタグを、選ばれたセットの N 個すべてのラインのタグと並列に比較し、有効なラインのいずれかが一致すればヒットとなる (図 12.5)。データは一致したラインから得られる。

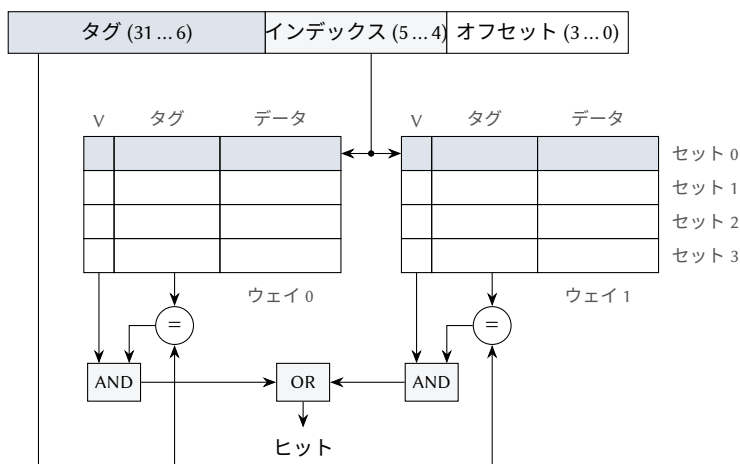


図 12.5: 4 セットの 2 ウェイセットアソシアティブキャッシュ (例 12.18 の構成) における探索。インデックス (ここではその例と同じ 0) がセットを選び、セット内の 2 つのラインはそれぞれ、有効でタグが一致すればヒットとなる。

例 12.18. 例 12.16 の 8 ラインを 2 ウェイセットアソシアティブキャッシュとして構成する。セットは 4 個なので、インデックスはビット 5-4、タグはビット 31-6 である。ブロック 0x100 はタグ 0x40 でセット 0 に、ブロック 0x108 はタグ 0x42 でセット 0 に入る。2 つのブロックは依然として同じセットを共有するが、セットには 2 つのラインがある。各ブロックへの最初のアクセスがミスしてそのブロックを読み込んだ後は、以降のアクセスはすべてヒットする。

同じサイズのダイレクトマップキャッシュと比べると、 N ウェイセットアソシアティブキャッシュには N 個の比較器と、一致したラインのデータの選択が必要であり、そのためヒット時間は長くなる。しかし競合が少ないので、ミス率は低い。

12.9 フルアソシアティブキャッシュ

もう一方の極端では、ブロックをどこにでも置ける。

定義 12.19 (フルアソシアティブキャッシュ). フルアソシアティブキャッシュでは、どのブロックもどのラインにでも置ける。

選ぶものがないので、インデックスは存在しない。タグはブロック番号全体であり、探索ではそれをキャッシュ内のすべてのラインのタグと比較する。空いているラインがある限り 2 つのブロックが 1 つのラインを奪い合うことはないが、ラインごとに比較器が必要になるので、大きなフルアソシアティブキャッシュは低速で高価になる。

12.9.1 3 つの構成に共通の枠組み

ダイレクトマップキャッシュとフルアソシアティブキャッシュは、セットアソシアティブキャッシュの両極端である。ダイレクトマップキャッシュは、ラインと同数のセットをもつ 1 ウェイセットアソシアティブキャッシュであり、フルアソシアティブキャッシュは、すべてのラインを含むただ 1 つのセットをもつ。アド

出典 Lesson 12, 動画 32-33 ; H&P §B.1.

▷ fully associative cache

レスが a ビットするとき、どの構成についても次が成り立つ。

$$\begin{aligned} \text{オフセットビット数} &= \log_2(\text{ブロックサイズ}), \\ \text{インデックスビット数} &= \log_2(\text{セット数}) = \log_2(\text{ライン数}/N), \\ \text{タグビット数} &= a - \text{インデックスビット数} - \text{オフセットビット数}, \end{aligned} \quad (12.4)$$

ここで、ダイレクトマップキャッシュでは $N = 1$ (インデックスビット数 = $\log_2(\text{ライン数})$)、フルアソシアティブキャッシュでは $N = \text{ライン数}$ (インデックスは 0 ビット) である。例 12.15 の 128 バイトのブロックをもつ 64 KB のキャッシュでは、タグはダイレクトマップの 16 ビットからフルアソシアティブの 25 ビットまで増える。その中間の場合は 12.12 節の表 12.4 に示す。

要点

連想度はヒット時間とミス率のトレードオフを決める。セット当たりのウェイトが多いほど、毎回のアクセスでのタグ比較は増えるが、競合は減る。ブロックが追い出されるのは、そのセットのすべてのラインが他のブロックで占められているときだけだからである。

ヒント C バイトのキャッシュのタグは $a - \log_2(C/N)$ ビットである。 N を 2 倍にするたびに、1 ビットがインデックスからタグに移る。

12.10 キャッシュの置換

ミスの際には、新しいブロックを、それに許されたラインのいずれかに置かなければならない。それらのラインのうち有効でないものがあれば、そのラインを使う。すべてが有効なブロックを保持していれば、1 つのブロックを追いつ出さなければならない。ダイレクトマップキャッシュには選択の余地がない。セットアソシアティブキャッシュとフルアソシアティブキャッシュでは、追いつ出すブロックを置換アルゴリズムを選ぶ。

ランダム セット内のブロックを無作為に選んで追いつ出す。単純だが、ブロックの使われ方を無視する。

FIFO (first in, first out) キャッシュに最も長く存在しているブロックを、まだ使われているかどうかにかかわらず追いつ出す。

LRU 最後にアクセスされたのが最も昔であるブロックを追いつ出す。

定義 12.20 (LRU) LRU 方式は、置き換えてよいブロックのうち、最も長い間アクセスされていないものを追いつ出す。

LRU は時間的局所性を利用する。最近アクセスされたブロックは近いうちに再びアクセスされる可能性が高いので、最も長い間使われていないブロックが、必要とされる可能性の最も低いブロックである。

12.10.1 カウンタによる LRU の実装

LRU は、ラインごとのカウンタで実装できる。 N ウェイセットアソシアティブキャッシュでは、各ラインが $\log_2 N$ ビットのカウンタをもつ。起動時に各セットのカウンタを任意の順序で値 $0, 1, \dots, N-1$ に初期化し、以下の更新規則はこの状態を保つ。すなわち、各セットでどの値もちょうど 1 回ずつ現れる。カウンタが 0 のラインが最も長い間使われていないラインであり、カウンタが $N-1$ のラインが直近に使われたラインである。カウンタが c であるラインへのアクセスのたびに、次を行う。

1. セット内で c より大きいカウンタをすべて 1 減らす。
2. アクセスしたラインのカウンタを $N-1$ にする。

ミスの際には、カウンタが 0 のラインを追いつ出す。新しいブロックがそこに入った後は、そのアクセスを他のアクセスと同様に扱い、上のようにカウンタを更新する。表 12.1 は 4 ウェイのセット 1 つを追いつしたものである。

この実装のコストは、ラインごとの $\log_2 N$ ビットと、より重要なことに、すべてのアクセスで生じる動作である。ヒットでさえセットの N 個のカウンタすべてを変更しうる。連想度の高いキャッシュでは、これはかなりのハードウェアとエネルギーを要し、第 14 章の、より安価な LRU の近似の動機となる。

出典 Lesson 12, 動画 34–36 ; H&P §B.1.

▷ replacement policy

▷ least recently used

注意 カウンタはヒットを含むすべてのアクセスで更新すること。読み込み時にだけ更新すると FIFO になる。

表 12.1: ブロック A~D を保持する 4 ウェイのセット 1 つの LRU カウンタ. 4 つのラインそれぞれについて, ブロック: カウンタの形で示す.

アクセス	結果	ライン 0	ライン 1	ライン 2	ライン 3
(初期状態)		A:0	B:1	C:2	D:3
C	ヒット	A:0	B:1	C:3	D:2
A	ヒット	A:3	B:0	C:2	D:1
E	ミス, B を追い出す	A:2	E:3	C:1	D:0
D	ヒット	A:1	E:2	C:0	D:3
B	ミス, C を追い出す	A:0	E:1	B:3	D:2

12.11 書き込み方針

書き込みには, 読み出しでは生じない問題が 2 つある.

1. 書き込みミスの際にブロックをキャッシュに持ち込むか. 持ち込むキャッシュは**ライトアロケート方式**に従う. ノーライトアロケートのキャッシュは値を主記憶だけに書き込む. 大半のキャッシュはライトアロケートである. 局所性により, 書き込まれたばかりのブロックは近いうちに再び読み書きされる可能性が高いからである.
2. 主記憶が書き込まれた値を受け取るのはいつか. これには 2 つの方式がある.

定義 12.21 (ライトスルーとライトバック). **ライトスルー方式**では, 書き込みはキャッシュ内のブロックを更新し, ただちに主記憶内のブロックも更新する.

ライトバック方式では, 書き込みはキャッシュ内のブロックだけを更新し, 主記憶はそのブロックがキャッシュから追い出されるときに更新される.

ライトスルーでは主記憶が常に最新に保たれるが, そのかわりすべての書き込みが主記憶のアクセス時間を払うことになり, これはまさにキャッシュが避けようとしたものである. そのためキャッシュはライトバックを用いる. ブロックがキャッシュにある間の多数の書き込みは, 追い出し時の高々 1 回の主記憶への書き込みで済む. ライトバックのキャッシュは通常ライトアロケートでもある. そうでなければ, キャッシュにないブロックへの書き込みは, そのブロックがどれほど頻繁に書き込まれても, すべて主記憶に向かうことになる.

しかし, 追い出すブロックをすべて書き戻すと, 読み出されただけでメモリ上の写しと同一のブロックまで書き込むことになる.

定義 12.22 (ダーティビット). ラインの**ダーティビット**は, そのブロックが読み込まれてから書き込まれたかどうかを示す. ブロックを読み込んだときにクリアし, ブロックが書き込まれたときにセットする. 追い出しの際, ダーティなブロックは主記憶に書き込み, クリーンなブロックは単に破棄する.

例 12.23. ブロック X と Y はダイレクトマップキャッシュの同じラインに対応し, このラインは最初は無効である. 表 12.2 は, ライトバック方式のもとでの一連のアクセスを追ったものである. Y への連続した 2 回の書き込みは, ステップ 6 での 1 回のメモリ書き込みで済み, ステップ 8 で Y を追い出してもメモリ書き込みは生じない. Y はステップ 7 で読み込まれた後に書き込まれていないからである. ライトバックに必要なメモリ書き込みは合計 3 回であり, ライトスルーなら書き込みごとに 1 回, すなわち 4 回必要になる.

12.12 全体のまとめ

キャッシュはいくつかの設計上の選択によって完全に記述され, その構造はそこから導かれる.

キャッシュサイズとブロックサイズ ライン数とオフセットのビット数が決まる.

連想度 セット数, したがってインデックスとタグのビット数が決まる (式 (12.4)).

置換アルゴリズム LRU は各ラインに $\log_2 N$ ビットのカウンタを加える (ダイレクトマップキャッシュでは不要).

書き込み方針 ライトバックは各ラインにダーティビットを加える.

出典 Lesson 12, 動画 37-40 ; H&P §B.1.

▷ write-allocate policy

▷ write-through policy

▷ write-back policy

▷ dirty bit

出典 Lesson 12, 動画 41-45 ; H&P §B.1.

表 12.2: ライトバック方式のもとでの 1 つのライン (例 12.23). D はアクセス後のダーティビットである.

ステップ	アクセス	結果	メモリ操作	アクセス後のライン
1	X 読み出し	ミス	X 読み込み	X, D=0
2	X 書き込み	ヒット	—	X, D=1
3	Y 読み出し	ミス	X 書き戻し, Y 読み込み	Y, D=0
4	Y 書き込み	ヒット	—	Y, D=1
5	Y 書き込み	ヒット	—	Y, D=1
6	X 書き込み	ミス	Y 書き戻し, X 読み込み	X, D=1
7	Y 読み出し	ミス	X 書き戻し, Y 読み込み	Y, D=0
8	X 読み出し	ミス	X 読み込み	X, D=0

こうして各ラインは、データに加えて有効ビット、ダーティビット、LRU カウンタ、タグを格納する。図 12.6 は、1 回のアクセスでこれらがどのように使われるかを示す。ヒットはデータを読み書きし、LRU カウンタを更新する。ミスはまず追い出すラインを選び、それが有効かつダーティであれば書き戻してから、新しいブロックを読み込む。その後、アクセスはヒットと同様に進む。

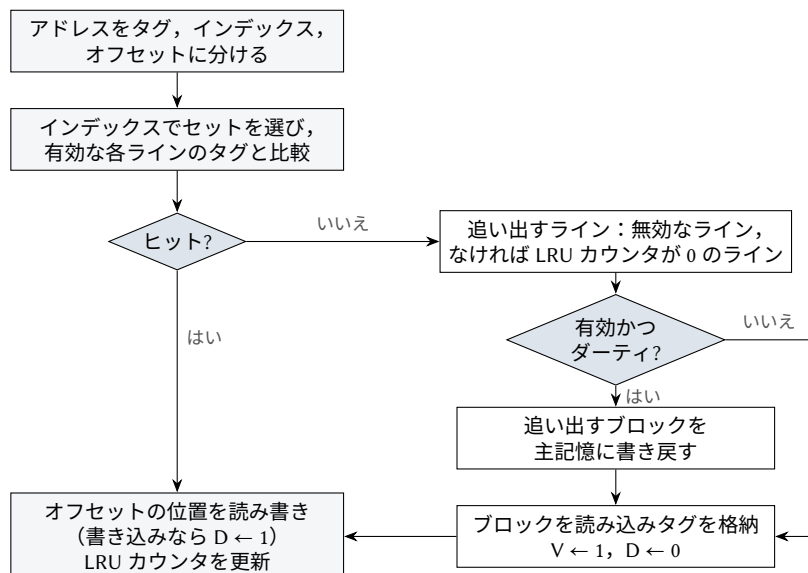


図 12.6: LRU 置換とライトバック方式を用いるセットアソシアティブキャッシュにおける 1 回のメモリアクセス。

例 12.24. 例 12.18 の 2 ウェイセットアソシアティブキャッシュ (16 バイトのブロック、インデックスはビット 5-4、タグはビット 31-6) に、LRU 置換とライトバック方式を加えたものを考える。表 12.3 は、そのセット 1 を 5 回のアクセスにわたって追ったものである。セットの 2 つのラインは終始有効である。最初のアクセスはヒットし、カウンタを更新するだけである。2 番目のアクセスはミスし、カウンタが 0 のウェイ 1 が追い出される。そのブロックはダーティなので、新しいブロックを読み込む前に書き戻される。4 番目のアクセスは書き込みミスであり、追い出されるブロックはクリーンなので何も書き戻されず、読み込まれたブロックはただちにダーティになる。最後のアクセスは、3 番目のアクセスが書き込んだタグ 0x40 のブロックを追い出し、それを書き戻す。

例 12.25. 表 12.4 は、128 バイトのブロック、32 ビットのアドレス、LRU 置換、ライトバックを用いる 64 KB のキャッシュの構成を、いくつかの連想度について求めたものである。8 ウェイのキャッシュでは、アドレス 0x0040A7C6 (ブロック番号 0x814F) のインデックスは $0x814F \& 0x3F = 15$ 、タグは $0x814F \gg 6 = 0x205$ である。その 512 個のラインには $512 \times 24 = 12288$ ビット、すなわち 1536 バイトのメタデータが必要であり、これは 64 KB のデータの約 2.3% にあたる。

表 12.3: LRU 置換とライトバックを用いる 2 ウェイセットアソシアティブキャッシュのセット 1 (例 12.24). 各アクセス後の各ウェイのタグ、ダーティビット、LRU カウンタを示す。メモリ操作のものも含め、タグはすべて 16 進数である。

アクセス	結果	ウェイ 0			ウェイ 1			メモリ操作
		タグ	D	LRU	タグ	D	LRU	
(初期状態)		40	0	0	43	1	1	
読み出し 0x1018	ヒット	40	0	1	43	1	0	—
読み出し 0x1054	ミス	40	0	0	41	0	1	43 書き戻し, 41 読み込み
書き込み 0x101C	ヒット	40	1	1	41	0	0	—
書き込み 0x1098	ミス	40	1	0	42	1	1	42 読み込み
読み出し 0x1058	ミス	41	0	1	42	1	0	40 書き戻し, 41 読み込み

表 12.4: 128 バイトのブロックと 32 ビットのアドレスをもつ 64 KB のキャッシュのアドレスフィールドと、ライン当たりのメタデータのビット数 (有効ビット + ダーティビット + LRU カウンタ + タグ)。

構成	セット数	オフセット	インデックス	タグ	ライン当たりのメタデータ
ダイレクトマップ	512	7	9	16	$1 + 1 + 0 + 16 = 18$
2 ウェイ	256	7	8	17	$1 + 1 + 1 + 17 = 20$
8 ウェイ	64	7	6	19	$1 + 1 + 3 + 19 = 24$
フルアソシアティブ	1	7	0	25	$1 + 1 + 9 + 25 = 36$

第 13 章では同じ考え方—ブロック、タグ、置換—を仮想アドレスの変換に適用し、第 14 章では、ここで述べた基本的なキャッシュのヒット時間、ミス率、ミスペナルティを下げる技法を扱う。

要点 — 章のまとめ

プログラムは時間的局所性と空間的局所性をもってメモリにアクセスするので、最近使われたブロックを保持する小容量で高速なキャッシュが大半のアクセスに応える。その効果は $AMAT = \text{ヒット時間} + \text{ミス率} \times \text{ミスペナルティ}$ (式 (12.1)) で測られる。メモリは 2 のべき乗の大きさのブロックに分割されてキャッシュのラインに置かれる。アドレスはタグ、インデックス、オフセットに分かれ、ラインが有効でそのタグが一致すればヒットとなる。ダイレクトマップ、セットアソシアティブ、フルアソシアティブの各キャッシュは、ブロックが占めうるラインの数が異なり、ヒット時間とミス率をトレードオフする。ミスの際には LRU が最も長い間使われていないブロックを追いつ出し、ダーティビットを用いるライトバック方式は、変更されたブロックが追いつ出されるときにだけブロックをメモリに書き込む。

仮想記憶

講義動画	Lesson 13, 動画 1-27
教科書	H&P [1] §B.4-B.5
キーワード	仮想記憶, 物理メモリ, ページ, ページフレーム, ページテーブル, ページフォルト, 仮想ページ番号, ページオフセット, 多段ページテーブル, ページサイズ, 内部断片化, TLB, TLB ミス, ページテーブルウォーク

第 12 章では、プロセッサが生成するアドレスをすべて主記憶上の位置として扱った。実際のシステムでは、プログラムが使うアドレスは仮想である。すなわち、どのプログラムも自分専用の大きなアドレス空間を見ており、オペレーティングシステムとプロセッサが協力して、そのうちプログラムが実際に使う部分を搭載されたメモリに対応づける。この章では、なぜ仮想記憶が必要か、ページテーブルがどのように仮想アドレスを物理アドレスに変換するか、それらのテーブルがどれほど大きく、多段ページテーブルがどのようにそれを小さく保つか、そして、プロセッサが最近の変換を TLB に保持しなければ、変換がすべてのメモリアクセスを遅くしてしまうのはなぜかを述べる。第 14 章は、この TLB を第 12 章のキャッシュと組み合わせる。

13.1 なぜ仮想記憶が必要か

ハードウェアとソフトウェアでは、メモリの見え方が異なる。ハードウェアにとって、メモリとはマシンに搭載されているもの、すなわちメモリモジュールが提供するバイト数だけの配列である。プログラムにとって、メモリとは 32 ビットプロセッサなら 2^{32} バイト、64 ビットプロセッサなら 2^{64} バイトのアドレス空間であり、そのすべてがそのプログラムのものである。コード、データ、スタックは、搭載されたメモリの量にも、同時に走る他のプログラムにも関係なく、プログラムとコンパイラが選んだアドレスに置かれる。

この 2 つの見方は、2 つの点で食い違う。

- 1 つのプログラムのアドレス空間は、搭載されたメモリよりはるかに大きい。32 ビットのアドレス空間は 4 GB であり、マシンがそれだけのメモリをもつとは限らない。64 ビットのアドレス空間は 16 EB (1 EB = 2^{60} バイト) で、どのマシンがもつ量よりも何桁も大きい。
- 多数のプログラムが同時に走り、そのそれぞれが、アドレス空間全体を所有しているかのように同じアドレスを使う。

定義 13.1 (仮想記憶)。すべてのプログラムに、大きな自分専用のメモリという見え方を与え、そのうちプログラムが実際に使う部分をマシンに搭載されたメモリに対応づける機構を**仮想記憶**という。

各プログラムの各部分をどこに置くかはオペレーティングシステムが決め、プロセッサはその決定をすべてのメモリアクセスのすべてのアドレスに適用する。これにより、プログラムはメモリの大きさも、一緒に走ることになる他のプログラムも知らずに書ける。また、オペレーティングシステムが取り計らわない限り、あるプログラムが別のプログラムのメモリにアクセスすることはできず、さらに、プログラム全体では搭載量を超えるメモリを使うことができ、あふれた分はディスクに置かれる (13.4 節)。

13.2 物理メモリと仮想アドレス空間

2 つの見方は、2 種類のアドレスに対応する。

定義 13.2 (物理メモリ)。マシンに実際に搭載されているメモリを、そのマシンの**物理メモリ**という。その各バイトには 0 から物理メモリの大きさより 1 小さい値まで連番が付けられ、バイトを識別するこの番号を、そのバイトの**物理アドレス**

出典 Lesson 13, 動画 1-3 ; H&P §B.4 ; [23].

▷ virtual memory

出典 Lesson 13, 動画 4-5 ; H&P §B.4.

▷ physical memory

▷ physical address

という。

物理メモリの大きさを決めるのは搭載されたモジュールであって、プロセッサのアドレスの幅ではない。32 ビットのマシンは 4 GB をアドレス指定できるが、搭載されているのは 2 GB かもしれない。64 ビットのマシンはおそらく数十ギガバイトをもつが、これはアドレス指定可能な 2^{64} バイトのごく一部である。この章の例では、物理アドレスのビット数は搭載されたメモリが要求するちょうどの数とする。2 GB なら 31 ビットである。

定義 13.3 (仮想アドレス空間). プログラムの**仮想アドレス空間**とは、物理メモリとは無関係に、そのプログラムが使えるアドレスの全範囲、すなわち n ビットプロセッサでは 0 から $2^n - 1$ までである。その中のアドレスを**仮想アドレス**という。

プログラムが扱うアドレスはすべて仮想である。プログラムカウンタも、データ中に格納されたポインタも、ロードとストアのためにプロセッサが計算するアドレスもそうである。プログラムが使うのは、仮想アドレス空間のうちのいくつかの領域だけである (図 13.1 の左と右)。

- ・ 低位アドレスにあるコードとグローバルなデータ。
- ・ その上にあるヒープ。プログラムが動的にメモリを確保するにつれて高位アドレスの方向へ伸びる。
- ・ アドレス空間の最上部にあるスタック。低位アドレスの方向へ伸びる。

ヒープとスタックの間には、プログラムが決して使わない膨大なアドレスの範囲がある。走っているプログラムはどれもこのように配置された自分専用の仮想アドレス空間をもつので、異なるプログラムがまったく異なる内容に対して同じ仮想アドレスを使うことになる。

13.3 ページとページテーブル

仮想アドレスごとに対応づけを記録すると、アドレスの数だけテーブルのエントリが必要になる。そこで仮想記憶は、キャッシュが個々のバイトではなくブロックを移動させるのと同じように、固定サイズのアドレスの範囲に対応づける。

定義 13.4 (ページとページフレーム). 仮想アドレス空間は等しい大きさの連続した範囲に分割され、その 1 つ 1 つを**ページ**という。物理メモリも同じ大きさの範囲に分割され、その 1 つ 1 つを**ページフレーム**、あるいは単にフレームという。その大きさは 2 のべき乗で、典型的には 4 KB であり、どのページもどのフレームにもちょうど収まる。

定義 13.5 (ページテーブル). プログラムの**ページテーブル**は、その仮想アドレス空間のページごとに、そのページが物理メモリにあるかどうかと、ある場合はどのフレームがそれを保持しているかを記録する。

オペレーティングシステムは、走っているプログラムごとに 1 つのページテーブルを管理し、そのプログラムのページをフレームに置いたり取り除いたりするたびに更新する。走っているプログラムのページテーブルは、オペレーティングシステムが設定したレジスタを通じて見つけられ、そのプログラムのアドレスを変換するために読まれる。読むのは通常プロセッサ自身であり、システムによってはオペレーティングシステムである (13.10 節)。図 13.1 は、2 つのプログラムについて、こうして得られる対応づけを示す。

- ・ どのページもどのフレームにでも置ける。仮想アドレス空間で隣り合うページが物理メモリで隣り合っている必要はなく、異なるプログラムのフレームは互いに入り混じる。
- ・ プログラムが決して使わないページは対応づけられず、フレームを占めない。物理メモリを消費するのは、アドレス空間のうち使われる領域だけである。
- ・ 2 つのプログラムのページテーブルが、同じフレームにページを対応づけることもある。これによりプログラムはメモリを共有でき、例えば同じプログラムが 2 つ走っているとき、そのコードを 1 つのコピーで済ませられる。
- ・ 対応づけられたページがすべて物理メモリにある必要はない。残りはディスクにある (13.4 節)。

ページテーブルは、使われるかどうかによらず仮想アドレス空間のすべての

▷ virtual address space

▷ virtual address

出典 Lesson 13, 動画 6-7 ; H&P §B.4.

▷ page

▷ page frame

▷ page table

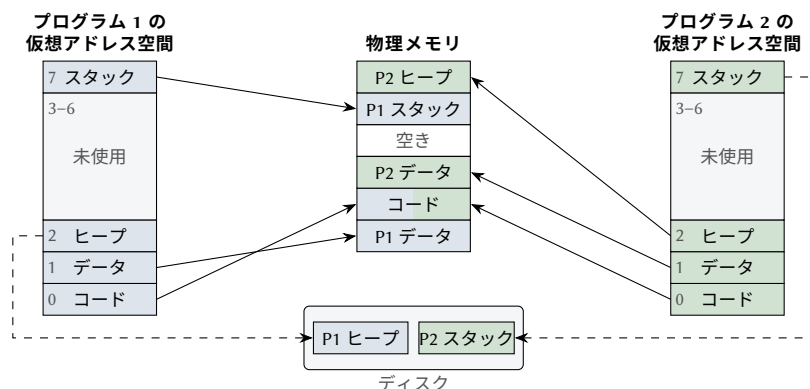


図 13.1: 2つのプログラムの仮想アドレス空間（ページ 0-7, 下が低位アドレス）を物理メモリの 6つのフレームに対応づけたもの。両プログラムのページ 0 は同じフレームに対応づけられている。各プログラムは 1 ページをディスクにもち（破線）、使われないページ 3-6 は対応づけられていない。

ページを記述する。最も単純な形（13.6 節）では、そのそれぞれにエントリをもつ。したがって、記述するページの数を決めるのは仮想アドレス空間の大きさとページの大きさであり、物理メモリの大きさは、1つのエントリが指せるフレームの数を決めるにすぎない。

13.4 ページフォールト

走っているプログラム全体では、通常、物理メモリのフレーム数より多くのページが対応づけられる。収まりきらないページはディスクに置かれ、物理メモリは最近使われたページを保持する。

定義 13.6 (ページフォールト) ページテーブルがフレームに対応づけていないページへのアクセスは**ページフォールト**、すなわち制御をオペレーティングシステムに移す例外を引き起こす。

そのページがディスクにある場合、オペレーティングシステムは 3つの手順でページフォールトを処理する。

1. そのページのためにフレームを 1つ選ぶ。空きフレームがなければ、最近使われていないページを追い出す。そのページが読み込まれてから書き込まれていれば、先にディスクに書き戻す。追い出されたページのページテーブルは、そのページがもうメモリにないことを示すように更新する。
2. フォールトしたページをディスクからそのフレームに読み込み、ページをこのフレームに対応づけるようにページテーブルを更新する。
3. フォールトを起こした命令を再実行する。今度はそのページがメモリに見つかる。これには正確な例外（第 8 章）が必要である。

ディスクアクセスにはミリ秒、すなわち数百万クロックサイクルがかかるので、ページフォールトはキャッシュミスよりはるかに高くつく。ページフォールトは稀でなければならず、追い出すページを、オペレーティングシステムがソフトウェアで入念なアルゴリズムを用いて選ぶだけの価値がある。

すべてのページフォールトがディスクを伴うわけではない。アドレスが、そのプログラムに割り当てられたどの領域にも属さない場合、読み込むものは何もなく、オペレーティングシステムはそのアクセスを誤りとして扱い、通常はプログラムを終了させる。

注 13.7. 物理メモリは、ディスク上のページのキャッシュとして働く。ページは第 12 章のブロックに、フレームはラインに、ページフォールトはキャッシュミスに、変更されたページだけをディスクに書き戻すことはダーティビットをもつライトバックキャッシュに対応する。どのページもどのフレームにでも置けるので配置はフルアソシアティブであるが、タグを探索する必要はない。ページテーブルはページを添字として直接引けるからである。

注意 ページテーブルの添字は仮想ページであって、フレームではない。物理メモリが少なればフレームは少なくなるが、ページテーブルが記述すべきページの数是不変である。

出典 Lesson 13, 動画 8 ; H&P §B.4.

▷ page fault

13.5 アドレス変換

ページはページサイズの倍数から始まるので、仮想アドレスは、キャッシュアドレスのブロック番号とブロックオフセットとちょうど同じように、2つのフィールドに分かれる。

定義 13.8 (仮想ページ番号とページオフセット). 2^k バイトのページでは、仮想アドレスの下位 k ビットが**ページオフセット**、すなわちそのバイトのページ内での位置を表す。残りの上位ビットが**仮想ページ番号** (VPN) であり、ページを識別する。

フレームも同じように番号づけられる。物理アドレス $j \times 2^k$ から始まるフレームの**物理フレーム番号** (PFN) は j である。ページテーブルは VPN を PFN に対応づけ、変換はオフセットを保ったまま前者を後者で置き換える (図 13.2)。

$$\begin{aligned} \text{VPN} &= \lfloor \text{VA} / 2^k \rfloor, & \text{オフセット} &= \text{VA} \bmod 2^k, \\ \text{PFN} &= \text{ページテーブル}[\text{VPN}], & \text{PA} &= \text{PFN} \times 2^k + \text{オフセット}, \end{aligned} \quad (13.1)$$

ここで VA と PA はそれぞれ仮想アドレスと物理アドレスである。ページとそのフレームは同じ大きさであり、バイトはそこでの位置を保つので、オフセットはそのまま通り抜ける。2進表現では、物理アドレスは PFN のビットの後にオフセットのビットが続いたものである。VPN は仮想アドレスより k ビット少なく、PFN は物理アドレスより k ビット少ないので、仮想アドレスと物理アドレスの幅が異なれば、この2つの番号の幅も異なる。そのページに対する PFN がページテーブルにない場合、そのアクセスは代わりにページフォールトを引き起こす。

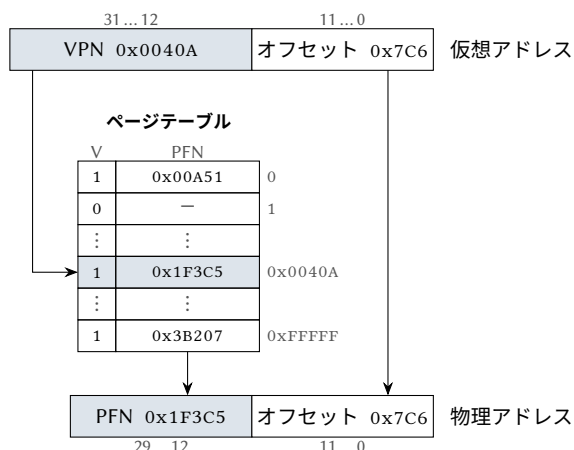


図 13.2: 4 KB のページと 30 ビットの物理アドレスのもとでの仮想アドレス `0x0040A7C6` の変換 (例 13.9)。VPN がページテーブルエントリを選び、その PFN が VPN を置き換え、オフセットはそのまま写される。V = 0 のエントリであればページフォールトとなる。

例 13.9. 4 KB のページ ($k = 12$) をもつ 32 ビットプロセッサが 1 GB の物理メモリをもつとすると、物理アドレスは 30 ビットである。ページオフセットはビット 11-0, VPN はビット 31-12 (20 ビット) であり、PFN は 18 ビットである。仮想アドレス `0x0040A7C6` の VPN は `0x0040A`, オフセットは `0x7C6` である。図 13.2 のページテーブルはページ `0x0040A` をフレーム `0x1F3C5` に対応づけるので、物理アドレスは $0x1F3C5 \times 2^{12} + 0x7C6 = 0x1F3C57C6$ である。仮想アドレス `0x000017FF` はページ `0x00001` に属し、同じページテーブルはこれをメモリにないと記している。このアドレスへのアクセスはページフォールトを引き起こす。

要点

変換は仮想ページ番号を物理フレーム番号に置き換え、ページオフセットをそのまま残す。フレーム番号を与えるのは、オペレーティングシステムが管

出典 Lesson 13, 動画 9-10 ; H&P §B.4.

▷ page offset
▷ virtual page number

▷ physical frame number

ヒント 4 KB のページで 16 進表記のアドレスを使うと、下 3 桁がオフセットである。変換はその手前の桁を置き換える。

理するページテーブルである。そのページに対するフレーム番号がなければ、そのアクセスはページフォールトを引き起こす。

13.6 単一レベルページテーブルの大きさ

最も単純なページテーブルは、1つの配列である。

定義 13.10 (単一レベルページテーブル). 単一レベルページテーブルとは、仮想ページごとに1つのエントリをもち、VPNで直接添字づけされる配列である。あるページに対応するその要素を、そのページのページテーブルエントリ (PTE) という。

PTEは、そのページのPFNといくつかの状態ビット、すなわちページが物理メモリにあるときだけセットされる有効ビットと、そのページにどのようにアクセスしてよいか (読み出し、書き込み、実行) を記録するビットを保持する。PTEの大きさは典型的にはアドレスと同程度、32ビットプロセッサで4バイト、64ビットプロセッサで8バイトであり、PFNと状態ビットを収めるだけの余地がある。どの仮想ページにもエントリがあるので、

$$\begin{aligned} & \text{単一レベルページテーブルの大きさ} \\ &= \frac{\text{仮想アドレス空間の大きさ}}{\text{ページサイズ}} \times \text{PTEの大きさ}. \end{aligned} \quad (13.2)$$

式 (13.2) に現れない量が2つある。物理メモリの大きさは、エントリのうちPFNが占めるビット数に影響するだけである。プログラムが実際に使うメモリの量はまったく関係しない。使われないページにもエントリがあるからである。走っているプログラムはどれも、この大きさのページテーブルを必要とする。

例 13.11. 32ビットの仮想アドレス、4KBのページ、4バイトのエントリでは、単一レベルページテーブルは $2^{32}/2^{12} = 2^{20}$ 個のエントリをもち、走っているプログラム1つにつき $2^{20} \times 4$ バイト = 4 MB を占める。数キロバイトしか使わないプログラムでも同じである。64ビットの仮想アドレス、4KBのページ、8バイトのエントリなら 2^{52} 個のエントリをもち、 2^{55} バイト (32 PB)、すなわちどのメモリにもディスクにも収まらない量を占めることになる。64ビットのアドレスのうち48ビットしか使わない場合でも、単一レベルページテーブルは $2^{36} \times 8 = 2^{39}$ バイト、すなわちプログラム1つにつき 512 GB を占める。

13.7 多段ページテーブル

単一レベルページテーブルが大きいのは、プログラムがアドレス空間の下端と上端しか使わないにもかかわらず、空間全体に対するエントリをもつからである。多段ページテーブルは、その間の使われない範囲に対するエントリを保持しないようにする。

定義 13.12 (多段ページテーブル). 多段ページテーブルとは、小さなテーブルからなる木である。VPNは段ごとに1つのフィールドに分割される。最初のフィールドは外側ページテーブルに添字を与え、そのエントリは次の段のテーブルを指す。最後の段のテーブル、すなわち内側ページテーブルのエントリがPFNを保持する。担当するアドレス空間の部分がまったく使われていないエントリは無効と記され、その下にはテーブルが存在しない。

2段の場合、仮想アドレスは外側の添字 p_1 、内側の添字 p_2 、ページオフセットに分かれ (図 13.3)、変換は2つの手順を踏む。

1. 外側ページテーブルのエントリ p_1 が、内側ページテーブルの位置を与える。
2. その内側ページテーブルのエントリ p_2 がPFNを与え、これが式 (13.1) のようにオフセットと組み合わされる。

どちらかのエントリが無効ならPFNは得られず、そのアクセスはページフォールトを引き起こす。テーブルの大きさは、通常、1つがちょうど1ページに収まるように定められる。

出典 Lesson 13, 動画 11–12 ; H&P §B.4.

▷ flat page table

▷ page table entry

出典 Lesson 13, 動画 13–17 ; H&P §B.4–B.5.

▷ multi-level page table

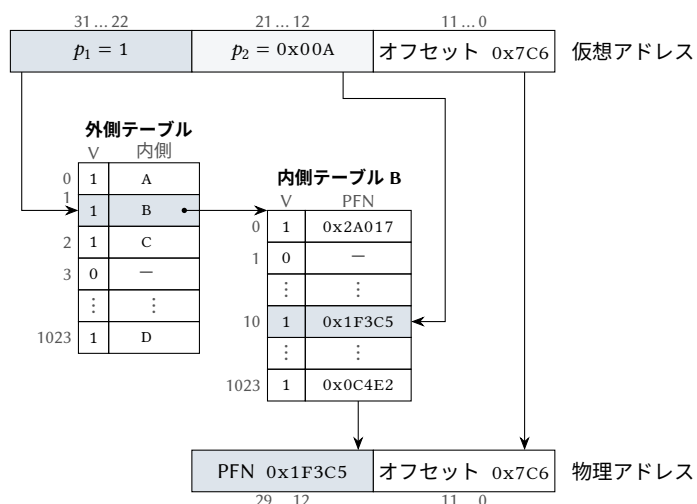


図 13.13: 仮想アドレス 0x0040A7C6 の 2 段の変換 (例 13.13)。外側のエントリ 1 は内側テーブル B を指す。A, C, D のエントリは図示していない内側テーブルを指す。エントリ 3 のような無効な外側エントリには、内側テーブルがそもそも存在しない。

例 13.13. 例 13.9 の 20 ビットの VPN を, 10 ビットの外側の添字 (ビット 31-22) と 10 ビットの内側の添字 (ビット 21-12) に分割する。4 バイトのエントリなら, どのテーブルも 2^{10} 個のエントリをもち 4 KB, すなわちちょうど 1 ページを占める。仮想アドレス 0x0040A7C6 の VPN は $0x0040A = 0000000001\ 0000001010_2$ なので, $p_1 = 1$, $p_2 = 0x00A = 10$ である。外側のエントリ 1 は内側テーブル B を指し, B のエントリ 10 が PFN 0x1F3C5 を保持する。物理アドレスは単一レベルページテーブルの場合と同じ 0x1F3C57C6 である。多段ページテーブルは, 同じ対応づけを別の形で格納しているにすぎない。

13.7.1 多段ページテーブルの大きさ

外側ページテーブルは常に存在するが, 内側ページテーブルが存在するのは, その担当するアドレス空間の部分で少なくとも 1 つのページが使われている場合だけである。例 13.13 の構成では, 内側テーブル 1 つが 2^{10} ページ, すなわち仮想アドレス空間の 4 MB を担当する。

例 13.14. あるプログラムが, 32 ビットのアドレス空間の下端の 10 MB をコード, データ, ヒープに (アドレス 0x00000000-0x009FFFFF), 上端の 1 MB をスタックに (0xFFFF0000-0xFFFFFFFF) 使うとする。例 13.13 の 2 段ページテーブルでは, 下端の 10 MB はそれぞれ 4 MB を担当する外側エントリ 0, 1, 2 の範囲に, スタックは外側エントリ 1023 の範囲に入る。ページテーブルは外側テーブル 1 つと内側テーブル 4 つからなり, 単一レベルページテーブルの 4 MB ではなく $5 \times 4\text{ KB} = 20\text{ KB}$ である。

2 段ページテーブルが単一レベルのものより大きくなるのは, すべての外側エントリに内側テーブルが必要な場合, すなわち 4 MB の 1024 個の範囲のそれぞれで少なくとも 1 つのページが使われている場合だけである。そのとき $4\text{ KB} + 1024 \times 4\text{ KB} \approx 4.004\text{ MB}$ を占め, 単一レベルのテーブルより外側テーブルの分だけ大きくなる。実際のプログラムはアドレス空間の連続したいくつかの領域を使うので, 必要な内側テーブルはわずかである。

13.7.2 64 ビットアドレスのためのより多くの段

64 ビットのアドレス空間には 2 段では足りない。8 バイトのエントリからなる 4 KB のテーブルは 2^9 個のエントリをもつので, 1 ページに収まるテーブルからなる各段が消費できるのは VPN の 9 ビットだけである。48 ビットの仮想アドレスでは VPN は 36 ビットである。9 ビットの内側テーブルをもつ 2 段ページテーブルなら 27 ビットの外側の添字が必要で, 外側テーブルはプログラム 1 つにつ

き $2^{27} \times 8$ バイト = 1 GB になる。代わりに 36 ビットを 9 ビットずつ 4 つのフィールドに分割すると、どのテーブルも 1 ページに収まる 4 段ページテーブルが得られる。x86-64 プロセッサはこの構成を用いる。段を 1 つ増やすごとに、仮想アドレスをさらに 9 ビット分収められるようになる。

例 13.15. このような 4 段ページテーブルでは、最後の段のテーブル 1 つが 2^9 ページ、すなわちアドレス空間の 2 MB を担当し、第 3 段のテーブルはその 2^9 倍の 1 GB を、第 2 段のテーブルは 512 GB を、そして第 1 段のただ 1 つのテーブルは 2^{48} バイト (256 TB) すべてを担当する。アドレス空間の下端で 8 MB、上端で 8 MB を使うプログラムは、それぞれの端について最後の段のテーブル 4 つと第 3 段および第 2 段のテーブルを 1 つずつ必要とし、それらはすべて第 1 段のただ 1 つのテーブルの下にぶら下がる。合計で $1 + 2 \times (4 + 1 + 1) = 13$ 個のテーブル、すなわち 13×4 KB = 52 KB である。単一レベルページテーブルなら 512 GB を占めることになる (例 13.11)。

要点

ページテーブルは仮想アドレス空間全体を記述しなければならないが、多段ページテーブルが記憶領域を必要とするのは、使われている部分についてだけである。その代償は、変換のたびに段の数だけテーブルを読むことである。

13.8 ページサイズの選択

ページをどれだけ大きくすべきかは、2 つの効果の折り合いで決まる。

- ・ **ページが大きい**ほどページ数は少なくなる。PTE が減ってページテーブルが小さくなり、さらに 13.10 節で示すように、プロセッサが保持する変換 1 つが覆うメモリが増える。
- ・ **ページが小さい**ほど、部分的にしか使われないページで無駄になるメモリが減る。

1 つのページと 1 つのフレームのバイト数である**ページサイズ**は、この両者の均衡をとらなければならない。

定義 13.16 (内部断片化). メモリはページ単位で対応づけられる。したがって、大きさがページサイズの倍数でない領域は、最後のページの一部を使わないまま残り、その部分は他のどのプログラムも使えない。このようにして失われるメモリを**内部断片化**という。

大きさが任意の領域は平均して半ページを使わないまま残すので、内部断片化はページサイズに比例して増える。

例 13.17. 4 バイトの PTE をもつ 32 ビットプロセッサ上のあるプログラムが、3 つの領域 (コードとデータ、ヒープ、スタック) をもち、そのそれぞれが部分的にしか使われないページで終わっているとする。表 13.1 は 3 つのページサイズを比較したものである。4 KB のページから 4 MB のページにすると、単一レベルページテーブルは 2^{10} 分の 1、すなわち 4 MB から 4 KB に縮むが、内部断片化の期待値は同じ倍率で、6 KB から 6 MB に増える。

表 13.1: 例 13.17 のプログラムについての、ページテーブルの大きさと内部断片化の期待値 (領域当たり半ページ)。

ページサイズ	ページ数	単一レベルページテーブル	内部断片化
4 KB	2^{20}	4 MB	3×2 KB = 6 KB
64 KB	2^{16}	256 KB	3×32 KB = 96 KB
4 MB	2^{10}	4 KB	3×2 MB = 6 MB

ページはキャッシュブロック (第 12 章) よりはるかに大きい。ページごとに PTE が必要であり、ディスクから持ち込むページごとにディスクアクセスがかかるからである。通常のページサイズは 4 KB である。プロセッサは、x86-64 の 2 MB や 1 GB のように、はるかに大きなページも備えている。オペレーティングシステムは、プログラムが完全に埋める大きな領域にこれを使うことができ、

出典 Lesson 13, 動画 18 ; H&P §B.4.

▷ page size

▷ internal fragmentation

その場合、内部断片化はほとんど問題にならない。

13.9 変換を伴うメモリアクセス時間

プログラムが行うメモリアクセスは、命令フェッチであれロードやストアであれ、すべて仮想アドレスを使うので、物理メモリにアクセスする前に変換されなければならない。ページテーブル自体もメモリに置かれているので、変換はメモリ読み出しからなる。2 段ページテーブルの場合、プロセッサは LW R1, 4(R2) のようなロードを 4 つの手順で実行する。

1. R2 に 4 を加えて仮想アドレスを求め、その VPN を取り出す。
2. 外側の PTE の物理アドレスを（外側テーブルのアドレスに p_1 と PTE の大きさの積を加えて）計算し、その PTE を読む。
3. 同じようにして、外側の PTE にあるテーブルのアドレスと p_2 から内側の PTE のアドレスを計算し、その PTE を読む。
4. PFN にオフセットを付け加え、この物理アドレスにあるデータにキャッシュを通じてアクセスする。

PTE の読み出しはいずれも通常のメモリアクセスであり、キャッシュを通り、ヒットすることもミスすることもある。PTE がキャッシュに保持されていなければ、1 回ごとに主記憶への完全なアクセスがかかる。L 段のページテーブルでは、プログラムのアクセス 1 回が $L+1$ 回のメモリアクセスになる。PTE の読み出しがデータアクセスと同じ平均メモリアクセス時間（AMAT）をもつとすれば、

$$\text{変換を伴うアクセス時間} \approx (L+1) \times \text{AMAT}. \quad (13.3)$$

例 13.18. あるキャッシュのヒット時間が 2 サイクル、ミス率が 5%，ミスペナルティが 40 サイクルなら、 $\text{AMAT} = 2 + 0.05 \times 40 = 4$ サイクルである（式 (12.1)）。変換がなければメモリアクセスは平均 4 サイクルだが、単一レベルページテーブルでは $2 \times 4 = 8$ サイクル、4 段ページテーブルでは $5 \times 4 = 20$ サイクルになる。個々のアクセスのばらつきは大きい。上位 3 段の PTE がヒットし（ 3×2 サイクル）、最後の段の PTE がミスし（ $2 + 40$ サイクル）、データがヒットする（2 サイクル）ロードは 50 サイクルかかり、これは変換がなければキャッシュヒットで済んだ時間の 25 倍である。

上位の段のわずかなエントリはどの変換でも読まれるので、たいていキャッシュに保持される。数の多い最後の段のエントリはより頻繁にミスする。いずれにせよ、より速く変換する方法がなければ、仮想記憶はすべての命令フェッチ、ロード、ストアを大きな倍率で遅くしてしまう。

13.10 TLB

あるページにアクセスしたプログラムは、近いうちに同じページに再びアクセスする可能性が高いので、ある時点で必要な変換はわずかであり、それが何度も繰り返し使われる。そこでプロセッサは、それをキャッシュする。

定義 13.19 (TLB). TLB とは、最近の変換を保持する小容量で高速なキャッシュである。各エントリは、有効ビット、VPN から取ったタグ（フルアソシアティブの TLB では VPN 全体）、そのページが対応づけられている PFN、およびその PTE のアクセスビットを保持する。アドレスの VPN が有効なエントリと一致すれば、PFN はそのエントリから得られ、ページテーブルは読まれない。

TLB は第 12 章の意味でのキャッシュであるが、データキャッシュよりはるかに小さくできる。理由は 3 つある。

- 格納するのは変換だけでデータではない。1 エントリは数バイトで済む。
- 1 つのエントリが、1 つのキャッシュブロックではなく 1 ページ全体に対応するので、わずかなエントリでプログラムが現に使っているメモリを覆える。
- 格納するのは変換の最終結果であって、個々の段のエントリではない。ページテーブルが何段であろうと、TLB ヒットは 1 回の探索で済む。

出典 Lesson 13, 動画 19–21 ; H&P §B.4.

出典 Lesson 13, 動画 22–24 ; H&P §B.4.

▷ translation lookaside buffer

例 13.20. 64 バイトのブロックをもつ 128 KB のデータキャッシュには 2048 本のラインがある。4 KB のページなら、 $128/4 = 32$ 個の TLB エントリでキャッシュが保持するのと同じだけのメモリを覆える。キャッシュされたブロックがすべて別々のページにある場合には 2048 個が必要になるが、空間的局所性により実際に必要な数はこの上限をはるかに下回る。

13.10.1 TLB ミス

アドレスの VPN に一致する有効な TLB エントリがないとき、**TLB ミス**が生じる。このとき変換はページテーブルから得なければならない。

定義 13.21 (ページテーブルウォーク). ページテーブルウォークとは、13.7 節で述べたように、あるアドレスのページテーブルエントリを、外側ページテーブルから PFN を保持するエントリまで段ごとに読むことである。

TLB ミスの後、ウォークが PFN を見つけ、その変換が TLB に書き込まれ (TLB が満杯なら既存のエントリを置き換え)、アクセスが進む (図 13.4)。

TLB はページテーブルと一致していなければならない。オペレーティングシステムが対応づけを変更したとき、例えばページを追い出したときには、そのページの TLB エントリを無効化する。また、プロセッサが別のプログラムに切り替わるときには、前のプログラムのエントリを無効化するか、プログラムの識別子をタグとして付けておく。

▷ TLB miss

▷ page table walk

注意 TLB ミスはページフォールトではない。ウォークはたいてい有効な PTE を見つける。

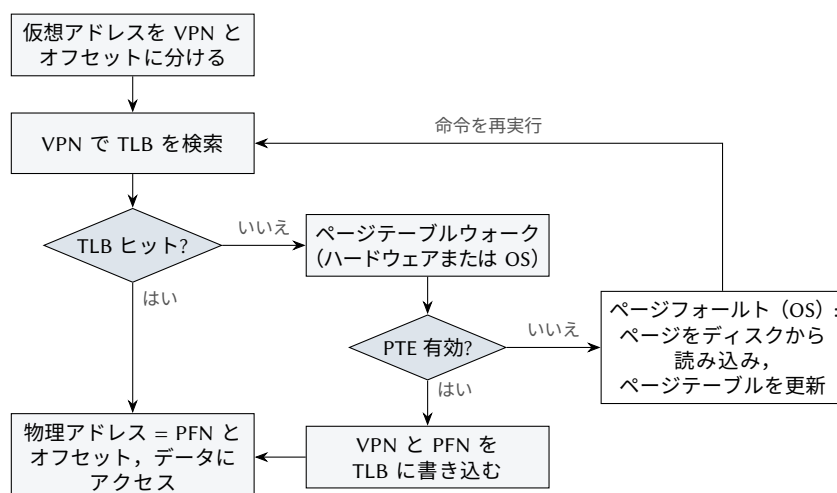


図 13.4: メモリアクセスにおける変換。TLB ヒットならただちに PFN が得られ、TLB ミスならページテーブルウォークが必要になる。PTE が無効であればページフォールトとなり、ページがメモリに入った後に命令が再実行される。

ウォークは 2 通りの方法で行える。

ソフトウェアによる方法 TLB ミスが例外を起こし、オペレーティングシステム内のハンドラがページテーブルをたどって変換を TLB に書き込む。ハードウェアは単純で済み、ページテーブルをどのように構成してもよいが、TLB ミスのたびに例外の代償を払う。MIPS プロセッサは TLB ミスをこの方法で処理する。

ハードウェアによる方法 プロセッサがページテーブルウォーク、すなわち PTE を読んで自ら TLB を埋める論理回路を内蔵する。TLB ミスの処理ははるかに速いが、ページテーブルはハードウェアが想定する形式でなければならない。x86 系のような高性能プロセッサは、ページテーブルウォークをハードウェアで行う。

要点

どのメモリアクセスにも変換が必要である。TLB は、よくある場合である TLB ヒットを 1 回の高速な探索で済ませる。ページテーブルウォークは、ハードウェアによるものであれオペレーティングシステムによるものであれ TLB ミスのときだけ必要であり、ページフォールトハンドラは、ウォークが PFN

を見つけられなかったときだけ必要である。

13.11 TLB の構成と性能

TLB の構成は、その役割から導かれる。

連想度 TLB ミスは 1 回ごとに数回のメモリ読み出しからなるウォークを要するので、TLB はフルアソシアティブか、非常に連想度の高いセットアソシアティブとする。これだけ少ないエントリ数であれば比較器の費用は許容でき、変換どうしがエントリを奪い合うこともなくなる。

サイズ TLB のヒット率はキャッシュと同程度であるべきなので、キャッシュが保持するデータのページを覆えなければならない (例 13.20)。これには 64 個から 512 個ほどのエントリが必要である。すべてのアクセスで参照されるので、L1 キャッシュのヒットと同程度に高速でもなければならず、そのためこれ以上大きくできない。

階層 それで十分なメモリを覆えない場合は、キャッシュを階層化するのと同じように (第 14 章)、数百から数千のエントリをもつ、より大きく低速な第 2 レベルの TLB を第 1 レベルの後ろに置く。これは第 1 レベルでミスしたときにだけ参照され、両方の TLB がミスしたときにだけウォークが必要になる。

例 13.22. 例 13.18 のプロセッサ (どのメモリ読み出しも AMAT は 4 サイクル、4 段ページテーブル) に、1 サイクルで参照でき、アクセスの 1% でミスする TLB を与える。どのアクセスも探索とデータアクセスの代償を払い、100 回に 1 回は $4 \times 4 = 16$ サイクルのウォークの代償も払うので、アクセス 1 回は平均して 20 サイクルではなく $1 + 0.01 \times 16 + 4 = 5.16$ サイクルかかる。変換の費用は 16 サイクルではなく 1.16 サイクルとなり、その大半は探索の分である。第 14 章では、この探索をキャッシュの探索と重ね合わせる。

要点 – 章のまとめ

仮想記憶は、各プログラムの仮想アドレス空間のページを物理メモリのフレームに対応づけ、残りをディスクに置く。フレームのないページへのアクセスはページフォールトである。変換は、仮想ページ番号を、ページテーブルから得たフレーム番号で置き換え、オフセットはそのまま残す (式 (13.1))。多段ページテーブルは、単一レベルのテーブルが使われないアドレス空間に費やすエントリを不要にする (式 (13.2))。ページサイズはテーブルの大きさと内部断片化のトレードオフを決める。どの段もアクセス 1 回につきメモリ読み出しを 1 回加えるので、小さく連想度の高い TLB が変換をキャッシュし、ページテーブルウォークは TLB ミスのときだけ行われる。

出典 Lesson 13, 動画 25–26 ; H&P §B.4.

高度なキャッシュ

講義動画	Lesson 14, 動画 1-40
教科書	H&P [1] §2.3, §B.1, §B.3
キーワード	パイプライン化キャッシュ, VIPT キャッシュ, エイリアシング, ウェイ予測, NMRU, 疑似 LRU, 競合性ミス, プリフェッチ, ループ交換, ミスアンダーミス対応, MSHR, キャッシュ階層, 大域ミス率, 包含性

第 12 章では基本的なキャッシュを述べ、平均メモリアクセス時間でそれを測った。第 13 章では、仮想記憶をもつシステムでどのメモリアクセスにも必要となるアドレス変換を加えた。この章では、実際のキャッシュを高速にする技法をまとめる。技法は、平均メモリアクセス時間のどの項を下げるかによって整理する。すなわち、ヒット時間（パイプライン化キャッシュ, TLB に対するキャッシュの置き方, ウェイ予測, 安価な置換アルゴリズム）、ミス率（ブロックの大型化, プリフェッチ, ループ交換）、そしてミスペナルティ（重なったミスの処理と、何よりも多段のキャッシュ階層）である。

14.1 キャッシュ性能の改善

改善すべき量は、第 12 章の平均メモリアクセス時間（AMAT）である。

$$AMAT = \text{ヒット時間} + \text{ミス率} \times \text{ミスペナルティ} \tag{14.1}$$

3 つの項のそれぞれに固有の攻め方があり、表 14.1 にはこの章の技法を、対象とする項ごとに挙げる。代償を伴わないものはほとんどない。一方の項を下げる技法は、しばしば別の項を上げる—キャッシュを小さくすればヒット時間は短くなるがミス率は高くなる—ので、AMAT 全体が減る場合にのみ価値がある。

表 14.1: この章の技法を、式 (14.1) のどの項を下げるかで分類したもの。

項	技法	節
ヒット時間	パイプライン化キャッシュ, VIPT キャッシュ, ウェイ予測, NMRU 置換と PLRU 置換	14.2-14.6
ミス率	ブロックの大型化, プリフェッチ, ループ交換	14.7-14.10
ミスペナルティ	ミスアンダーミス対応, キャッシュ階層	14.11-14.14

14.2 パイプライン化キャッシュ

ヒット時間を下げる最も直接的な方法は、キャッシュを小さく単純に—たとえばダイレクトマップに—保つことである。デコードすべきラインが少なく、比較すべきタグも少なく済む。しかし小さなキャッシュはミスが多く、ダイレクトマップキャッシュは競合を多く受けるので、ヒットで節約した時間はミスで容易に失われる。この節と続く 4 つの節の技法は、ミス率を犠牲にすることなく、ヒット時間またはその代償を減らす。

第 12 章で述べたように、L1 のヒットには 1~3 クロックサイクルかかる。数サイクルを要する探索が、完了するまでキャッシュを占有するとすれば、どのメモリアクセスも直前のアクセスを待たなければならない。アクセスから見たヒット時間は自身の探索時間にこの待ち時間を加えたものとなり、連続するロードとストアはプロセッサを止めてしまう。探索が 1 サイクルに収まるキャッシュにはこの問題はない。2~3 サイクルかかる L1 キャッシュは、通常パイプライン化される。

定義 14.1 (パイプライン化キャッシュ). パイプライン化キャッシュとは、探索を

出典 Lesson 14, 動画 1-2 ; H&P §2.3.

出典 Lesson 14, 動画 3-4 ; H&P §2.3.

▷ pipelined cache

ラッチで区切られた段に分割し、1 回の探索には数クロックサイクルかかるが、毎サイクル新しい探索を開始できるようにしたキャッシュである。

セットアソシアティブキャッシュでは、探索は自然に 3 つの手順に分かれ、そのそれぞれが段になりうる (図 14.1)。

1. インデックスがセットを選び、そのセットのラインの有効ビットとタグを読み出す。
2. タグをアドレスのタグと比較して、ヒットかミスかを決めるとともに一致したウェイを特定し、データの読み出しを開始する。
3. データの読み出しが完了し、一致したウェイのデータが選択され、ブロックオフセットがブロックから要求されたバイトを取り出して引き渡す。

ミスは第 2 段の終わりに判明する。パイプライン化は、1 回の探索にかかるサイクル数を減らすものではない。そのサイクル数がアクセスを発行できる頻度を制限しないようにするものである。あるアクセスがタグを比較している間に、次のアクセスはすでに自身のセットを読み出ししており、これは第 3 章のパイプラインで命令が重なるのとまったく同じである。そしてクロック周期は、探索全体ではなく最も遅い段だけを収めればよい。

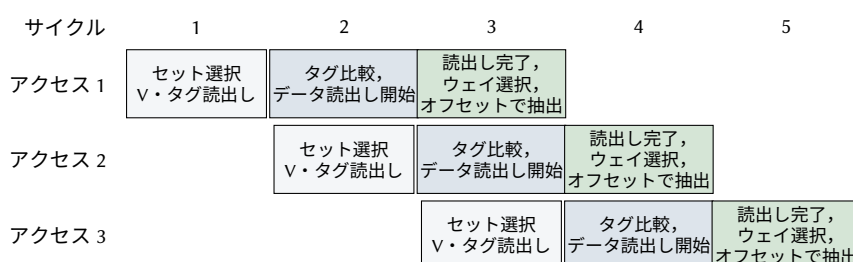


図 14.1: 探索を 3 段に分割したキャッシュへの、連続する 3 回のアクセス。各アクセスには 3 サイクルかかるが、新しいアクセスが毎サイクル開始される。

例 14.2. ある L1 キャッシュの探索論理の遅延は 0.75 ns であり、プロセッサの他の段は 0.3 ns のクロック周期を許す。パイプライン化しなければ、プロセッサ全体のクロック周期が 0.75 ns 以上に伸びるか、各アクセスが 3 サイクルにわたってキャッシュを占有してメモリアクセスが 3 サイクルに 1 回しか開始できなくなるかである。ラッチのオーバーヘッドの余地を残して約 0.25 ns の 3 段に分割すれば、探索は 0.3 ns のクロックに収まる。データはアクセス開始の 3 サイクル後に到着し、アクセスは毎サイクル開始できる。

14.3 アドレス変換とヒット時間

仮想記憶 (第 13 章) のもとでは、プロセッサは仮想アドレスを発行するが、主記憶のデータは物理アドレスで位置づけられる。キャッシュの探索は、変換の後に行うことも、前に行うことも、一部を変換と並列に行うこともでき (図 14.2)、この選択がヒット時間に影響する。

出典 Lesson 14, 動画 5-7 ; H&P §B.3.

14.3.1 物理アドレスキャッシュ

物理アドレスで探索されるキャッシュを**物理アドレスキャッシュ**という。どのアクセスもまず TLB で仮想アドレスを変換し、得られた物理アドレスだけが探索に使われる (図 14.2a)。したがってヒット時間には変換が含まれる。

▷ physically accessed cache

$$\text{ヒット時間} = \text{TLB の探索時間} + \text{キャッシュの探索時間} \quad (14.2)$$

TLB は小さく高速だが、その遅延はヒットを含むすべてのアクセスで支払われ、まさに我々が減らそうとしている量に上乗せされる。

14.3.2 仮想アドレスキャッシュ

定義 14.3 (仮想アドレスキャッシュ). **仮想アドレスキャッシュ**は、インデックス

▷ virtually accessed cache

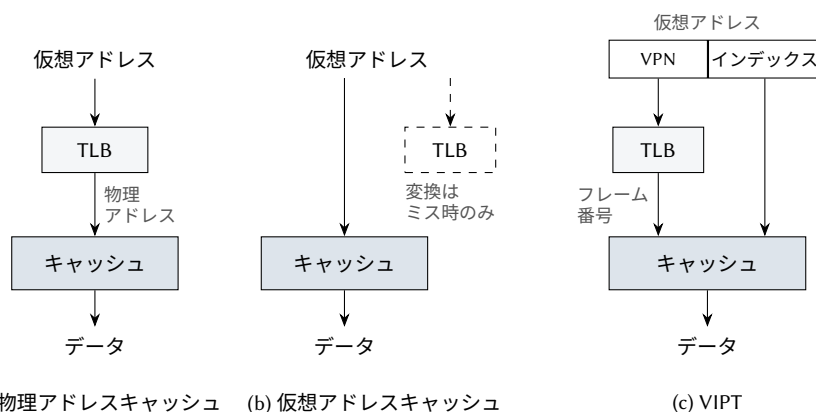


図 14.2: TLB とキャッシュ探索を組み合わせた 3 つの方法. (b) では変換はミス時にだけ必要なので、TLB はヒットの経路に入らない（権限の確認のためには依然として参照される）。(c) では、インデックスがすでにセットを選んでいる間に TLB が仮想ページ番号を変換する。

もタグも仮想アドレスから取る。アドレスが変換されるのは、ブロックを主記憶から取得しなければならないミスときだけである。

ヒット時には TLB は経路に入らず、ヒット時間はキャッシュの探索時間そのものである（図 14.2b）。この利点には問題が伴う。

- ・ **権限.** ページの TLB エントリは、そのページを読めるか、書けるか、実行できるかも記録しており、権限はすべてのアクセスで確認しなければならない。そのため実際には、ヒット時にも TLB が参照される。
- ・ **コンテキストスイッチ.** 各プロセスは自分の仮想アドレス空間をもつので、同じ仮想アドレスが異なるプロセスでは異なるデータを指す。オペレーティングシステムがプロセッサをあるプロセスから別のプロセスへ切り替える（コンテキストスイッチ）と、キャッシュ内のブロックは古いプロセスの仮想アドレスで識別されており、新しいプロセスがそれらを見つけてはならない。すべての有効ビットを落としてキャッシュをフラッシュするか、すべてのタグにアドレス空間の識別子を付け加えるかである（H&P §B.3）。コンテキストスイッチは毎秒何度も起こり、フラッシュする方式では、次に走るプロセスが毎回、空のキャッシュとミスの多発から始めることになる。
- ・ **エイリアシング.** これは 14.4 節の主題である。

14.3.3 仮想インデックス物理タグキャッシュ

両者を組み合わせると、短いヒット時間を保ちながらフラッシュを避けられる。

定義 14.4 (VIPT キャッシュ). 仮想インデックス物理タグキャッシュ (VIPT キャッシュ) は、インデックスを仮想アドレスから、タグを物理アドレスから取る。

▷ virtually indexed physically tagged cache

仮想アドレスのインデックスビットが直ちにセットを選ぶ（図 14.3）。そのセットの有効ビットとタグが読み出されている間に、TLB が仮想ページ番号を物理フレーム番号に変換し、物理的な値であるセットのタグが TLB からのフレーム番号と比較される。

- ・ ヒット時間は本質的にキャッシュの探索時間である。TLB の探索はセットの選択と並列に走り、タグが比較される時点までにフレーム番号を届けられればよく、小さな TLB なら通常これができる。
- ・ コンテキストスイッチでキャッシュをフラッシュする必要がある。タグはブロックの由来する物理メモリを表しており、どのプロセスが走っていてもそれは正しいままである。
- ・ いずれにせよ TLB はすべてのアクセスで参照されるので、権限の確認に追加の費用はかからない。

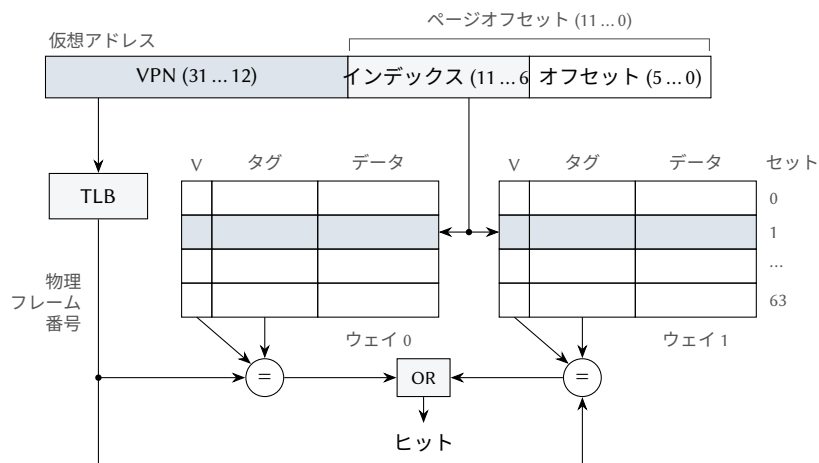


図 14.3: 4 KB のページ、64 バイトのブロック、64 セット (8 KB) をもつ 2 ウェイ VIPT キャッシュにおける探索。インデックスは仮想アドレスのページオフセットから取られる。タグは物理フレーム番号であり、TLB の出力と比較される。ラインは、有効ビットが立っていてタグが一致すればヒットする。

14.4 エイリアシング

仮想記憶では、同じ物理メモリが複数の仮想アドレスに現れうる。

定義 14.5 (エイリアシング) ．同じアドレス空間内の、あるいは異なるアドレス空間の 2 つ以上の仮想アドレスが同じ物理アドレスに対応づけられる状況を**エイリアシング**という。このとき、それらの仮想アドレスは互いにエイリアスである。

エイリアシングはよく起こる。メモリを共有するプロセスは、通常は異なる仮想アドレスで同じフレームを見るし、1 つのプロセスが 1 つのフレームを 2 つのアドレスに対応づけることもある。仮想アドレスキャッシュでは、2 つのエイリアスは単に 2 つの異なるアドレスである。一般にインデックスもタグも異なるので、キャッシュは同じ物理ブロックの写しをエイリアスごとに 1 つずつ保持でき、写し同士を結びつけるものは何もない。

例 14.6．仮想アドレス V1 と V2 は物理アドレス P のエイリアスであり、仮想アドレスキャッシュでは両者のインデックスが異なる。P のメモリには値 5 が入っている。表 14.2 は 4 回のアクセスを追う。最後のアクセスは V2 を通して 5 を読むが、同じメモリには V1 を通して 7 が書き込まれている。ライトスルー方式であれば主記憶は 7 を保持するが、それでも V2 用の古い写しが返される。

表 14.2: ライトバック方式の仮想アドレスキャッシュにおける 2 つのエイリアス (例 14.6)．各アクセスの後の、2 つの写しと主記憶の内容。

手順	アクセス	結果	V1 の写し	V2 の写し	P のメモリ
1	V1 を読む	ミス	5	—	5
2	V2 を読む	ミス	5	5	5
3	V1 に 7 を書く	ヒット	7	5	5
4	V2 を読む	ヒット, 5 を返す	7	5	5

正しく動作するためには、どの書き込みも同じ物理ブロックのキャッシュ内の他のすべての写しを見つけ出し、更新するか無効化しなければならないが、キャッシュにはそれらを効率的に探す手段がない。したがってエイリアシングは仮想アドレスキャッシュのもう 1 つの問題である。

14.4.1 VIPT キャッシュにおけるエイリアシング

VIPT キャッシュではタグが物理的なので、ブロックの 2 つの写しは同じタグをもつことになる。それでも両者は異なるセットに入りうる。インデックスが仮想アドレスから取られるからである。それが起こりうるかどうかは、インデックス

出典 Lesson 14, 動画 8–11 ; H&P §B.3.

▷ aliasing

ビットがどこにあるかで決まる。変換は仮想ページ番号をフレーム番号に置き換え、ページオフセットはそのまま残す。したがってエイリアスは、仮想ページ番号こそ異なるが、ページオフセットは常に等しい。

- インデックスが仮想ページ番号のビットを含む場合、2つのエイリアスは異なるセットを選ぶるので、キャッシュは2つの写しを保持でき、例 14.6 の問題が残る。
- インデックスがページオフセットのビットだけからなる場合、エイリアスは常に同じセットを選ぶ。そこには自分のブロックの物理タグをもつラインが1つあるだけなので、写しはたかだか1つしか存在せず、エイリアシングは無害である。このとき仮想アドレスのインデックスビットは物理アドレスのインデックスビットでもあり、キャッシュは物理アドレスキャッシュとまったく同じに、ただしより高速に振る舞う。

14.4.2 エイリアシングの回避

したがって VIPT キャッシュは、ブロックオフセットとインデックスがページオフセットに収まるとき、エイリアシングから自由である。

$$\text{オフセットビット数} + \text{インデックスビット数} \leq \text{ページオフセットビット数}. \quad (14.3)$$

左辺は $\log_2(\text{セット数} \times \text{ブロックサイズ})$ 、右辺は $\log_2(\text{ページサイズ})$ なので、この条件は $\text{セット数} \times \text{ブロックサイズ} \leq \text{ページサイズ}$ と読める。両辺に連想度 N を掛けると、キャッシュサイズの上界が得られる。

$$\text{キャッシュサイズ} \leq N \times \text{ページサイズ}. \quad (14.4)$$

ブロックサイズは消えた。VIPT キャッシュを制限するのは連想度とページサイズだけである。

例 14.7. 4 KB のページではページオフセットは 12 ビットである。64 バイトのブロック（オフセット 6 ビット）をもつ VIPT キャッシュは、インデックスを最大 6 ビット、すなわち 64 セットまでしか使えない。2 ウェイならこれは図 14.3 の 8 KB のキャッシュであり、4 ウェイなら 16 KB である。16 KB の 2 ウェイキャッシュは 128 セットを必要とし、その 7 番目のインデックスビットは仮想アドレスのビット 12、すなわち仮想ページ番号の最下位ビットとなるので、エイリアスが異なるセットに置かれうる。

14.4.3 実際の VIPT キャッシュ

ページサイズはアーキテクチャとオペレーティングシステムが定めるもので、典型的には 4 KB である。式 (14.4) により、VIPT であり、かつエイリアシングから自由であろうとする L1 キャッシュは、連想度を上げることによってしか大きくできない。ダイレクトマップなら 4 KB が限界であり、32 KB のキャッシュには 8 ウェイが必要である。Intel の Core 2, Nehalem, Sandy Bridge, Haswell はいずれも 32 KB の 8 ウェイ L1 データキャッシュをもち、ちょうどこの上界にある。したがって実際の L1 キャッシュは、ヒット時間だけでは正当化できないほど連想度が高く、そのことが次の 2 つの節の技法を重要にしている。

要点

VIPT キャッシュは、TLB が並列に変換する間に仮想アドレスでインデックスし、物理タグを比較する。ヒット時間は仮想アドレスキャッシュとほぼ同じで、コンテキストスイッチでのフラッシュを必要とせず、キャッシュサイズ $\leq N \times \text{ページサイズ}$ である限りエイリアシングの影響を受けない。

14.5 ウェイ予測

連想度はヒット時間とミス率を逆向きに動かす。セット当たりのウェイが多いほど競合は減ってミス率は下がり—VIPT キャッシュでは許されるサイズも大きくなる（式 (14.4)）—が、探索ではより多くのタグを比較し、より多くのラインから

選択しなければならないので、ヒット時間は長くなる。ダイレクトマップキャッシュはヒット時間が最も短く、ミス率が最も高い。目標は、セットアソシアティブキャッシュのミス率を、ダイレクトマップキャッシュのヒット時間で得ることである。

定義 14.8 (ウェイ予測). ウェイ予測を行うセットアソシアティブキャッシュは、選ばれたセットのどのラインが要求されたブロックを保持しているかを推測し、まずそのラインだけを調べる。そのタグが一致すれば、アクセスはダイレクトマップキャッシュのヒット時間で完了する。そうでなければ、通常のセットアソシアティブキャッシュと同様にセットの他のラインを調べる。

サイズ C の N ウェイキャッシュの最初の確認はセット当たり 1 つのラインを調べるので、同じセットをもつサイズ C/N のダイレクトマップキャッシュのように振る舞い、そのようなキャッシュのヒット率が、推測の当たる頻度を見積もる。推測がセット内で最も最近アクセスされたラインであれば、この見積もりは厳密である。そのラインはセット内で最も最近アクセスされたブロックを保持しており、それはまさにサイズ C/N のダイレクトマップキャッシュが保持するブロックだからである。推測が外れれば、素早い確認に加えて通常の探索の分も費やし、ミスもまた通常の探索の後にしか判明しない。予測されたラインがヒットするアクセスの割合を p 、1 つのラインを調べる時間を t_1 、セット全体の探索時間を t_N とすると、平均探索時間は次で与えられる。

$$t = pt_1 + (1-p)(t_1 + t_N) = t_1 + (1-p)t_N. \quad (14.5)$$

例 14.9. ある L1 キャッシュの後ろには L2 キャッシュがあり、L1 のミスペナルティは 20 ns である。3 つの L1 の設計を比較する。4 KB のダイレクトマップキャッシュは、ウェイ予測が最初に調べる 8 ウェイキャッシュの部分集合なので、予測されたラインは全アクセスの $p = 80\%$ でヒットし、残りの 20% (他のラインでのヒットとすべてのミス) は $0.8 + 1.5 = 2.3$ ns を要する。

L1 キャッシュ	探索時間	ミス率	AMAT
4 KB ダイレクトマップ	0.8 ns	20 %	$0.8 + 0.20 \times 20 = 4.8$ ns
32 KB 8 ウェイ	1.5 ns	5 %	$1.5 + 0.05 \times 20 = 2.5$ ns
32 KB 8 ウェイ, 予測あり	0.8 または 2.3 ns	5 %	$1.1 + 0.05 \times 20 = 2.1$ ns

式 (14.5) により、予測ありの平均探索時間は $0.8 + 0.2 \times 1.5 = 1.1$ ns である。ウェイ予測は 8 ウェイキャッシュのミス率を保ったまま、その平均探索時間をダイレクトマップキャッシュのそれに近づける。

14.6 置換アルゴリズムとヒット時間

置換アルゴリズムはミス率だけでなくヒット時間にも影響する。第 12 章のアルゴリズムのうち 2 つが両極端をなす。

ランダム置換 ヒット時には何もせず、ミス時に犠牲を無作為に選ぶ。ヒット時には何も加えないが、これから使われるブロックを追い出してしまうことがあるので、ミス率は高くなる。

LRU はミス率が低いが、各ラインに $\log_2 N$ ビットのカウンタを保持し、ヒットであれミスであれ 1 回のアクセスがセットの N 本すべてのラインのカウンタを変えうる。すべてのアクセスでのこの活動は論理とエネルギーとヒット時間を費やし、連想度が高いほどその度合いは大きい。

以下のアルゴリズムは、はるかに少ない管理情報で LRU のミス率に近づく。

14.6.1 NMRU

定義 14.10 (NMRU). NMRU 方式は、各セットについてどのラインが最も最近アクセスされたかだけを記録し、ミス時にはそれ以外のラインの 1 つを無作為に選んで追い出す。

時間的局所性により、最も最近使われたブロックは次にアクセスされる可能性が最も高い。それを追い出すことは、ランダム置換がなしうる最も有害な選択で

▷ way prediction

出典 Lesson 14, 動画 16–19 ; H&P §B.1.

▷ not most recently used

あり、NMRU はまさにその選択を除外する。費用はセット当たり $\log_2 N$ ビットのポインタ 1 つであり、1 回のアクセスが変えるのはただかこのポインタだけである。8 ウェイキャッシュには、LRU の $8 \times 3 = 24$ ビットではなく、セット当たり 3 ビットで足りる。NMRU は LRU よりいくらか多くミスする。その犠牲が、最も最近使われたものよりわずかに古いだけのブロックでありうるからである。

14.6.2 疑似 LRU

定義 14.11 (PLRU). 疑似 LRU (PLRU) 方式は、ライン当たり 1 ビットを初期値 0 で保持する。アクセスは自分のラインのビットを立てる。これによってセット内でまだ 0 であった最後のビットが立つと、他のすべてのラインのビットが落とされる。ミスは、ビットが 0 のラインを無作為に選んで追い出す。

ラインのビットは、そのラインが最後のリセット以降にアクセスされたかどうかを示す。最も最近使われたラインのビットは常に立っているので、PLRU の候補は常に NMRU の候補に含まれ、PLRU は NMRU と同等以上である。LRU にどれだけ近いかは、いくつのビットが立っているかによる。リセットの直後には、NMRU と同じく、最も最近使われたラインだけが除外される。 $N-1$ 個のビットが立っているときは、唯一の候補は最後のアクセスが最も昔であるラインであり、これはまさに LRU の選択である。費用はセット当たり N ビットで、1 回のアクセスはリセットの場合を除いて 1 ビットを立てるだけである。

例 14.12. ある 4 ウェイのセットが、ライン 0-3 にブロック A, B, C, D を保持している。最も最近使われたのは D であり、PLRU のビットは D へのアクセスで最後にリセットされた。表 14.3 は、3 つのアルゴリズムのもとでのこのセットを追う。E でのミスでは、LRU は A を追い出さなければならず、NMRU は A, C, D のいずれかを、PLRU は A か C を追い出さう。ここでは 3 つとも A を追い出す。次のアクセス F もミスすれば、LRU は C を追い出し、NMRU は B, C, D のいずれかを追い出さう。PLRU はビットが 0 のラインがライン 2 だけなので、やはり C を追い出す。

表 14.3: 1 つの 4 ウェイセットについて LRU, NMRU, PLRU が保持する状態 (例 14.12)。LRU の順は最も長く使われていないブロックから最も最近使われたブロックまで、NMRU は最も最近使われたラインの記録、PLRU はライン 0-3 のビットである。

アクセス	結果	ライン 0-3	LRU の順	NMRU	PLRU のビット
(初期状態)		A B C D	A B C D	3	0 0 0 1
B	ヒット	A B C D	A C D B	1	0 1 0 1
D	ヒット	A B C D	A C B D	3	0 1 0 1
B	ヒット	A B C D	A C D B	1	0 1 0 1
E	ミス	E B C D	C D B E	0	1 1 0 1

要点

すべてのアクセスで行われる作業は、すべてのヒットを長くする。ウェイ予測は他のラインより先に 1 つのラインだけを調べ、NMRU と PLRU は LRU のカウンタを、セット当たり 1 つのポインタ (NMRU) またはライン当たり 1 ビット (PLRU) に置き換える。いずれも、連想度や LRU によるミス率の利点の大半を、ヒット時間の代償のわずかな割合で保つ。

14.7 ミスの原因

ミス率を下げるには、まずミスがなぜ起こるかを知ることから始める。ミスは 3 種類に分類され、しばしば 3C と呼ばれる。

定義 14.13 (初期参照ミス, 容量性ミス, 競合性ミス). 初期参照ミスとは、あるブロックへの最初のアクセスで生じるミスである。これはどんなキャッシュでも、無限に大きなキャッシュでさえ生じる。

容量性ミスとは、同じサイズのフルアソシアティブキャッシュでも生じるであ

▷ pseudo-LRU

出典 Lesson 14, 動画 20; H&P §B.3; [24].

▷ compulsory miss

▷ capacity miss

ろうミスである。キャッシュがプログラムの使っているすべてのブロックを保持できず、場所が足りないためにそのブロックが追い出されていたのである。

競合性ミスとは、連想度が限られているためだけに生じるミスである。同じサイズのフルアソシアティブキャッシュならまだそのブロックを保持しているが、実際のキャッシュでは、同じセットを争う別のブロックによって追い出されていた。

▷ conflict miss

3つの個数は、同じアクセス列を3つのキャッシュでシミュレートすれば測定できる。無限のキャッシュのミスが初期参照ミスであり、実際のサイズのフルアソシアティブキャッシュがそれに容量性ミスを加え、実際のキャッシュがさらに競合性ミスを加える。

例 14.14. あるプログラムについて、無限のキャッシュは 1500 回、32 KB のフルアソシアティブキャッシュは 6000 回、実際の 4 ウェイ 32 KB キャッシュは 7800 回ミスする。7800 回のミスのうち 1500 回が初期参照ミス、 $6000 - 1500 = 4500$ 回が容量性ミス、 $7800 - 6000 = 1800$ 回が競合性ミスである。

この分類は対処法を示す。キャッシュを大きくすれば容量性ミスが減り、連想度を上げるか置換アルゴリズムを改善すれば競合性ミスが減るが、いずれもヒット時間を長くする。以下の節では、ヒットを長くすることなくミス率を下げる技法を述べる。

14.8 キャッシュブロックの大型化

与えられたサイズのキャッシュでは、ブロックサイズを大きくすることが、ミス率に相反する 2 通りの影響を与える。

- 空間的局所性が良い場合、アクセスされたアドレスの周辺のデータはすぐに使われる。ブロックが大きければ 1 回のミスでそれらを多くキャッシュに持ち込め、そうでなければミスしていた後続のアクセスがヒットするようになる。
- 空間的局所性が乏しい場合、大きなブロックの大半は一度も使われない。キャッシュが保持できるブロックの数が減り、したがって実際に必要とされるブロックも減って、容量性ミスと競合性ミスが増える。

そのためブロックサイズを大きくしていくと、ミス率はまず下がり、その後再び上がる（図 14.4）。最小値は、キャッシュが大きいほど大きなブロックサイズに現れる。たとえば 4 KB のキャッシュでは約 64 バイト、256 KB のキャッシュでは約 256 バイトである。大きなキャッシュはブロックが大きくても多くのブロックを収める余地があるので、使われないデータに費やす場所の害が小さい。

出典 Lesson 14, 動画 21-22 ; H&P §B.3.

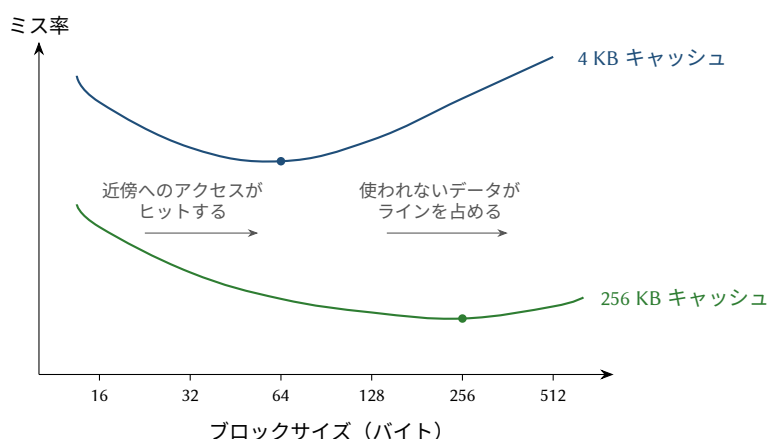


図 14.4: 4 KB と 256 KB のキャッシュについての、ブロックサイズの関数としてのミス率（形状のみ）。点は最良のブロックサイズを示す。

14.9 プリフェッチ

ブロックをプログラムが要求する前にキャッシュへ持ち込めば、ミスをもそもも回避できる。

定義 14.15 (プリフェッチ). 近いうちにアクセスされると予想されるブロックを、アクセスされる前にキャッシュへ読み込むことを**プリフェッチ**という。

プリフェッチは推測であり、その効果は推測が当たるかどうかで決まる。

- 推測が当たれば、そのブロックへの最初のアクセスはヒットになる。メモリからの取得自体は行われるが、プロセッサが他の仕事をしている間に前倒しで行われるので、そのアクセスがミスペナルティを払うことはない。
- 推測が外れれば、その取得は無駄になる。さらに悪いことに、プリフェッチされたブロックは別のブロックの場所を奪っており、そのブロックが後で必要になればミスする。外れた推測はキャッシュを汚染する。

プリフェッチが割に合うのは、推測の大半が当たる場合だけである。

14.9.1 プリフェッチ命令

プログラムは、次にどのデータにアクセスするかを知っていることが多い。**プリフェッチ命令**とは、与えられたアドレスを含むブロックをキャッシュへ持ち込むようプロセッサに求める ISA の命令であり、データは変更しない。コンパイラやプログラマは、あらかじめ分かっているアクセスの手前に、典型的には配列を巡るループの中にこれを置く [25]。

```
for (i = 0; i < n; i++) {
    prefetch(&a[i + D]);    /* D 反復先 */
    sum = sum + a[i];
}
```

難しいのはプリフェッチ距離 D である。

- D が小さすぎると、ループが要素 $i+D$ に達したときにプリフェッチが完了しておらず、そのアクセスは結局ブロックを待つので、得るものは少ない。
- D が大きすぎると、ブロックは使われるずっと前に到着する。その間ラインを占有し、必要になる前に再び追い出されるかもしれない。

適切な距離は、取得にかかる時間を 1 回の反復の時間で割ったもの程度である。どちらもマシンに依存するので、あるプロセッサに合う距離でコンパイルされたプログラムは、別のプロセッサではプリフェッチが遅すぎたり早すぎたりする。

例 14.16. プリフェッチされたブロックは、プリフェッチ命令の 50 ns 後にキャッシュに入り、上のループの 1 回の反復には 4 ns かかる。距離は少なくとも $50/4 = 12.5$ でなければならないので、距離 16 ならわずかな余裕がある。同じメモリで 1 回の反復を 2 ns で実行するプロセッサでは、ループが要素 $i+16$ に達するのはそのプリフェッチの 32 ns 後であり、ブロックの到着より 18 ns 早い。このときループはメモリを待たなければならない。およそ 16 回に 1 回のアクセスが 18 ns 待つので、16 回の反復には 32 ns ではなく約 50 ns かかり、速いはずのプロセッサはこのループを 1 反復当たり約 3.1 ns でしか実行できない。少なくとも 25 の距離が必要である。

14.9.2 ハードウェアプリフェッチ

ハードウェアプリフェッチでは、プロセッサ自身がメモリアクセスの列を観測し、どのブロックが必要になるかを推測する。プログラムを変更する必要はなく、推測はプログラムが走るマシンに適応する。3 つの設計が異なるアクセスパターンを利用する (図 14.5)。

ストリームバッファ ストリームバッファ [26] は逐次的なプリフェッチャである。ブロックが順にアクセスされると、続く数ブロックをキャッシュの脇の小さなバッファに取得する。アクセスはメモリアクセスなしにそこからデータを取得。そしてバッファが消費されるにつれて、さらに先のブロックを取得する。

ストライドプリフェッチャ ストライドプリフェッチャは、ループが配列の要素

出典 Lesson 14, 動画 23–26 ; H&P §2.3.

▷ prefetching

▷ prefetch instruction

注意 プリフェッチ距離は 1 つのマシンに合わせて調整されている。同じメモリでプロセッサが速くなれば、より大きな距離が必要になる。

▷ hardware prefetching

▷ stream buffer

▷ stride prefetcher

を k 個ごとに訪れる場合のように、一定の距離—ストライド—で進むアクセスを検出し、1 つ以上のストライド先のアドレスをプリフェッチする。

関連プリフェッチャ 関連プリフェッチャは、あるブロックについて、その次にどのブロックがアクセスされたかを記録し、そのブロックが再びアクセスされたときに、記録された後続ブロックをプリフェッチする。メモリ中に散在するノードをもち、繰り返し辿られる連結リストのような、規則的な間隔をもたないパターンを扱える。

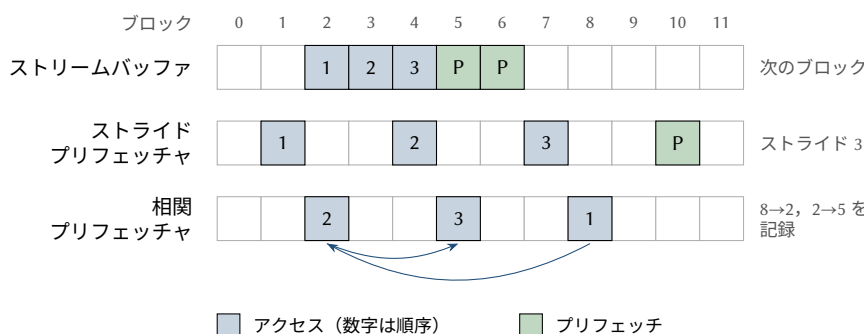


図 14.5: 3 つのハードウェアプリフェッチャが利用するアクセスパターン。図示した 3 回のアクセスの後、ストリームバッファは続くブロックを、ストライドプリフェッチャは 1 ストライド先のブロックを取得し、相関プリフェッチャは次にブロック 8 がアクセスされたときにブロック 2 を取得する。

14.10 ループ交換

コンパイラは、プログラムがデータにアクセスする順序を変えることによってミス率を下げられる。C は 2 次元配列を行ごとに格納する。 $m[j][i]$ と $m[j][i+1]$ は隣接するが、 $m[j+1][i]$ は 1 行分先にある。次のループの入れ子は、配列を列ごとに走査する。

```
double m[2000][1000];
for (i = 0; i < 1000; i++)
    for (j = 0; j < 2000; j++)
        m[j][i] = 2 * m[j][i];
```

内側のループのアクセスは、どれも直前のアクセスから 1000 要素、8000 バイト先へ飛ぶ。空間的局所性はまったくない。内側のループのアクセスはどれも異なるブロックに触れ、内側のループの 1 巡が訪れる 2000 個のブロックは L1 キャッシュが保持できる数をはるかに超えるので、ほとんどすべてのアクセスがミスする。

定義 14.17 (ループ交換) 入れ子になったループの順序を入れ替え、最も内側のループが配列の最後の添字を走ってデータが逐次的にアクセスされるようにするコンパイラ変換を**ループ交換**という。

交換の後、外側のループは j を、内側のループは i を走る。内側のループは隣接する要素を訪れるようになり、1 つのブロックが連続する複数のアクセスに応え、逐次プリフェッチャやストライドプリフェッチャが次のブロックを予測できる。コンパイラがループを交換してよいのは、結果が変わらないことを証明できる場合だけである。ここでは各反復が自分の要素だけを読み書きするので、反復はどの順序で実行してもよい。

要点

プリフェッチはブロックへの最初のアクセスのミスさえ回避できるが、役に立つのは推測が正しく、かつ間に合う場合だけである。ループ交換は、アクセスの順序をメモリ上のデータの配置に合わせることで、プログラムの側からミスを攻める。

▷ correlating prefetcher

出典 Lesson 14, 動画 27 ; H&P §2.3.

▷ loop interchange

14.11 ミスの重なり

ミスペナルティは、ブロックの取得に要する時間より大きくなりうる。アウトオブオーダープロセッサ（第 7 章）は多くの命令を同時に処理しており、そのロードとストアのいくつかは数サイクルのうちにメモリへアクセスしうる。欠けているブロックの取得には多くのサイクルがかかり、その間に別のアクセスもミスしうる。

定義 14.18 (オーバーラップミス). 先行するミスがまだ処理されている間に生じるキャッシュミスを**オーバーラップミス**という。

ロードがミスした後も、アウトオブオーダープロセッサは他の命令を実行し続ける。しかしそのロードはコミットできないので、プロセッサはやがて ROB エントリリザベーションステーションを使い果たして停止する。その時点より前に発行された 2 つ目のミスは、1 つ目と重なる。それがどれだけの代償になるかはキャッシュによる。

- ・ **ブロッキングキャッシュ**は、ミスを処理している間は他に何もしない。後続のアクセスは先行するミスが完了するまで探索さえされないで、重なったミスは自身のペナルティに加えて先行するミスの残り時間を払う (図 14.6a)。
- ・ **ノンブロッキングキャッシュ**は、ミスの処理中にもヒットに応え (ヒットアンダーミス)、さらに別のミスを開始することもできる。このとき複数の要求がメモリで同時に処理され、メモリが要求を並列に処理できる限り、メモリレベル並列性が活用される。

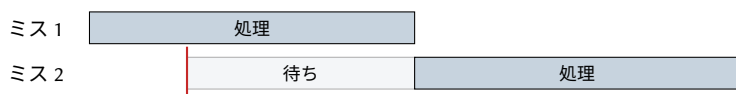
定義 14.19 (ミスアンダーミス対応). **ミスアンダーミス対応**をもつキャッシュは、複数のミスを同時に処理できる。処理中の各ミスを記録し、それらの要求をメモリ階層の次のレベルへ並行して送る。

処理中の各ミスは**ミス状態管理レジスタ (MSHR)**に記録される。キャッシュでミスしたアクセスは、まず自身のブロックアドレスを MSHR と比較する。

一致なし そのアクセスは新しいミスである。空いている MSHR を 1 つ確保し、そこにブロックと起こすべき命令を記録して、要求をメモリへ送る。

一致あり そのブロックはすでに到着の途中である。これを**ハーフミス**という。2 つ目の要求は送られず、命令は一致した MSHR に追加されるだけである。データが到着すると、それがキャッシュに置かれ、その MSHR に記録されたすべての命令が起こされ、MSHR が解放される。MSHR が 2 つあるだけでもブロッキングキャッシュより大きく改善し、4 つあればさらに良い。そしてメモリアクセスは非常に長くかかるので、16~32 個の MSHR でも効果が続く。

(a) ブロッキングキャッシュ



(b) ミスアンダーミス対応

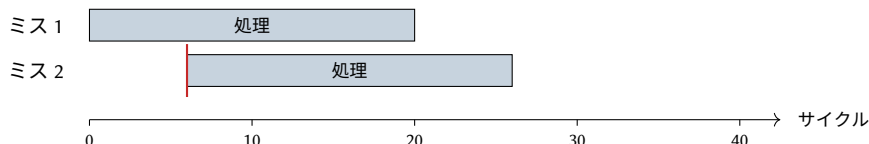


図 14.6: ミスペナルティが 20 サイクルの 2 つのミス。2 つ目は 1 つ目の 6 サイクル後に生じる (縦線)。(a) ブロッキングキャッシュでは、2 つ目のミスに 20 サイクルではなく 34 サイクルかかる。(b) ミスアンダーミス対応があれば、どちらのミスも 20 サイクルで済む。

ミスアンダーミス対応には追加のハードウェアが必要である。多くのメモリアクセスを同時に発行するプロセッサ、とりわけアウトオブオーダープロセッサで割に合う。

出典 Lesson 14, 動画 28-30.

▷ overlap miss

▷ blocking cache

▷ non-blocking cache

▷ miss-under-miss support

▷ miss status handling register

▷ half-miss

14.12 キャッシュ階層

L1 キャッシュのミスペナルティは、主記憶のアクセス時間に支配される。これは、そもそも L1 キャッシュを生んだのと同じ発想で下げられる。すなわち、L1 と主記憶の間にもう 1 つキャッシュを置く。L1 でミスしたアクセスは **L2 キャッシュ** を探索する。L2 は L1 より大きく遅いが、それでも主記憶よりはるかに速い。そして L2 でミスしたもののだけがさらに先へ進む。

定義 14.20 (キャッシュ階層) **キャッシュ階層**とは、L1, L2, L3, ... という複数のレベルのキャッシュからなり、各レベルは直前のレベルがミスしたときにだけ探索されるものである (図 14.7)。主記憶の直前のレベルを **最終レベルキャッシュ (LLC)** という。

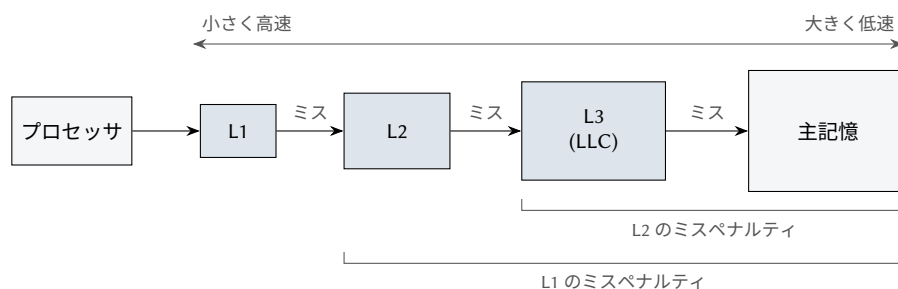


図 14.7: 3 レベルのキャッシュ階層。各レベルのミスペナルティは、その後ろにあるレベル群の平均アクセス時間である。

14.12.1 キャッシュ階層の AMAT

あるレベルのミスペナルティは、その後ろのレベルからブロックを得るのにかかる平均時間、すなわち階層の残りの部分の AMAT である。式 (14.1) をレベルごとに適用すると次が得られる。

$$\begin{aligned}
 \text{AMAT} &= h_1 + m_1 \times P_1, \\
 P_1 &= h_2 + m_2 \times P_2, \\
 &\vdots \\
 P_{\text{LLC}} &= \text{主記憶のアクセス時間},
 \end{aligned}
 \tag{14.6}$$

ここで h_i はレベル i のヒット時間、 m_i はミス率、 P_i はミスペナルティである。ミス率 m_i は、レベル i に到達したアクセスについて数える (14.13 節)。

14.12.2 L1 と L2

式 (14.6) では、L1 のヒット時間 h_1 はすべてのアクセスが払うが、L2 のヒット時間 h_2 は L1 でミスしたアクセスの割合 m_1 だけが払う。したがって L1 は、前の節までの技法によって短いヒット時間を目指して設計され、L2 はヒット時間が長くなる代わりにミス率を下げてよい。

例 14.21. 主記憶は 1 回のアクセスに 50 ns かかる。32 KB のキャッシュはヒット時間 1 ns、ミス率 10%、256 KB のキャッシュはヒット時間 5 ns、ミス率 3% である。階層の中でも 256 KB のキャッシュは単独の場合と同じアクセスでミスする、すなわちそこに到達するアクセスの $0.03/0.10 = 30\%$ でミスすると仮定する。

構成	AMAT
キャッシュなし	50 ns
32 KB キャッシュのみ	$1 + 0.10 \times 50 = 6 \text{ ns}$
256 KB キャッシュのみ	$5 + 0.03 \times 50 = 6.5 \text{ ns}$
L1 32 KB, L2 256 KB	$1 + 0.10 \times (5 + 0.30 \times 50) = 3 \text{ ns}$

出典 Lesson 14, 動画 31–34 ; H&P §B.3.

▷ L2 cache

▷ cache hierarchy

▷ last-level cache

この階層は、どちらのキャッシュ単独よりも 2 倍速い。すべてのアクセスが払う小さなキャッシュの短いヒット時間と、長いヒット時間をアクセスの 10% だけが払う大きなキャッシュの低いミス率とを兼ね備えている。

14.13 局所ミス率と大域ミス率

例 14.21 では、L2 キャッシュはそこに到達するアクセスの 30% でミスし、これは L1 の 3 倍の率である。だからといって L2 が劣ったキャッシュだということにはならない。L2 が見るのは L1 がミスしたアクセスだけであり、それらは局所性が最も弱いものである。応えるのが容易なアクセスは L1 より先へは行かない。したがって下位レベルのミス率は、その基準とともに述べなければならない。

定義 14.22 (局所率と大域率)。キャッシュの**局所ヒット率**とは、そのヒット数を、そこに到達したアクセス数で割ったものであり、**局所ミス率**は $1 - \text{局所ヒット率}$ である。

キャッシュの**大域ミス率**とは、そのミス数を、プロセッサのすべてのメモリアクセス数で割ったものであり、**大域ヒット率**は $1 - \text{大域ミス率}$ である。

L1 では局所率と大域率は一致する。下位のレベルでは、そこに到達するアクセスは上のレベルのミスなので、レベル n の大域ミス率は局所ミス率の積になる。

$$\text{大域ミス率}_n = \prod_{i=1}^n \text{局所ミス率}_i. \quad (14.7)$$

式 (14.6) のミス率は局所ミス率である。あるレベルの大域ミス率は、全アクセスのうちどれだけの割合がそのレベルより先へ進まなければならないかを述べるので、レベルや階層を比較するのに適した尺度である。

定義 14.23 (1000 命令あたりミス数)。**1000 命令あたりミス数 (MPKI)** という指標は、実行された 1000 命令当たりのキャッシュのミス数を数える。

大域ミス率と同様に MPKI もあるレベルのすべてのミスを数えるが、基準がメモリアクセスではなく命令である。

$$\text{MPKI} = 1000 \times \text{大域ミス率} \times \frac{\text{メモリアクセス数}}{\text{命令}}. \quad (14.8)$$

これにより、そのレベルのミスペナルティをプログラムが命令当たりどれだけの頻度で払うかが直接に分かる。

例 14.24。あるプログラムが 1 000 000 命令を実行し、メモリアクセスは 1 500 000 回である。L1 は 90 000 回、L2 は 27 000 回ミスする。

- L1: 局所ミス率と大域ミス率は $90\,000 / 1\,500\,000 = 6\%$ 、MPKI は $90\,000 / 1000 = 90$ 。
- L2: 局所ミス率は $27\,000 / 90\,000 = 30\%$ (局所ヒット率 70%)、大域ミス率は $27\,000 / 1\,500\,000 = 1.8\% = 0.06 \times 0.30$ (大域ヒット率 98.2%)、MPKI は 27。

要点

AMAT は局所ミス率で計算する。レベルや階層の比較には大域ミス率か MPKI を用いる。上のレベルのミスだけを見るレベルでは、局所ヒット率が低いのは当然だからである。

14.14 包含性

L1 にあるブロックは、L2 にもあるかもしれないし、ないかもしれない。レベル間の関係には 3 つの可能性がある。

包含 L1 にあるブロックはすべて L2 にもある。

排除 L1 にあるブロックは L2 にはない。

どちらでもない L1 にあるブロックの一部は L2 にもあり、他はない。

出典 Lesson 14, 動画 35–37; H&P §B.3.

▷ local hit rate
▷ local miss rate

▷ global miss rate
▷ global hit rate

▷ misses per 1000 instructions

出典 Lesson 14, 動画 38–40; H&P §B.3; [27].

定義 14.25 (包含性). あるレベルにあるブロックがすべて、次の、より大きなレベルにも存在するとき、そのキャッシュ階層は**包含性**をもつという。

▷ inclusion property

LRU 置換をもつ大きな L2 なら、L1 のブロックを自動的にすべて保持しように思える。そうではない。L1 でのヒットは L2 に届かないので、頻繁に使われ、そのため L1 に留まるブロックは、L2 の LRU カウンタからは使われていないように見える。やがてそれは L2 で最も長く使われていないブロックとなり、L1 に残ったまま L2 では追い出される。

例 14.26. L1 は 2 ブロック、L2 は 3 ブロックを保持する。どちらもフルアソシアティブで LRU 置換をもち、初期状態は空であり、両方のレベルでミスしたブロックは両方に読み込まれる。表 14.4 はアクセス P, Q, P, R, P, S を追う。P への後のアクセスは L1 でヒットするので、L2 が P を最後に見るのは最初のアクセスである。S でのミスの時点で P は L2 で最も長く使われていないブロックであり、まだ L1 にあるにもかかわらず追い出される。

表 14.4: 両方のレベルが LRU 置換をもつ 2 レベルの階層 (例 14.26)。内容は、最も長く使われていないものから最も最近使われたものの順に並べる。最後のアクセスで包含性が破れる。

アクセス	L1	L2	後の L1	後の L2
P	ミス	ミス	P	P
Q	ミス	ミス	P Q	P Q
P	ヒット	—	Q P	P Q
R	ミス	ミス	P R	P Q R
P	ヒット	—	R P	P Q R
S	ミス	ミス	P S	Q R S

したがって包含性が欲しければ、それを強制しなければならない。1 つの方法は、L2 のすべてのラインに包含ビットを設け、そのブロックが L1 にもある間これを立てておくことである。L2 は、このビットが立っているラインを犠牲に選ばない。

この章で、単一プロセッサにおけるキャッシュの議論は完結する (第 19 章ではマルチプロセッサのキャッシュに戻る)。第 15 章では、キャッシュと主記憶を作るメモリ技術に話を移す。

要点 — 章のまとめ

AMAT = ヒット時間 + ミス率 × ミスペナルティであり、各項にはそれぞれの技法がある。ヒット時間は、探索のパイプライン化、仮想アドレスでインデックスしながら TLB が変換し、キャッシュサイズ $\leq N \times$ ページサイズならエイリアシングを避けられる VIPT キャッシュ、ウェイ予測、そして安価な置換アルゴリズムである NMRU と PLRU によって短くなる。ミスは初期参照ミス、容量性ミス、競合性ミスのいずれかであり、ミス率は適切なブロックサイズ、プリフェッチ (プリフェッチ命令、ストリームバッファ、ストライドプリフェッチャ、相関プリフェッチャ)、ループ交換によって下がる。ミスペナルティは、処理中のミスを MSHR で追跡するミスアンダーミス対応と、レベルごとに局所ミス率で AMAT を計算するキャッシュ階層によって下がる。大域ミス率と MPKI は、あるレベルより先へ進むアクセスがどれだけあるかを表す。各レベルが LRU を用いる場合、包含性は自動的に成り立たない。

メモリ

講義動画	Lesson 15, 動画 1-14
教科書	H&P [1] §2.2
キーワード	SRAM, DRAM, メモリセル, ワード線, ビット線, センスアンプ, 破壊読み出し, トレンチセル, リフレッシュ, 行デコーダ, RAS と CAS, ファストページモード, フロントサイドバス, ノースブリッジ

第 12~14 章では、主記憶をキャッシュの背後に隠れた大きく遅い装置として扱ってきた。また第 1 章では、プロセッサとメモリの速度の差、すなわちメモリウォールが広がり続けることを見た。この章ではメモリそのものの内部を見る。まず、メモリを作る 2 つの技術—キャッシュ用の静的 RAM と主記憶用の動的 RAM—を、1 ビットを蓄える回路まで掘り下げて述べる。続いて、DRAM セルがどのようにチップ上に構成されアクセスされるか、そしてチップがどのようにプロセッサに接続されるかを示す。これらを合わせると、主記憶が遅い理由と、主記憶がキャッシュのより速い技術で作られていない理由が分かる。

15.1 メモリ技術

キャッシュも主記憶も、ランダムアクセスメモリである。

定義 15.1 (ランダムアクセス). どの位置もそのアドレスによって直接、しかもどの位置についてもほぼ同じ時間で読み書きできるとき、そのメモリは**ランダムアクセス**を提供するという。

その反対が順次アクセス (第 16 章) である。磁気テープのような媒体では、現在の位置と要求された位置の間にあるデータすべてを通り過ぎるまで媒体を動かさなければならない。ランダムアクセスメモリ (RAM) は 2 つの技術で作られ、その名前は、電源が入っている間に、記憶されたビットに何が起こるかに由来する。

定義 15.2 (SRAM と DRAM). **SRAM** では、記憶されたビットは電源が入っている限りその値を保つ。**DRAM** では、記憶されたビットは、電源が入っていても、周期的に書き直さない限り短時間で値を失う。

これらの名前は、電源がないときに何が起こるかについては何も言っていない。どちらの技術も揮発性であり、電源を切れば内容をすべて失う。

2 つの技術は 1 ビットを蓄える回路が異なり、この違いが速度とコストを決める。SRAM は 1 ビット当たり複数のトランジスタを使い、通常の設計では 6 個である (15.2 節)。DRAM は 1 個のトランジスタしか使わない (15.3 節)。したがって SRAM は 1 ビット当たりのチップ面積が大きくビット当たりの価格も高いが、速い。そのセルがビット線を能動的に駆動するのに対し、DRAM セルはわずかな電荷を保持するだけで、その電荷は検出し、書き戻し、リフレッシュしなければならないからである。DRAM は同じ面積により多くのビットを詰め込め、ビット当たりの価格も安い。速くなくてはならず小さくてよいキャッシュは SRAM で作られ、大きく安くなければならない主記憶は DRAM で作られる。

15.2 SRAM セル

メモリはビットを、同一の回路からなる 2 次元配列に蓄える。1 ビットを蓄える回路を**メモリセル**という。セルは行と列に並べられ、配列を横切る 2 組の配線を通じてアクセスされる。

定義 15.3 (ビット線とワード線). **ビット線**はセルの列に沿って走り、その列のセルから読まれる値、あるいはそのセルに書かれる値を運ぶ。**ワード線**はセルの行に沿って走り、活性化されると、その行のすべてのセルをそれぞれの列のビット線に接続する。

出典 Lesson 15, 動画 1-3 ; H&P §2.2.

▷ random access

▷ static random-access memory

▷ dynamic random-access memory

出典 Lesson 15, 動画 4 ; H&P §2.2.

▷ memory cell

▷ bit line

▷ word line

SRAM セル (図 15.1a) では、ビットは輪をなすように接続された 2 つのインバータが保持する。各インバータの出力が他方の入力を駆動する。最初のインバータが 1 を出力すれば 2 番目は 0 を出力し、それが今度は最初のインバータを 1 に保つ。この輪は 2 つの状態のどちらでも安定で、インバータに電源が入っている限りその状態を保つ。これが SRAM を静的たらしめている。ゲートがワード線に接続されたアクセストランジスタが、輪の一方のノードをビット線に接続する。

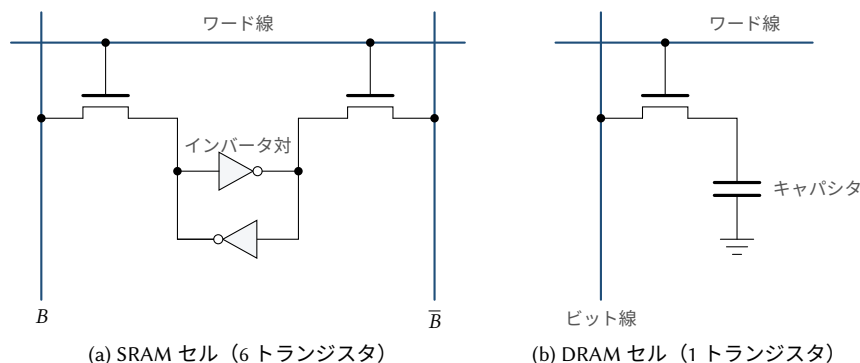


図 15.1: 1 ビットの記憶. (a) SRAM セルは 2 つのインバータの輪にビットを保持する。輪のノードは、2 つのアクセストランジスタによって相補的なビット線 B と $\bar{B}$ に接続される. (b) DRAM セルは、1 個のアクセストランジスタの奥にあるキャパシタの電荷としてビットを保持する。

書き込み ビット線を新しい値に強く駆動し、ワード線を活性化する。ビット線は、ノードを駆動している小さなインバータを押し負かす。輪はビット線に一致する状態に落ち着き、ワード線を非活性化した後もその状態を保つ。

読み出し ワード線を活性化すると、輪がビット線を駆動する。セルは小さくしなければならぬのでインバータのトランジスタは小さいが、ビット線は列のすべてのセルに接続された長い配線である。1 個のセルはビット線の電圧をゆっくりとしか変えられない。

読み出しを速めるために、ビットの反転値を保持する輪のもう一方のノードを、2 つ目のアクセストランジスタを通じて 2 本目のビット線 $\bar{B}$ に接続する。ワード線が活性化されると、セルは 2 本のビット線を互いに逆向きに引っ張り、値は両者を比較することで得られる。

定義 15.4 (センスアンプ). センスアンプは、ビット線上の小さな電圧変化、あるいは 1 対のビット線の間の小さな差を検出し、それを完全な論理 0 または 1 に増幅する。

▷ sense amplifier

相補的な対があれば、センスアンプは 2 本のビット線を比較するだけでよい。セルが 2 本を逆向きに動かすので、ごくわずかな振れの後には差の符号が明らかであり、基準電圧も要らない。1 本のビット線だけであれば、その値を確実に判定できるまでにはるかに大きく動かなければならない。この対は書き込みにも役立つ。値とその反転値が輪の両側に同時に働くからである。こうしてできるセルは、各インバータに 2 個とアクセストランジスタ 2 個の計 6 個のトランジスタをもち、ワード線 1 本、ビット線 2 本、そしてインバータの電源接続を必要とする。

15.3 DRAM セル

DRAM セルはビットをキャパシタ上の電荷として蓄える (図 15.1b)。ワード線が制御する 1 個のアクセストランジスタが、キャパシタをビット線に接続する。充電されたキャパシタが 1 を、放電されたキャパシタが 0 を表す。書き込むにはビット線をその値にしてワード線を活性化する。するとキャパシタはトランジスタを通じて充電または放電される。ワード線を非活性化するとトランジスタが遮断し、電荷を孤立させる。

しかし、遮断したトランジスタは完全なスイッチではない。静的電力 (第 1 章) を生むのと同じ不完全さによってわずかな電流が流れ続け、記憶された 1 の電荷

出典 Lesson 15, 動画 5-6 ; H&P §2.2 ; [28].

は少しずつ漏れ出して、やがてそのセルは 0 と区別できなくなる。これが DRAM を動的たらしめている。ビットは、電荷が失われすぎる前に書き直されたときだけ生き延びる。

定義 15.5 (リフレッシュ). DRAM セルのビットを読み出し、電荷が漏れ出してしまう前に完全な電荷で書き戻すことを**リフレッシュ**という。どのセルも、そのデータが使われているかどうかに関わらず、周期的にリフレッシュしなければならない。

DRAM セルの読み出しも繊細である。まずビット線を 0 と 1 の中間の電圧にし、それからワード線を活性化する。キャパシタはビット線と電荷を分け合い、セルが 1 を保持していればビット線の電圧はわずかに上がり、0 を保持していればわずかに下がる。セルのキャパシタは長いビット線の容量に比べて極めて小さいので、この変化は非常に小さく、検出にはセンスアンプが必要である。電荷を分け合った後、キャパシタは中間の電圧の近くに取り残される。すなわち、記憶されていた値は失われている。

定義 15.6 (破壊読み出し). **破壊読み出し**とは、記憶された値を壊してしまう読み出しのことである。したがって、DRAM セルの読み出しの後には必ず、読み出した値をそのセルに書き戻す動作が続く。

SRAM の読み出しは破壊的ではない。インバータの輪がノードを駆動し続け、値を自力で復元するからである。

キャパシタは面積の問題も引き起こす。チップ表面に平らに作った場合、確実に検出できるだけの大きさのキャパシタはトランジスタよりはるかに広い面積を占め、1 トランジスタのセルがもつ密度上の利点の多くが失われてしまう。

定義 15.7 (トレンチセル). **トレンチセル**とは、アクセストランジスタの隣のシリコンに垂直に掘った、深く狭い溝（トレンチ）の中にキャパシタを形成した DRAM セルである（図 15.2）。

トレンチによって、キャパシタはチップ表面をほとんど使わずに広い電極面積を得る。もう 1 つのよく使われる設計は、現在の DRAM の大半が採用しているもので、キャパシタをアクセストランジスタの上に積み重ねる。いずれにせよ、セルが占める面積はトランジスタよりわずかに大きい程度である。また、SRAM セルより配線も少なく済む。ビット線は 2 本ではなく 1 本であり、インバータのための電源接続も要らない。表 15.1 に 2 つのセルの比較をまとめる。

表 15.1: SRAM と DRAM の比較。

	SRAM	DRAM
記憶素子	インバータの輪	キャパシタ
1 ビット当たりトランジスタ数	6	1
セル当たりビット線数	2	1
電源投入中の値の保持	保つ	リフレッシュすれば保つ
読み出し	非破壊	破壊的、書き戻しが続く
速度	速い	遅い
1 ビット当たり面積とコスト	大きい	小さい
用途	キャッシュ	主記憶

要点

DRAM はビットを 1 個のトランジスタの奥のわずかな電荷として蓄える。そのため密度が高く安価だが遅い。読み出しは必ず電荷を壊すので書き戻しが続かなければならず、漏れ電流が周期的なリフレッシュを強いるからである。SRAM は 1 ビット当たり 6 個のトランジスタという代償で、そのどちらも避けている。

▷ refresh

▷ destructive read

▷ trench cell

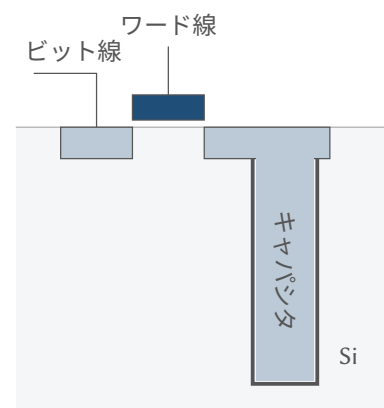


図 15.2: トレンチセルの断面（模式図）。

15.4 メモリチップの構成

メモリチップのセルは 2^r 行 2^c 列の配列をなし (図 15.3), $r+c$ ビットのアドレスは r ビットの行アドレスと c ビットの列アドレスに分割される。例えば, アドレスの上位ビットと下位ビットである。DRAM チップの 1 ビットの読み出しは 4 つの段階で進む。

1. **行デコーダ**が行アドレスを, 2^r 本のワード線のうちちょうど 1 本の活性化に変換する。
2. 選ばれた行のすべてのセルがそれぞれのビット線に接続される。ビット線 1 本につき 1 個のセンスアンプが値を検出し, 行全体が行ラッチに保持される。
3. **列マルチプレクサ**が列アドレスを使ってラッチの 1 ビットを選び, データ出力へ送り出す。
4. 読み出しは行のすべてのセルにとって破壊的だったので, ラッチの内容をビット線を通じて行に書き戻す。

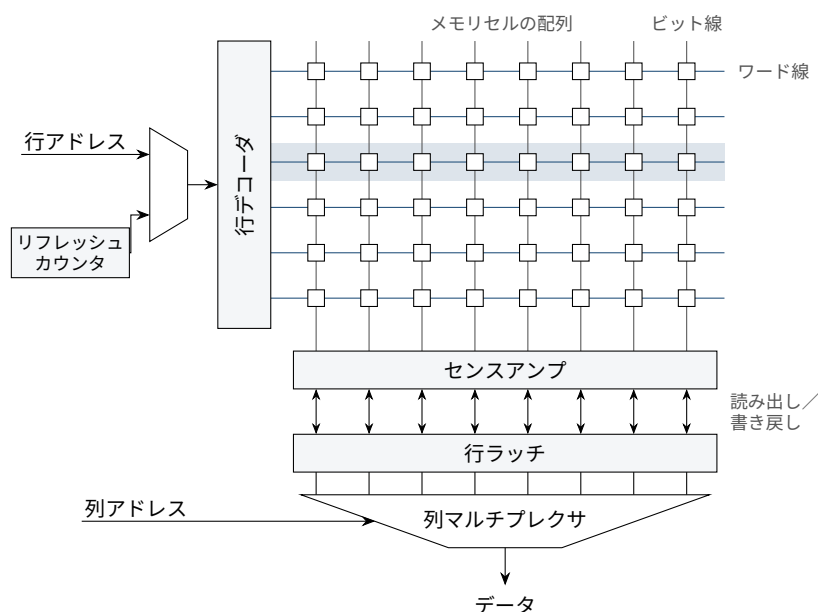


図 15.3: DRAM チップの構成。行デコーダは, 行アドレスまたはリフレッシュカウンタ (15.5 節) が選ぶワード線を活性化する。センスアンプが行全体を行ラッチに読み出し, 列マルチプレクサが 1 ビットを選び, ラッチが行に書き戻される。

書き込みも同じ経路を使う。上と同じように行をラッチに読み出し, 列アドレスが選ぶビットをラッチ上で書き込むべき値に置き換え, 書き戻しによって変更された行を格納する。このように, DRAM チップにおける書き込みとは, 書き戻す前にラッチを変更する読み出しである。

注 15.8. 行を閉じるときに書き戻される行ラッチは, この章を通じて用いる単純化したモデルである。実際の DRAM チップでは, センスアンプ自身が行を保持する。ワード線が活性のままの間にセンスアンプがビット線を完全なレベルまで駆動し, それによって行のセルが復元される。また, ラッチに書き込まれたビットはただちにそのセルに届く。行を閉じる操作はワード線を落とし, ビット線を次の読み出しに備えさせる。

アドレスの 2 つの部分にかかる時間は大きく異なる。行を選ぶには, デコードし, 長いワード線を駆動し, 長いビット線上のわずかな電荷をセンスアンプが検出するのを待たなければならない。列を選ぶのは, すでにラッチにあるビットの間でマルチプレクサを切り替えるだけである。15.6 節と 15.7 節はこの差を利用する。

注 15.9. SRAM チップも同じように構成され, 行デコーダ, センスアンプ, 列マルチプレクサをもつ。ただし, その読み出しは破壊的ではないので, 読み出しの後に書き戻しは続かず, またそのセルにリフレッシュも要らない。

出典 Lesson 15, 動画 7 ; H&P §2.2.

▷ row decoder

▷ column multiplexer

15.5 リフレッシュ

DRAM チップのどの行も、長さ T の区間ごとに少なくとも 1 回はリフレッシュしなければならない。 T は普通、数十ミリ秒である。リフレッシュは通常の読み出し経路を使う。すなわち、出力を使わずにラッチを書き戻す読み出し（15.4 節）である。

プログラムによる読み出しは、触れた行をリフレッシュする。しかし、そのような読み出しは当てにできない。長い間、あるいはまったく読まれない行があるからである。そこで、すべての行を系統的にリフレッシュする。リフレッシュカウンタが行番号を保持する。周期的に、行アドレスの代わりにカウンタの値が行デコーダに与えられ（図 15.3）、その行が読み出されて書き戻され、カウンタは次の行へ進み、最後の行の後は先頭に戻る。行数を R とし、そのそれぞれを区間 T ごとに 1 回リフレッシュしなければならないとすると、チップは、次の長さの区間ごとに少なくとも 1 回、1 つの行をリフレッシュしてカウンタを進めなければならない。

$$t_{\text{row}} = \frac{T}{R}. \quad (15.1)$$

ある行をリフレッシュしている間、チップは読み出しにも書き込みにも応じられず、その間に到着したアクセスは待たなければならない。こうしてリフレッシュは DRAM が遅い理由をもう 1 つ増やし、そのコストは行数とともに増える。

例 15.10. ある DRAM チップは 8192 行をもち、そのセルは 64 ms ごとにリフレッシュしなければならない。式 (15.1) により、チップは $64 \text{ ms} / 8192 \approx 7.8 \mu\text{s}$ ごとに 1 行をリフレッシュしなければならない。すなわち毎秒 128 000 行である。1 行のリフレッシュに 50 ns かかるとすると、チップは毎秒のうち $128\,000 \times 50 \text{ ns} = 6.4 \text{ ms}$ 、すなわち時間の 0.64 % をリフレッシュに費やす。行数が 4 倍で T が同じチップでは 2.56 % となる。

15.6 アドレスの多重化

2^{r+c} 個のセルをもつチップには $r+c$ ビットのアドレスが必要であり、並列に送るビットはどれもパッケージのピン 1 本とチップへの配線 1 本を消費する。しかし、行アドレスと列アドレスが同時に必要になるわけではない。行アドレスは、遅い行の選択を始めるために先に必要である。列アドレスは、行がラッチに入ってから初めて必要になる。そこで DRAM チップは両方の部分を同じピンから順に受け取り、2 本の制御信号が、どちらの部分が来ているかをチップに伝える。

定義 15.11 (RAS と CAS). **行アドレスストローブ (RAS)** がアサートされると、チップはアドレスピン上の値を行アドレスとして取り込み、ワード線を活性化して行をラッチに読み出す。**列アドレスストローブ (CAS)** がアサートされると、チップはアドレスピン上の値を列アドレスとして取り込み、ラッチのそのビットを選ぶ。

読み出しでは、選ばれたビットがデータ出力に現れる。CAS と同時にライトイネーブル信号がアサートされていれば、代わりに、ラッチ中のそのビットがデータ入力の値に置き換えられる。RAS をデアサートするとアクセスが終わり、（注 15.8 のモデルでは）ラッチが書き戻されて行が閉じられる。図 15.4a は、この種のアクセスを 2 回示している。

多重化により、アドレスピンは $r+c$ 本から $\max(r, c)$ 本に減る。行数と列数が等しければ半分である。これによってパッケージは小さく安くなり、チップと、それを駆動するメモリコントローラ（15.8 節）との間の配線も減る。例えば、16 384 行 16 384 列に 2^{28} 個のセルをもつチップは、28 本ではなく 14 本のアドレスピンで済む。

15.7 ファストページモード

図 15.4a の流れでは、どのアクセスも行を開き、1 つの列を使い、また行を閉じる。次のアクセスが同じ行に対するものであってもそうである。しかし、いった

出典 Lesson 15, 動画 8–9 ; H&P §2.2.

出典 Lesson 15, 動画 10 ; H&P §2.2.

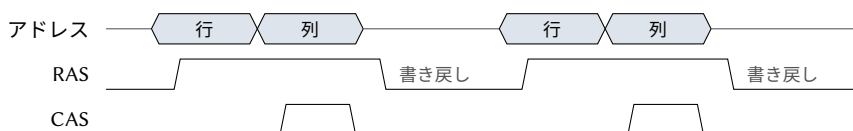
▷ row address strobe

▷ column address strobe

実際のチップでは RAS と CAS は負論理である。図 15.4 ではアサートされた信号を高レベルで描いている。

出典 Lesson 15, 動画 11–12.

(a) 同じ行への 2 回のアクセス



(b) ファストページモード

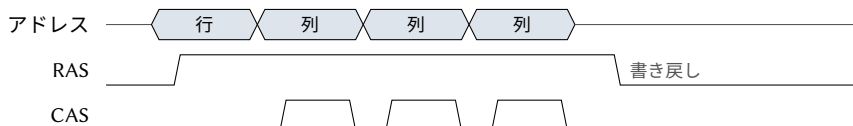


図 15.4: アドレスピンを多重化した DRAM チップの信号. (a) 各アクセスは RAS で行を開き, CAS で列を選び, 行を閉じる. (b) ファストページモード (15.7 節) では, 行を一度だけ開いて 3 つの列にアクセスし, RAS をデアサートしたときに行を書き戻す.

ん行を開いてしまえば, その行のビットはすべてラッチにあり, その行の別のビットに届くには新しい列アドレスがあればよい.

定義 15.12 (ファストページモード). ファストページモードでは, RAS で一度開いた行を開いたままにし, 続けざまの CAS パルスで複数の列アドレスを与えて, そのたびに行の 1 ビットを読み書きする. 行が書き戻されるのは, 最後に RAS がデアサートされるときに 1 回だけである (図 15.4b).

ファストページモードでは, 高価な段階—行の選択とセンス, そして書き戻し—をアクセスのまとまり全体につき 1 回だけ行い, 各アクセスに要するのは速い列の選択だけである. 空間的局所性 (第 12 章) によって頻繁に起こる近いアドレスへのアクセスは, 普通はアドレスの下位ビットだけが異なるので, 同じ行を共有する. 行をどれだけ長く開いたままにするかは, RAS と CAS を駆動するコントローラが決める. 後続のアクセスが同じ行を使う間は行を開いたままにでき, 別の行にアクセスするときや, どれかの行をリフレッシュするときには, その前に行を閉じなければならない. したがって, 節約できる量は, 連続するアクセスのうちどれだけが同じ行を共有するかによる.

例 15.13. ある DRAM チップでは, 行を開く (RAS, デコードとセンス) のに 30 ns, CAS で列を選ぶのに 5 ns, 行を閉じる (書き戻し) のに 10 ns かかる. 表 15.2 の順に 5 つのアクセスが到着する. 表中の組は (行, 列) を表す. ファストページモードがなければ, どのアクセスも自分の行を開いて閉じるので $5 \times 45 \text{ ns} = 225 \text{ ns}$ かかる. ファストページモードでは行を開いて閉じるのが 3 回で済み, 表の合計は 145 ns となる. 行 22 へのアクセスが最後に来ていれば, 行 7 への 4 つのアクセスが 1 回の開きを共有し, 合計は 105 ns だったはずである.

表 15.2: 例 15.13 のアクセスをファストページモードで実行した場合.

アクセス	動作	時間 (ns)
読み出し (7, 12)	行 7 を開く, 列 12 を CAS	35
読み出し (7, 13)	列 13 を CAS	5
書き込み (7, 14)	ライトイネーブルとともに列 14 を CAS	5
読み出し (22, 3)	行 7 を閉じる, 行 22 を開く, 列 3 を CAS	45
読み出し (7, 15)	行 22 を閉じる, 行 7 を開く, 列 15 を CAS	45
—	行 7 を閉じる	10
合計		145

15.8 DRAM とプロセッサの接続

プロセッサが主記憶を待つ時間は, DRAM チップのアクセス時間だけではない.

▷ fast page mode

注意 ここでいう「ページ」はチップの行のことであり, 仮想記憶のページ (第 13 章) ではない.

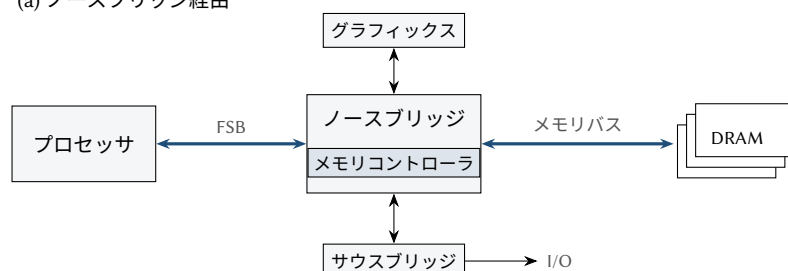
両者の間には他のチップとバスを通る経路があり、その伝統的な形を 図 15.5a に示す。

定義 15.14 (フロントサイドバスとノースブリッジ). フロントサイドバス (FSB) は、プロセッサをノースブリッジに接続する。ノースブリッジは、プロセッサを主記憶と、グラフィックスアダプタのような高速なデバイスと、そして遅い I/O デバイスが接続されるサウスブリッジ (第 16 章) とに結ぶチップである。

ノースブリッジはメモリコントローラを内蔵する。メモリコントローラは、FSB を通じて届くメモリアクセスのそれぞれを DRAM チップの信号に翻訳する。すなわち、アドレスを行アドレスと列アドレスに分割し、メモリバス上に RAS, CAS, ライトイネーブルを駆動する。したがって、キャッシュミスによるメモリアクセスは、プロセッサから FSB を通ってノースブリッジへ、そこからメモリバスを通して DRAM チップへと進み、データは同じ道に戻る。この各段階がレイテンシを加える上に、FSB は、プロセッサとノースブリッジの先のデバイスとの間のそれ以外のトラフィックもすべて運ぶ。

- ▷ front-side bus
- ▷ northbridge
- ▷ memory controller

(a) ノースブリッジ経由



(b) メモリコントローラ統合



図 15.5: プロセッサと DRAM の間の経路。(a) ノースブリッジ内のメモリコントローラには、フロントサイドバスを通じて到達する。(b) メモリコントローラをプロセッサに統合すると、メモリバスはプロセッサに直接接続される。

新しいプロセッサは、メモリコントローラをプロセッサチップ上に統合し、メモリバスをプロセッサに直接接続する (図 15.5b)。こうすると FSB とノースブリッジはメモリへの経路から消え、主記憶へのすべてのアクセスのレイテンシが下がる。それでもなお、この章にまとめた理由により、DRAM のアクセスはプロセッサよりはるかに遅く、第 12~14 章のキャッシュは欠かせないままである。第 16 章では、さらに遅いが電源がなくてもデータを保つ記憶装置に話を移す。

要点 – 章のまとめ

RAM は、6 個のトランジスタからなるセルがインバータの輪にビットを保持する SRAM として、あるいは 1 個のトランジスタからなるセル (例えばトレンチセル) がキャパシタ上の電荷としてビットを保持する DRAM として作られる。SRAM は速くて高価であり、キャッシュに使われる。DRAM は密度が高くて安価であり、主記憶に使われるが、その読み出しは破壊的で書き戻しが続く、漏れ電流がすべての行のリフレッシュを強いるので、 T/R の区間ごとに 1 行をリフレッシュしなければならない。DRAM チップは、行デコーダで行を選び、センスアンプでそれをラッチに読み出し、列マルチプレクサで 1 ビットを選ぶ。行アドレスと列アドレスはアドレスピンを共有し、RAS と CAS で区別される。そしてファストページモードは、1 つの行への複数のアクセスを 1 回の開きで処理する。メモリアクセスは、伝統的にはフロントサイドバスを通してノースブリッジ内のメモリコントローラへ、さらにメモリバスを通して進む。新しいプロセッサはメモリコントローラを統合し、メモリバスを直接駆動する。

参考文献

- [1] John L. Hennessy and David A. Patterson. *Computer Architecture: A Quantitative Approach*. 6th ed. Morgan Kaufmann, 2017.
- [2] Gordon E. Moore. “Cramming More Components onto Integrated Circuits”. In: *Electronics* 38.8 (1965).
- [3] William A. Wulf and Sally A. McKee. “Hitting the Memory Wall: Implications of the Obvious”. In: *ACM SIGARCH Computer Architecture News* 23.1 (1995), pp. 20–24.
- [4] Standard Performance Evaluation Corporation. *SPEC Benchmarks*. URL: <https://www.spec.org/> (visited on 09/14/2026).
- [5] Gene M. Amdahl. “Validity of the Single Processor Approach to Achieving Large Scale Computing Capabilities”. In: *AFIPS Conference Proceedings* 30 (1967), pp. 483–485.
- [6] Johnny K. F. Lee and Alan Jay Smith. “Branch Prediction Strategies and Branch Target Buffer Design”. In: *Computer* 17.1 (1984), pp. 6–22.
- [7] James E. Smith. “A Study of Branch Prediction Strategies”. In: *Proceedings of the 8th Annual International Symposium on Computer Architecture (ISCA)*. 1981, pp. 135–148.
- [8] Tse-Yu Yeh and Yale N. Patt. “Two-Level Adaptive Training Branch Prediction”. In: *Proceedings of the 24th Annual International Symposium on Microarchitecture (MICRO)*. 1991, pp. 51–61.
- [9] Scott McFarling. *Combining Branch Predictors*. WRL Technical Note TN-36. Digital Equipment Corporation, Western Research Laboratory, 1993.
- [10] David R. Kaeli and Philip G. Emma. “Branch History Table Prediction of Moving Target Branches Due to Subroutine Returns”. In: *Proceedings of the 18th Annual International Symposium on Computer Architecture (ISCA)*. 1991, pp. 34–42.
- [11] J. R. Allen et al. “Conversion of Control Dependence to Data Dependence”. In: *Proceedings of the 10th ACM SIGACT-SIGPLAN Symposium on Principles of Programming Languages (POPL)*. 1983, pp. 177–189.
- [12] Scott A. Mahlke et al. “A Comparison of Full and Partial Predicated Execution Support for ILP Processors”. In: *Proceedings of the 22nd Annual International Symposium on Computer Architecture (ISCA)*. 1995, pp. 138–150.
- [13] Robert M. Keller. “Look-Ahead Processors”. In: *ACM Computing Surveys* 7.4 (1975), pp. 177–195.
- [14] David W. Wall. “Limits of Instruction-Level Parallelism”. In: *Proceedings of the Fourth International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*. 1991, pp. 176–188.
- [15] Robert M. Tomasulo. “An Efficient Algorithm for Exploiting Multiple Arithmetic Units”. In: *IBM Journal of Research and Development* 11.1 (1967), pp. 25–33.
- [16] James E. Smith and Andrew R. Pleszkun. “Implementing Precise Interrupts in Pipelined Processors”. In: *IEEE Transactions on Computers* 37.5 (1988), pp. 562–573.
- [17] Monica Lam. “Software Pipelining: An Effective Scheduling Technique for VLIW Machines”. In: *Proceedings of the ACM SIGPLAN 1988 Conference on Programming Language Design and Implementation (PLDI)*. 1988, pp. 318–328.
- [18] Joseph A. Fisher. “Trace Scheduling: A Technique for Global Microcode Compaction”. In: *IEEE Transactions on Computers* C-30.7 (1981), pp. 478–490.
- [19] Joseph A. Fisher. “Very Long Instruction Word Architectures and the ELI-512”. In: *Proceedings of the 10th Annual International Symposium on Computer Architecture (ISCA)*. 1983, pp. 140–150.

- [20] Michael S. Schlansker and B. Ramakrishna Rau. “EPIC: Explicitly Parallel Instruction Computing”. In: *Computer* 33.2 (2000), pp. 37–45.
- [21] Peter J. Denning. “The Locality Principle”. In: *Communications of the ACM* 48.7 (2005), pp. 19–24.
- [22] Maurice V. Wilkes. “Slave Memories and Dynamic Storage Allocation”. In: *IEEE Transactions on Electronic Computers* EC-14.2 (1965), pp. 270–271.
- [23] T. Kilburn et al. “One-Level Storage System”. In: *IRE Transactions on Electronic Computers* EC-11.2 (1962), pp. 223–235.
- [24] Mark D. Hill and Alan Jay Smith. “Evaluating Associativity in CPU Caches”. In: *IEEE Transactions on Computers* 38.12 (1989), pp. 1612–1630.
- [25] David Callahan, Ken Kennedy, and Allan Porterfield. “Software Prefetching”. In: *Proceedings of the Fourth International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS-IV)*. 1991, pp. 40–52.
- [26] Norman P. Jouppi. “Improving Direct-Mapped Cache Performance by the Addition of a Small Fully-Associative Cache and Prefetch Buffers”. In: *Proceedings of the 17th Annual International Symposium on Computer Architecture (ISCA)*. 1990, pp. 364–373.
- [27] Jean-Loup Baer and Wen-Hann Wang. “On the Inclusion Properties for Multi-Level Cache Hierarchies”. In: *Proceedings of the 15th Annual International Symposium on Computer Architecture (ISCA)*. 1988, pp. 73–80.
- [28] Robert H. Dennard. “Field-Effect Transistor Memory”. U.S. pat. 3387286. International Business Machines Corporation. June 4, 1968.

索引

記号

1000 命令当たりミス数, 112

1BP, → 1 ビット予測器

1 ビット予測器, 16

2BC, → 2 ビット予測器

2BP, → 2 ビット予測器

2 ビット予測器, 17

A

AMAT, → 平均メモリアクセス時間

B

BHT, → 分岐履歴テーブル

BTB, → 分岐先バッファ

C

CAS, → 列アドレスストロープ

CDB, → 共通データバス

CMOVcc, 26

D

DRAM, 114

DSP, 77

F

FSB, → フロントサイドバス

G

GShare 予測器, 19

I

if 変換, 25, 69, 77

ILP, → 命令レベル並列性

IPC, → サイクル当たり命令数

Itanium, 28, 77

L

L1 キャッシュ, 81

L2 キャッシュ, 111

LLC, → 最終レベルキャッシュ

LRU, 86

LSQ, → ロードストアキュー

M

MOVN, 26

MOVZ, 26

MPKI, → 1000 命令当たりミス数

MSHR, → ミス状態管理レジスタ

N

NMRU, 105

N ビット履歴予測器, 18

P

PFN, → 物理フレーム番号

PHT, → パターン履歴テーブル

PLRU, → 疑似 LRU

PShare 予測器, 19

PTE, → ページテーブルエントリ

R

RAS, → リターンアドレススタック

RAT, → レジスタエイリアステーブル

RAW 依存, 31

ROB, → リオーダーバッファ

RS, → リザベーションステーション

S

SRAM, 114

T

TLB, 97

TLB ミス, 98

V

VIPT, → 仮想インデックス物理タグ
キャッシュ

VLIW, 74

VPN, → 仮想ページ番号

W

WAR 依存, 31

WAW 依存, 31

ア

アーキテクチャレジスタ, 31

アウトオブオーダー実行, 35, 37, 56

Amdahl の法則, 6

インオーダー実行, 35

インデックス, 83

インライン展開, 71, 77

ウェイ予測, 105

ウェハ, 3

エイリアシング, 103

オーバーラップミス, 110

カ

階層型予測器, 21

仮想アドレス, 91

仮想アドレスキャッシュ, 101

仮想アドレス空間, 91

仮想インデックス物理タグキャッ
シュ, 102

仮想記憶, 90

仮想ページ番号, 93

完全プレディケーション, 28, 77

幾何平均, 5

疑似 LRU, 106

偽の依存, 10, 31

木の高さ削減, 66, 76

キャッシュ, 80
 キャッシュ階層, 111
 キャッシュヒット, 80
 キャッシュブロック, 81
 キャッシュミス, 80
 キャッシュライン, 82
 行アドレスストローブ, 118
 競合性ミス, 107
 共通データバス, 39
 行デコーダ, 117
 局所性の原理, 79
 局所ヒット率, 112
 局所ミス率, 112
 空間的局所性, 79
 グローバル分岐履歴, 19
 結果書き込み, 39
 コード肥大化, 71
 高速化率, 4
 後方互換性, 76
 コミット, 50, 63
 コミットポインタ, 49
 コンピュータアーキテクチャ, 1

サ

サイクル当たり命令数, 35
 最古優先, 40
 最終レベルキャッシュ, 111
 時間的局所性, 79
 収穫通減, 6
 述語レジスタ, 28
 条件コード, 26
 条件付きムーブ, 26
 初期参照ミス, 106
 スーパースカラ, 56, 74
 ステージ, 8
 ストール, 9
 ストアロードフォワーディング, 60
 ストップビット, 77
 スライドプリフェッチャ, 108
 ストリームバッファ, 108
 スループット, 4
 正確な例外, 47
 制御依存, 9
 静的電力, 3
 性能の鉄則, 5
 成立分岐, 12
 セットアソシアティブキャッシュ,
 84
 センスアンプ, 115
 相関プリフェッチャ, 109
 ソフトウェアパイプラインニング, 73

タ

ターティビット, 87
 ダイ, 3
 大域ヒット率, 112
 大域ミス率, 112
 ダイレクトマップキャッシュ, 83
 タグ, 82

多段ページテーブル, 94
 単一レベルページテーブル, 94
 置換アルゴリズム, 86
 データ依存, 10
 ディスパッチ, 39
 トーナメント予測器, 20
 統合リザベーションステーション,
 55
 動的電力, 2
 Tomasulo のアルゴリズム, 38, 49
 トレーススケジューリング, 73
 トレンチセル, 116

ナ

内部断片化, 96
 ノースブリッジ, 120
 ノンブロッキングキャッシュ, 110

ハ

ハードウェアプリフェッチ, 108
 ハーフミス, 110
 パイプライン化キャッシュ, 100
 破壊読み出し, 116
 ハザード, 10
 パターン履歴テーブル, 19
 発行, 39
 発行ポインタ, 49
 ヒット時間, 80
 ビット線, 114
 ヒット率, 80
 ファストページモード, 119
 ファントム例外, 48
 フォワーディング, 11
 不成立分岐, 12
 不成立予測, 14
 物理アドレス, 90
 物理アドレスキャッシュ, 101
 物理フレーム番号, 93
 物理メモリ, 90
 物理レジスタ, 32
 歩留まり, 3
 フラッシュ, 9
 プリデコード, 22
 プリフェッチ, 108
 プリフェッチ命令, 108
 フルアソシアティブキャッシュ, 85
 古くなった結果, 41
 プレディケーション, 24
 ブロードキャスト, → 結果書き込み
 ブロッキングキャッシュ, 110
 ブロックオフセット, 82
 ブロックサイズ, 81
 ブロック番号, 82
 フロントサイドバス, 120
 分岐先アドレス, 12
 分岐先バッファ, 15
 分岐ヒント, 77
 分岐予測, 12
 分岐予測精度, 13

分岐予測ミス, 12
分岐予測ミス回復, 51
分岐履歴テーブル, 16
ページ, 91
ページオフセット, 93
ページサイズ, 96
ページテーブル, 91
ページテーブルウォーク, 98
ページテーブルエントリ, 94
ページフォールト, 92
ページフレーム, 91
平均メモリアクセス時間, 80
ベンチマーク, 4
包含性, 113
方向予測器, 15
飽和カウンタ, 17

マ

ミスアンダーミス対応, 110
ミス状態管理レジスタ, 110
ミスペナルティ, 80
ミス率, 80
ムーアの法則, 1
命令キュー, 38
命令スケジューリング, 67
命令レベル並列性, 33
メタ予測器, 20
メモリウォール, 2
メモリコントローラ, 120
メモリセル, 114

ヤ

有効ビット, 83

容量性ミス, 106
予測ミスペナルティ, 13

ラ

ライトアロケート方式, 87
ライトスルー方式, 87
ライトバック方式, 87
ランダムアクセス, 114
リオーダーバッファ, 49
リザベーションステーション, 38, 50
理想プロセッサ, 33
リターンアドレススタック, 22
リタイア, → コミット
リフレッシュ, 116
履歴ベース予測器, 18
ループ交換, 109
ループ展開, 69, 75
レイテンシ, 4
レジスタエイリアステーブル, 32, 52
レジスタリネーミング, 32
列アドレスストローブ, 118
列マルチプレクサ, 117
連想度, 83
ローカル分岐履歴, 19
ロードストアキュー, 59
ロードストアのアウトオブオーダー
実行, 60
ロードストアのインオーダー実行, 61

ワ

ワード線, 114